TRƯỜNG ĐẠI HỌC NHA TRANG

KHOA XÂY DỰNG

CƠ HỌC KẾT CẤU CÔNG TRÌNH XÂY DỰNG TỐI ƯU KẾT CẤU

Trương Thành Chung

Lê Thanh Cao

MỤC LỤC

LỜI NÓI ĐẦU 4

CHƯƠNG 1. NHẬP MÔN TỐI ƯU KẾT CẤU 6

1.1. Giới thiệu 6

1.2. Phát biểu tổng quát của bài toán tối ưu kết cấu 7

1.3. Công thức đồng thời và công thức lồng ghép của bài toán tối ưu kết cấu . . 8

1.4. Phân loại bài toán tối ưu 8

1.4.1. Tối ưu hóa kích thước 8

1.4.2. Tối ưu hóa hình dạng 9

1.4.3. Tối ưu hóa cấu trúc 9

CHƯƠNG 2. TỐI ƯU HÓA KẾT CẤU BẰNG PHƯƠNG PHÁP ĐỒ THỊ. 12

2.1. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất 12

2.2. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và ổn định . 13

2.3. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và chuyển vị 15

2.4. Tối ưu trọng lượng dầm công-xôn chịu ràng buộc chuyển vị 17

2.5. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc ứng suất 19

2.6. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc độ cứng 26

2.7. Bài tập 27

CHƯƠNG 3. CƠ SỞ TOÁN HỌC QUY HOẠCH LỒI 32

3.1. Cực trị địa phương và cực trị toàn cục 32

3.2. Tính lồi 33

3.2.1. Định nghĩa tập lồi và hàm lồi 33

3.2.2. Tính chất của bài toán tối ưu lồi 34

3.2.3. Kiểm tra tính lồi của hàm khả vi 34

3.3. Điều kiện tối ưu KKT của bài toán tối ưu hóa có ràng buộc 35

3.4. Đối ngẫu Lagrange 41

3.5. Bài tập 45

CHƯƠNG 4. CÁC THUẬT TOÁN XẤP XỈ LỒI TƯỜNG MINH TUẦN TỰ 50

4.1. Quy trình giải tổng quát 50

4.2. Quy hoạch tuyến tính tuần tự SLP 50

4.3. Quy hoạch toàn phương tuần tự SQP 51

4.4. Tuyến tính hóa lồi CONLIN 51

4.5. Phương pháp tiệm cận động MMA 52

4.6. Bài tập 55

CHƯƠNG 5. TỐI ƯU ĐỘ CỨNG GIÀN. 57

5.1. Thiết lập bài toán tối ưu độ cứng giàn 57

5.1.1. Công thức lồng ghép 57

5.1.2. Phân tích độ nhạy 58

5.2. Tối ưu độ cứng giàn sử dụng phương pháp tiêu chuẩn tối ưu 59

5.2.1. Thiết kế đồng ứng suất 59

5.2.2. Phương pháp tiêu chuẩn tối ưu OC 60

5.2.3. Tối ưu độ cứng giàn sử dụng phương pháp tiệm cận động 61

5.3. Bài tập 63

CHƯƠNG 6. TỐI ƯU CẤU TRÚC 68
6.1. Phát biểu bài toán tối ưu cấu trúc 68
6.2. Thuật toán tối ưu cấu trúc SIMP 69
CHƯƠNG 7. TỐI ƯU HÓA ĐA MỤC TIÊU 72
7.1. Giới thiệu 72
7.2. Phát biểu bài toán đa mục tiêu 73
7.3. Khái niệm Pareto tối ưu 73
7.4. Một số cách tiếp cận trong tối ưu hóa đa mục tiêu 75
7.4.1. Phương pháp tổng trọng số (weighted sum method) 76
7.4.2. Phương pháp ràng buộc epsilon (epsilon-constraint method) 76
7.4.3. Phương pháp tiến hóa đa mục tiêu 76
7.5. Ví dụ minh họa 77
7.6. Bài tập 79
CHƯƠNG 8. THUẬT TOÁN TỐI ƯU METAHEURISTIC 81
8.1. Giới thiệu 81
8.2. Thuật toán di truyền 82
8.2.1. Nguyên lý cơ bản 82
8.2.2. Quy trình vận hành 82
8.3. Thuật toán luyện kim mô phỏng 83
8.3.1. Nguyên lý cơ bản 83
8.3.2. Quy trình vận hành 84
8.4. Ví dụ minh họa 84
8.5. Bài tập 89
PHỤ LỤC CODE 92
TÀI LIỆU THAM KHẢO 160

LỜI NÓI ĐẦU

Cuốn sách này trình bày các cơ sở và phương pháp tối ưu hóa kết cấu trong kỹ thuật xây dựng, từ các bài toán minh họa bằng phương pháp đồ thị đến các thuật toán tối ưu hiện đại. Nội dung được biên soạn theo hướng ngắn gọn, hệ thống và thực hành, giúp người đọc nắm được mô hình toán học, điều kiện ràng buộc và cách triển khai tính toán cho các dạng kết cấu thường gặp.

Trong thiết kế kết cấu truyền thống, kỹ sư thường bắt đầu từ một phương án sơ bộ, sau đó kiểm tra và hiệu chỉnh nhiều lần để thỏa mãn các yêu cầu về độ bền, độ cứng, ổn định và khả năng thi công. Cách tiếp cận này phụ thuộc nhiều vào kinh nghiệm và thường chỉ cho một phương án chấp nhận được, chứ chưa chắc là phương án hiệu quả nhất. Tối ưu hóa kết cấu cung cấp một khuôn khổ toán học và tính toán để biến quá trình hiệu chỉnh đó thành một bài toán có mục tiêu rõ ràng, có ràng buộc cụ thể và có tiêu chí đánh giá định lượng.

Mục tiêu của cuốn sách là giới thiệu các khái niệm nền tảng và các phương pháp tính thường dùng trong tối ưu hóa kết cấu, đồng thời liên hệ trực tiếp với các bài toán cơ học kết cấu quen thuộc như hệ giàn, dầm, khung và bài toán tối ưu topo. Tài liệu không chỉ trình bày công thức toán học, mà còn nhấn mạnh ý nghĩa kỹ thuật của các biến thiết kế, hàm mục tiêu, ràng buộc và nghiệm tối ưu. Nhiều ví dụ được xây dựng theo hướng từng bước để người đọc thấy rõ cách chuyển từ mô hình kết cấu sang mô hình tối ưu, cách diễn giải nghiệm và cách kiểm tra điều kiện tối ưu.

Cuốn sách hướng đến sinh viên ngành kỹ thuật xây dựng, học viên cao học, kỹ sư thiết kế và những người quan tâm đến ứng dụng tối ưu hóa trong cơ học kết cấu. Người đọc nên có kiến thức cơ bản về cơ học kết cấu, sức bền vật liệu, đại số tuyến tính và giải tích. Tuy nhiên, các khái niệm tối ưu hóa quan trọng đều được nhắc lại ở mức cần thiết để tài liệu có thể dùng như một phần nhập môn cho chủ đề thiết kế tối ưu.

Cấu trúc cuốn sách được sắp xếp theo trình tự từ trực quan đến tổng quát. Chương 1 giới thiệu bức tranh tổng quan về tối ưu hóa kết cấu, các loại bài toán tối ưu thường gặp và vai trò của tối ưu hóa trong quy trình thiết kế. Chương 2 trình bày phương pháp đồ thị thông qua các bài toán giàn và dầm đơn giản, giúp người đọc hình dung miền khả dĩ, đường ràng buộc và vị trí nghiệm tối ưu. Chương 3 hệ thống hóa các cơ sở toán học của quy hoạch lồi, điều kiện tối ưu KKT và phương pháp đối ngẫu Lagrange. Chương 4 giới thiệu các thuật toán xấp xỉ lồi tường minh tuần tự như SLP, SQP, CONLIN và MMA, là nền tảng cho nhiều phương pháp tối ưu kết cấu quy mô lớn.

Chương 5 tập trung vào bài toán tối ưu độ cứng của hệ giàn, trong đó phương pháp tiêu chuẩn tối ưu và phương pháp tiệm cận động được trình bày cùng các ví dụ tính toán. Chương 6 mở rộng sang tối ưu cấu trúc và tối ưu topo, đặc biệt là phương pháp SIMP và các vấn đề thực tế như lọc độ nhạy, dạng ô cờ và cập nhật biến thiết kế. Chương 7 giới thiệu tối ưu hóa đa mục tiêu, khái niệm Pareto, phương pháp tổng trọng số, phương pháp ràng buộc epsilon và cách diễn giải đánh đổi giữa các tiêu chí

thiết kế. Chương 8 trình bày các thuật toán tối ưu metaheuristic như thuật toán di truyền và luyện kim mô phỏng, kèm các bài toán kết cấu có nhiều biến và ràng buộc phức tạp.

Phần phụ lục mã nguồn được đưa vào nhằm hỗ trợ người đọc tự kiểm tra lại các ví dụ và mở rộng thành các chương trình tính riêng. Các đoạn mã không nhằm thay thế phần lý thuyết, mà đóng vai trò minh họa cách triển khai thuật toán, cách tổ chức dữ liệu kết cấu và cách biểu diễn kết quả. Người đọc được khuyến khích thay đổi tham số, điều kiện ràng buộc hoặc mô hình kết cấu để hiểu rõ hơn ảnh hưởng của từng giả thiết đến nghiệm tối ưu.

Mặc dù đã cố gắng trình bày nội dung một cách mạch lạc và kiểm tra các công thức, hình vẽ cũng như mã nguồn, cuốn sách khó tránh khỏi thiếu sót. Các góp ý của người đọc về nội dung, ký hiệu, ví dụ và cách trình bày sẽ rất hữu ích cho việc hoàn thiện tài liệu trong các lần biên soạn tiếp theo.

Các tác giả

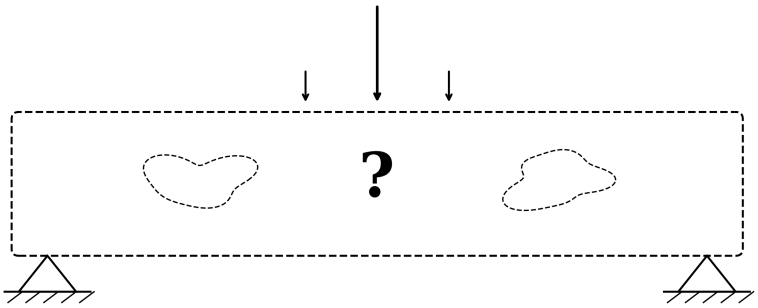
CHƯƠNG 1

NHẬP MÔN TỐI ƯU KẾT CẤU

Chương này cung cấp cái nhìn tổng quan về các khái niệm và thuật ngữ nền tảng trong tối ưu hóa kết cấu, từ cách mô tả bài toán thiết kế, biến thiết kế, hàm mục tiêu và ràng buộc đến các dạng tối ưu thường gặp trong cơ học kết cấu. Nội dung chương đóng vai trò nền tảng để người đọc hiểu cách một yêu cầu thiết kế kỹ thuật được chuyển thành mô hình toán học có thể giải bằng các phương pháp tối ưu hóa ở các chương sau.

1.1. Giới thiệu

Một kết cấu (structure) có thể hiểu đơn giản là một hệ thống vật liệu được bố trí nhằm mục đích chịu tải trọng. Theo đó, tối ưu hóa kết cấu (Structural Optimization SO) là quá trình sắp xếp vật liệu hoặc tinh chỉnh hệ thống đó sao cho khả năng chịu tải đạt mức "tốt nhất".



Hình 1.1. Bài toán tối ưu hóa kết cấu.

Khái niệm "tốt nhất"ở đây cần được định lượng cụ thể thông qua một hàm mục tiêu (objective function). Chẳng hạn như tối thiểu hóa trọng lượng để kết cấu nhẹ nhất, tối đa hóa độ cứng để kết cấu ít biến dạng nhất hoặc tối đa hóa độ bền để kết cấu ổn định và an toàn nhất. Tuy nhiên, quá trình tìm kiếm phương án tối ưu này không diễn ra tự do mà bị giới hạn bởi các ràng buộc (constraints) về ứng suất, chuyển vị, hoặc kích thước hình học nhằm đảm bảo tính khả thi và an toàn cho công trình.

Tối ưu hóa là một mắt xích quan trọng trong quy trình thiết kế tổng thể gồm 4 bước:

- Chức năng (function): xác định mục đích sử dụng của sản phẩm. Ví dụ với một cây cầu: cần xác định chiều dài, bề rộng, số lượng làn xe và các loại tải trọng dự kiến tác động lên nó.
- Thiết kế ý tưởng (conceptual design): lựa chọn giải pháp kết cấu sơ bộ. Đối với

cây cầu, kỹ sư cần quyết định nên sử dụng hệ kết cấu nào: cầu giàn (truss), cầu treo (suspension) hay cầu vòm (arch)...
- Tối ưu hóa (optimization): từ phương án ý tưởng đã chọn và các giới hạn về công năng, tiến hành tinh chỉnh để sản phẩm đạt hiệu quả cao nhất. Với cầu, mục tiêu thường là giảm thiểu chi phí, gián tiếp thực hiện qua việc sử dụng lượng vật liệu tối thiểu mà vẫn đảm bảo an toàn.
- Chi tiết (details): hoàn thiện thiết kế dựa trên các yếu tố phi kỹ thuật như thị trường, xã hội hoặc thẩm mỹ (ví dụ: lựa chọn màu sắc hoặc các chi tiết trang trí).

1.2. Phát biểu tổng quát của bài toán tối ưu kết cấu

Trong mọi bài toán tối ưu hóa kết cấu, ba thành phần cơ bản sau đây luôn được xác định:
- Hàm mục tiêu f: là hàm dùng để đánh giá mức độ "tốt" của một phương án thiết kế. Với mỗi thiết kế khả dĩ, hàm mục tiêu trả về một giá trị số phản ánh chất lượng của thiết kế đó. Thông thường, bài toán được xây dựng theo dạng cực tiểu hóa, trong đó giá trị nhỏ hơn tương ứng với thiết kế tốt hơn. Trong tối ưu kết cấu, hàm mục tiêu thường biểu diễn khối lượng, chuyển vị theo một phương nhất định, năng lượng biến dạng hoặc chi phí sản xuất.
- Biến thiết kế x: là hàm hoặc vector mô tả các đặc trưng của thiết kế và có thể thay đổi trong quá trình tối ưu hóa. Biến thiết kế có thể đại diện cho hình học của kết cấu hoặc cho sự lựa chọn vật liệu. Khi mô tả hình học, biến thiết kế có thể liên quan đến các tham số nội suy phức tạp của hình dạng, hoặc đơn giản là diện tích tiết diện của một thanh hay chiều dày của một tấm.
- Biến trạng thái y: với một kết cấu xác định, tương ứng với một thiết kế x cho trước, biến trạng thái là hàm hoặc vector biểu diễn phản ứng của kết cấu. Trong cơ học kết cấu, phản ứng thường bao gồm chuyển vị, ứng suất, biến dạng hoặc nội lực.

Một bài toán tối ưu hóa kết cấu tổng quát có thể được phát biểu như sau:

$$(\mathbb{SO})\begin{cases}\text{Cực tiểu hóa } f(x,y) \text{ theo các biến } x \text{ và } y,\\ \text{Thỏa mãn các điều kiện} \begin{cases}\text{Ràng buộc về ứng xử theo } y,\\ \text{Ràng buộc về thiết kế theo } x,\\ \text{Ràng buộc cân bằng.}\end{cases}\end{cases} \tag{1.1}$$

Ba nhóm ràng buộc thường gặp trong bài toán tối ưu kết cấu bao gồm:
- Ràng buộc ứng xử (behavioral constraints): là các ràng buộc đặt lên biến trạng thái y. Chúng thường được biểu diễn dưới dạng $g(y) \leqslant 0$, trong đó g là hàm mô tả một đại lượng phản ứng của kết cấu, chẳng hạn như chuyển vị tại một vị trí theo một phương xác định.
- Ràng buộc thiết kế (design constraints): là các ràng buộc liên quan trực tiếp đến biến thiết kế x, ví dụ như giới hạn hình học, giới hạn tiết diện hoặc giới hạn vật liệu.
- Ràng buộc cân bằng (equilibrium constraint): trong các bài toán rời rạc tự nhiên hoặc các bài toán đã được rời rạc hóa tuyến tính, điều kiện cân bằng của kết cấu có dạng $\mathbf{K}(x)\mathbf{u} = \mathbf{F}(x)$, trong đó $\mathbf{K}(x)$ là ma trận độ cứng của kết cấu, $\mathbf{u}$ là vector chuyển vị, và $\mathbf{F}(x)$ là vector ngoại lực.

1.3. Công thức đồng thời và công thức lồng ghép của bài toán tối ưu kết cấu

Trong công thức (1.1), các biến thiết kế x và biến trạng thái y được xem là độc lập. Cách phát biểu bài toán theo dạng này thường được gọi là công thức đồng thời (simultaneous formulation sf), trong đó điều kiện cân bằng được giải đồng thời với bài toán tối ưu hóa.

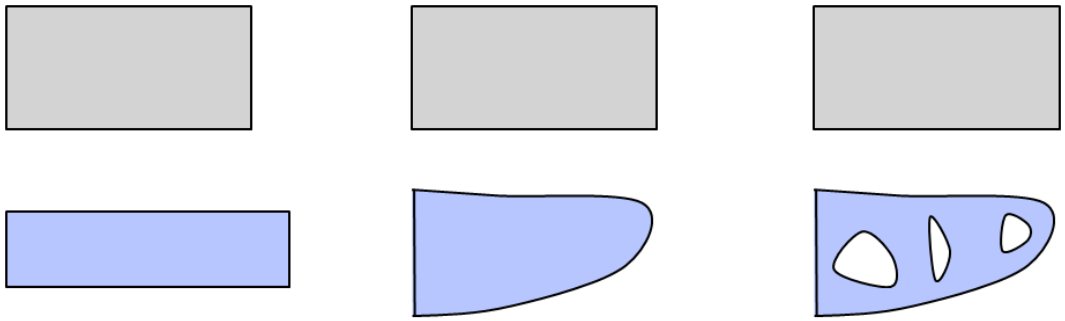
Tuy nhiên, trong nhiều trường hợp thực tế, bài toán trạng thái có nghiệm duy nhất đối với mỗi thiết kế x. Chẳng hạn, nếu ma trận độ cứng $\mathbf{K}(x)$ là khả nghịch với mọi x, ta có thể xác định chuyển vị dưới dạng hàm của thiết kế: $\mathbf{u} = \mathbf{u}(x) = \mathbf{K}(x)^{-1}\mathbf{F}(x)$. Khi đó, bằng cách xem $\mathbf{u}(x)$ như một hàm đã biết, điều kiện cân bằng có thể được loại bỏ khỏi bài toán ($\mathbb{SO}$), và hàm này được thay thế cho biến trạng thái. Bài toán tối ưu khi đó có thể được viết lại dưới dạng:

$$(\mathbb{SO})_{\text{nf}} \quad \begin{cases} \min\limits_{x} f(x, \mathbf{u}(x)) \\ \text{s.t. } g(x, \mathbf{u}(x)) \leqslant 0, \end{cases} \tag{1.2}$$

trong đó ký hiệu "s.t." (subject to) biểu thị các ràng buộc, và giả thiết rằng toàn bộ các ràng buộc ứng xử và ràng buộc thiết kế đều có thể được viết gộp dưới dạng $g(x, \mathbf{u}) \leqslant 0$. Cách phát biểu này được gọi là công thức lồng ghép (nested formulation nf).

1.4. Phân loại bài toán tối ưu

Các bài toán tối ưu hóa kết cấu có thể được phân thành ba nhóm chính: tối ưu hóa kích thước (sizing optimization), tối ưu hóa hình dạng (shape optimization) và tối ưu hóa cấu trúc (topology optimization).



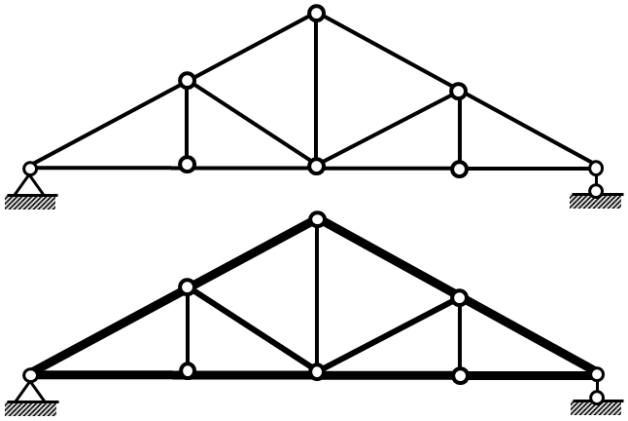
(a) Tối ưu kích thước (b) Tối ưu hình dạng (c) Tối ưu cấu trúc

Hình 1.2. Ba cấp độ của bài toán tìm bố cục tối ưu (lay-out optimization).

1.4.1. Tối ưu hóa kích thước

Tối ưu hóa kích thước là dạng bài toán phổ biến nhất trong thiết kế kết cấu, trong đó biến thiết kế đại diện cho các thông số kích thước đặc trưng của cấu kiện, ví dụ như diện tích mặt cắt ngang của thanh giàn, tiết diện dầm/cột, hoặc chiều dày của tấm vỏ. Hình học tổng thể và dạng của mô hình thiết kế được giữ nguyên (không thay đổi lưới phần tử). Quá trình tối ưu chỉ thực hiện việc tăng hoặc giảm các kích thước cục bộ. Mục tiêu là xác định tiết diện hợp lý nhất cho các cấu kiện dưới tác dụng của tải trọng thiết kế. Kết quả tối ưu phải thỏa mãn các ràng buộc về độ bền, độ cứng, độ ổn định, cũng như các tiêu chí về khả năng chế tạo và hiệu quả kinh tế.

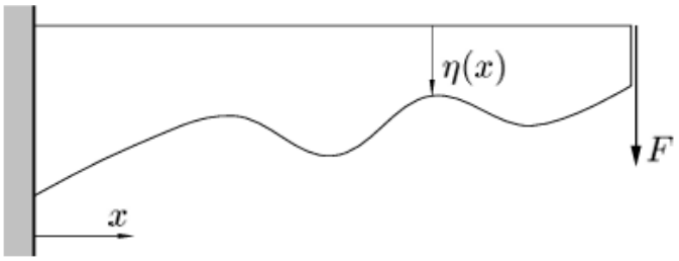
Ví dụ bài toán tìm diện tích tiết diện tối ưu cho các thanh trong một kết cấu giàn minh họa tại Hình 1.3 để chịu lực tốt nhất với trọng lượng nhẹ nhất.



Hình 1.3. Tối ưu hóa kích thước các thanh giàn.

1.4.2. Tối ưu hóa hình dạng

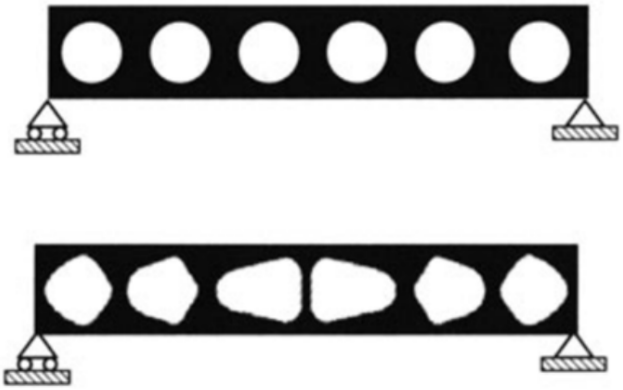
Trong bài toán tối ưu hóa hình dạng, biến thiết kế được sử dụng để mô tả đường bao hình học của một miền kết cấu, và quá trình tối ưu thực chất là việc xác định các tọa độ biên lý tưởng bằng cách dịch chuyển chúng theo những phương phù hợp. Một nguyên tắc cốt lõi của phương pháp này là tính liên kết (connectivity) và cấu trúc (topology) của hệ thống luôn được bảo toàn; nghĩa là hình dáng các biên hoặc lỗ rỗng có thể thay đổi để chịu lực tốt hơn, nhưng số lượng lỗ và cấu trúc tổng thể vẫn giữ nguyên. Các ví dụ điển hình cho bài toán này bao gồm tối ưu hóa hình dạng dầm (Hình 1.4), tối ưu hóa hình dáng các lỗ giảm trọng lượng trên dầm (Hình 1.5) hay thiết kế nhíp xe ô tô (Hình 1.6).



Hình 1.4. Tối ưu hóa hình dạng dầm.

1.4.3. Tối ưu hóa cấu trúc

Tối ưu hóa cấu trúc là cấp độ tổng quát và linh hoạt nhất trong thiết kế kết cấu, cho phép thay đổi đồng thời cả biên dạng bên ngoài lẫn sự phân bố vật liệu bên trong miền thiết kế. Nguyên lý cốt lõi của phương pháp này là xác định sự tồn tại của vật liệu tại mỗi điểm trong không gian thiết kế: đối với kết cấu rời rạc (như hệ giàn), biến thiết kế là diện tích mặt cắt ngang, và khi giá trị này về 0, thanh tương ứng sẽ bị loại bỏ hoàn toàn khỏi hệ thống (Hình 1.7); đối với kết cấu liên tục (như tấm hoặc khối rắn), mật độ vật liệu tại mỗi phần tử được xem là biến số, thường nhận giá trị nhị phân (0 là lỗ rỗng, 1 là vật liệu đặc). Ví dụ điển hình là bài toán tìm bố cục cho dầm console trong miền thiết kế hình chữ nhật (lưới 20×10) với lượng vật liệu giới hạn (Hình 1.8); thuật toán sẽ tự động phân bố các phần tử vật liệu để tạo ra kết cấu có độ cứng lớn nhất, hình thành nên các lỗ rỗng đặc trưng bên trong mà các phương

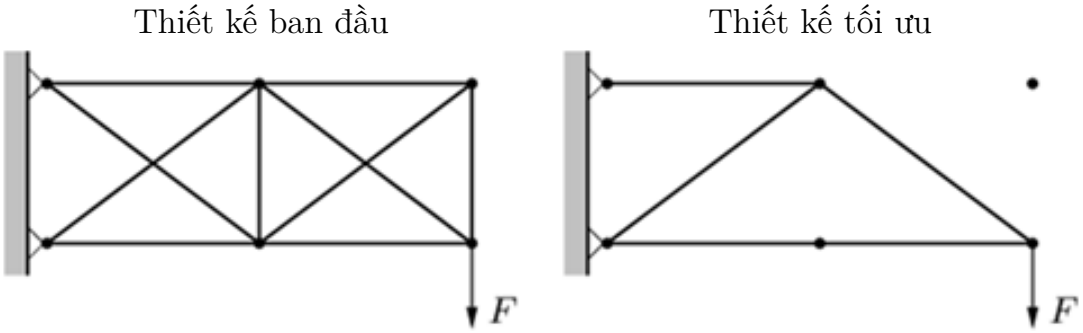


Hình 1.5. Tối ưu hóa hình dạng lỗ rỗng trong thanh dầm đơn giản.

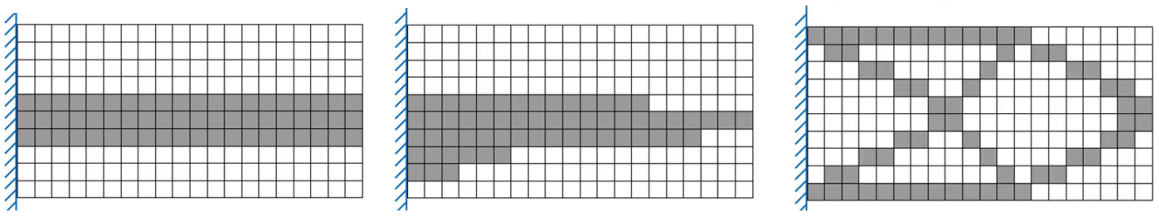


Hình 1.6. Tối ưu hóa hình dạng nhíp xe chịu lực tập trung giữa nhịp.

pháp tối ưu hóa kích thước hay hình dạng truyền thống không thể thực hiện được.



Hình 1.7. Tối ưu topo hệ giàn.



(a) Thiết kế 1 (b) Thiết kế 2 (c) Thiết kế 3

Hình 1.8. Các dạng thiết kế khác nhau của kết cấu dầm console với hàm mục tiêu tối đa độ cứng của dầm sử dụng 60 ô vuông nhỏ phần tử vật liệu.

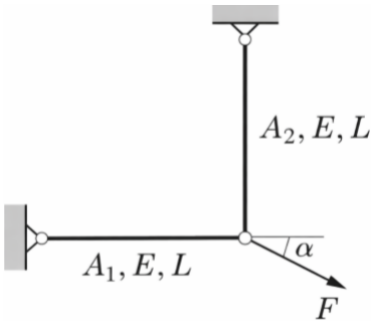
CHƯƠNG 2

TỐI ƯU HÓA KẾT CẤU BẰNG PHƯƠNG PHÁP ĐỒ THỊ

Chương này giới thiệu một số ví dụ minh họa cho bài toán tối ưu kích thước của thanh và dầm sử dụng phương pháp đồ thị. Phương pháp này có thể áp dụng cho các bài toán đơn giản, có ít biến thiết kế; các phương pháp tổng quát hơn sẽ được trình bày trong các chương tiếp theo.

2.1. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất

Hãy xem xét một hệ giàn hai thanh như minh họa tại Hình 2.1. Hai thanh này có cùng chiều dài L và mô-đun đàn hồi E. Hệ chịu tác dụng của một lực tập trung F tại nút tự do, hợp với phương ngang một góc α ($0 \leqslant \alpha \leqslant 90°$). Mục tiêu của bài toán là tìm phương án thiết kế sao cho trọng lượng của hệ là nhỏ nhất (tối thiểu hóa trọng lượng) nhưng vẫn phải thỏa mãn các điều kiện bền về ứng suất.



Hình 2.1. Tối ưu hóa trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất.

Chúng ta chọn diện tích mặt cắt ngang của hai thanh, A_1 và A_2, làm các biến thiết kế. Hàm mục tiêu (trọng lượng tổng của giàn) được xác định như sau:

$$f(A_1, A_2) = (A_1 + A_2)\rho L \tag{2.1}$$

trong đó ρ là khối lượng riêng của vật liệu. Vì ρ và L là các hằng số dương, việc tối thiểu hóa trọng lượng tương đương với việc tối thiểu hóa tổng diện tích mặt cắt ngang $(A_1 + A_2)$, với điều kiện hiển nhiên:

$$A_1 \geqslant 0, \quad A_2 \geqslant 0 \tag{2.2}$$

Nội lực và ứng suất trong các thanh có thể được xác định trực tiếp thông qua các

phương trình cân bằng lực tại nút:

$$\begin{aligned} F\cos\alpha - \sigma_1 A_1 &= 0 \\ F\sin\alpha - \sigma_2 A_2 &= 0 \end{aligned} \tag{2.3}$$

Để đảm bảo an toàn cho kết cấu, trị tuyệt đối của ứng suất trong mỗi thanh không được vượt quá giới hạn cho phép σ_0:

$$|\sigma_i| \leqslant \sigma_0, \quad i = 1, 2 \tag{2.4}$$

Kết hợp phương trình cân bằng và điều kiện bền, ta chuyển hóa các ràng buộc ứng suất thành các ràng buộc trực tiếp lên biến thiết kế:

$$A_1 \geqslant \frac{F\cos\alpha}{\sigma_0}, \qquad A_2 \geqslant \frac{F\sin\alpha}{\sigma_0} \tag{2.5}$$

Từ các phân tích trên, bài toán tối ưu hóa ban đầu được đưa về dạng rút gọn như sau:

$$(\mathbb{SO})^1_{\text{nf}} \quad \begin{cases} \min\limits_{A_1, A_2} \ (A_1 + A_2) \\ \text{s.t.} \quad \begin{cases} A_1 \geqslant \frac{F\cos\alpha}{\sigma_0} \\ A_2 \geqslant \frac{F\sin\alpha}{\sigma_0} \end{cases} \end{cases} \tag{2.6}$$

Dễ dàng nhận thấy hàm mục tiêu đạt giá trị nhỏ nhất khi các biến thiết kế chạm ngưỡng giới hạn dưới của chúng. Nghiệm tối ưu là:

$$A_1^* = \frac{F\cos\alpha}{\sigma_0}, \qquad A_2^* = \frac{F\sin\alpha}{\sigma_0} \tag{2.7}$$

Lời giải bằng đồ thị của bài toán $(\mathbb{SO})^1_{\text{nf}}$ ở Hình 2.2.

2.2. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và ổn định

Xét một hệ giàn Hình 2.3 gồm hai thanh tiết diện tròn vuông góc nhau, cùng chiều dài L và mô-đun đàn hồi E. Lực F tác dụng tại nút tự do với góc nghiêng 45° so với phương ngang. Tìm diện tích mặt cắt ngang của thanh A_1 và thanh A_2 để trọng lượng giàn là nhỏ nhất. Kết cấu phải thỏa mãn điều kiện bền và điều kiện ổn định (buckling).

Lực dọc trong hai thanh có độ lớn bằng nhau nhưng trái dấu. Thanh 1 chịu kéo và thanh 2 chịu nén với giá trị lực dọc: $N = \frac{F}{\sqrt{2}}$. Cả hai thanh đều không được vượt quá giới hạn ứng suất cho phép σ_0:

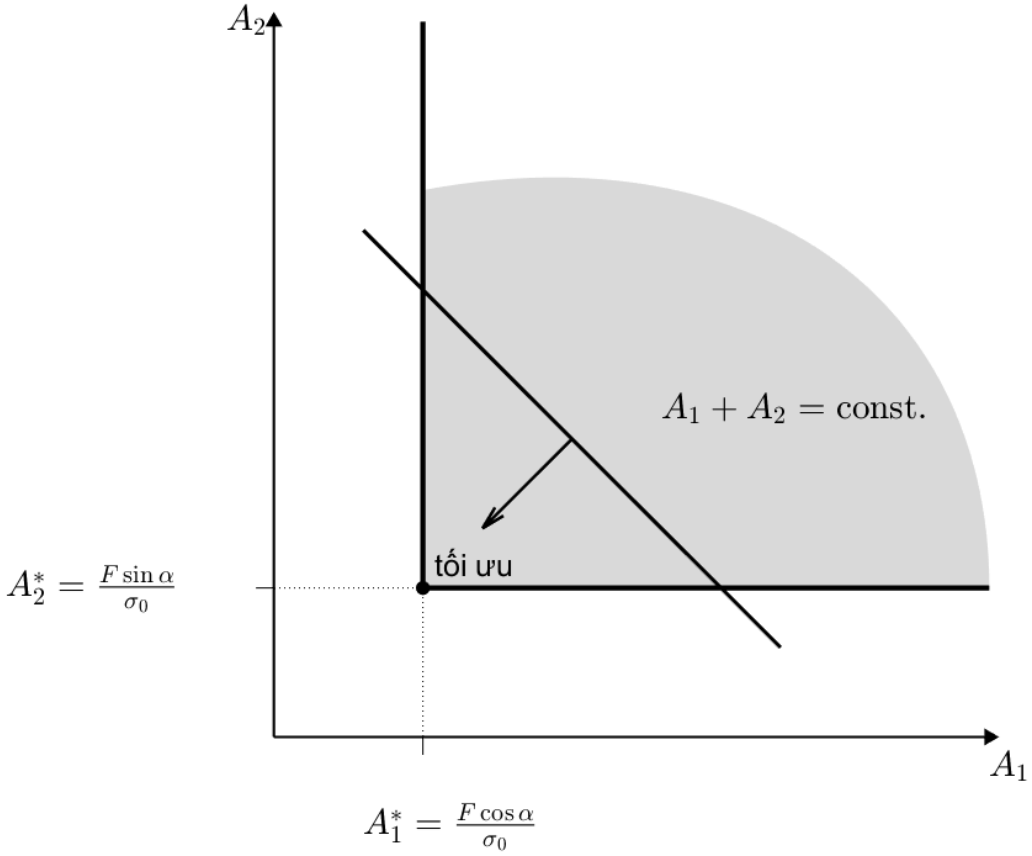
$$A_1 \geqslant \frac{F}{\sqrt{2}\sigma_0} \text{ và } A_2 \geqslant \frac{F}{\sqrt{2}\sigma_0} \tag{2.8}$$

Ràng buộc về ổn định chỉ áp dụng cho thanh 2 vì thanh này chịu nén. Công thức lực tới hạn Euler:

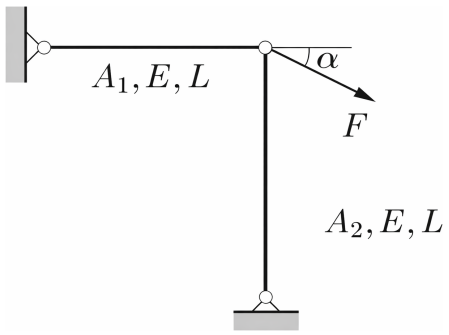
$$P_c = \pi^2 \frac{EI}{L^2} \tag{2.9}$$

trong đó mô-men quán tính tiết diện tròn: $I = \frac{A^2}{4\pi}$

Hệ số an toàn chống oằn được chọn là 4. Tức là khả năng chịu lực tới hạn phải lớn



Hình 2.2. Lời giải bằng đồ thị của bài toán $(\mathbb{SO})^1_{\text{nf}}$.



Hình 2.3. Tối ưu hóa trọng lượng hệ giàn hai thanh chịu ràng buộc điều kiện bền và ổn định.

gấp 4 lần lực nén thực tế: $P_c \geqslant 4 \times$ lực nén trong thanh 2.

$$\frac{\pi^2 E(A_2^2/4\pi)}{L^2} \geqslant 4\frac{F}{\sqrt{2}} \tag{2.10}$$

Rút gọn biểu thức trên, ta có ràng buộc phi tuyến đối với A_2:

$$A_2^2 \geqslant \frac{16FL^2}{\sqrt{2}\pi E} \tag{2.11}$$

Bài toán được tóm tắt dưới dạng công thức giải tích như sau:

$$(\mathbb{SO})^2_{\text{nf}} \quad \begin{cases} \min\limits_{A_1,A_2} (A_1 + A_2) \\ \text{s.t.} \begin{cases} A_1 \geqslant \frac{F}{\sqrt{2}\sigma_0} \\ A_2 \geqslant \frac{F}{\sqrt{2}\sigma_0} \\ A_2 \geqslant \sqrt{\frac{16FL^2}{\sqrt{2}\pi E}} \end{cases} \end{cases} \tag{2.12}$$

Việc xác định ràng buộc nào trong số các ràng buộc trên đóng vai trò ràng buộc chủ động (active constraint), tức là tại đó dấu bằng xảy ra sẽ phụ thuộc vào tương quan giữa các tham số vật lý. Để minh họa, hãy xem xét một trường hợp cụ thể với các thông số giả định sau:

$$\sigma_0 = \frac{E}{100}, \qquad \sqrt{\frac{F}{\sigma_0}} = \frac{L}{4} \tag{2.13}$$

Khi thay các giá trị này vào hệ điều kiện của bài toán $(\mathbb{SO})^2_{\text{nf}}$, ta thu được các giới hạn cụ thể cho tiết diện:

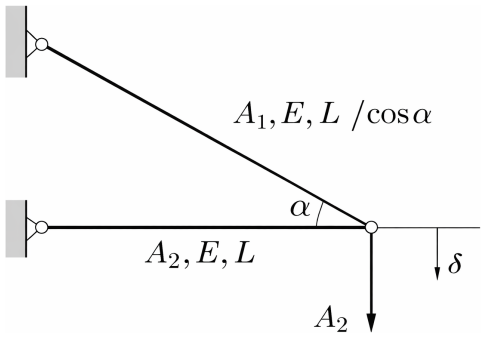
$$A_1 \geqslant \frac{L^2}{16\sqrt{2}}, \qquad A_2 \geqslant \frac{L^2}{16\sqrt{2}}, \qquad A_2 \geqslant \frac{L^2}{10\sqrt{2\pi}} \tag{2.14}$$

Tiếp theo, ta so sánh hai giới hạn dưới áp dụng cho thanh chịu nén. Dễ thấy rằng ràng buộc thứ ba sẽ chi phối thiết kế của thanh 2. Kết quả cuối cùng là:

$$A_1^* = \frac{L^2}{16\sqrt{2}} \approx 0,044L^2, \qquad A_2^* = \frac{L^2}{10\sqrt{2\pi}} \approx 0,047L^2 \tag{2.15}$$

2.3. Tối ưu trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và chuyển vị

Xét hệ giàn hai thanh như Hình 2.4, góc $\alpha = 30°$. Tìm diện tích mặt cắt ngang A_1, A_2 để trọng lượng giàn là nhỏ nhất. Ứng suất không vượt quá giới hạn σ_0 và chuyển vị đứng tại đầu mút δ không được vượt quá giá trị cho phép $\delta_0 = \sigma_0 L/E$.



Hình 2.4. Tối ưu hóa trọng lượng hệ giàn hai thanh chịu ràng buộc ứng suất và chuyển vị.

Hàm mục tiêu

$$f(A_1, A_2) = \rho L \left(\frac{2}{\sqrt{3}} A_1 + A_2 \right) \tag{2.16}$$

Sử dụng phương pháp phân tích kết cấu, ta thiết lập được mối quan hệ giữa chuyển vị đầu mút và diện tích tiết diện như sau (người học tự chứng minh):

$$\delta = \frac{FL}{E} \left(\frac{8}{\sqrt{3} A_1} + \frac{3}{A_2} \right) \tag{2.17}$$

Ràng buộc chuyển vị $\delta \leqslant \delta_0$ trở thành:

$$\frac{8}{\sqrt{3} A_1} + \frac{3}{A_2} \leqslant \frac{\sigma_0}{F} \tag{2.18}$$

Do tính chất phi tuyến của ràng buộc (2.18) đối với các biến A_1 và A_2, việc tìm kiếm lời giải tối ưu trở nên phức tạp. Nhằm đơn giản hóa bài toán, ta thực hiện chuyển đổi các biến sang dạng không thứ nguyên:

$$x_1 = \frac{2F}{\sigma_0 A_1}, \qquad x_2 = \frac{\sqrt{3} F}{\sigma_0 A_2} \tag{2.19}$$

Khi đó ràng buộc chuyển vị trở nên tuyến tính:

$$\frac{4}{\sqrt{3}} x_1 + \sqrt{3} x_2 \leqslant 1 \tag{2.20}$$

Bài toán tối ưu hóa được phát biểu như sau:

$$(\mathbb{SO})^3_{\text{nf}} \quad \begin{cases} \min\limits_{x_1, x_2} \hat{f}(x_1, x_2) = \frac{4}{3x_1} + \frac{1}{x_2} \\ \text{s.t.} \quad \begin{cases} \frac{4}{\sqrt{3}} x_1 + \sqrt{3} x_2 \leqslant 1 \\ x_1 > 0, \ x_2 > 0. \end{cases} \end{cases} \tag{2.21}$$

Bài toán này được minh họa trong Hình 2.5, từ đó chúng ta kết luận rằng ràng buộc (2.20) là hoạt động (active). Chúng ta viết (2.20) dưới dạng đẳng thức và giải tìm x_2 để thu được:

$$x_2 = \frac{1}{\sqrt{3}} - \frac{4}{3} x_1 \tag{2.22}$$

Thế phương trình này vào hàm mục tiêu $\hat{f}(x_1, x_2)$, ta đưa bài toán về hàm một biến theo x_1. Lấy đạo hàm và cho bằng 0 để tìm cực trị:

$$x_1 = \pm \left(\frac{1}{\sqrt{3}} - \frac{4}{3} x_1 \right) \tag{2.23}$$

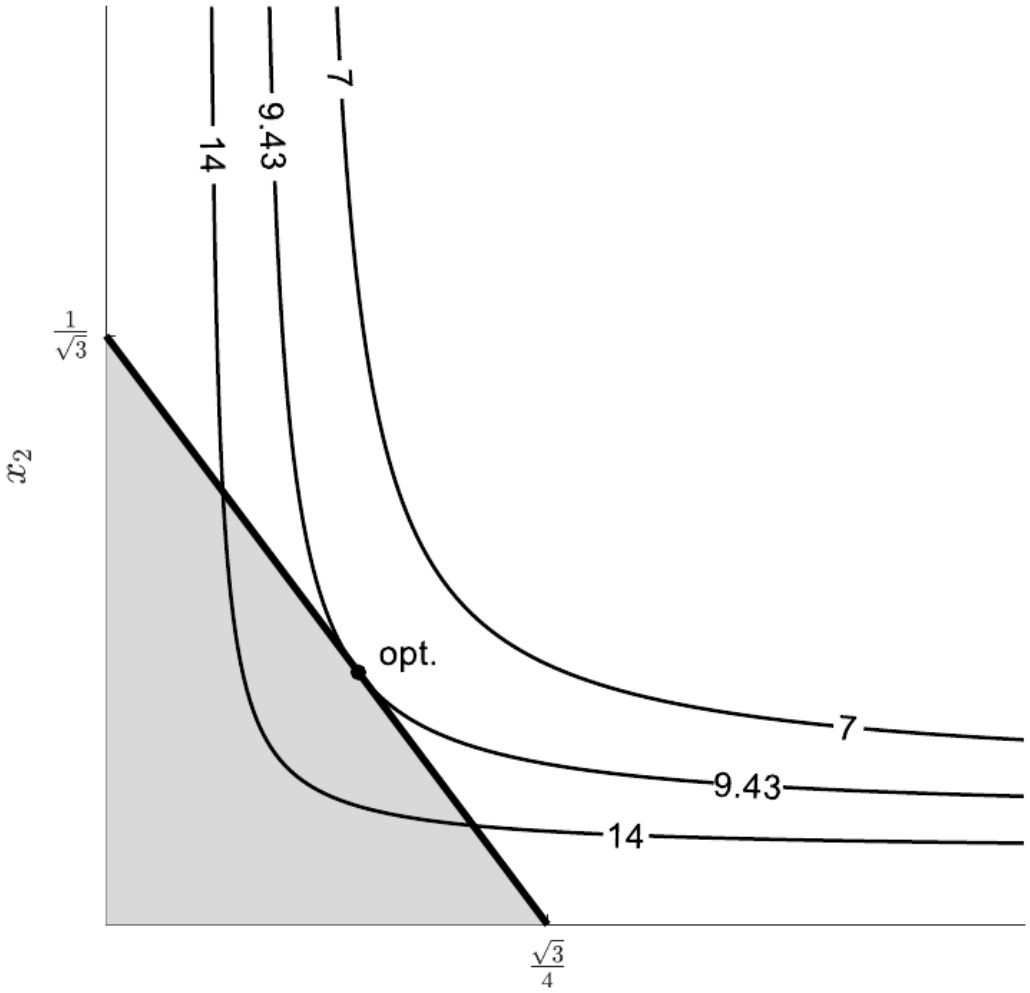
Giải phương trình trên (loại nghiệm âm và nghiệm làm $x > 1$), ta thu được nghiệm tối ưu:

$$x_1^* = \frac{\sqrt{3}}{7}, \quad x_2^* = \frac{\sqrt{3}}{7}, \tag{2.24}$$

Chuyển đổi ngược lại về biến diện tích, ta có thiết kế tối ưu:

$$A_1^* = \frac{14F}{\sqrt{3}\sigma_0} \approx \frac{8.1F}{\sigma_0}, \quad A_2^* = \frac{7F}{\sigma_0} \tag{2.25}$$

Lời giải bằng đồ thị của bài toán $(\mathbb{SO})^3_{\text{nf}}$ ở Hình 2.5.



Hình 2.5. Hình mô tả bài toán tối ưu $(\mathbb{SO})^3_{\text{nf}}$.

2.4. Tối ưu trọng lượng dầm công-xôn chịu ràng buộc chuyển vị

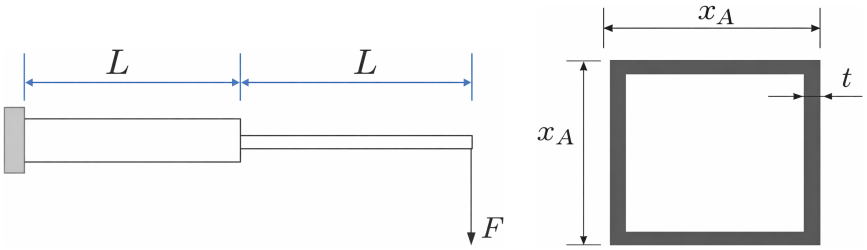
Xét dầm công-xôn ngàm tại đầu trái và chịu lực tập trung $F > 0$ tại đầu phải. Dầm gồm N đoạn có chiều dài L bằng nhau với tiết diện hình hộp rỗng. Giả sử chiều dày thành t không đổi, trong khi kích thước cạnh x_A thay đổi theo từng đoạn.

Với giả thiết $t \ll x_A$, mô-men quán tính I_A được xấp xỉ bằng công thức:

$$I_A = \frac{x_A^4}{12} - \frac{(x_A - 2t)^4}{12} \approx \frac{2tx_A^3}{3} \tag{2.26}$$

Mục tiêu là cực tiểu hóa trọng lượng dầm sao cho chuyển vị đầu mút δ không vượt quá giới hạn δ_0. Hàm mục tiêu trọng lượng được viết dưới dạng:

$$f(x_1, \ldots, x_N) = 4L\rho t \sum_{A=1}^{N} x_A \tag{2.27}$$



Hình 2.6. Tối ưu hóa trọng lượng dầm công-xôn N đoạn chịu ràng buộc chuyển vị (hình vẽ với N = 2).

Áp dụng nguyên lý cộng tác dụng, chuyển vị tổng cộng δ tại đầu tự do là tổng đóng góp chuyển vị từ mỗi đoạn (khi coi các đoạn khác là tuyệt đối cứng). Với giả thiết biến dạng bé ($\sin\theta_A \approx \theta_A$), ta có hệ thức hình học:

$$\delta^{(A)} = \delta_A + (A-1)L\theta_A \tag{2.28}$$

trong đó δ_A và θ_A là độ võng và góc xoay tại đầu phải của đoạn A, được xác định theo lý thuyết dầm cơ bản:

$$\delta_A = \frac{M_A L^2}{2EI_A} + \frac{F_A L^3}{3EI_A}, \quad \theta_A = \frac{M_A L}{EI_A} + \frac{F_A L^2}{2EI_A} \tag{2.29}$$

Với nội lực $M_A = (A-1)LF$ và lực cắt $F_A = F$.

Thay thế các biểu thức trên vào phương trình tổng quát, ta thu được công thức chuyển vị tường minh:

$$\delta = \frac{3FL^3}{2Et}\sum_{A=1}^{N}\left(A^2 - A + \frac{1}{3}\right)\frac{1}{x_A^3} \tag{2.30}$$

Xét trường hợp đơn giản với 2 đoạn, bài toán tối ưu hóa $(\mathbb{SO})^4_{\text{nf}}$ được phát biểu như sau:

$$(\mathbb{SO})^4_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} f(x_1,x_2) = C_1(x_1+x_2) \\ \text{s.t.} \begin{cases} \frac{1}{x_1^3} + \frac{7}{x_2^3} \leqslant C_2 \\ x_1 > 0, \quad x_2 > 0, \end{cases} \end{cases} \tag{2.31}$$

trong đó $C_1 = 4\rho Lt$ và $C_2 = \frac{2\delta_0 Et}{FL^3}$.

Giải bài toán ta thu được nghiệm tối ưu:

$$x_1^* = \left(\frac{1+7^{1/4}}{C_2}\right)^{1/3}, \quad x_2^* = 7^{1/4}\left(\frac{1+7^{1/4}}{C_2}\right)^{1/3} \tag{2.32}$$

Xét bài toán đối ngẫu: cực tiểu hóa chuyển vị đầu mút với ràng buộc về tổng trọng lượng W cho trước.

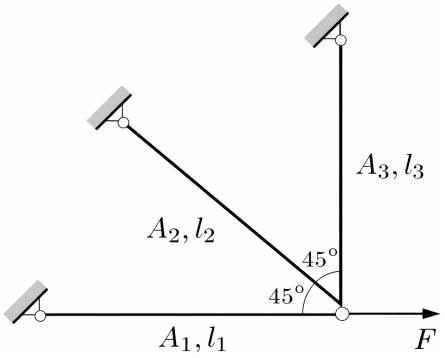
$$\begin{cases} \min\limits_{x_1,x_2} \frac{1}{x_1^3} + \frac{7}{x_2^3} \\ \text{s.t.} \begin{cases} C_1(x_1+x_2) \leqslant W \\ x_1 > 0,\ x_2 > 0 \end{cases} \end{cases} \tag{2.33}$$

Bài toán này có thể được giải theo cách tương tự như $(\mathbb{SO})^4_{\text{nf}}$. Ta tìm được lời giải:

$$x_1^{**} = \frac{W}{C_1}\left(\frac{1}{1+7^{1/4}}\right), \quad x_2^{**} = \frac{W}{C_1}\left(\frac{7^{1/4}}{1+7^{1/4}}\right), \tag{2.34}$$

2.5. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc ứng suất

Xét một hệ giàn ba thanh như Hình 2.7, chịu tác dụng của lực F. Các thanh có cùng mô đun đàn hồi E. Chiều dài các thanh lần lượt là $l_1 = l_2 = L$ và $l_3 = L/\beta$ (với $\beta = 1$), diện tích tiết diện $A_1 = A_3$. Bài toán đặt ra là tìm các biến thiết kế A_1 và A_2 để tối thiểu trọng lượng của hệ giàn, đồng thời thỏa mãn các ràng buộc về ứng suất.



Hình 2.7. Tối ưu hóa trọng lượng hệ giàn ba thanh chịu ràng buộc ứng suất.

Tổng trọng lượng của giàn được xác định bởi:

$$f(A_1, A_2) = \rho_1 L A_1 + \rho_2 L A_2 + \rho_3 \frac{L}{\beta} A_1 = L\left(\rho_1 + \frac{\rho_3}{\beta}\right) A_1 + L\rho_2 A_2 \tag{2.35}$$

trong đó ρ_i là mật độ khối lượng của các thanh. Các biến thiết kế phải không âm: $A_1 \geqslant 0, A_2 \geqslant 0$.

Do hệ giàn là siêu tĩnh, nội lực trong các thanh phụ thuộc vào độ cứng của chúng. Giải bài toán cân bằng dựa trên chuyển vị, ta thu được công thức tính chuyển vị tại điểm đặt lực như sau:

$$\begin{aligned} u_x &= \tfrac{FL}{EA_1}\left(\tfrac{2\beta A_1 + A_2}{2\beta A_1 + (1+\beta)A_2}\right), \\ u_y &= \tfrac{FL}{EA_1}\left(\tfrac{A_2}{2\beta A_1 + (1+\beta)A_2}\right). \end{aligned} \tag{2.36}$$

và ứng suất trong các thanh như sau:

$$\begin{aligned} \sigma_1 &= \frac{F}{2\beta A_1 + (1+\beta)A_2}\left(2\beta + \frac{A_2}{A_1}\right) \\ \sigma_2 &= \frac{\sqrt{2}F\beta}{2\beta A_1 + (1+\beta)A_2} \\ \sigma_3 &= -\frac{F\beta\frac{A_2}{A_1}}{2\beta A_1 + (1+\beta)A_2} \end{aligned} \tag{2.37}$$

Từ kết quả trên, dễ thấy thanh 1 và 2 chịu kéo, còn thanh 3 chịu nén. Do đó, các điều kiện bền cần kiểm tra là $\sigma_1 \leqslant \sigma_1^{max}$, $\sigma_2 \leqslant \sigma_2^{max}$ và $-\sigma_3 \leqslant \sigma_3^{max}$.

Lưu ý rằng thanh 2 có thể có diện tích bằng 0 mà kết cấu vẫn tĩnh định, còn thanh 1 và 3 không thể đồng thời có diện tích bằng 0 do giả thiết $A_1 = A_3$ vì khi đó kết cấu biến hình. Các ràng buộc ứng suất được viết lại cụ thể:

Thanh 1:

$$\frac{F(2A_1 + A_2)}{2A_1(A_1 + A_2)} \leqslant \sigma_1^{max} \tag{2.38}$$

Thanh 2 (chỉ xét khi $A_2 > 0$):

$$\frac{F}{\sqrt{2}(A_1 + A_2)} \leqslant \sigma_2^{max} \tag{2.39}$$

Thanh 3:

$$\frac{FA_2}{2A_1(A_1 + A_2)} \leqslant \sigma_3^{max} \tag{2.40}$$

Tổng hợp lại, ta có phát biểu bài toán tối ưu hóa đầy đủ như sau:

$$(\mathbb{SO})^5_{\text{nf}} \quad \begin{cases} \min\limits_{A_1,A_2} (\rho_1 A_1 + \rho_2 A_2 + \rho_3 A_1)L \\ \text{s.t.} \begin{cases} \frac{F(2A_1+A_2)}{2A_1(A_1+A_2)} - \sigma_1^{\max} \leqslant 0 \\ \frac{F}{\sqrt{2}(A_1+A_2)} - \sigma_2^{\max} \leqslant 0 \quad \text{if } A_2 > 0 \\ \frac{FA_2}{2A_1(A_1+A_2)} - \sigma_3^{\max} \leqslant 0 \\ A_1 > 0, \quad A_2 \geqslant 0. \end{cases} \end{cases} \tag{2.41}$$

Để minh họa tính chất đa dạng của lời giải tối ưu trong kết cấu, các phần tiếp theo sẽ khảo sát bài toán này dưới 5 trường hợp khác nhau của vật liệu và giới hạn bền.

Trường hợp a: $\rho_1 = 2\rho_0, \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_2^{\max} = \sigma_3^{\max} = \sigma_0$.

Với các thông số vật liệu $\rho_1 = 2\rho_0$, $\rho_2 = \rho_3 = \rho_0$ và giới hạn ứng suất $\sigma^{max} = \sigma_0$, ta sử dụng các biến không thứ nguyên:

$$x_1 = \frac{A_1\sigma_0}{F}, \quad x_2 = \frac{A_2\sigma_0}{F} \tag{2.42}$$

Bài toán tối ưu hóa $(\mathbb{SO})^{5a}_{\text{nf}}$ được phát biểu như sau:

$$(\mathbb{SO})^{5a}_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} 3x_1 + x_2 \\ \text{s.t.} \begin{cases} \frac{2x_1+x_2}{2x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_1) \\ \frac{1}{\sqrt{2}(x_1+x_2)} - 1 \leqslant 0 \quad \text{if } x_2 > 0 \quad (\sigma_2) \\ \frac{x_2}{2x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_3) \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.43}$$

Quan sát Hình 2.8, ta thấy tại điểm tối ưu, chỉ có ràng buộc σ_1 là hoạt động (active), trong khi các ràng buộc khác đều không hoạt động. Do đó, ta xét phương trình đẳng thức của σ_1:

$$2x_1 + x_2 - 2x_1(x_1 + x_2) = 0 \tag{2.44}$$

Biến đổi để biểu diễn x_2 theo x_1:

$$x_2 = \frac{2x_1(x_1 - 1)}{1 - 2x_1} \tag{2.45}$$

Thế vào hàm mục tiêu, bài toán quy về việc cực tiểu hóa hàm số theo biến x_1. Việc tìm điểm dừng dẫn đến phương trình bậc hai:

$$8x_1^2 - 8x_1 + 1 = 0 \tag{2.46}$$

Giải phương trình, ta có hai nghiệm:

$$x_1^* = \frac{1}{2} \pm \frac{\sqrt{2}}{4} \tag{2.47}$$

Để thỏa mãn điều kiện $x_2 > 0$ khi thế vào (2.45), ta loại nghiệm dấu trừ và chọn nghiệm dấu cộng. Kết quả các biến tối ưu không thứ nguyên là:

$$x_1^* = \frac{2 + \sqrt{2}}{4}, \quad x_2^* = \frac{\sqrt{2}}{4} \tag{2.48}$$

Chuyển đổi ngược về các biến diện tích tiết diện thực tế (A_1, A_2) ta có diện tích tiết diện tối ưu:

$$A_1^* = \frac{F}{2\sigma_0}\left(1 + \frac{1}{\sqrt{2}}\right), \quad A_2^* = \frac{F}{2\sqrt{2}\sigma_0} \tag{2.49}$$

Trường hợp b: $\rho_1 = \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$

Sử dụng cùng các biến không thứ nguyên như trường hợp trước, chúng ta có thể viết bài toán là:

$$(\mathbb{SO})_{\text{nf}}^{5b} \quad \begin{cases} \min\limits_{x_1,x_2} & 2x_1 + x_2 \\ \text{s.t.} & \begin{cases} \frac{2x_1+x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_1) \\ \frac{1}{\sqrt{2}(x_1+x_2)} - 1 \leqslant 0 \quad \text{if } x_2 > 0 \quad (\sigma_2) \\ \frac{x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_3) \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.50}$$

Quan sát Hình 2.9, thoạt nhìn lời giải tối ưu dường như nằm tại điểm A, là giao điểm của hai đường ràng buộc σ_1 và σ_2. Tuy nhiên, cần lưu ý rằng ràng buộc σ_2 chỉ có hiệu lực khi $x_2 > 0$.

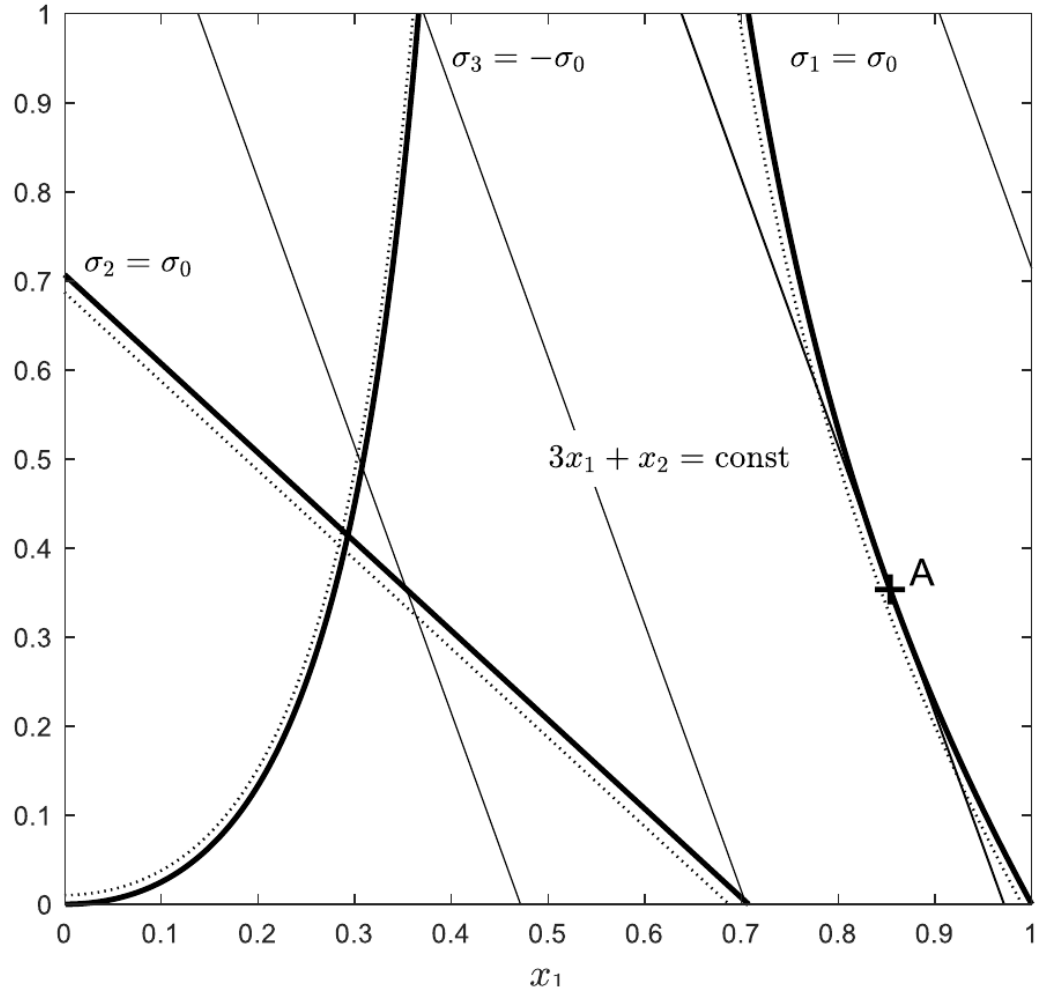
Nếu loại bỏ điều kiện này, dựa vào đồ thị, ta thấy điểm B nằm trên đường cong ràng buộc σ_1 (tại vị trí $x_2 = 0$) mới chính là điểm mang lại trọng lượng nhỏ nhất. Tọa độ điểm này được xác định bằng cách thay $x_2 = 0$ vào phương trình ràng buộc σ_1 đang hoạt động (active constraint):

$$2x_1 - 4x_1^2 = 0 \tag{2.51}$$

Phương trình trên cho kết quả $x_1^* = 1/2$ (loại bỏ nghiệm $x_1^* = 0$ vì đây không phải là một phương án thiết kế hợp lệ). Khi chuyển đổi trở lại các biến ban đầu, lời giải tối ưu là:

$$A_1^* = \frac{F}{2\sigma_0}, \quad A_2^* = 0 \tag{2.52}$$

Trường hợp c: $\rho_1 = (2\sqrt{2} - 1)\rho_0, \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$



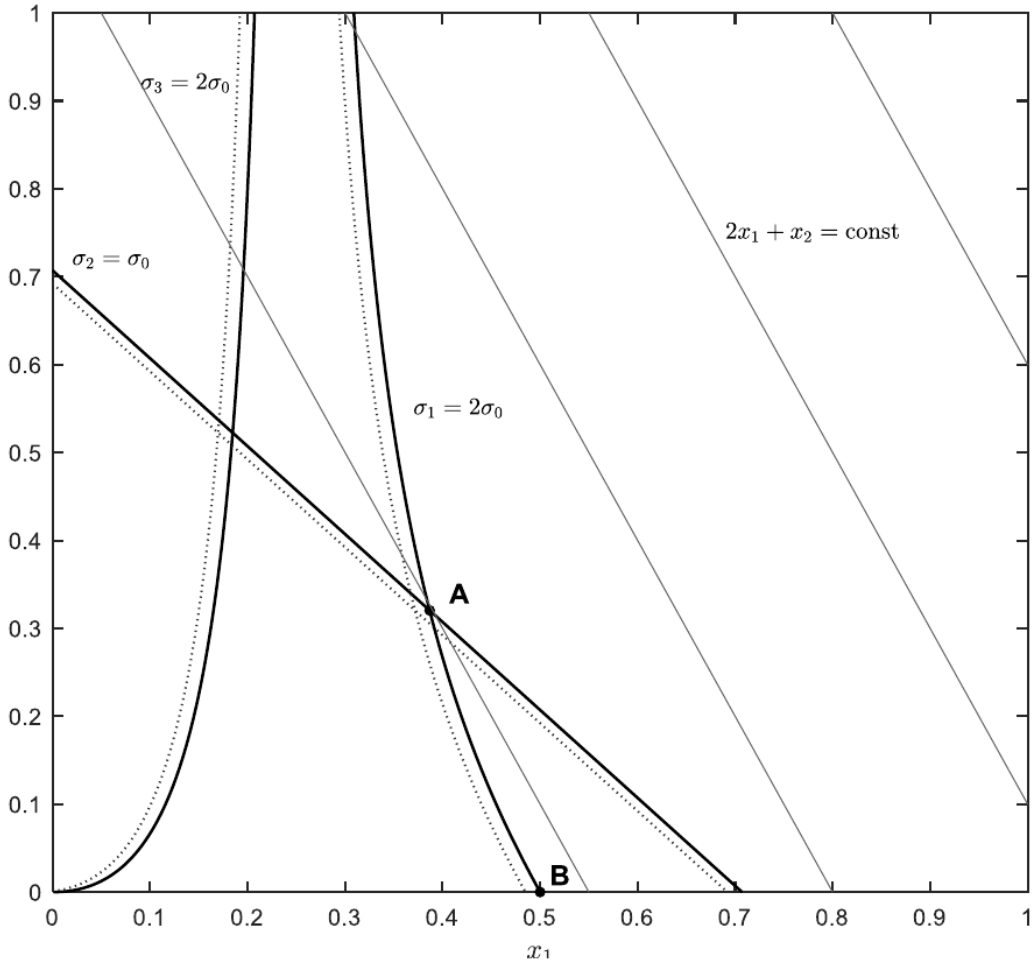
Hình 2.8. Trường hợp a). Đường liền nét đậm đi kèm với đường nét chấm biểu thị một ràng buộc; vùng nằm cùng phía với đường nét chấm so với đường đậm không thuộc miền thiết kế. Các đường liền nét mảnh là các đường đồng mức, nghĩa là tất cả các điểm nằm trên cùng một đường mảnh sẽ cho cùng một giá trị hàm mục tiêu. Điểm A là lời giải của bài toán.

Trong trường hợp này, mật độ vật liệu của thanh số 1 được giả định tăng nhẹ so với trường hợp b. Sự thay đổi này dẫn đến việc hàm mục tiêu biến đổi, trong khi các ràng buộc của bài toán vẫn giữ nguyên như $(\mathbb{SO})^{5b}_{\text{nf}}$. Bài toán tối ưu hóa mới được thiết lập như sau:

$$(\mathbb{SO})^{5c}_{\text{nf}} \quad \begin{cases} \min_{x_1,x_2} 2\sqrt{2}x_1 + x_2 \\ \text{s.t. các ràng buộc trong } (\mathbb{SO})^{5b}_{\text{nf}}, \end{cases} \tag{2.53}$$

Bài toán được minh họa trực quan trong Hình 2.10. Quan sát đồ thị, việc xác định lời giải tối ưu chỉ bằng mắt thường trở nên mơ hồ; không rõ liệu điểm tối ưu thực sự nằm tại giao điểm A của hai đường biên ràng buộc (σ_1 và σ_2), hay tại điểm B (nơi $x_2 = 0$, tương ứng với thiết kế loại bỏ thanh 2).

Tại điểm A: Đây là giao điểm của hai ràng buộc đang hoạt động. Tọa độ của A được xác định bằng cách giải hệ phương trình khi các bất đẳng thức σ_1 và σ_2 trở thành đẳng thức:



Hình 2.9. Trường hợp b. Điểm B là lời giải tối ưu.

$$x_1^* = \frac{4+\sqrt{2}}{14}, \quad x_2^* = \frac{6\sqrt{2}-4}{14} \tag{2.54}$$

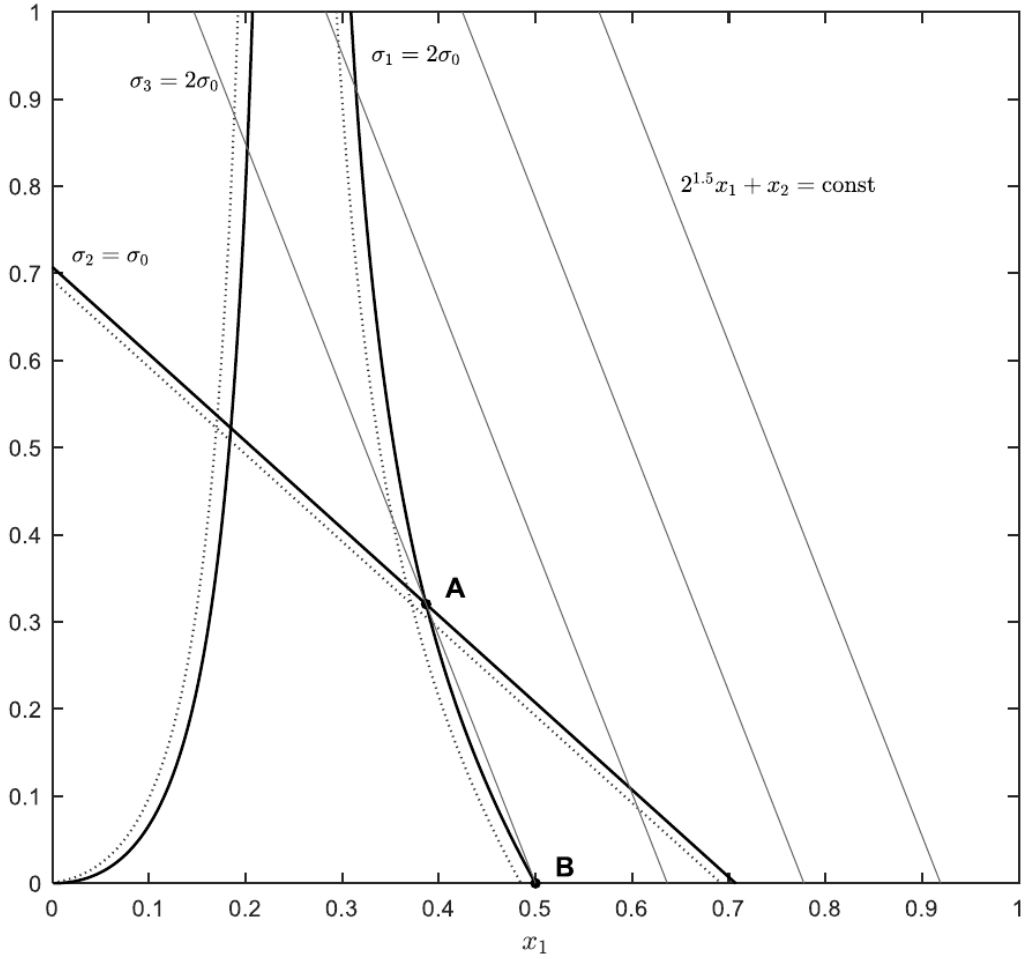
Tại điểm B: Tương ứng với thiết kế không có thanh 2, ta có: $x_1^{**} = 1/2, \quad x_2^{**} = 0$. Một kết quả thú vị xuất hiện khi thay các giá trị trên vào hàm mục tiêu: cả hai điểm A và B đều mang lại cùng một giá trị hàm mục tiêu. Điều này đồng nghĩa với việc bài toán tồn tại song song hai lời giải tối ưu toàn cục. Khi chuyển đổi trở lại các biến thiết kế ban đầu, hai phương án tối ưu được biểu diễn như sau: phương án 1, tương ứng với điểm A, có $A_1^* = \frac{F}{\sigma_0}\left(\frac{4+\sqrt{2}}{14}\right)$ và $A_2^* = \frac{F}{\sigma_0}\left(\frac{6\sqrt{2}-4}{14}\right)$; phương án 2, tương ứng với điểm B, có $A_1^{**} = \frac{F}{2\sigma_0}$ và $A_2^{**} = 0$.

Cả hai phương án thiết kế này đều cho cùng một trọng lượng tối ưu của hệ là:

$$W_{min} = \frac{\sqrt{2}FL\rho_0}{\sigma_0} \tag{2.55}$$

Trường hợp d: $\rho_1 = 3\rho_0, \rho_2 = \rho_3 = \rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$

Tiếp tục khảo sát sự ảnh hưởng của mật độ vật liệu, trong trường hợp này, ta giả định mật độ của thanh số 1 tăng lên đáng kể, cụ thể là gấp 3 lần so với ban đầu ($\rho_1 = 3\rho_0$). Sự thay đổi này làm biến đổi hàm mục tiêu, trong khi tập hợp các ràng



Hình 2.10. Trường hợp c. Điểm A và B là lời giải tối ưu.

buộc vẫn giữ nguyên như trong mô hình $(SO)^{5b}_{\text{nf}}$. Bài toán tối ưu hóa lúc này được thiết lập lại như sau:

$$(\mathbb{SO})^{5d}_{\text{nf}} \quad \begin{cases} \min_{x_1,x_2} 4x_1 + x_2 \\ \text{s.t. các ràng buộc trong } (\mathbb{SO})^{5b}_{\text{nf}}. \end{cases} \tag{2.56}$$

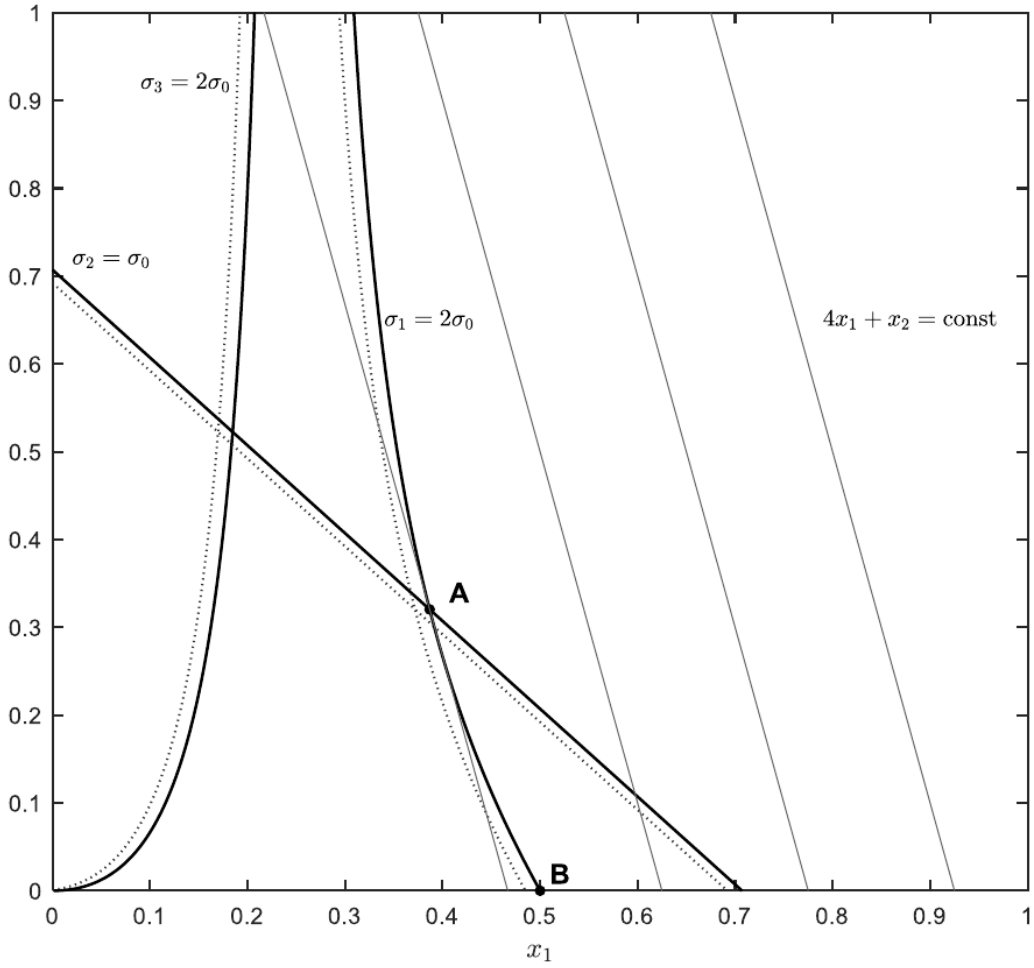
Quan sát đồ thị minh họa Hình 2.11, ta dễ dàng nhận thấy lời giải tối ưu nằm tại giao điểm của hai đường cong ràng buộc σ_1 và σ_2. Điều này chỉ ra rằng cả hai ràng buộc ứng suất này đều đang ở trạng thái "hoạt động"(active) tại điểm tối ưu.

Đáng chú ý, tọa độ của điểm tối ưu này hoàn toàn trùng khớp với kết quả điểm A mà chúng ta đã tính toán được trong trường hợp c. Khi chuyển đổi trở lại các biến thiết kế ban đầu, lời giải tối ưu là:

$$A_1^* = \frac{F}{\sigma_0}\left(\frac{4+\sqrt{2}}{14}\right), \quad A_2^* = \frac{F}{\sigma_0}\left(\frac{6\sqrt{2}-4}{14}\right), \tag{2.57}$$

Tương ứng với phương án thiết kế này, trọng lượng tối ưu của hệ kết cấu được xác định bởi công thức:

$$W_{min} = \frac{FL\rho_0}{\sigma_0}\left(\frac{6+5\sqrt{2}}{7}\right) \tag{2.58}$$



Hình 2.11. Trường hợp d. Điểm A là lời giải tối ưu.

Trường hợp e: $\rho_1 = \rho_3 = \rho_0, \rho_2 = 2\rho_0, \sigma_1^{\max} = \sigma_3^{\max} = 2\sigma_0, \sigma_2^{\max} = \sigma_0$

Cuối cùng, chúng ta xét Trường hợp e bằng cách điều chỉnh lại mô hình của Trường hợp b. Cụ thể, mật độ vật liệu của thanh số 2 được tăng lên gấp đôi, trong khi các thông số khác giữ nguyên. Khi sử dụng các biến không thứ nguyên tương tự như trước, bài toán tối ưu hóa được thiết lập như sau:

$$(\mathbb{SO})^{5e}_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} \quad x_1 + x_2 \\ \text{s.t.} \quad \begin{cases} \frac{2x_1+x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_1) \\ \frac{1}{\sqrt{2}(x_1+x_2)} - 1 \leqslant 0 \quad \text{if } x_2 > 0 \quad (\sigma_2) \\ \frac{x_2}{4x_1(x_1+x_2)} - 1 \leqslant 0 \quad (\sigma_3) \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.59}$$

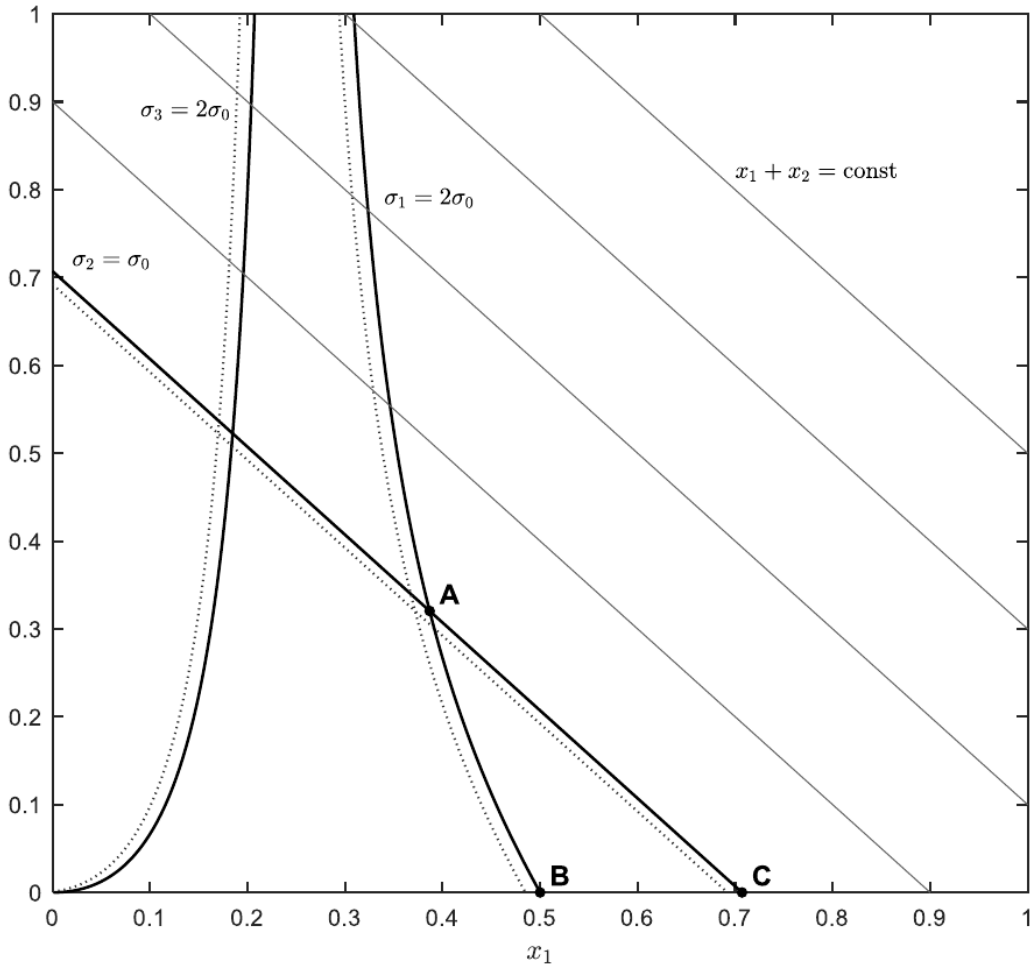
Quan sát Hình 2.12, điểm lời giải tối ưu nằm tại vị trí B. Tại điểm này, kết cấu tối ưu đạt được bằng cách loại bỏ hoàn toàn thanh số 2. Khi chuyển đổi trở lại các biến ban đầu, lời giải tối ưu là:

$$A_1^* = \frac{F}{2\sigma_0}, \quad A_2^* = 0 \tag{2.60}$$

Trọng lượng tối ưu tương ứng của hệ là:

$$W_{min} = \frac{FL\rho_0}{\sigma_0} \tag{2.61}$$

Một điểm đáng chú ý là kết quả này hoàn toàn trùng khớp với lời giải của Trường hợp b. Nguyên nhân cốt lõi là do thanh số 2 không tham gia vào cấu trúc tối ưu, nên việc thay đổi mật độ của nó (dù tăng gấp đôi) không ảnh hưởng đến hàm mục tiêu hay các ràng buộc đang hoạt động.



Hình 2.12. Trường hợp e. Điểm B là lời giải tối ưu.

2.6. Tối ưu trọng lượng hệ giàn ba thanh chịu ràng buộc độ cứng

Trong phần này, chúng ta tiếp tục bài toán cực tiểu hóa trọng lượng của hệ giàn ba thanh đã khảo sát trước đó, nhưng thay vì ràng buộc về ứng suất, chúng ta sẽ áp đặt ràng buộc về độ cứng. Cụ thể, yêu cầu đặt ra là chuẩn Euclid của vector chuyển vị $\mathbf{u}$ không được vượt quá một giới hạn $\delta_0 > 0$ cho trước, tức là $\mathbf{u}^T\mathbf{u} \leqslant \delta_0^2$.

Xét trường hợp cụ thể với tham số $\beta = 1/10$, tức giả định rằng thanh số 3 có chiều dài gấp 10 lần các thanh 1 và 2. Thay giá trị này vào các biểu thức (2.36), ta xác định

được vector chuyển vị $\mathbf{u}$:

$$\mathbf{u} = \frac{FL}{EA_1(2A_1 + 11A_2)} \begin{pmatrix} 2A_1 + 10A_2 \\ 10A_2 \end{pmatrix}, \tag{2.62}$$

Khi đó, ràng buộc độ cứng được triển khai cụ thể như sau:

$$\mathbf{u}^T\mathbf{u} = \frac{F^2L^2(4A_1^2 + 200A_2^2 + 40A_1A_2)}{E^2A_1^2(2A_1 + 11A_2)^2} \leqslant \delta_0^2 \tag{2.63}$$

Giả sử khối lượng riêng của tất cả các thanh đều là ρ_0, hàm mục tiêu (trọng lượng W của hệ) được xác định bởi:

$$W = \rho_0 L(11A_1 + A_2) \tag{2.64}$$

Để thuận tiện cho việc tính toán, ta đưa vào các biến thiết kế không thứ nguyên:

$$x_i = \frac{E\delta_0}{FL}A_i, \quad i = 1,\ 2 \tag{2.65}$$

Bài toán tối ưu hóa được thiết lập lại theo các biến này như sau:

$$(\mathbb{SO})^6_{\text{nf}} \quad \begin{cases} \min\limits_{x_1,x_2} & 11x_1 + x_2 \\ \text{s.t.} & \begin{cases} \frac{4x_1^2+200x_2^2+40x_1x_2}{x_1^2(2x_1+11x_2)^2} - 1 \leqslant 0 \\ x_1 > 0, \quad x_2 \geqslant 0, \end{cases} \end{cases} \tag{2.66}$$

Bài toán được minh họa trong Hình 2.13. Quan sát sơ bộ, thoạt nhìn lời giải tối ưu dường như nằm tại điểm $x_1 = 1, x_2 = 0$. Tuy nhiên, các hình vẽ phóng to Hình 2.14 cho thấy nhận định này không chính xác.

Để tìm nghiệm thực sự, ta giả định ràng buộc độ cứng là ràng buộc hoạt động. Từ phương trình ràng buộc này, ta biểu diễn x_2 theo x_1, sau đó thế vào hàm mục tiêu để giải bài toán tối ưu hóa một biến đối với x_1. Kết quả tính toán cho ra nghiệm tối ưu là:

$$x_1^* = 0,995, \qquad x_2^* = 0,0169 \tag{2.67}$$

Bằng cách sử dụng công thức (2.65), ta có thể tìm lại được các diện tích mặt cắt ngang tối ưu tương ứng và chuyển nghiệm không thứ nguyên về kích thước thiết kế ban đầu.

2.7. Bài tập

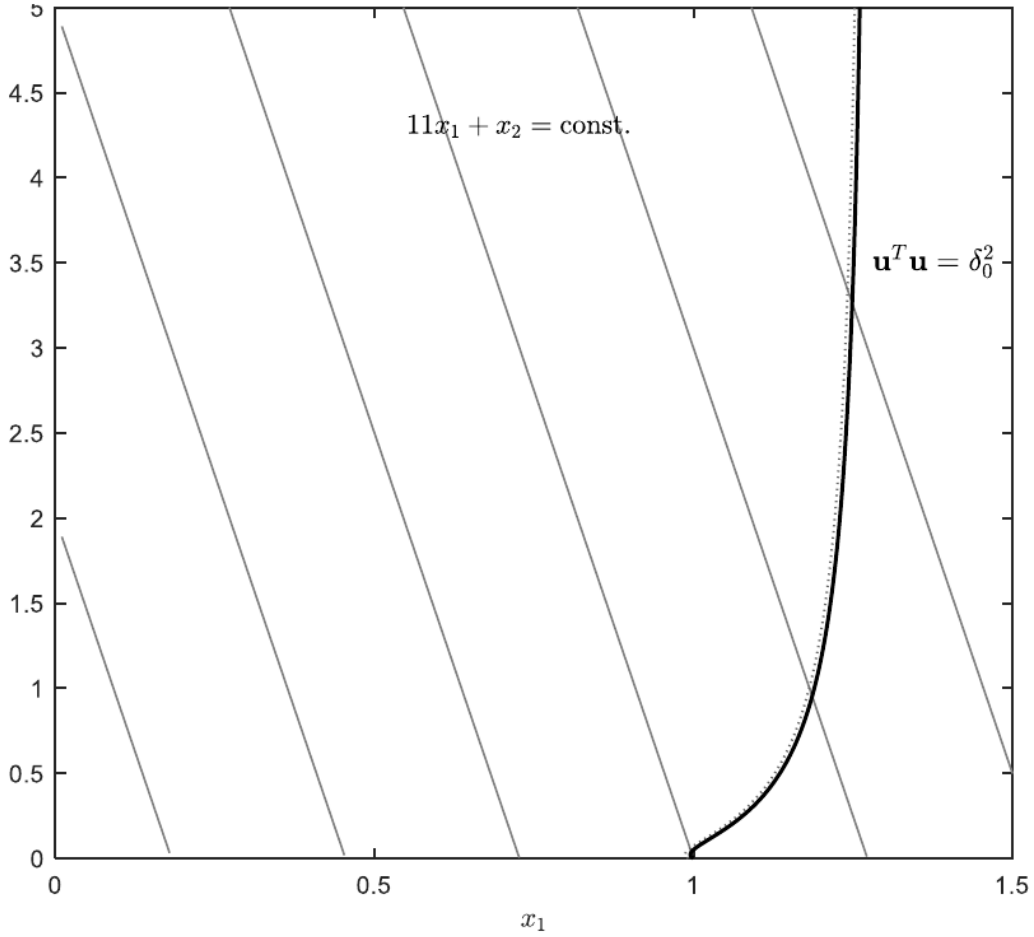
Bài 2.1: Điều gì xảy ra nếu $F < 0$ trong ví dụ ở Mục 2.1?

Bài 2.2: Kiểm chứng các bước suy ra nghiệm (x_1^*, x_2^*) và (x_1^{**}, x_2^{**}) trong Mục 2.4.

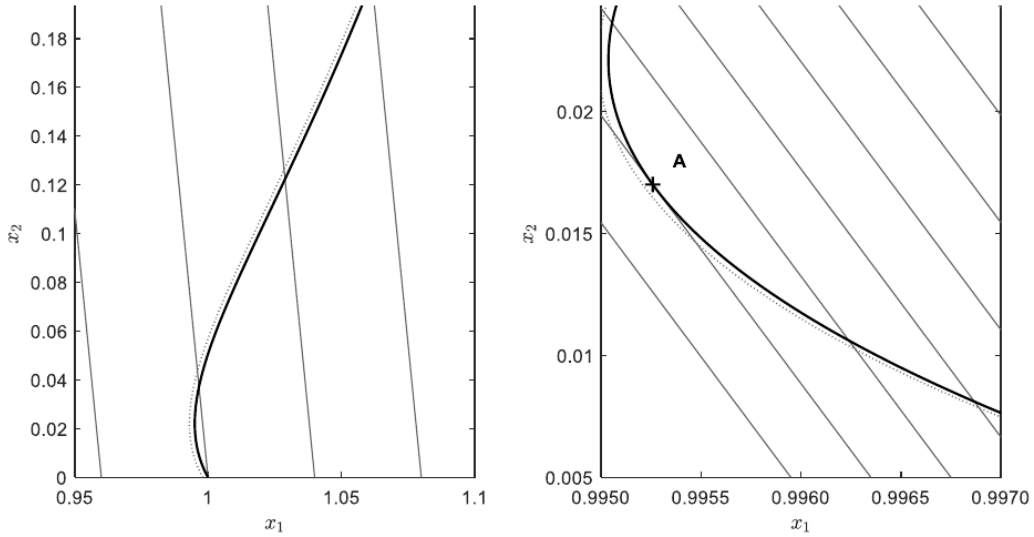
Bài 2.3: Nếu chiều dài của thanh thứ hai trong ví dụ ở Mục 2.5 thay đổi, thì cấu trúc tối ưu của hệ giàn cũng thay đổi: diện tích tối ưu của thanh thứ hai bằng 0 khi $l_2 \geqslant L$ với $\beta = 1$, $\rho_i = \rho_0$ và $\sigma_i^{\max} = \sigma_0,\ i = 1, 2, 3$. Hãy kiểm chứng điều này cho một trường hợp đặc biệt, ví dụ $l_2 = \sqrt{2}L$.

Bài 2.4: Nghiệm của ví dụ ở Mục 2.5 thay đổi như thế nào nếu ứng suất nén cho phép lớn nhất nhỏ hơn ứng suất kéo cho phép?

Bài 2.5: Độ cứng của hệ giàn hai thanh chịu lực $P > 0$ trong Hình 2.15 được cực đại hóa bằng cách cực tiểu hóa chuyển vị $\mathbf{u}$ của nút tự do. Mô đun đàn hồi của cả

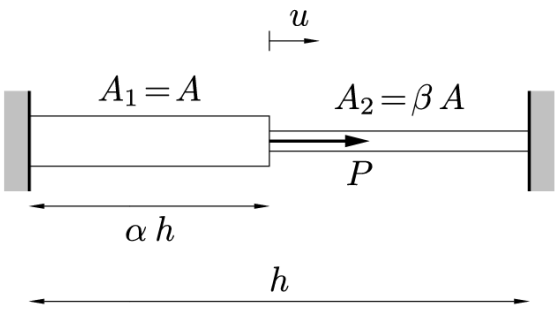


Hình 2.13. Tối ưu bài toán $(\mathbb{SO})^6_{\text{nf}}$.



Hình 2.14. Điểm A là lời giải tối ưu bài toán $(\mathbb{SO})^6_{\text{nf}}$.

hai thanh là E. Thể tích của giàn không được phép vượt quá giá trị V_0. Tổng chiều dài của các thanh là h, trong đó thanh 1 có chiều dài αh, với α là một số nằm trong khoảng từ $\alpha_{\min}$ đến $\alpha_{\max}$. Diện tích mặt cắt ngang của các thanh lần lượt là $A_1 = A$ và $A_2 = \beta A$, với β là một số.



Hình 2.15.

a) Chứng minh rằng bài toán có thể được lập công thức dưới dạng bài toán quy hoạch toán học sau:

$$\begin{cases} \min\limits_{\alpha,\beta} & \dfrac{\alpha(1-\alpha)}{1-\alpha+\alpha\beta} \\ \text{s.t.} & g_1 = \alpha + (1-\alpha)\beta - \dfrac{V_0}{Ah} \leqslant 0 \\ & \alpha_{\min} \leqslant \alpha \leqslant \alpha_{\max} \\ & \beta \geqslant 0 \end{cases}$$

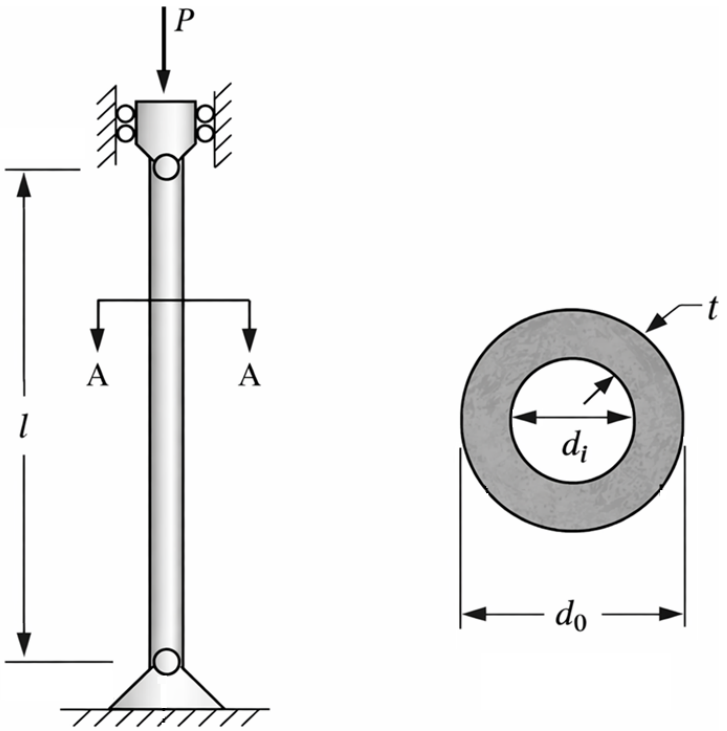
b) Cho $V_0/(Ah) = 1$ và $\alpha_{\max} = 1$. Giải bài toán cho trường hợp $\alpha_{\min}$ bất kỳ.

c) Cho $V_0/(Ah) = 1,2$ và $\alpha_{\min} = 0,2$. Giải bài toán với $\alpha_{\max} = 0,6$ và $\alpha_{\max} = 0,8$.

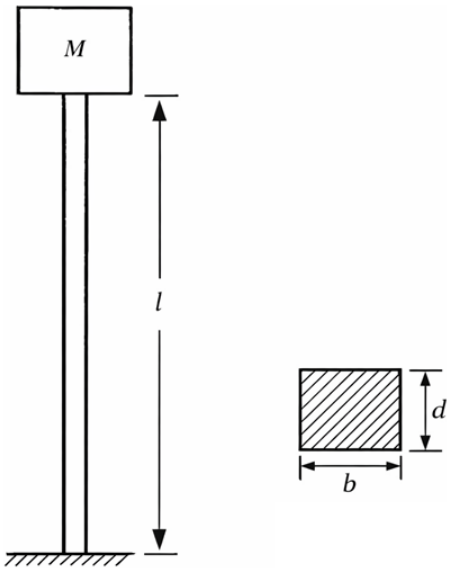
Bài 2.6: Thiết kế một cột có tiết diện hình ống, hai đầu liên kết bản lề, để chịu một tải trọng nén P = 25 kN với chi phí thấp nhất. Cột được làm từ vật liệu có ứng suất chảy là 50 MPa, mô-đun đàn hồi là 80 GPa, và khối lượng riêng là 2500 kg/m3. Chiều dài của cột là 2,5 m. Ứng suất phát sinh trong cột phải nhỏ hơn ứng suất uốn dọc tới hạn cũng như ứng suất chảy. Đường kính trung bình của cột bị giới hạn trong khoảng từ 0,02 m đến 0,14 m, và cột có độ dày thành ống nằm trong khoảng 2 mm đến 8 mm. Chi phí của cột bao gồm chi phí vật liệu và chi phí chế tạo, được tính bằng công thức 5W + 2d, trong đó W là trọng lượng tính bằng kg và d là đường kính trung bình của cột tính bằng cm.

Bài 2.7: Một cột đồng nhất có tiết diện hình chữ nhật được thiết kế để đỡ một bồn nước có khối lượng M. Yêu cầu đặt ra là phải tối thiểu hóa khối lượng của cột để tiết kiệm chi phí, và tối đa hóa tần số dao động tự nhiên của dao động ngang nhằm tránh khả năng xảy ra cộng hưởng do gió. Hãy thiết lập bài toán thiết kế cột để tránh hư hỏng do nén trực tiếp và mất ổn định (buckling). Giả sử ứng suất nén cho phép là σ_{max}.

Bài 2.8: Một dầm chữ nhật hẹp có tiết diện kích thước $b \times h$ được ngàm một đầu, chiều dài $L = 6$ m, chịu lực tập trung ở đầu tự do $p = 45$ kN. Ngoài phá hoại do uốn, dầm còn có thể mất ổn định ngang kèm xoắn; tải trọng tới hạn được cho bởi



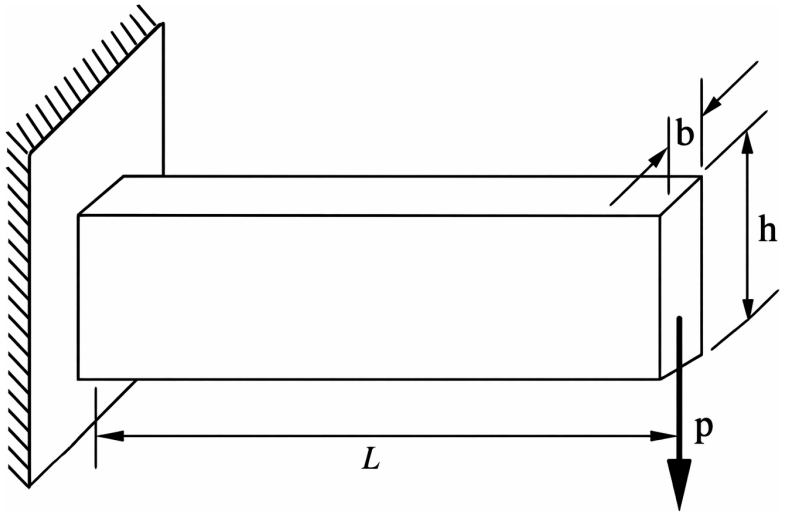
Hình 2.16.



Hình 2.17.

$$p_{cr} = 4,013\frac{\sqrt{EI_{\text{least}}c}}{L^2}$$

trong đó E là mô đun đàn hồi, I_{least} là mô men quán tính nhỏ nhất của tiết diện, và $c = 0,312hb^3G$ là độ cứng xoắn của dầm với G là mô-đun trượt. Yêu cầu thiết kế dầm có khối lượng nhỏ nhất sao cho không bị phá hoại cả về uốn lẫn xoắn. Giả thiết $E = 200$ GPa, $G = 80$ GPa và ứng suất cho phép $[\sigma] = 517$ MPa (kéo và nén).



Hình 2.18.

CHƯƠNG 3

CƠ SỞ TOÁN HỌC QUY HOẠCH LỒI

Phương pháp tiếp cận bằng đồ thị ở chương trước dựa trên giả định rằng ta có thể dễ dàng xác định các ràng buộc hoạt động (active constraints) tại điểm nghiệm tối ưu. Tuy nhiên, các bài toán thực tế thường có quy mô lớn và phức tạp, khiến việc xác định trực quan trở nên bất khả thi. Để giải quyết vấn đề đó, chúng ta cần những công cụ mạnh mẽ hơn từ lĩnh vực quy hoạch toán học (mathematical programming).

Chương này trình bày các cơ sở toán học làm nền tảng cho phần lớn các thuật toán tối ưu hóa kết cấu được sử dụng trong cuốn sách. Trước hết, ta phân biệt cực trị địa phương với cực trị toàn cục và làm rõ vì sao tính lồi (convexity) lại đóng vai trò then chốt: đối với bài toán lồi, mọi cực tiểu địa phương đều là cực tiểu toàn cục, nhờ đó việc giải bài toán trở nên đáng tin cậy. Tiếp theo, ta xây dựng các tiêu chuẩn kiểm tra tính lồi của tập hợp và hàm số, phát biểu hệ điều kiện tối ưu Karush–Kuhn–Tucker (KKT) cho bài toán có ràng buộc, và cuối cùng giới thiệu lý thuyết đối ngẫu Lagrange (Lagrangian duality) như một công cụ để phân rã và giải các bài toán quy mô lớn. Những khái niệm này sẽ được vận dụng xuyên suốt các chương sau.

3.1. Cực trị địa phương và cực trị toàn cục

Xét bài toán tối ưu hóa dưới dạng chuẩn tắc ($\mathbb{P}$):

$$(\mathbb{P}) \quad \begin{cases} \min\limits_{x} \; g_0(\mathbf{x}) \\ \text{s.t.} \;\; g_i(\mathbf{x}) \leqslant 0, \quad i = 1, \dots, l \\ \qquad \mathbf{x} \in \mathcal{X}, \end{cases} \tag{3.1}$$

trong đó $g_i(\mathbf{x})$ là các hàm khả vi liên tục và $\mathcal{X}$ là tập hợp các ràng buộc hộp (giới hạn trên và dưới của biến, có thể là vô hạn):

$$\mathcal{X} = \{\mathbf{x} \in \mathbb{R}^n : x_j^{\min} \leqslant x_j \leqslant x_j^{\max}, \quad j = 1, \dots, n\}. \tag{3.2}$$

Lưu ý mọi bài toán cực đại hóa $\max g_0(\mathbf{x})$ đều có thể chuyển về bài toán cực tiểu hóa thông qua phép biến đổi tương đương: $\max g_0(\mathbf{x}) = -\min(-g_0(\mathbf{x}))$. Trong miền thiết kế này, điểm khả dĩ (feasible point) là điểm $\bar{\mathbf{x}}$ nằm trong tập khả dĩ (feasible set), thỏa mãn tất cả các ràng buộc $g_i(\bar{\mathbf{x}}) \leqslant 0$ và $\bar{\mathbf{x}} \in \mathcal{X}$. Cực tiểu toàn cục (global minimum) là điểm khả dĩ $\mathbf{x}^*$ tốt nhất trong toàn bộ không gian tìm kiếm, sao cho $g_0(\mathbf{x}^*) \leqslant g_0(\bar{\mathbf{x}})$ với mọi điểm khả dĩ $\bar{\mathbf{x}}$ khác.

Để hiểu rõ hơn bản chất của việc tìm kiếm nghiệm tối ưu, cần phân biệt giữa bài toán không ràng buộc và bài toán có ràng buộc. Với bài toán không ràng buộc (unconstrained problem), các cực trị thường xuất hiện tại các điểm dừng (stationary points), nơi đạo hàm (gradient) triệt tiêu, tức là $\nabla g_0(\mathbf{x}^*) = \mathbf{0}$. Ngược lại, với bài toán có ràng buộc (constrained problem), cực tiểu địa phương không nhất thiết phải là

điểm dừng, bởi nghiệm tối ưu có thể nằm ngay trên biên của miền khả dĩ. Đối với các bài toán tổng quát, cực tiểu địa phương cũng không nhất thiết là cực tiểu toàn cục; đây là thách thức lớn nhất của tối ưu hóa. Tuy nhiên, trong một lớp bài toán đặc biệt quan trọng là bài toán lồi (convex problems), tính chất này được đảm bảo: mọi cực tiểu địa phương đều là cực tiểu toàn cục.

3.2. Tính lồi

3.2.1. Định nghĩa tập lồi và hàm lồi

Một tập hợp $S \subset \mathbb{R}^n$ được gọi là tập lồi (convex set) nếu với mọi $\mathbf{x}_1, \mathbf{x}_2 \in S$ và mọi $\lambda \in (0, 1)$, ta có:

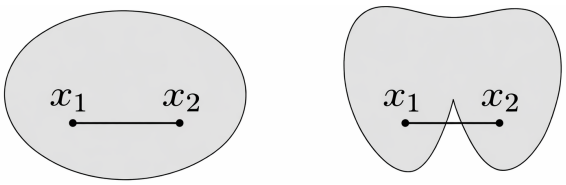
$$\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2 \in S \tag{3.3}$$

Nghĩa là, một tập hợp được gọi là lồi nếu tất cả các điểm nằm trên đoạn thẳng nối hai điểm bất kỳ thuộc tập hợp đó cũng thuộc về tập hợp đó (xem Hình 3.1).

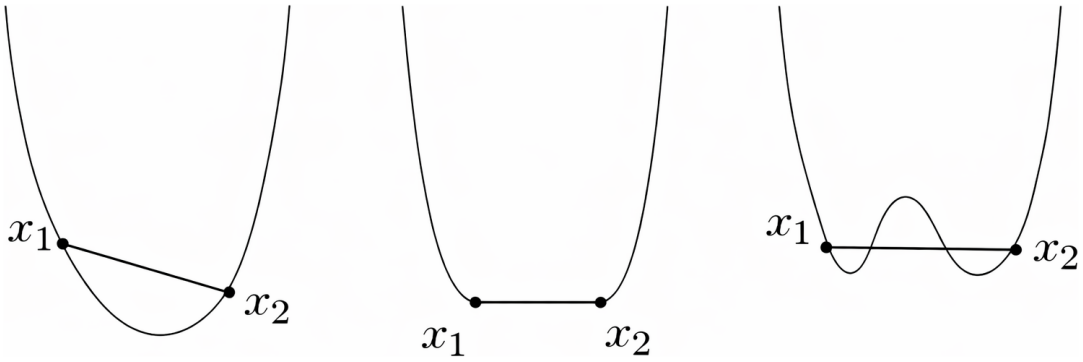
Một hàm số $f : S \to \mathbb{R}$ được gọi là lồi (trên tập lồi $S \subset \mathbb{R}^n$) nếu với mọi $\mathbf{x}_1, \mathbf{x}_2 \in S$ với $\mathbf{x}_1 \neq \mathbf{x}_2$ và mọi $\lambda \in (0, 1)$, ta có:

$$f(\lambda \mathbf{x}_1 + (1 - \lambda)\mathbf{x}_2) \leqslant \lambda f(\mathbf{x}_1) + (1 - \lambda) f(\mathbf{x}_2) \tag{3.4}$$

Hàm số f là lồi chặt (strictly convex) nếu bất đẳng thức chặt ($<$) xảy ra trong biểu thức trên. Hàm số f là lõm chặt (strictly concave) nếu $-f$ là lồi chặt.



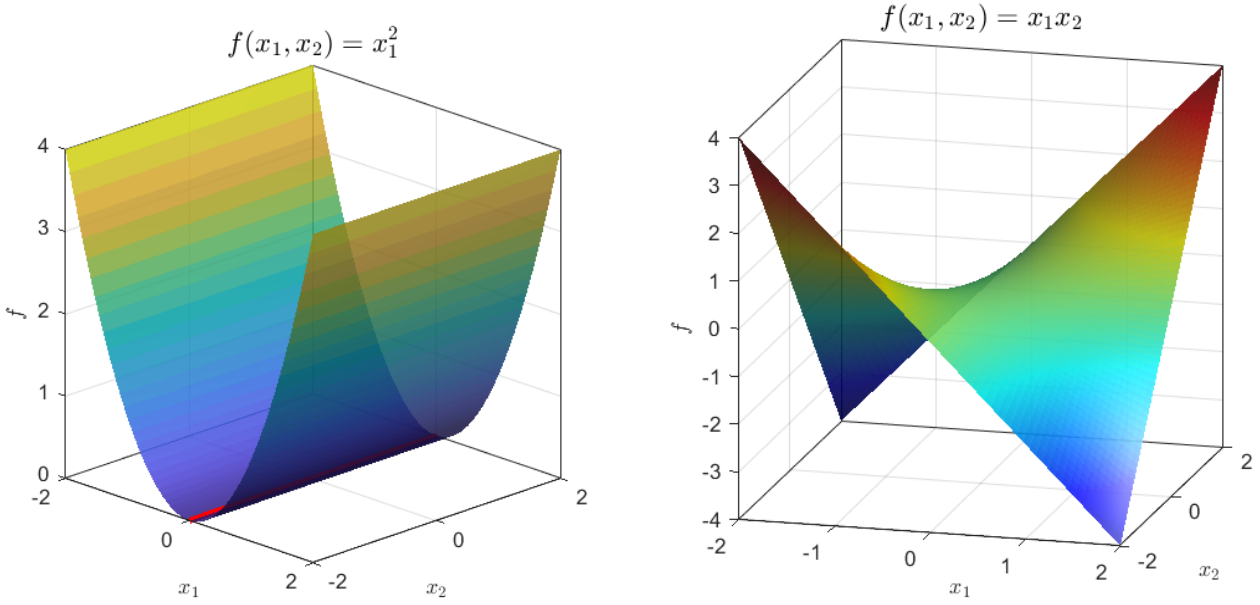
Hình 3.1. Tập lồi (trái) và không lồi (phải).



Hình 3.2. Hàm lồi chặt (trái), lồi (giữa) và không lồi (phải).

Ví dụ 3.1: Kiểm tra tính lồi của các hàm số sau:
$f : \mathbb{R} \to \mathbb{R}, f(x_1) = x_1^2$
$f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1^2$
$f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1 x_2$
Hàm $f : \mathbb{R} \to \mathbb{R}, f(x_1) = x_1^2$ là hàm lồi chặt.

Hàm $f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1^2$ là hàm lồi.
Hàm $f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1 x_2$ là hàm không lồi cũng không lõm.



Hình 3.3. Kiểm tra tính lồi của các hàm số trong Ví dụ 3.1.

3.2.2. Tính chất của bài toán tối ưu lồi

Bằng cách áp dụng các định nghĩa về tập lồi và hàm lồi, ta dễ dàng thu được các tính chất sau:

(i) Tập $S = \{\mathbf{x} \in \mathcal{X} : g_i(\mathbf{x}) \leqslant 0, i = 1, \ldots, l\}$ là lồi nếu các hàm $g_i : \mathbb{R}^n \to \mathbb{R}, i = 1, \ldots, l$ là các hàm lồi.

(ii) Cho S là một tập lồi. Nếu $f : S \to \mathbb{R}$ và $g : S \to \mathbb{R}$ là các hàm lồi và $h : S \to \mathbb{R}$ là lồi chặt, thì αf là hàm lồi với $\alpha \geqslant 0$ là một số tùy ý, $f + g$ là hàm lồi và $f + h$ là hàm lồi chặt.

Nếu cả hàm mục tiêu và tập khả dĩ của bài toán $(\mathbb{P})$ đều lồi, bài toán được gọi là bài toán lồi.

3.2.3. Kiểm tra tính lồi của hàm khả vi

Để xác định xem một hàm khả vi liên tục có lồi hay không, chúng ta có thể xem xét gradient (đạo hàm cấp 1) hoặc ma trận Hessian (đạo hàm cấp 2) của nó; đây là cơ sở cho các tiêu chuẩn kiểm tra tính lồi được trình bày trong các định lý tiếp theo.

Định lý 3.1: Cho $f : S \to \mathbb{R}$, trong đó S là tập lồi và f khả vi liên tục. Khi đó f là lồi (hoặc lồi chặt) khi và chỉ khi gradient ∇f là đơn điệu (hoặc đơn điệu chặt).

Một hàm $g : S \to \mathbb{R}^n$ gọi là đơn điệu (monotone) trên S nếu với mọi $\mathbf{x}_1, \mathbf{x}_2 \in S$ với $\mathbf{x}_1 \neq \mathbf{x}_2$ ta có:

$$(\mathbf{x}_2 - \mathbf{x}_1)^T (g(\mathbf{x}_2) - g(\mathbf{x}_1)) \geqslant 0 \tag{3.5}$$

Hàm g là đơn điệu chặt (strictly monotone) trên S nếu bất đẳng thức ở (3.5) thực sự xảy ra.

Ví dụ 3.2: Hàm $f : \mathbb{R} \to \mathbb{R}, f(x) = x^4$ là lồi chặt trên $\mathbb{R}$ vì gradient $\nabla f(x) = 4x^3$ là đơn điệu chặt trên $\mathbb{R}$:

$$(x_2 - x_1)(x_2^3 - x_1^3) = (x_2 - x_1)^2(x_1^2 + x_1x_2 + x_2^2)$$
$$= (x_2 - x_1)^2 \left[\left(x_1 + \frac{1}{2}x_2 \right)^2 + \frac{3}{4}x_2^2 \right] > 0,\ x_1 \neq x_2$$

Định lý 3.2: Cho $f : S \to \mathbb{R}$, trong đó S là lồi và f khả vi liên tục hai lần. Khi đó:

(i) f là lồi khi và chỉ khi ma trận Hessian $\nabla^2 f$ là nửa xác định dương (positive semidefinite).

(ii) f là lồi chặt nếu ma trận Hessian $\nabla^2 f$ là xác định dương (positive definite).

trong đó ma trận Hessian $\nabla^2 f(\mathbf{x})$ được định nghĩa bởi:

$$\nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f(\mathbf{x})}{\partial \mathbf{x}_1^2} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial \mathbf{x}_1 \partial \mathbf{x}_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f(\mathbf{x})}{\partial \mathbf{x}_n \partial \mathbf{x}_1} & \cdots & \frac{\partial^2 f(\mathbf{x})}{\partial \mathbf{x}_n^2} \end{bmatrix} \tag{3.6}$$

Một ma trận $\mathbf{A} \in \mathbb{R}^{n \times n}$ là nửa xác định dương nếu $\mathbf{y}^T \mathbf{A}\mathbf{y} \geqslant 0$ với mọi $\mathbf{y} \in \mathbb{R}^n$ và xác định dương nếu $\mathbf{y}^T \mathbf{A}\mathbf{y} > 0$ với mọi $\mathbf{y} \in \mathbb{R}^n$ mà $\mathbf{y} \neq \mathbf{0}$. Tính xác định dương của một ma trận đối xứng có thể được kiểm tra bằng tiêu chuẩn Sylvester như sau:

Định lý 3.3: Một ma trận đối xứng $\mathbf{A} \in \mathbb{R}^{n \times n}$ là xác định dương khi và chỉ khi tất cả các định thức con cấp k ở góc trên bên trái đều dương với mọi $k = 1, \ldots, n$.

Bên cạnh tiêu chuẩn Sylvester, tính xác định dương còn có thể được kiểm chứng thông qua các phương pháp khác như phân rã Cholesky hoặc kiểm tra các trị riêng (eigenvalues) của ma trận đều dương. Tuy nhiên, để giữ sự tập trung vào nội dung chính, cuốn sách này sẽ không đi sâu vào chi tiết các thuật toán đại số tuyến tính đó.

Ví dụ 3.3: Xét hàm $f : \mathbb{R}^2 \to \mathbb{R}, f(x_1, x_2) = x_1^2 + x_2^2$. Khi đó:

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 \\ 2x_2 \end{bmatrix}, \quad \nabla^2 f(x_1, x_2) = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}.$$

Ma trận Hessian là xác định dương theo tiêu chuẩn Sylvester vì $2 > 0$ và $2 \times 2 - 0 \times 0 > 0$. Do đó, Định lý 3.2 suy ra rằng f là lồi chặt trên $\mathbb{R}^2$.

3.3. Điều kiện tối ưu KKT của bài toán tối ưu hóa có ràng buộc

Để xác định các điểm cực tiểu địa phương (và cũng là cực tiểu toàn cục đối với bài toán lồi), chúng ta sử dụng hệ điều kiện Karush–Kuhn–Tucker (KKT).

Xét bài toán tối ưu (3.1) ($\mathbb{P}$) có hàm mục tiêu $g_0(\mathbf{x})$ và các ràng buộc bất đẳng thức $g_i(\mathbf{x}) \leqslant 0, i = 1, \ldots, l$. Hàm Lagrange $\mathcal{L} : \mathbb{R}^n \times \mathbb{R}^l \to \mathbb{R}$ của bài toán được định nghĩa là:

$$\mathcal{L}(\mathbf{x}, \lambda) = g_0(\mathbf{x}) + \sum_{i=1}^{l} \lambda_i g_i(\mathbf{x}) \tag{3.7}$$

trong đó λ_i là các nhân tử Lagrange tương ứng với ràng buộc thứ i.

Một điểm $\mathbf{x}^* \in \mathbb{R}^n$ được gọi là điểm KKT nếu tồn tại vector nhân tử $\lambda^* \in \mathbb{R}^l$ thỏa mãn hệ điều kiện sau:

1. Điều kiện dừng (stationarity)

Đạo hàm của hàm Lagrange theo các biến thiết kế phải thỏa mãn:

$$\nabla_{\mathbf{x}}\mathcal{L}(\mathbf{x}^*, \lambda^*) = \nabla g_0(\mathbf{x}^*) + \sum_{i=1}^{l} \lambda_i^* \nabla g_i(\mathbf{x}^*) = \mathbf{0} \tag{3.8}$$

2. Điều kiện khả dĩ nguyên thủy (primal feasibility)
Điểm giải phải thỏa mãn mọi ràng buộc của bài toán:

$$g_i(\mathbf{x}^*) \leqslant 0, \quad \forall i = 1, \ldots, l \tag{3.9}$$

3. Điều kiện khả dĩ đối ngẫu (dual feasibility)
Các nhân tử Lagrange phải không âm:

$$\lambda_i^* \geqslant 0, \quad \forall i = 1, \ldots, l \tag{3.10}$$

4. Điều kiện độ lệch bù (complementary slackness)
Nhân tử Lagrange chỉ được phép dương tại các ràng buộc đang hoạt động:

$$\lambda_i^* g_i(\mathbf{x}^*) = 0, \quad \forall i = 1, \ldots, l \tag{3.11}$$

Trong thực tế kỹ thuật, các biến thiết kế thường bị giới hạn bởi cận trên và cận dưới $x_j^{\min} \leqslant x_j \leqslant x_j^{\max}$. Thay vì đưa các giới hạn này vào danh sách $g_i(\mathbf{x})$, ta có thể xử lý chúng trực tiếp trong điều kiện dừng để giảm số lượng nhân tử Lagrange cần tìm. Khi đó, điều kiện $\nabla_{\mathbf{x}}\mathcal{L} = 0$ được thay thế bằng:

$$\begin{aligned}
\frac{\partial \mathcal{L}}{\partial x_j} &\geqslant 0 \text{ nếu } x_j = x_j^{\min} \\
\frac{\partial \mathcal{L}}{\partial x_j} &\leqslant 0 \text{ nếu } x_j = x_j^{\max} \\
\frac{\partial \mathcal{L}}{\partial x_j} &= 0 \text{ nếu } x_j^{\min} < x_j < x_j^{\max}
\end{aligned} \tag{3.12}$$

Đối với các bài toán không lồi, điều kiện KKT chỉ là điều kiện cần. Nghĩa là, mọi cực trị địa phương đều là điểm KKT, nhưng một điểm KKT chưa chắc đã là cực trị (nó có thể là điểm yên ngựa). Tuy nhiên, đối với bài toán lồi, KKT trở thành điều kiện đủ.

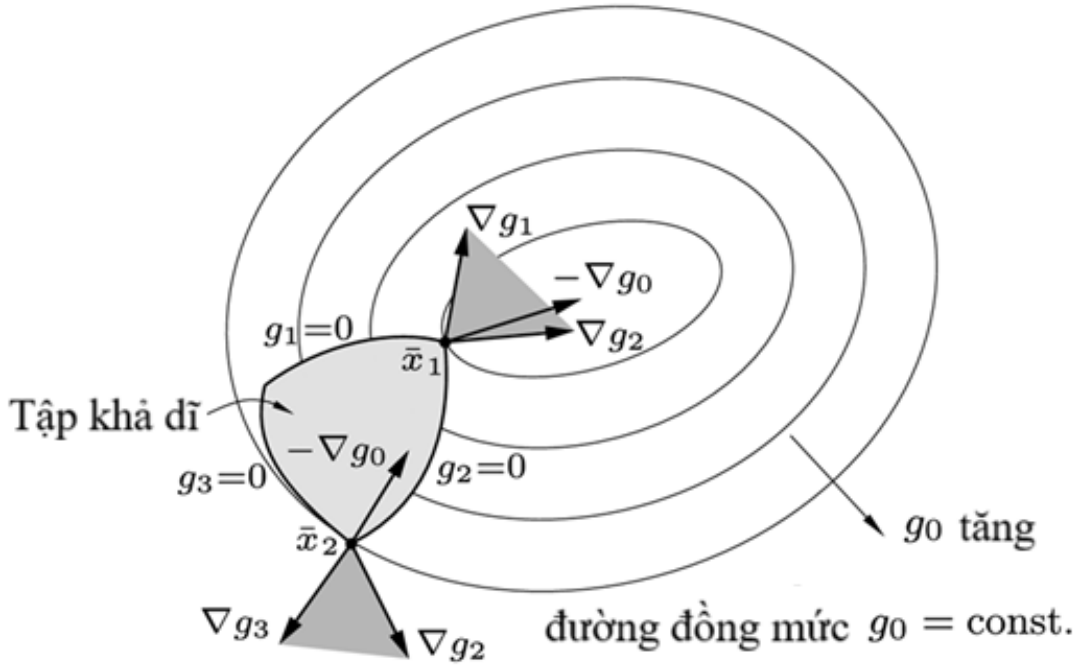
Minh họa trực quan cho định lý KKT được thể hiện tại Hình 3.4. Theo điều kiện KKT, tại một điểm khả dĩ $\bar{x}$, hướng xuống dốc nhanh nhất của hàm mục tiêu phải nằm trong nón được sinh bởi các vector gradient của các ràng buộc đang hoạt động tại điểm đó.

Xét điểm $\bar{x}_1$: Tại đây, hai ràng buộc g_1 và g_2 đang ở trạng thái hoạt động, tức là $g_1(\bar{x}_1) = 0$ và $g_2(\bar{x}_1) = 0$. Quan sát hình vẽ, ta thấy vector $-\nabla g_0(\bar{x}_1)$ nằm lọt vào giữa góc tạo bởi hai vector $\nabla g_1(\bar{x}_1)$ và $\nabla g_2(\bar{x}_1)$. Điều này tương đương với việc tồn tại các hệ số không âm $\lambda_1 \geqslant 0$ và $\lambda_2 \geqslant 0$ thỏa mãn phương trình cân bằng:

$$-\nabla g_0(\bar{x}_1) = \lambda_1 \nabla g_1(\bar{x}_1) + \lambda_2 \nabla g_2(\bar{x}_1) \tag{3.13}$$

Do đó, $\bar{x}_1$ thỏa mãn các điều kiện KKT và là nghiệm tối ưu của bài toán.

Xét điểm $\bar{x}_2$: Tại vị trí này, các ràng buộc g_2 và g_3 đang hoạt động. Tuy nhiên, vector $-\nabla g_0(\bar{x}_2)$ lại nằm lệch ra ngoài nón tạo bởi $\nabla g_2(\bar{x}_2)$ và $\nabla g_3(\bar{x}_2)$. Nói cách khác, không tồn tại bất kỳ cặp số thực không âm λ_2, λ_3 nào để biểu diễn $-\nabla g_0$ dưới dạng tổ hợp tuyến tính của hai vector gradient ràng buộc:



Hình 3.4. Minh họa hình học của các điều kiện KKT.

$$-\nabla g_0(\bar{x}_2) \neq \lambda_2 \nabla g_2(\bar{x}_2) + \lambda_3 \nabla g_3(\bar{x}_2), \quad \forall \lambda_2, \lambda_3 \geqslant 0 \tag{3.14}$$

Vì vậy, $\bar{x}_2$ vi phạm điều kiện KKT, dẫn đến việc điểm này chắc chắn không phải là nghiệm tối ưu.

Ví dụ 3.4: Tìm nghiệm tối ưu của bài toán sau

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^2} \quad & g_0(\mathbf{x}) = x_1^2 + x_2^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = 1 - x_1 - x_2 \leqslant 0 \end{aligned}$$

Hàm mục tiêu g_0 là lồi (dạng paraboloid) và hàm ràng buộc g_1 là tuyến tính (lồi). Do đó, đây là bài toán tối ưu lồi. Bất kỳ điểm nào thỏa mãn điều kiện KKT đều là nghiệm tối ưu toàn cục.

Hàm Lagrange của bài toán được thiết lập như sau:

$$\mathcal{L}(\mathbf{x}, \lambda) = x_1^2 + x_2^2 + \lambda(1 - x_1 - x_2)$$

Hệ điều kiện KKT để tìm điểm cực trị $(\mathbf{x}^*, \lambda^*)$ bao gồm:

1. Điều kiện dừng:

Đạo hàm của hàm Lagrange theo biến $\mathbf{x}$ phải bằng 0:

$$\nabla_{\mathbf{x}} \mathcal{L} = \begin{bmatrix} 2x_1 - \lambda \\ 2x_2 - \lambda \end{bmatrix} = \mathbf{0} \Rightarrow x_1 = x_2 = \frac{\lambda}{2} \quad (*)$$

2. Điều kiện độ lệch bù:

$$\lambda(1 - x_1 - x_2) = 0$$

Điều kiện này dẫn đến hai trường hợp để biện luận:

Trường hợp 1: Ràng buộc không hoạt động ($\lambda = 0$).
Thay vào $(*)$, ta có $x_1 = x_2 = 0$.
Kiểm tra điều kiện khả dĩ: $g_1(0,0) = 1 - 0 - 0 = 1 \not\leqslant 0$. (Vi phạm ràng buộc) $\Rightarrow$ Loại.
Trường hợp 2: Ràng buộc hoạt động ($\lambda > 0$).
Khi đó, biểu thức trong ngoặc phải bằng 0:

$$1 - x_1 - x_2 = 0$$

Thay $(*)$ vào phương trình trên:

$$1 - \frac{\lambda}{2} - \frac{\lambda}{2} = 0 \Rightarrow 1 - \lambda = 0 \Rightarrow \lambda = 1$$

Với $\lambda = 1$, ta tính được nghiệm tương ứng:

$$x_1^* = x_2^* = \frac{1}{2}$$

Kiểm tra điều kiện khả dĩ đối ngẫu: $\lambda^* = 1 > 0$ (Thỏa mãn).
Vậy, bộ nghiệm KKT duy nhất là:

$$\mathbf{x}^* = \left(\frac{1}{2}, \frac{1}{2}\right), \quad \lambda^* = 1$$

Vì đây là bài toán lồi, $\mathbf{x}^* = \left(\frac{1}{2}, \frac{1}{2}\right)$ chính là nghiệm tối ưu toàn cục.

Ví dụ 3.5: Tìm nghiệm tối ưu của bài toán sau

$$\begin{aligned} \min \quad & f(x_1, x_2) = (x_1 - 5)^2 + (x_2 - 4)^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = x_1 + 2x_2 - 10 \leqslant 0 \\ & g_2(\mathbf{x}) = x_1 - 4 \leqslant 0 \end{aligned}$$

Bước 1: Thiết lập hàm Lagrange

$$\mathcal{L}(\mathbf{x}, \lambda) = (x_1 - 5)^2 + (x_2 - 4)^2 + \lambda_1(x_1 + 2x_2 - 10) + \lambda_2(x_1 - 4)$$

Bước 2: Thiết lập hệ phương trình KKT
Điều kiện dừng:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 2(x_1 - 5) + \lambda_1 + \lambda_2 = 0 & (1) \\ \frac{\partial \mathcal{L}}{\partial x_2} = 2(x_2 - 4) + 2\lambda_1 = 0 & (2) \end{cases}$$

Điều kiện khả dĩ nguyên thủy:

$$g_1(\mathbf{x}) \leqslant 0, \ g_2(\mathbf{x}) \leqslant 0$$

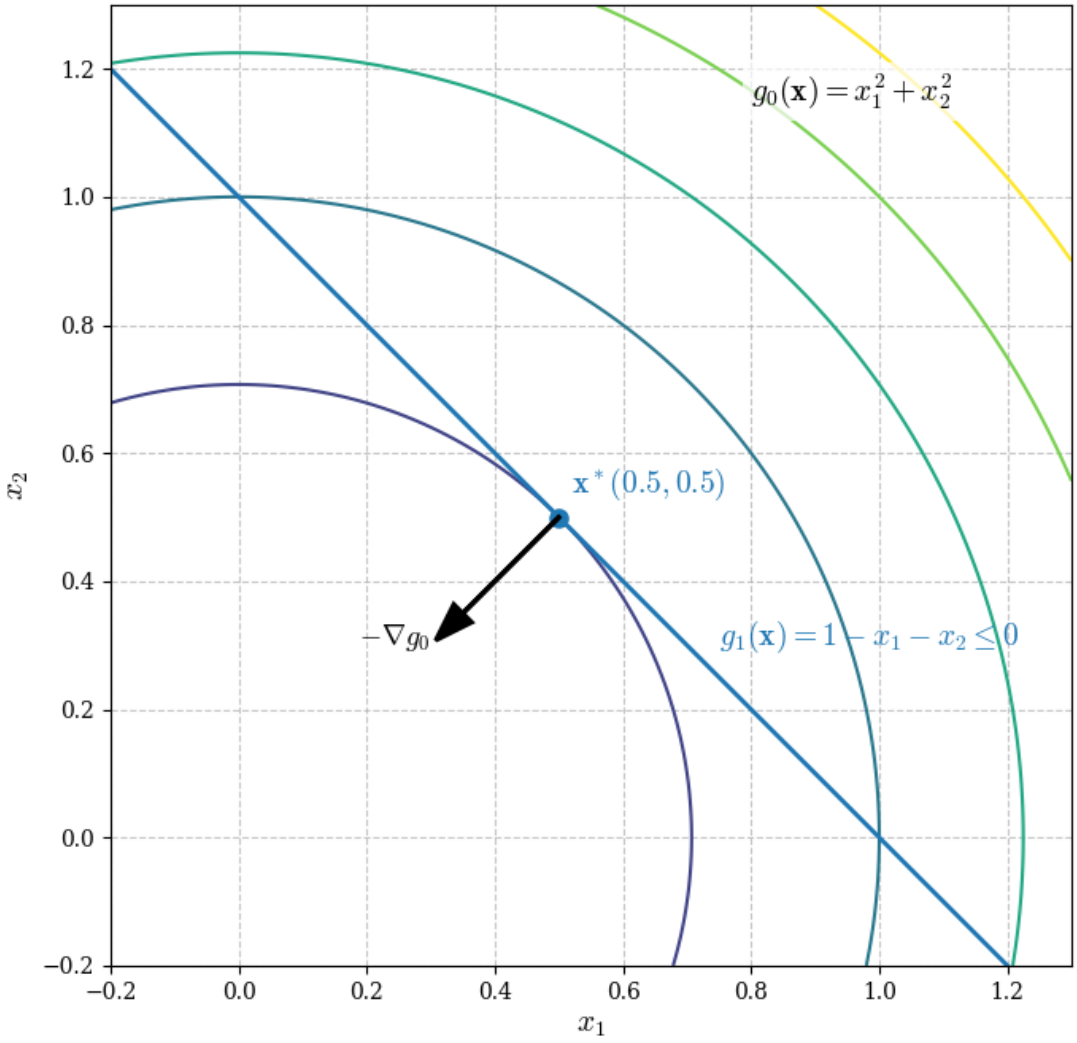
Điều kiện khả dĩ đối ngẫu:

$$\lambda_1 \geqslant 0, \ \lambda_2 \geqslant 0$$

Điều kiện độ lệch bù:

$$\lambda_1 g_1(\mathbf{x}) = 0, \ \lambda_2 g_2(\mathbf{x}) = 0$$

Bước 3: Giải hệ phương trình

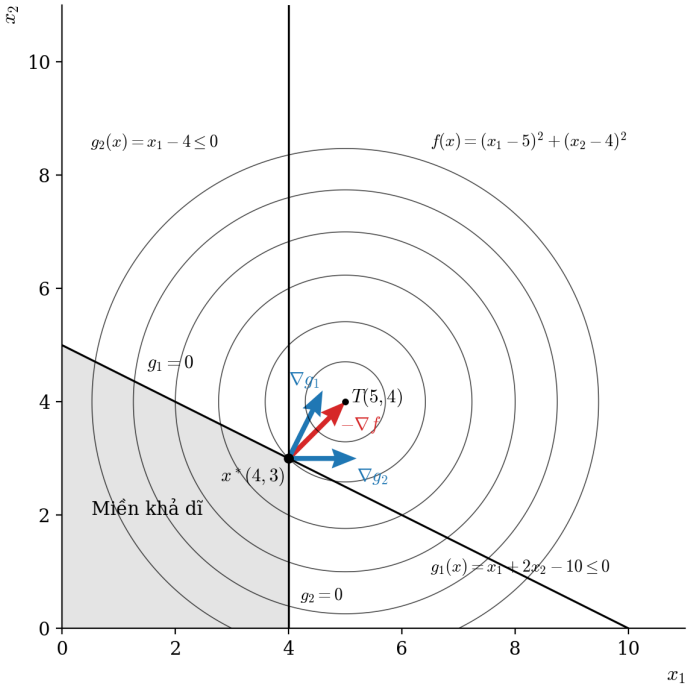


Hình 3.5. Miền khả dĩ và nghiệm KKT của Ví dụ 3.4.

Giả sử cả hai ràng buộc đều hoạt động. Khi đó $g_1 = 0$ và $g_2 = 0$.
Từ $g_2(\mathbf{x}) = x_1 - 4 = 0 \Rightarrow \mathbf{x_1 = 4}$.
Thay vào $g_1(\mathbf{x}) = 4 + 2x_2 - 10 = 0 \Rightarrow 2x_2 = 6 \Rightarrow \mathbf{x_2 = 3}$.
Vậy nghiệm dự đoán là $\mathbf{x}^* = (4, 3)$. Ta cần tìm λ_1, λ_2 để kiểm tra xem điểm này có thỏa mãn KKT không.
Thay $x_1 = 4, x_2 = 3$ vào phương trình (1) và (2):
Từ (2): $2(3 - 4) + 2\lambda_1 = 0 \Rightarrow -2 + 2\lambda_1 = 0 \Rightarrow \mathbf{\lambda_1 = 1}$.
Từ (1): $2(4 - 5) + \lambda_1 + \lambda_2 = 0 \Rightarrow -2 + 1 + \lambda_2 = 0 \Rightarrow \mathbf{\lambda_2 = 1}$.
Kiểm tra điều kiện:
Cả $\lambda_1 = 1 > 0$ và $\lambda_2 = 1 > 0$ đều thỏa mãn điều kiện khả dĩ đối ngẫu.
Kết luận:

$$\mathbf{x}^* = (4, 3)$$

là nghiệm tối ưu.



Hình 3.6. Các ràng buộc hoạt động tại nghiệm của Ví dụ 3.5.

Ví dụ 3.6: Tìm nghiệm tối ưu của bài toán sau

$$\begin{aligned} \min \quad & f(x) : x^2 + y^2 + z^2 = 3 \\ \text{s.t.} \quad & g(x) : x + y + z \geqslant 3 \end{aligned}$$

Bước 1: Thiết lập hàm Lagrangian:

$$\mathcal{L} = x^2 + y^2 + z^2 + \lambda(3 - x - y - z)$$

Bước 2: Thiết lập hệ phương trình KKT
Điều kiện dừng:

$$\nabla \mathcal{L} = 0 \Rightarrow 2x = \lambda,\ 2y = \lambda,\ 2z = \lambda \Rightarrow x = y = z = \frac{\lambda}{2}$$

Điều kiện độ lệch bù:

$$\lambda(3 - x - y - z) = 0$$

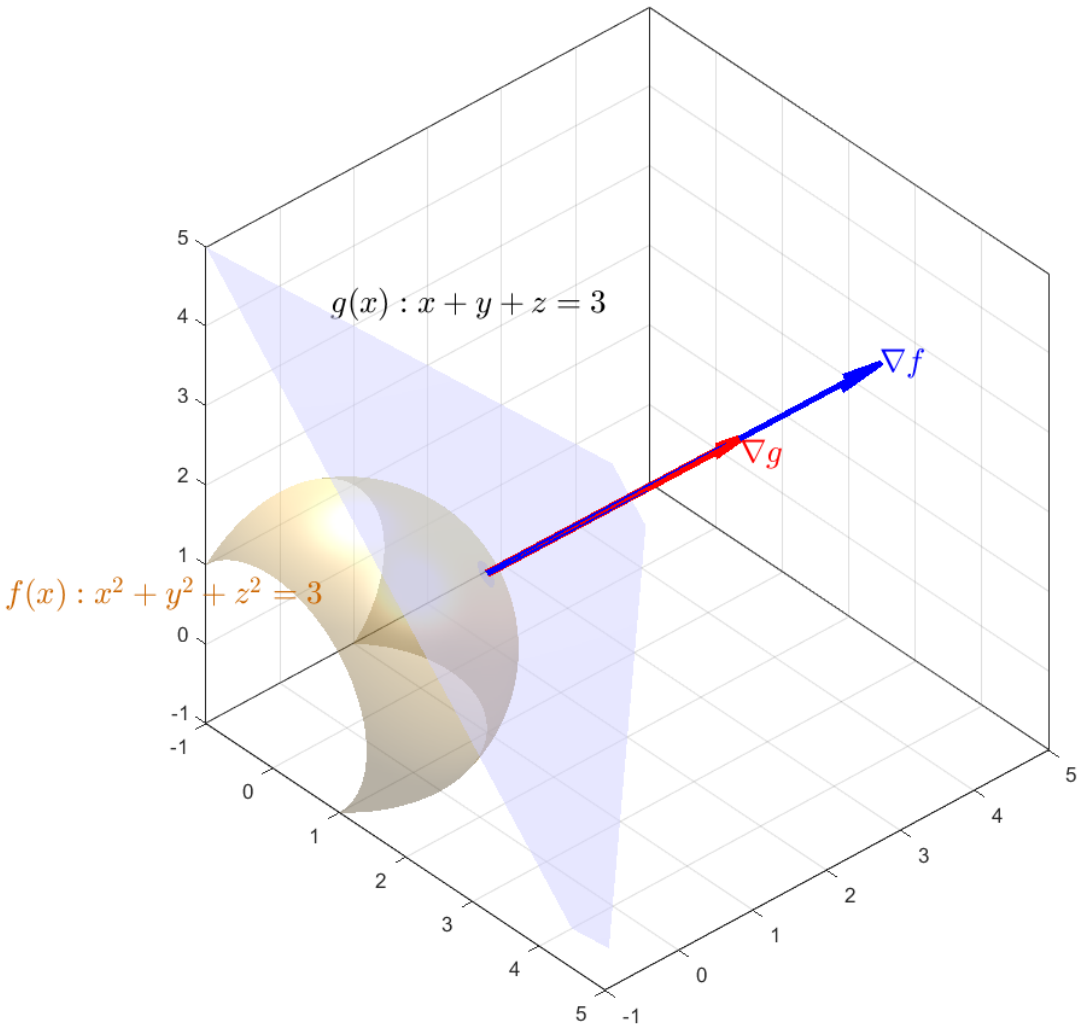
Điều kiện khả thi:

$$\lambda \geqslant 0, \qquad x + y + z \geqslant 3$$

Nếu $\lambda = 0 \Rightarrow x = y = z = 0$ (Loại vì $0 \ngeqslant 3$).
Nếu $\lambda > 0 \Rightarrow x + y + z = 3$. Thay $x = y = z = \frac{\lambda}{2}$ vào:

$$\frac{3\lambda}{2} = 3 \Rightarrow \lambda = 2 \Rightarrow x = y = z = 1$$

Kết luận: Nghiệm tối ưu $(1, 1, 1)$, giá trị nhỏ nhất $f_{\min} = 3$.



Hình 3.7. Mặt mục tiêu và ràng buộc của Ví dụ 3.6.

3.4. Đối ngẫu Lagrange

Xét một bài toán tối ưu hóa lồi $(\mathbb{P})$ với các biến số $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^n$, trong đó tập ràng buộc $\mathcal{X}$ được định nghĩa bởi các giới hạn biên $x_j^{\min} \leqslant x_j \leqslant x_j^{\max}$:

$$(\mathbb{P}) \quad \begin{cases} \min\limits_{\mathbf{x}} \quad g_0(\mathbf{x}) \\ \text{s.t.} \quad g_i(\mathbf{x}) \leqslant 0, \quad i = 1, \ldots, l \\ \qquad \mathbf{x} \in \mathcal{X} = \{\mathbf{x} \in \mathbb{R}^n : x_j^{\min} \leqslant x_j \leqslant x_j^{\max}, \quad j = 1, \ldots, n\}, \end{cases} \tag{3.15}$$

trong đó các hàm g_i $(i = 0, \ldots, l)$ là các hàm khả vi liên tục và lồi, g_0 lồi chặt. Xét trường hợp các hàm mục tiêu và hàm ràng buộc là tách được (separable), nghĩa là có thể được biểu diễn dưới dạng tổng của các hàm một biến thành phần:

$$g_i(\mathbf{x}) = \sum_{j=1}^{n} g_{ij}(x_j), \quad i = 0, \ldots, l. \tag{3.16}$$

Hàm Lagrange của $(\mathbb{P})$ trở thành

$$\begin{aligned}\mathcal{L}(\mathbf{x},\lambda) &= g_0(\mathbf{x}) + \sum_{i=1}^{l} \lambda_i g_i(\mathbf{x}) \\ &= \sum_{j=1}^{n} g_{0j}(x_j) + \sum_{i=1}^{l} \lambda_i \left(\sum_{j=1}^{n} g_{ij}(x_j) \right) \\ &= \sum_{j=1}^{n} \underbrace{\left(g_{0j}(x_j) + \sum_{i=1}^{l} \lambda_i g_{ij}(x_j) \right)}_{\mathcal{L}_j(x_j,\lambda)},\end{aligned} \tag{3.17}$$

với $\lambda_i \geqslant 0,\ i = 1, ..., l$.

Chính cấu trúc này làm cho phương pháp đối ngẫu Lagrange trở nên đặc biệt hấp dẫn, cho phép phân rã một bài toán lớn phức tạp thành chuỗi các bài toán con đơn giản hơn. Theo lý thuyết đối ngẫu, bài toán gốc ($\mathbb{P}$) có thể được giải thông qua bài toán đối ngẫu ($\mathbb{D}$):

$$(\mathbb{D}) \quad \begin{cases} \max\limits_{\lambda} & \varphi(\lambda) \\ \text{s.t.} & \lambda \geqslant \mathbf{0}, \end{cases} \tag{3.18}$$

trong đó hàm mục tiêu đối ngẫu $\varphi(\lambda)$ được định nghĩa là giá trị cực tiểu của hàm Lagrange theo $\mathbf{x}$:

$$\varphi(\lambda) = \min_{\mathbf{x}\in\mathcal{X}} \mathcal{L}(\mathbf{x},\lambda) \tag{3.19}$$

Việc cực tiểu hóa hàm Lagrange $\mathcal{L}$ trên không gian n chiều $\mathcal{X}$ có thể được quy về việc thực hiện n phép cực tiểu hóa độc lập trên không gian một chiều:

$$\varphi(\lambda) = \min_{\mathbf{x}\in\mathcal{X}} \sum_{j=1}^{n} \mathcal{L}_j(x_j,\lambda) = \sum_{j=1}^{n} \min_{x_j^{\min} \leqslant x_j \leqslant x_j^{\max}} \mathcal{L}_j(x_j,\lambda) \tag{3.20}$$

Với một λ cố định, nghiệm tối ưu x_j^* của từng bài toán con được xác định bằng cách xét đạo hàm riêng của $\mathcal{L}_j$ tại các biên:

$$\begin{aligned} &\text{Nếu } \frac{\partial \mathcal{L}_j(x_j^{\min},\lambda)}{\partial x_j} \geqslant 0 \text{ , thì } x_j^* = x_j^{\min} \\ &\text{Nếu } \frac{\partial \mathcal{L}_j(x_j^{\max},\lambda)}{\partial x_j} \leqslant 0 \text{ , thì } x_j^* = x_j^{\max} \\ &\text{Các trường hợp còn lại, } x_j^* \text{ là nghiệm của } \frac{\partial \mathcal{L}_j(x_j,\lambda)}{\partial x_j} = 0 \end{aligned} \tag{3.21}$$

Đạo hàm của hàm mục tiêu đối ngẫu $\varphi(\lambda)$ được tính toán trực tiếp từ các ràng buộc ban đầu:

$$\frac{\partial \varphi(\lambda)}{\partial \lambda_i} = g_i(\mathbf{x}^*(\lambda)), \quad i = 1, \ldots, l \tag{3.22}$$

Ví dụ 3.7: Giải lại Ví dụ 3.4 dùng phương pháp đối ngẫu Lagrange

$$\begin{aligned} \min_{\mathbf{x}\in\mathbb{R}^2} \quad & g_0(\mathbf{x}) = x_1^2 + x_2^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = 1 - x_1 - x_2 \leqslant 0 \end{aligned}$$

1. Thiết lập hàm Lagrange
Hàm Lagrange $\mathcal{L}(\mathbf{x}, \lambda)$ với nhân tử Lagrange $\lambda \geqslant 0$ là:

$$\mathcal{L}(x_1, x_2, \lambda) = x_1^2 + x_2^2 + \lambda(1 - x_1 - x_2)$$

2. Tìm hàm đối ngẫu $g(\lambda)$
Hàm đối ngẫu là giá trị nhỏ nhất của $\mathcal{L}$ theo biến $\mathbf{x}$: $g(\lambda) = \min_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \lambda)$.
Để tìm cực tiểu, ta lấy đạo hàm riêng theo x_1, x_2 và cho bằng 0:

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 2x_1 - \lambda = 0 \Rightarrow x_1 = \frac{\lambda}{2} \\ \frac{\partial \mathcal{L}}{\partial x_2} = 2x_2 - \lambda = 0 \Rightarrow x_2 = \frac{\lambda}{2} \end{cases}$$

Thay x_1, x_2 trở lại vào hàm Lagrange:

$$g(\lambda) = \left(\frac{\lambda}{2}\right)^2 + \left(\frac{\lambda}{2}\right)^2 + \lambda\left(1 - \frac{\lambda}{2} - \frac{\lambda}{2}\right)$$

$$g(\lambda) = \frac{\lambda^2}{2} + \lambda - \lambda^2 = \lambda - \frac{\lambda^2}{2}$$

3. Giải bài toán đối ngẫu
Bài toán đối ngẫu là tìm $\max_{\lambda\geqslant 0} g(\lambda)$:

$$\max_{\lambda\geqslant 0} \left(\lambda - \frac{\lambda^2}{2}\right)$$

Đạo hàm theo λ:

$$g'(\lambda) = 1 - \lambda$$

Cho $g'(\lambda) = 0 \Rightarrow \lambda^* = 1$ (thỏa mãn điều kiện $\lambda \geqslant 0$).
4. Tìm nghiệm tối ưu $\mathbf{x}^*$
Thay $\lambda^* = 1$ vào biểu thức x_1, x_2 đã tìm được ở bước 2:

$$x_1^* = \frac{1}{2}, \quad x_2^* = \frac{1}{2}$$

Kết luận:
Nghiệm tối ưu:

$$\mathbf{x}^* = (0.5, 0.5)$$

Giá trị tối ưu:

$$g_0(\mathbf{x}^*) = 0.5^2 + 0.5^2 = 0.5$$

Ví dụ 3.8: Giải lại Ví dụ 3.5 dùng phương pháp đối ngẫu Lagrange

$$\begin{aligned} \min \quad & f(x_1, x_2) = (x_1 - 5)^2 + (x_2 - 4)^2 \\ \text{s.t.} \quad & g_1(\mathbf{x}) = x_1 + 2x_2 - 10 \leqslant 0 \\ & g_2(\mathbf{x}) = x_1 - 4 \leqslant 0 \end{aligned}$$

Bước 1: Thiết lập hàm Lagrange

$$\mathcal{L}(x_1, x_2, \lambda_1, \lambda_2) = (x_1 - 5)^2 + (x_2 - 4)^2 + \lambda_1(x_1 + 2x_2 - 10) + \lambda_2(x_1 - 4)$$

Bước 2: Tìm biểu thức của $\mathbf{x}$ theo λ

Để tìm hàm đối ngẫu $g(\lambda) = \min_{\mathbf{x}} L$, ta lấy đạo hàm riêng của L theo x_1, x_2 và cho bằng 0:

$$\begin{cases} \frac{\partial L}{\partial x_1} = 2(x_1 - 5) + \lambda_1 + \lambda_2 = 0 \Rightarrow x_1 = 5 - \frac{\lambda_1 + \lambda_2}{2} \\ \frac{\partial L}{\partial x_2} = 2(x_2 - 4) + 2\lambda_1 = 0 \qquad \Rightarrow x_2 = 4 - \lambda_1 \end{cases}$$

Bước 3: Thiết lập hàm đối ngẫu $g(\lambda)$

Thay thế x_1, x_2 vừa tìm được ở trên ngược lại vào hàm L. Sau khi rút gọn đại số, ta được hàm đối ngẫu phụ thuộc hoàn toàn vào λ:

$$g(\lambda_1, \lambda_2) = -1.25\lambda_1^2 - 0.25\lambda_2^2 - 0.5\lambda_1\lambda_2 + 3\lambda_1 + \lambda_2$$

Bước 4: Giải bài toán đối ngẫu

Ta cần tìm cực đại của hàm $g(\lambda_1, \lambda_2)$ với điều kiện $\lambda \geqslant 0$. Lấy đạo hàm riêng theo λ_1, λ_2 và cho bằng 0 để tìm điểm cực trị:

$$\begin{cases} \frac{\partial g}{\partial \lambda_1} = -2.5\lambda_1 - 0.5\lambda_2 + 3 = 0 \\ \frac{\partial g}{\partial \lambda_2} = -0.5\lambda_1 - 0.5\lambda_2 + 1 = 0 \end{cases}$$

Suy ra $\lambda_1 = \lambda_2 = 1$.

Cả hai giá trị đều thỏa mãn điều kiện $\lambda \geqslant 0$. Vậy $\lambda_1^* = 1, \lambda_2^* = 1$.

Bước 5: Tìm nghiệm tối ưu $\mathbf{x}^*$

Thay $\lambda_1^* = 1, \lambda_2^* = 1$ trở lại công thức x ở bước 2:

$$\begin{cases} x_1^* = 5 - \frac{1+1}{2} = 5 - 1 = 4 \\ x_2^* = 4 - 1 = 3 \end{cases}$$

Kết luận:

Nghiệm tối ưu:

$$\mathbf{x}^* = (4, 3)$$

Giá trị tối ưu:

$$f(\mathbf{x}^*) = (4 - 5)^2 + (3 - 4)^2 = 1 + 1 = 2.$$

Ví dụ 3.9: Giải lại Ví dụ 3.6 dùng phương pháp đối ngẫu Lagrange

Bước 1: Xây dựng hàm Lagrangian

Hàm Lagrangian $\mathcal{L}(x, y, z, \lambda)$ với nhân tử Lagrange $\lambda \geqslant 0$:

$$\mathcal{L}(x, y, z, \lambda) = (x^2 + y^2 + z^2) + \lambda(3 - x - y - z)$$

Bước 2: Tìm hàm đối ngẫu

Hàm đối ngẫu $h(\lambda)$ là giá trị nhỏ nhất của $\mathcal{L}$ theo các biến x, y, z (coi λ là hằng số). Ta tìm cực tiểu bằng cách đạo hàm và cho bằng 0:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x - \lambda = 0 \Rightarrow x = \frac{\lambda}{2}$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2y - \lambda = 0 \Rightarrow y = \frac{\lambda}{2}$$

$$\frac{\partial \mathcal{L}}{\partial z} = 2z - \lambda = 0 \Rightarrow z = \frac{\lambda}{2}$$

Thay các giá trị x, y, z này ngược lại vào hàm $\mathcal{L}$ để tìm $h(\lambda)$:

$$h(\lambda) = \left(\frac{\lambda^2}{4} + \frac{\lambda^2}{4} + \frac{\lambda^2}{4}\right) + \lambda\left(3 - \frac{\lambda}{2} - \frac{\lambda}{2} - \frac{\lambda}{2}\right)$$

$$h(\lambda) = \frac{3\lambda^2}{4} + 3\lambda - \frac{3\lambda^2}{2}$$

$$h(\lambda) = -\frac{3\lambda^2}{4} + 3\lambda$$

Bước 3: Giải bài toán đối ngẫu

Bài toán đối ngẫu là tìm $\max h(\lambda)$ với điều kiện $\lambda \geqslant 0$:

$$\max_{\lambda \geqslant 0}\left(-\frac{3\lambda^2}{4} + 3\lambda\right)$$

Đây là hàm bậc 2 có hệ số $a < 0$ (parabol úp xuống), nên cực đại đạt được tại đỉnh đạo hàm bằng 0:

$$h'(\lambda) = -\frac{6\lambda}{4} + 3 = -\frac{3}{2}\lambda + 3 = 0 \Rightarrow \frac{3}{2}\lambda = 3 \Rightarrow \lambda = 2$$

Kiểm tra điều kiện: $\lambda = 2 > 0$ (thỏa mãn).

Vậy nhân tử tối ưu là $\lambda^* = 2$.

Bước 4: Tìm nghiệm bài toán gốc

Thay giá trị $\lambda^* = 2$ trở lại biểu thức của x, y, z

$$x^* = y^* = z^* = 1$$

Kết luận

Nghiệm tối ưu:

$$(x^*,\ y^*,\ z^*) = (1,\ 1,\ 1)$$

Giá trị nhỏ nhất:

$$f_{min} = 1^2 + 1^2 + 1^2 = 3$$

3.5. Bài tập

Bài 3.1: Tìm giá trị cực tiểu của hàm số sau:

$$f(X_1, X_2) = X_1^3 - X_2X_1 + X_2^2 - X_1 + X_2$$

Bài 3.2: Cho hàm số:

$$f = X_1^4 - 2X_2X_1^2 + X_2^2 + X_1^2 - 2X_1 + 5$$

Tính ∇f (vectơ gradient) và H (ma trận Hessian) tại các điểm $\mathbf{X}_1^T = \{0, 0\}$, $\mathbf{X}_2^T = \{2, 2\}$, $\mathbf{X}_3^T = \{1, 1\}$. Kiểm tra xem H có xác định dương tại mỗi điểm này hay không. Các điều kiện cần và đủ cho một điểm cực tiểu địa phương có được thỏa mãn tại bất kỳ điểm nào trong số này không?

Bài 3.3: Cho bài toán tối ưu hóa có ràng buộc:

$$\begin{aligned} \max \;\; & Z = 3X_1 + 2X_2 \\ \text{s.t.} \;\; & 2X_1 + X_2 \leqslant 10 \\ & X_1 + X_2 \leqslant 8 \\ & X_1 \leqslant 4 \\ & X_1, X_2 \geqslant 0 \end{aligned}$$

Kiểm tra điều kiện KKT tại các điểm: $\mathbf{X}_1^T = \{4, 2\}$, $\mathbf{X}_2^T = \{2, 6\}$, và $\mathbf{X}_3^T = \{4, 0\}$. Biểu diễn bằng đồ thị các vectơ gradient của các hàm ràng buộc và của hàm mục tiêu tại ba điểm nêu trên.

Bài 3.4: Cho bài toán:

$$\begin{aligned} (\min) \; & Z = X_1^2 + 2X_2^2 \\ \text{s.t.} \;\; & -X_1 - X_2 + 2 \leqslant 0 \end{aligned}$$

Giải bài toán này bằng phương pháp đối ngẫu.

Bài 3.5: Cho bài toán:

$$\begin{aligned} (\min) \; & Z = \frac{5}{X_1} + \frac{2}{X_2} \\ s.t. \;\; & X_1 + X_2 - 3 \leqslant 0 \\ & 4X_1 + X_2 - 6 \leqslant 0 \\ & X_1 + 3X_2 - 20 \leqslant 0 \\ & X_1, X_2 \geqslant 0 \end{aligned}$$

Giải bài toán này bằng phương pháp đối ngẫu.

Bài 3.6: Tối thiểu hóa trọng lượng của một giàn hai thanh Hình 3.8. Chiều dài của các thanh lần lượt là $5l$ và $3l$. Mô đun đàn hồi là E và khối lượng riêng là ρ cho cả hai thanh, và lực $P > 0$. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1 và A_2. Giàn phải đủ độ cứng hay độ mềm (compliance) phải thấp hơn một giá trị cho trước:

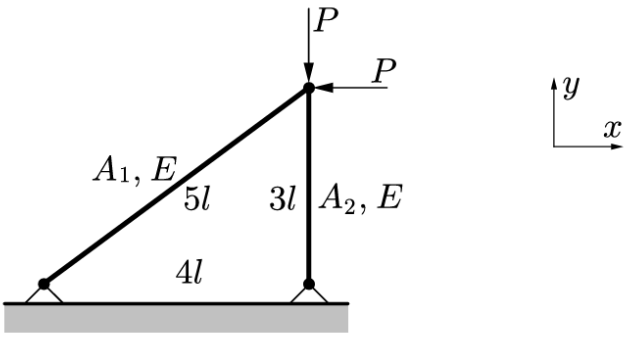
$$-Pu_x - Pu_y \leqslant c_0$$

trong đó (u_x, u_y) là các chuyển vị của nút tự do, và $c_0 > 0$ là một số cho trước.

a) Thiết lập phát biểu toán học bài toán tối ưu

b) Thay đổi biến số sang các biến không thứ nguyên là $x_i = P/(EA_i)$ với $i = 1, 2$, và giải bài toán tối ưu hóa bằng cách sử dụng các điều kiện KKT (Karush-Kuhn-Tucker).

c) Tương tự như câu b), nhưng giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange (Lagrangian duality).



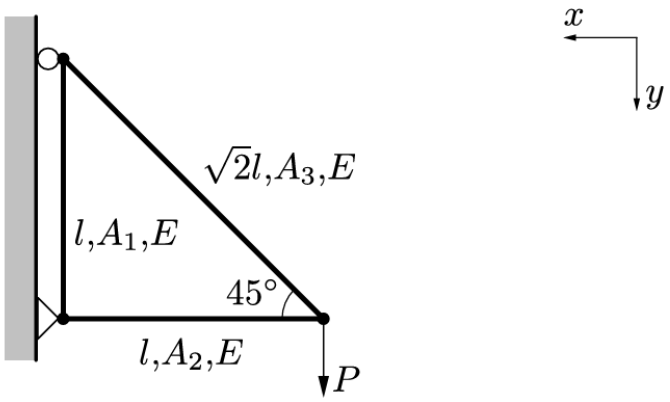
Hình 3.8.

Bài 3.7: Hệ giàn ba thanh Hình 3.9 chịu tác dụng của lực $P > 0$. Người ta muốn tối đa hóa độ cứng của giàn bằng cách tối thiểu hóa độ mềm (compliance) Pu_y của nó, trong đó u_y là chuyển vị theo phương y của nút nơi lực P được tác dụng. Thể tích của giàn không được vượt quá giá trị V_0. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1, A_2 và A_3.

a) Thiết lập phát biểu toán học bài toán tối ưu.

b) Giải bài toán tối ưu hóa bằng cách sử dụng các điều kiện KKT.

c) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.

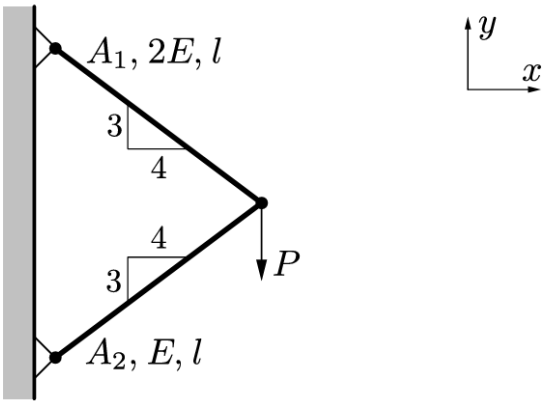


Hình 3.9.

Bài 3.8: Hệ giàn hai thanh Hình 3.10 chịu tác dụng của lực $P > 0$. Độ mềm $-Pu_y$ cần được tối thiểu hóa, trong đó u_y là chuyển vị theo phương y của nút tự do. Thể tích của giàn không được vượt quá giá trị V_0, và độ lớn của ứng suất trong mỗi thanh (cả kéo và nén) không được phép vượt quá giá trị $(5\alpha Pl)/(6V_0)$, với $\alpha > 0$ là một hằng số không thứ nguyên cho trước. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1 và A_2.

a) Thiết lập phát biểu toán học bài toán tối ưu.

b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange cho tất cả các trường hợp $\alpha > 0$.



Hình 3.10.

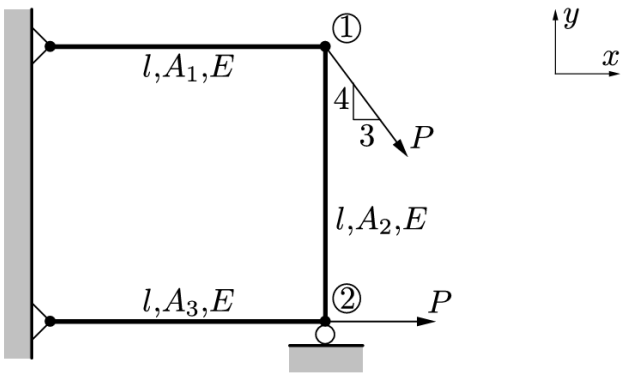
Bài 3.9: Độ cứng của hệ giàn ba thanh Hình 3.11 cần được tối đa hóa. Cụ thể hơn, người ta muốn tối thiểu hóa chuẩn 1 (1-norm) của vector chuyển vị, tức là:

$$|u_{1_x}| + |u_{1_y}| + |u_{2_x}|$$

Giàn chịu tác dụng của hai lực $P > 0$. Thể tích của giàn không được vượt quá giá trị V_0. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1, A_2 và A_3.

a) Thiết lập phát biểu toán học bài toán tối ưu.

b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.



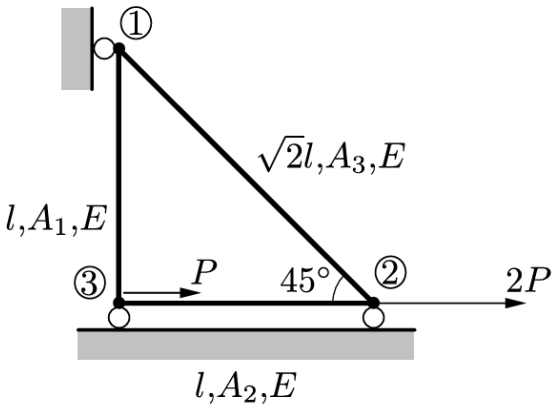
Hình 3.11.

Bài 3.10: Trọng lượng của hệ giàn ba thanh Hình 3.12 cần được tối thiểu hóa với điều kiện giàn phải đủ độ cứng; chuyển vị nút cực đại phải thấp hơn một giá trị định trước:

$$\max(|u_1|, |u_2|, |u_3|) \leqslant u_0$$

trong đó u_i là vector chuyển vị của nút i và $u_0 > 0$ là một đại lượng vô hướng cho trước. Giàn chịu tác dụng của hai lực đặt vào. Giả thiết $P > 0$. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: A_1, A_2 và A_3.

a) Thiết lập phát biểu toán học bài toán tối ưu
b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.
$(u_x^{\mathcal{N}}, u_y^{\mathcal{N}})$

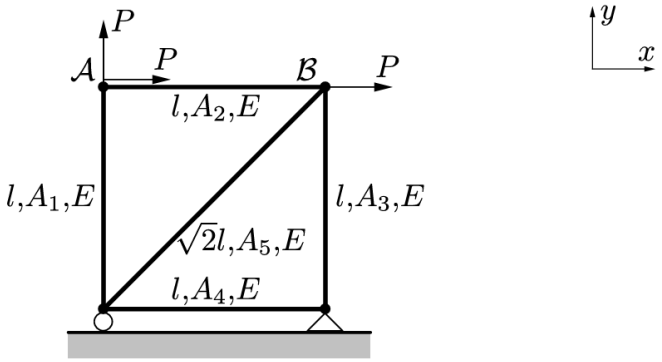


Hình 3.12.

Bài 3.11: Thể tích của hệ giàn năm thanh Hình 3.13 cần được tối thiểu hóa với điều kiện giàn phải đủ độ cứng. Cụ thể, độ mềm phải thấp hơn giá trị c_0. Tất cả các thanh đều có mô đun đàn hồi E. Các biến thiết kế là diện tích mặt cắt ngang của các thanh: $A_1, A_2, \ldots, A_5$. Giàn chịu tác dụng của ba lực $P > 0$, sao cho độ mềm có thể được viết dưới dạng:

$$Pu_x^{\mathcal{A}} + Pu_y^{\mathcal{A}} + Pu_x^{\mathcal{B}}$$

trong đó $u_x^{\mathcal{A}}, u_y^{\mathcal{A}}$ là các chuyển vị của nút $\mathcal{A}$ và $u_x^{\mathcal{B}}$ là chuyển vị theo phương x của nút $\mathcal{B}$.
a) Thiết lập phát biểu toán học bài toán tối ưu
b) Giải bài toán tối ưu hóa bằng cách sử dụng đối ngẫu Lagrange.



Hình 3.13.

CHƯƠNG 4

CÁC THUẬT TOÁN XẤP XỈ LỒI TƯỜNG MINH TUẦN TỰ

Đối với các bài toán tối ưu hóa kết cấu quy mô lớn, việc viết hàm mục tiêu và các ràng buộc dưới dạng hàm tường minh (hay hàm hiện - explicit functions) của các biến thiết kế là gần như không thể. Giải pháp là tạo ra một chuỗi các bài toán con dạng hiện (explicit subproblems) để xấp xỉ bài toán gốc và giải chúng theo trình tự.

4.1. Quy trình giải tổng quát

Xét công thức lồng ghép (nested formulation nf) của bài toán tối ưu hóa kết cấu như sau:

$$(\mathbb{SO})_{\text{nf}} \begin{cases} \min\limits_{x} g_0(\mathbf{x}, \mathbf{u}(\mathbf{x})) \\ \text{s.t. } g_i(\mathbf{x}, \mathbf{u}(\mathbf{x})) \leqslant 0, \quad i = 1, \dots, l \\ \quad \mathbf{x} \in \mathcal{X} \end{cases} \tag{4.1}$$

Bài toán này được giải bằng cách tạo ra và giải một chuỗi các bài toán con xấp xỉ. Quy trình giải lặp được mô tả qua các bước sau:

- Khởi tạo: bắt đầu với thiết kế ban đầu $\mathbf{x}^0$, đặt bộ đếm vòng lặp $k = 0$.
- Phân tích kết cấu: tính vectơ chuyển vị $\mathbf{u}(\mathbf{x}^k)$ cho thiết kế hiện tại bằng cách thực hiện phân tích phần tử hữu hạn (Finite Element Analysis FEA), tức là giải phương trình cân bằng $\mathbf{K}(\mathbf{x}^k)\mathbf{u}(\mathbf{x}^k) = \mathbf{F}(\mathbf{x}^k)$.
- Tính toán đạo hàm: tại thiết kế hiện tại $\mathbf{x}^k$, tính toán giá trị hàm mục tiêu $g_0(\mathbf{x}^k)$, các hàm ràng buộc $g_i(\mathbf{x}^k)$, và các gradient (đạo hàm bậc nhất) của chúng $\nabla g_i(\mathbf{x}^k)$ với $i = 0, \dots, l$.
- Xấp xỉ hóa: thiết lập một bài toán xấp xỉ dạng lồi, hiện $(\mathbb{SO})^k_{\text{nf}}$ tại điểm $\mathbf{x}^k$ để thay thế cho bài toán gốc $(\mathbb{SO})_{\text{nf}}$.
- Giải bài toán con: giải bài toán tối ưu hóa phi tuyến $(\mathbb{SO})^k_{\text{nf}}$ vừa thiết lập để tìm ra thiết kế mới $\mathbf{x}^{k+1}$.
- Lặp lại: tăng bộ đếm $k = k + 1$ và quay lại bước phân tích kết cấu, trừ khi tiêu chuẩn dừng (hội tụ) đã được thỏa mãn.

4.2. Quy hoạch tuyến tính tuần tự SLP

Trong phép xấp xỉ quy hoạch tuyến tính tuần tự (Sequential Linear Programming SLP) của bài toán $(\mathbb{SO})_{\text{nf}}$ tại $\mathbf{x}^k$, hàm mục tiêu và tất cả các phương trình ràng buộc đều được tuyến tính hóa tại điểm thiết kế $\mathbf{x}^k$; điều này dẫn đến bài toán con sau đây tại bước lặp k:

$$(\text{SLP}) \begin{cases} \min_{\mathbf{x}} \; g_0(\mathbf{x}^k) + \nabla g_0(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) \\ \text{s.t.} \quad g_i(\mathbf{x}^k) + \nabla g_i(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) \leqslant 0, \quad i = 1, \ldots, l \\ \qquad \mathbf{x} \in \mathcal{X} \\ \qquad -l_j^k \leqslant x_j - x_j^k \leqslant u_j^k, \quad j = 1, \ldots, n. \end{cases} \tag{4.2}$$

Ở đây, l_j^k và u_j^k, với $j = 1, \ldots, n$, được gọi là các giới hạn dịch chuyển (move limits); chúng được sử dụng vì phép tuyến tính hóa chỉ chính xác khi ở gần thiết kế hiện tại. Các giới hạn dịch chuyển được cập nhật theo một quy tắc do người dùng định nghĩa trong quá trình lặp. Thực tế cho thấy việc lựa chọn các giới hạn dịch chuyển ảnh hưởng đáng kể đến hiệu quả của thuật toán SLP.

Vì hàm mục tiêu và tất cả các ràng buộc trong SLP đều là các hàm affine của $\mathbf{x}$ (tức là chúng có thể được viết dưới dạng $\mathbf{a}^T\mathbf{x} + b$, trong đó $\mathbf{a}$ và b là các hằng số), nên chúng là các hàm lồi. Do đó SLP là một bài toán lồi.

Lời giải Ví dụ 3.5 bằng phương pháp SLP được minh họa qua chương trình ở Code 4.1.

4.3. Quy hoạch toàn phương tuần tự SQP

Bằng cách thêm một số hạng bậc hai vào khai triển Taylor của hàm mục tiêu trong SLP, ta thu được phép xấp xỉ quy hoạch toàn phương tuần tự (Sequential Quadratic Programming SQP):

$$(\text{SQP}) \begin{cases} \min_{\mathbf{x}} \; g_0(\mathbf{x}^k) + \nabla g_0(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^k)^T\mathbf{H}(\mathbf{x}^k)(\mathbf{x} - \mathbf{x}^k) \\ \text{s.t.} \quad g_i(\mathbf{x}^k) + \nabla g_i(\mathbf{x}^k)^T(\mathbf{x} - \mathbf{x}^k) \leqslant 0, \quad i = 1, \ldots, l \\ \qquad \mathbf{x} \in \mathcal{X}, \end{cases} \tag{4.3}$$

trong đó $\mathbf{H}(\mathbf{x}^k)$ biểu thị một xấp xỉ bậc nhất, xác định dương của ma trận Hessian của g_0 tại $\mathbf{x}^k$. Điều này sẽ dẫn đến một hàm mục tiêu lồi, do đó SQP là bài toán lồi.

Lời giải Ví dụ 3.5 bằng phương pháp SQP được minh họa qua chương trình ở Code 4.2.

4.4. Tuyến tính hóa lồi CONLIN

SLP và SQP là các phương pháp tổng quát nhưng không tận dụng được đặc điểm của bài toán kết cấu (ứng suất và chuyển vị thường tỉ lệ nghịch còn trọng lượng tỉ lệ thuận với diện tích). Phương pháp tuyến tính hóa lồi (Convex Linearization CONLIN) giải quyết vấn đề này bằng cách tuyến tính hóa hỗn hợp: sử dụng biến trực tiếp x_j hoặc biến nghịch đảo $1/x_j$ tùy thuộc vào đặc tính của hàm, trong đó giả thiết các biến thiết kế luôn dương ($x_j > 0$).

Xấp xỉ được xây dựng dựa trên biến trung gian y_j, nhờ đó mỗi thành phần có thể được tuyến tính hóa theo dạng phù hợp với dấu đạo hàm. Đạo hàm riêng theo y_j được tính bằng quy tắc chuỗi:

$$\frac{\partial g_i(\mathbf{x}^k)}{\partial y_j} = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \frac{1}{\left(\frac{dy_j}{dx_j}\right)} \tag{4.4}$$

Từ đó, ta xác định phần đóng góp của từng biến vào tổng xấp xỉ cho hai trường hợp:

Trường hợp $y_j = x_j$ (biến trực tiếp):

$$g_{ij}^{L,k}(\mathbf{x}) = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}(x_j - x_j^k) \tag{4.5}$$

Trường hợp $y_j = 1/x_j$ (biến nghịch đảo):

$$g_{ij}^{R,k}(\mathbf{x}) = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}\frac{1}{\left(-\frac{1}{(x_j^k)^2}\right)}\left(\frac{1}{x_j} - \frac{1}{x_j^k}\right) = \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}\frac{x_j^k(x_j - x_j^k)}{x_j} \tag{4.6}$$

Quy tắc chọn biến dựa trên dấu của đạo hàm riêng:
Nếu $\partial g_i/\partial x_j > 0$: dùng $g_{ij}^{L,k}$ (nhóm Ω_+).
Nếu $\partial g_i/\partial x_j \leqslant 0$: dùng $g_{ij}^{R,k}$ (nhóm Ω_-).
Công thức tổng quát tại bước lặp k:

$$g_i^{C,k}(\mathbf{x}) = g_i(\mathbf{x}^k) + \sum_{j\in\Omega_+}\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}(x_j - x_j^k) + \sum_{j\in\Omega_-}\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}\frac{x_j^k}{x_j}(x_j - x_j^k) \tag{4.7}$$

CONLIN luôn tạo ra xấp xỉ thận trọng (conservative) hơn SLP: $g_i^{C,k}(\mathbf{x}) \geqslant g_i^{L,k}(\mathbf{x})$. Tính chất này giúp thu hẹp tập khả thi quanh điểm lặp hiện tại, từ đó làm cho quá trình tối ưu hóa an toàn hơn khi các ràng buộc phi tuyến nhạy với biến thiết kế.

Do CONLIN là xấp xỉ lồi và tách biến, phương pháp đối ngẫu Lagrange thường được sử dụng. Để đảm bảo tính lồi chặt và ổn định, một số hạng $\alpha\sum(x_j - x_j^k)^2$ thường được thêm vào hàm mục tiêu.

4.5. Phương pháp tiệm cận động MMA

Phương pháp SLP và CONLIN đều có những hạn chế nhất định. Đó là SLP thường thiếu ổn định và có thể không hội tụ, còn CONLIN đôi khi lại quá thận trọng, làm chậm tốc độ hội tụ hoặc trong một số trường hợp vẫn chưa đủ thận trọng.

Để khắc phục, Svanberg (1987) đã đề xuất phương pháp tiệm cận động (Method of Moving Asymptotes MMA), giới thiệu các tham số linh hoạt gọi là tiệm cận (asymptotes), cho phép điều chỉnh độ cong (curvature) của hàm xấp xỉ để phù hợp với đặc điểm cụ thể của bài toán tại từng bước lặp. Thay vì tuyến tính hóa theo biến x_j hoặc $1/x_j$, MMA sử dụng các biến trung gian tổng quát dựa trên hai tiệm cận: tiệm cận dưới L_j và tiệm cận trên U_j.

Tại bước lặp k, hàm xấp xỉ $g_i^{M,k}(\mathbf{x})$ của hàm gốc $g_i(\mathbf{x})$ được định nghĩa như sau:

$$g_i^{M,k}(\mathbf{x}) = r_i^k + \sum_{j=1}^{n}\left(\frac{p_{ij}^k}{U_j^k - x_j} + \frac{q_{ij}^k}{x_j - L_j^k}\right) \tag{4.8}$$

trong đó, các hệ số p_{ij}^k và q_{ij}^k được xác định dựa trên dấu của đạo hàm riêng bậc nhất để đảm bảo tính lồi và sự phù hợp về độ dốc:

$$p_{ij}^k = (U_j^k - x_j^k)^2\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j}, \quad q_{ij}^k = 0 \text{ nếu } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} > 0 \tag{4.9}$$

$$p_{ij}^k = 0, \quad q_{ij}^k = -(x_j^k - L_j^k)^2\frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \text{ nếu } \frac{\partial g_i(\mathbf{x}^k)}{\partial x_j} \leqslant 0 r_i^k \;\; L_j^k = 0 \tag{4.10}$$

Thành phần tự do được xác định để đảm bảo giá trị hàm xấp xỉ bằng giá trị hàm gốc tại điểm thiết kế hiện tại $\mathbf{x}^k$:

$$r_i^k = g_i(\mathbf{x}^k) - \sum_{j=1}^{n} \left(\frac{p_{ij}^k}{U_j^k - x_j^k} + \frac{q_{ij}^k}{x_j^k - L_j^k} \right) \tag{4.11}$$

Tại mỗi bước lặp k, ta giải bài toán con tối ưu hóa sau:

$$\begin{cases} \min_{\mathbf{x}} \; g_0^{M,k}(\mathbf{x}) \\ \text{s.t.} \quad g_i^{M,k}(\mathbf{x}) \leqslant 0, \quad i = 1, \ldots, l \\ \qquad \alpha_j^k \leqslant x_j \leqslant \beta_j^k, \quad j = 1, \ldots, n \end{cases} \tag{4.12}$$

trong đó, α_j^k và β_j^k là các giới hạn di chuyển được thiết lập để đảm bảo biến thiết kế không bao giờ chạm vào các tiệm cận (tránh lỗi chia cho 0). Chúng được chọn như sau:

$$\begin{aligned} \alpha_j^k &= \max(x_j^{\min}, L_j^k + \mu(x_j^k - L_j^k)) \\ \beta_j^k &= \min(x_j^{\max}, U_j^k - \mu(U_j^k - x_j^k)) \end{aligned} \tag{4.13}$$

trong đó $0 < \mu < 1$.

Lưu ý để đảm bảo tính lồi chặt cho hàm mục tiêu, số hạng ổn định dạng $\alpha \sum_{j=1}^{n} \left(x_j - x_j^k\right)^2$ với α là một số dương nhỏ thường được thêm vào hàm $g_0^{M,k}$.

Quy tắc cập nhật tiệm cận: MMA cập nhật vị trí các tiệm cận L_j^k, U_j^k qua từng bước lặp để điều chỉnh độ thận trọng:

Bước khởi tạo ($k = 0, 1$):

$$\begin{aligned} L_j^k &= x_j^k - s_{\text{init}}(x_j^{\max} - x_j^{\min}) \\ U_j^k &= x_j^k + s_{\text{init}}(x_j^{\max} - x_j^{\min}) \end{aligned} \tag{4.14}$$

Bước lặp $k \geqslant 2$: Dựa trên xu hướng thay đổi của biến thiết kế $(x_j^k - x_j^{k-1})$ và $(x_j^{k-1} - x_j^{k-2})$.

Trường hợp dao động (oscillation): Nếu tích hai bước nhảy trái dấu (biến tăng rồi giảm hoặc ngược lại), ta cần ổn định quá trình bằng cách thu hẹp khoảng cách tiệm cận (tăng độ thận trọng):

$$\begin{aligned} L_j^k &= x_j^k - s_{\text{slower}}(x_j^{k-1} - L_j^{k-1}) \\ U_j^k &= x_j^k + s_{\text{slower}}(U_j^{k-1} - x_j^{k-1}) \end{aligned} \tag{4.15}$$

với $s_{\text{slower}} < 1$.

Trường hợp đơn điệu (monotonic): Nếu tích hai bước nhảy cùng dấu (biến tiếp tục tăng hoặc tiếp tục giảm), ta nới rộng khoảng cách tiệm cận để tăng tốc độ hội tụ:

$$\begin{aligned} L_j^k &= x_j^k - s_{\text{faster}}(x_j^{k-1} - L_j^{k-1}) \\ U_j^k &= x_j^k + s_{\text{faster}}(U_j^{k-1} - x_j^{k-1}) \end{aligned} \tag{4.16}$$

với $s_{\text{faster}} > 1$.

Có thể chứng minh MMA là dạng tổng quát bao hàm cả SLP và CONLIN: nếu $L_j^k \to -\infty$ và $U_j^k \to +\infty$MMA trở thành SLP; nếu $L_j^k = 0$ và $U_j^k \to +\infty$ MMA trở thành CONLIN.

Ví dụ 4.1: Xét hàm số $g(x) = x + x^2 - \frac{1}{40}x^4$. Chúng ta sẽ xác định các xấp xỉ CONLIN của hàm số này tại hai điểm $\bar{x} = 1$ và $\bar{x} = 6$.

Trước hết, ta xác định đạo hàm của hàm g:

$$g_x(x) = \frac{\partial g(x)}{\partial x} = 1 + 2x - \frac{1}{10}x^3$$

Tại điểm $\bar{x} = 1$:

Ta có $g(\bar{x}) = 1,975$ và $g_x(\bar{x}) = 2,9 > 0$. Do đạo hàm mang dấu dương, xấp xỉ CONLIN (g^C) sẽ trùng với xấp xỉ tuyến tính (g^L):

$$g^C(x) = g^L(x) = 1,975 + 2,9(x - 1)$$

Hình 4.1 biểu diễn đồ thị các hàm g, g^C cùng với xấp xỉ nghịch đảo $g^R(x) = 1,975 + \frac{2,9}{x}(x - 1)$.

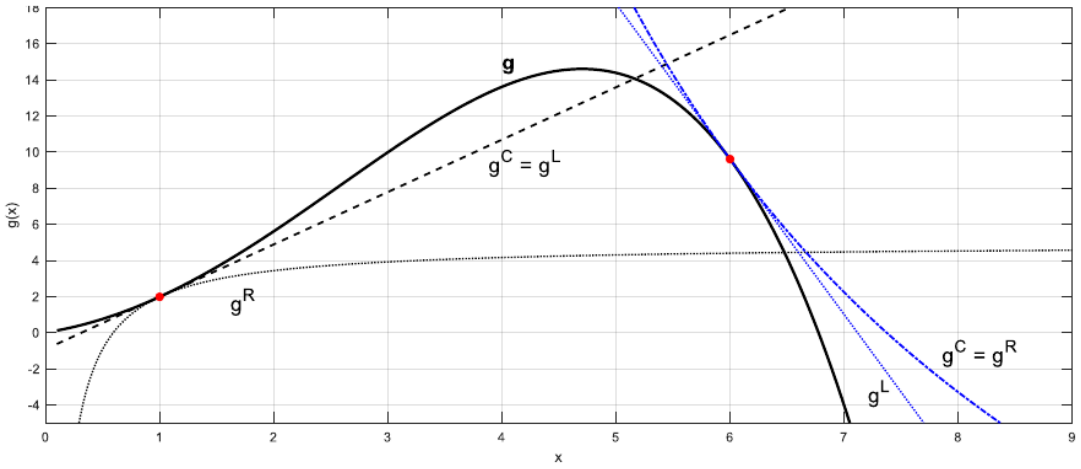
Tại điểm $\bar{x} = 6$:

Ta có $g(\bar{x}) = 9,6$ và $g_x(\bar{x}) = -8,6 < 0$. Do đạo hàm mang dấu âm, xấp xỉ CONLIN lúc này sẽ chuyển sang dạng xấp xỉ nghịch đảo (g^R):

$$g^C(x) = g^R(x) = 9,6 + \frac{51,6}{x}(x - 6)$$

Để tiện so sánh, ta cũng tính toán xấp xỉ tuyến tính tại điểm này:

$$g^L(x) = 9,6 - 8,6(x - 6)$$



Hình 4.1. Các xấp xỉ CONLIN của hàm số g.

Ví dụ 4.2: Làm lại Ví dụ 3.1, xác định xấp xỉ MMA của hàm số này tại hai điểm $\bar{x} = 1$.

Đạo hàm bậc nhất:

$$g'(x) = 1 + 2x - \frac{1}{10}x^3$$

Tại điểm khảo sát $x^0 = 1$, $g(1) = 1,975$ và $g'(1) = 1 + 2 \times 1 - 0,1 \times 1^3 = 2,9$

Theo nguyên lý của MMA, dạng của hàm xấp xỉ phụ thuộc vào dấu của đạo hàm bậc nhất tại điểm hiện tại. Vì $g'(1) = 2,9 > 0$ (đạo hàm dương), ta áp dụng trường

hợp đạo hàm dương trong công thức (4.9). Nghĩa là xấp xỉ MMA sẽ chỉ phụ thuộc vào tiệm cận trên U^0.

Do đó, công thức tổng quát của MMA:

$$g^{M,k}(x) = r^k + \frac{p^k}{U^k - x} + \frac{q^k}{x - L^k}$$

sẽ được rút gọn thành:

$$g^{M,0}(x) = r^0 + \frac{p^0}{U^0 - x}$$

Để hàm xấp xỉ này là một xấp xỉ bậc nhất (first order approximation), nó phải khớp với hàm gốc về cả giá trị và độ dốc tại x^0.

Tính hệ số p^0 (Độ cong):

Dựa theo công thức (4.9) cho trường hợp đạo hàm dương:

$$p^0 = (U^0 - x^0)^2 \frac{\partial g(x^0)}{\partial x} = 2,9(U^0 - 1)^2$$

Tính hệ số r^0 (Số hạng tự do):

Dựa theo công thức (4.11), r^0 được chọn sao cho $g^{M,0}(1) = g(1)$:

$$r^0 = g(x^0) - \frac{p^0}{U^0 - x^0} = 1,975 - \frac{p^0}{U^0 - 1}$$

Ví dụ này minh họa cách vị trí của tiệm cận U^0 ảnh hưởng đến hình dạng của hàm xấp xỉ:

Khi U^0 gần x^0 (ví dụ $U^0 = 1,5$): Mẫu số $(U^0 - x)$ biến thiên rất nhanh, tạo ra một hàm lồi có độ cong lớn. Đây là xấp xỉ rất thận trọng, an toàn nhưng bước nhảy hẹp.

Khi U^0 xa x^0 (ví dụ $U^0 = 20$): Độ cong giảm đi, hàm xấp xỉ phẳng hơn.

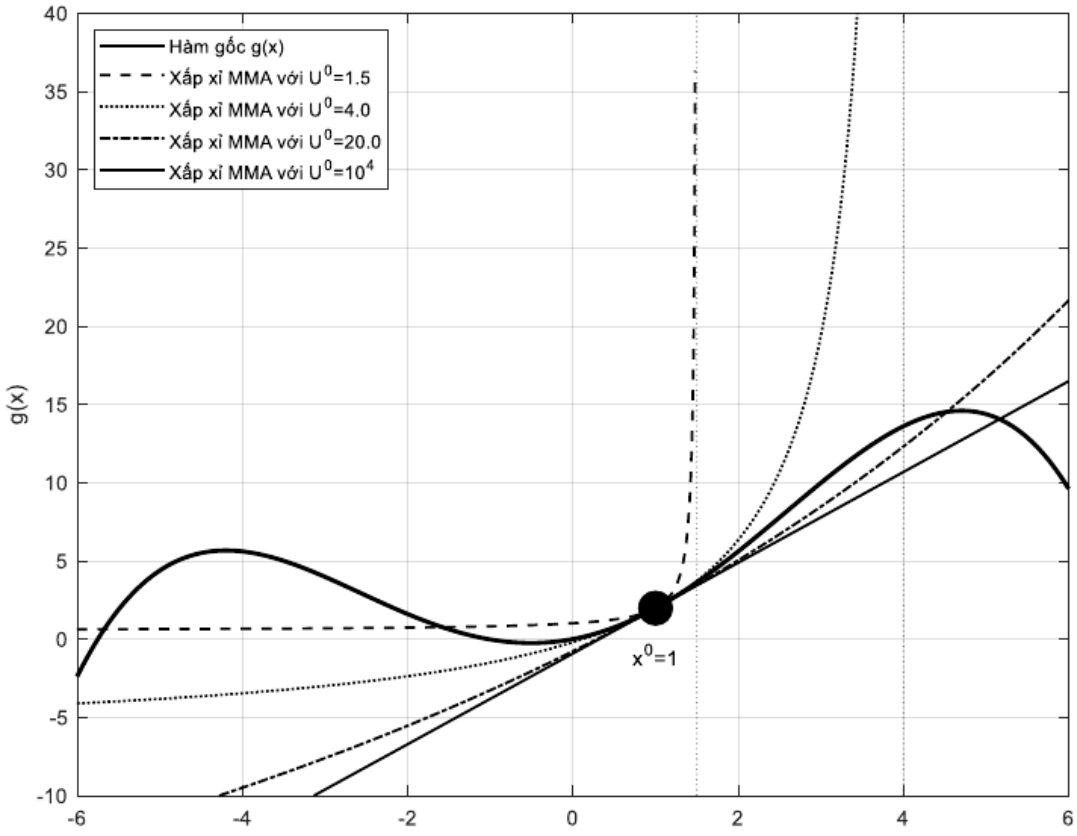
Khi $U^0 \to +\infty$ (ví dụ $U^0 = 10^4$): Số hạng $1/(U - x)$ ứng xử giống hàm tuyến tính. Xấp xỉ MMA lúc này trở về dạng SLP (xấp xỉ tuyến tính), ít thận trọng nhất.

4.6. Bài tập

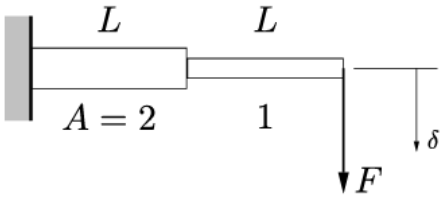
Bài 4.1: Xét bài toán tối thiểu hóa trọng lượng của một dầm gồm hai đoạn Hình 4.3. Bài toán tối ưu có thể được viết dưới dạng như sau:

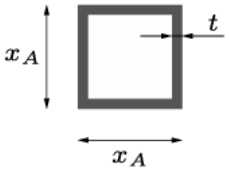
$$(\mathbb{P})_4 \begin{cases} \min\limits_{x_1,x_2} g_0(x_1,x_2) = x_1 + x_2 \\ \text{s.t.} \begin{cases} g_1(x_1,x_2) = \frac{1}{x_1^3} + \frac{7}{x_2^3} - 1 \leqslant 0 \\ 0.1 \leqslant x_1 \leqslant 10, \quad 0.1 \leqslant x_2 \leqslant 10 \end{cases} \end{cases}$$

Giải bài toán này bằng CONLIN và MMA



Hình 4.2. Xấp xỉ MMA.





Cross-section of segment A.

Hình 4.3.

CHƯƠNG 5

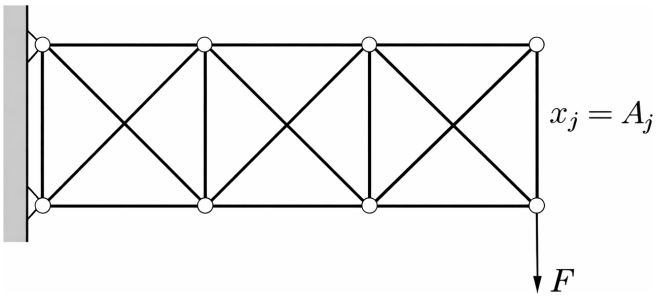
TỐI ƯU ĐỘ CỨNG GIÀN

5.1. Thiết lập bài toán tối ưu độ cứng giàn

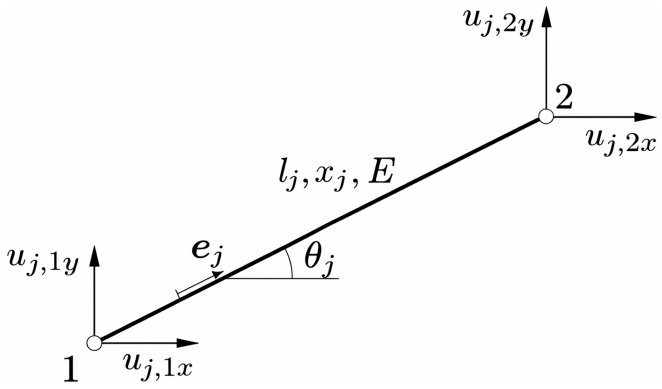
Mục tiêu của bài toán là tìm vector tiết diện các thanh giàn $\mathbf{x}$ sao cho hệ giàn cứng nhất, hay tương đương là có chuyển vị nhỏ nhất dưới tác dụng của ngoại lực, với một lượng vật liệu cho trước. Đây là dạng bài toán tối ưu độ cứng điển hình, trong đó biến thiết kế ảnh hưởng trực tiếp đến ma trận độ cứng và do đó ảnh hưởng đến đáp ứng chuyển vị của kết cấu.

5.1.1. Công thức lồng ghép

Thay vì giải đồng thời cả chuyển vị và thiết kế, ta sử dụng thiết lập lồng ghép (nested formulation). Tại mỗi bước thiết kế, chuyển vị $\mathbf{u}$ được coi là hàm ẩn của biến thiết kế x thông qua phương trình cân bằng $\mathbf{K}(\mathbf{x})\mathbf{u} = \mathbf{F}$.



Hình 5.1. Hệ giàn cần tối ưu.



Hình 5.2. Thanh giàn.

Bài toán tối ưu hóa $(\mathbb{P})_{\text{nf}}$ được viết như sau (người đọc tự chứng minh đây là bài toán lồi):

$$(\mathbb{P})_{\text{nf}} \quad \begin{cases} \min\limits_{x} \;\; C(\mathbf{x}) = \mathbf{F}^T\mathbf{u}(\mathbf{x}) \\ \text{s.t.} \;\; \sum\limits_{j=1}^{n} l_j x_j \leqslant V_{\max} \\ \qquad x_j^{\min} \leqslant x_j \leqslant x_j^{\max}, \quad j = 1, \ldots, n \end{cases} \tag{5.1}$$

Hàm mục tiêu $C(\mathbf{x})$ là độ mềm (compliance), giá trị càng nhỏ thì hệ càng cứng. Biến thiết kế x_j là diện tích mặt cắt ngang của thanh thứ j. Ràng buộc là tổng thể tích vật liệu không vượt quá $V_{\max}$ và giới hạn kích thước thanh trong đoạn $[x_{\min}, x_{\max}]$.

5.1.2. Phân tích độ nhạy

Lấy đạo hàm hàm mục tiêu $C(x)$ ở (5.1) theo diện tích một thanh x_j, kết hợp với phương trình cân bằng $\mathbf{K}(x)\mathbf{u}(x) = \mathbf{F}$, ta có (lưu ý $\mathbf{K}(x)$ đối xứng):

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = \mathbf{F}^T\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = (\mathbf{K}(\mathbf{x})\mathbf{u}(\mathbf{x}))^T\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = \mathbf{u}(\mathbf{x})^T\mathbf{K}(\mathbf{x})\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j}, \tag{5.2}$$

Lấy đạo hàm phương trình cân bằng $\mathbf{K}(x)\mathbf{u}(x) = \mathbf{F}$ theo x_j:

$$\frac{\partial \mathbf{K}(\mathbf{x})}{\partial x_j}\mathbf{u}(\mathbf{x}) + \mathbf{K}(\mathbf{x})\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = 0, \tag{5.3}$$

từ đó suy ra

$$\frac{\partial \mathbf{u}(\mathbf{x})}{\partial x_j} = -\mathbf{K}(\mathbf{x})^{-1}\frac{\partial \mathbf{K}(\mathbf{x})}{\partial x_j}\mathbf{u}(\mathbf{x}) = -\mathbf{K}(\mathbf{x})^{-1}\mathbf{K}_j^0\mathbf{u}(\mathbf{x}), \tag{5.4}$$

với $\mathbf{K}_j^0 = \frac{\partial \mathbf{K}(\mathbf{x})}{\partial x_j}$

Từ (5.2) và (5.4) ta có

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\mathbf{u}(\mathbf{x})^T\mathbf{K}_j^0\mathbf{u}(\mathbf{x}) \tag{5.5}$$

Vector chuyển vị trong hệ tọa độ địa phương $\mathbf{u}_j(\mathbf{x})$ liên hệ với vector chuyển vị tổng thể $u(x)$ như sau

$$\mathbf{u}_j(\mathbf{x}) \;=\; \mathbf{T}_j\mathbf{u}(\mathbf{x}) \tag{5.6}$$

Do đó

$$\mathbf{K}(\mathbf{x}) = \sum_{j=1}^{n}\mathbf{K}_j(\mathbf{x}) = \sum_{j=1}^{n} x_j\mathbf{K}_j^0 = \sum_{j=1}^{n} x_j\mathbf{T}_j^T\mathbf{k}_j^0\mathbf{T}_j \tag{5.7}$$

Biến đổi vế phải của (5.7)

$$\mathbf{u}(\mathbf{x})^T\mathbf{K}_j^0\mathbf{u}(\mathbf{x}) = \mathbf{u}(\mathbf{x})^T\mathbf{T}_j^T\mathbf{k}_j^0\mathbf{T}_j u(\mathbf{x}) = (\mathbf{T}_j\mathbf{u}(\mathbf{x}))^T\mathbf{k}_j^0\mathbf{T}_j\mathbf{u}(\mathbf{x}) = \mathbf{u}_j(\mathbf{x})^T\mathbf{k}_j^0\mathbf{u}_j(\mathbf{x}), \tag{5.8}$$

Từ (5.5) và (5.8) ta có

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\mathbf{u}_j(\mathbf{x})^T \mathbf{k}_j^0 \mathbf{u}_j(\mathbf{x}) \tag{5.9}$$

trong đó $\mathbf{k}_j^0$ là ma trận độ cứng đơn vị của thanh j.

$$\mathbf{k}_j^0 = \frac{E}{l_j} \begin{bmatrix} c^2 & sc & -c^2 & -sc \\ sc & s^2 & -sc & -s^2 \\ -c^2 & -sc & c^2 & sc \\ -sc & -s^2 & sc & s^2 \end{bmatrix} \tag{5.10}$$

trong đó $s = \sin\theta_j$ và $c = \cos\theta_j$ trong Hình 5.2.

5.2. Tối ưu độ cứng giàn sử dụng phương pháp tiêu chuẩn tối ưu

5.2.1. Thiết kế đồng ứng suất

Nguyên lý thiết kế đồng ứng suất (fully stressed design) là cơ sở vật lý quan trọng nhất của phương pháp tiêu chuẩn tối ưu (Optimality Criteria OC) trong bài toán tối ưu hóa độ cứng giàn. Nguyên lý này khẳng định rằng: để sử dụng vật liệu hiệu quả nhất, tất cả các thanh chịu lực đều phải làm việc ở cùng một mức độ ứng suất giới hạn. Cơ sở toán học của nguyên lý này được suy dẫn trực tiếp từ điều kiện tối ưu KKT (Karush-Kuhn-Tucker) như sau:

Do bài toán tối ưu hóa độ cứng dạng lồng ghép $(\mathbb{P})_{\text{nf}}$ là một bài toán lồi, các điều kiện KKT là điều kiện cần và đủ để tìm nghiệm tối ưu. Ta thiết lập hàm Lagrange để kết hợp hàm mục tiêu và ràng buộc thể tích như sau:

$$\mathcal{L}(\mathbf{x}, \lambda) = C(\mathbf{x}) + \lambda \left(\sum_{j=1}^{n} x_j l_j - V_{\max} \right) \tag{5.11}$$

trong đó λ là nhân tử Lagrange.

Viết lại công thức (5.9) đạo hàm riêng của hàm mục tiêu theo biến thiết kế x_j:

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\mathbf{u}_j(\mathbf{x})^T \mathbf{k}_j^0 \mathbf{u}_j(\mathbf{x}) = -\frac{1}{x_j} \mathbf{u}_j(\mathbf{x})^T \mathbf{k}_j \mathbf{u}_j(\mathbf{x}) = -\frac{2}{x_j} U_j \tag{5.12}$$

trong đó U_j là năng lượng biến dạng của thanh j.

Có thể viết lại U_j dưới dạng

$$U_j = \frac{1}{2} \int \sigma_j \varepsilon_j dV_j = \frac{1}{2} E \varepsilon_j^2 x_j l_j = \frac{1}{2} \frac{\sigma_j^2}{E} x_j l_j \tag{5.13}$$

trong đó σ_j là ứng suất trong thanh j, E là mô đun đàn hồi và l_j là chiều dài thanh.

Thế biểu thức (5.13) vào (5.12) ta được

$$\frac{\partial C(\mathbf{x})}{\partial x_j} = -\frac{\sigma_j^2}{E} l_j \tag{5.14}$$

Lấy đạo hàm hàm Lagrange (5.11) theo x_j

$$\frac{\partial \mathcal{L}(\mathbf{x}, \lambda)}{\partial x_j} = \frac{\partial C(\mathbf{x})}{\partial x_j} + \lambda l_j \tag{5.15}$$

Thay (5.14) vào (5.15):

$$\frac{\partial \mathcal{L}(\mathbf{x}, \lambda)}{\partial x_j} = -\frac{\sigma_j^2}{E} l_j + \lambda l_j \tag{5.16}$$

Áp dụng điều kiện KKT (3.12) ta có các điểm KKT (x^*, λ^*)

$$\begin{aligned} \sigma_j^2 &\leqslant \frac{\lambda^*}{E} \text{ nếu } x_j^* = x_j^{\min} \\ \sigma_j^2 &= \frac{\lambda^*}{E} \text{ nếu } x_j^{\min} < x_j^* < x_j^{\max} \\ \sigma_j^2 &\geqslant \frac{\lambda^*}{E} \text{ nếu } x_j^* = x_j^{\max} \end{aligned} \tag{5.17}$$

Ta thấy rằng đối với tất cả các thanh có kích thước nằm trong khoảng không chạm cận trên hoặc cận dưới, bình phương ứng suất σ_j^2 là một hằng số. Điều này có nghĩa là độ lớn của ứng suất trong các thanh này là như nhau:

$$|\sigma_j| = \sqrt{\frac{\lambda^*}{E}} = \text{const} \tag{5.18}$$

Đây chính là ý nghĩa của thiết kế đồng ứng suất. Điều này đảm bảo vật liệu được phân bố sao cho không có phần tử nào "làm việc quá sức"trong khi phần tử khác lại "nhàn rỗi", từ đó tối đa hóa độ cứng của toàn hệ kết cấu.

5.2.2. Phương pháp tiêu chuẩn tối ưu OC

Đối với bài toán tối ưu hóa độ cứng của hệ giàn chỉ chịu một ràng buộc duy nhất về thể tích vật liệu ($V \leqslant V_{max}$), phương pháp tiêu chuẩn tối ưu (Optimality Criteria OC) tỏ ra đặc biệt hiệu quả. Phương pháp này cho phép xây dựng các sơ đồ lặp đơn giản nhằm cập nhật biến thiết kế mà không cần giải các bài toán con phức tạp như trong phương pháp MMA. Đồng thời, OC vẫn đảm bảo tính ổn định và tốc độ hội tụ nhanh về nghiệm tối ưu toàn cục.

Điều kiện dừng của KKT yêu cầu đạo hàm của hàm Lagrange theo biến thiết kế (5.16) phải bằng 0 tại điểm tối ưu:

$$\frac{\partial \mathcal{L}(\mathbf{x}, \lambda)}{\partial x_j} = \frac{\partial C(\mathbf{x})}{\partial x_j} + \lambda l_j = 0 \quad \Rightarrow \quad -\frac{\partial C(\mathbf{x})}{\partial x_j} = \lambda l_j \tag{5.19}$$

Từ điều kiện tối ưu trên, ta định nghĩa một tỷ số dạng kinh nghiệm (heuristic) B_j đại diện cho tỷ lệ giữa lợi ích (giảm độ mềm) và chi phí (tăng thể tích):

$$B_j = \frac{-\frac{\partial C}{\partial x_j}}{\lambda l_j} \tag{5.20}$$

Sơ đồ lặp cập nhật biến thiết kế như sau:

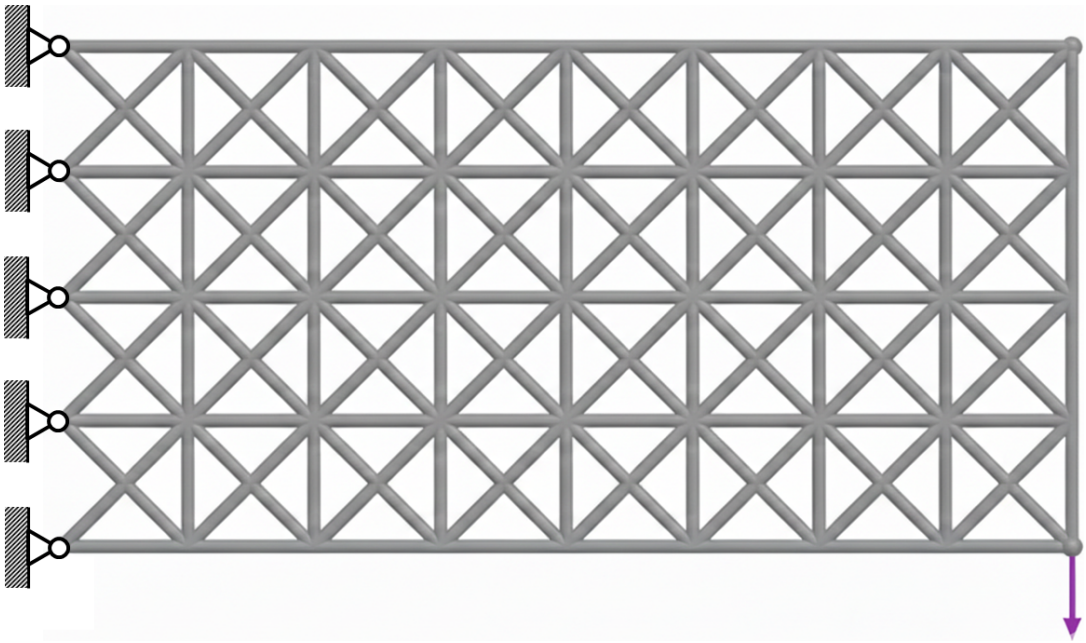
- Nếu $B_j > 1$: thanh chịu lực lớn, cần tăng diện tích.
- Nếu $B_j < 1$: thanh chịu lực nhỏ, cần giảm diện tích.

Công thức cập nhật biến thiết kế có tính đến hệ số cản η để ổn định hội tụ, thường chọn $\eta = 0,5$ như sau:

$$x_j^{new} = x_j^{old} (B_j)^{\eta} = x_j^{old} \sqrt{\frac{-\frac{\partial C}{\partial x_j}}{\lambda l_j}} \tag{5.21}$$

Nhân tử Lagrange λ trong công thức (5.20) được xác định tại mỗi vòng lặp bằng phương pháp chia đôi (bisection method) sao cho tổng thể tích của kết cấu thỏa mãn chính xác $V_{\max}$.

Ví dụ 5.1: Tối ưu hóa hệ giàn bằng phương pháp tiêu chuẩn tối ưu với các thông số: khoảng cách nút, lực tập trung và mô đun đàn hồi đều bằng 1 đơn vị. Ràng buộc tỷ lệ thể tích là 0,1; biến thiết kế (diện tích thanh) trong đoạn $\left[10^{-9},\ 1\right]$.



Hình 5.3. Hệ giàn.

Phát biểu lại bài toán tối ưu (5.1) với các thông số của Ví dụ 5.1:

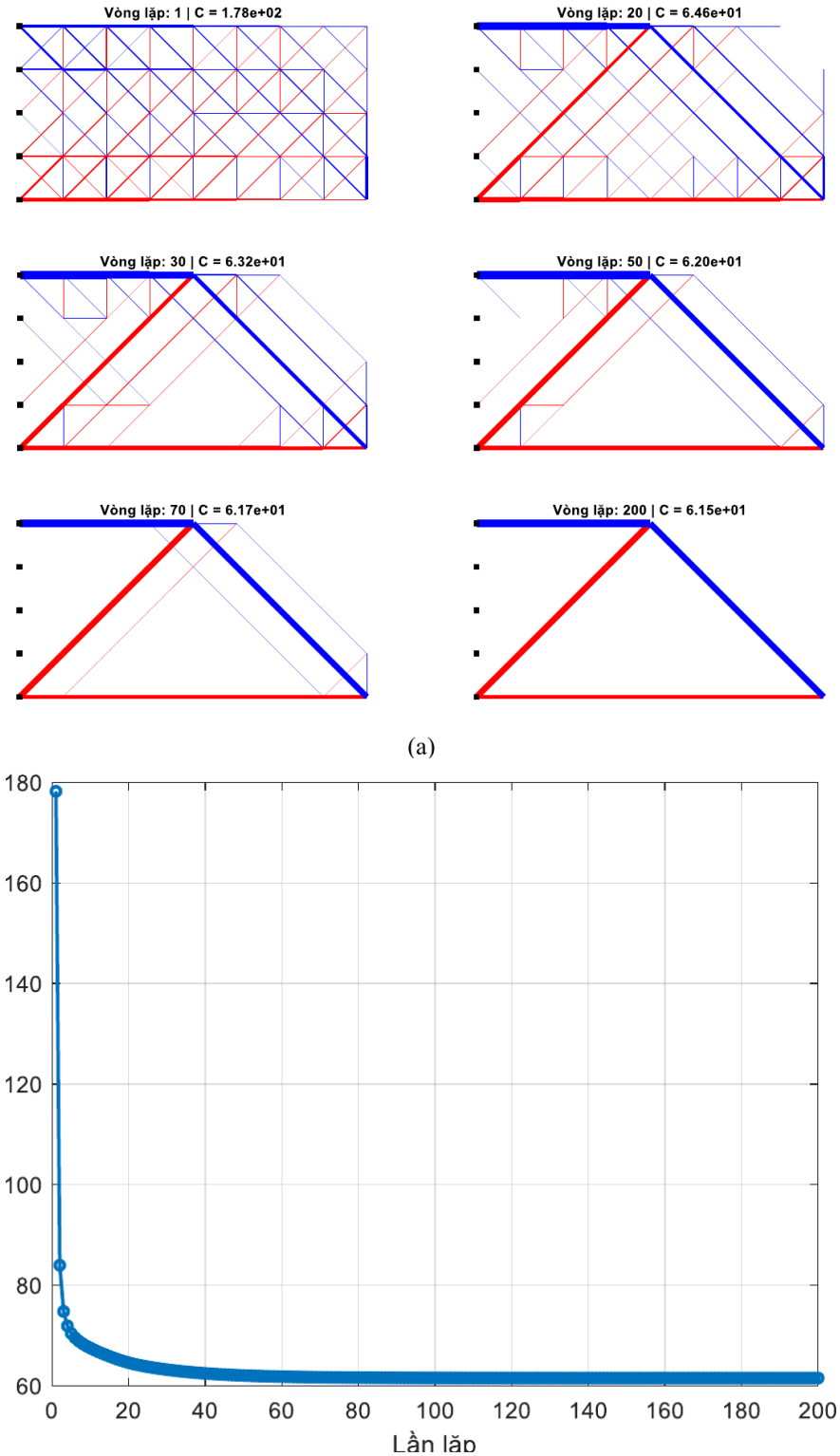
$$(\mathbb{P})\begin{cases}\min\limits_{\mathbf{x}} \ C(\mathbf{x}) = \mathbf{F}^T\mathbf{u}(\mathbf{x}) \\ \text{s.t.} \quad \sum\limits_{j=1}^{n} l_j x_j \leqslant 0,1\left(\sum\limits_{j=1}^{n} l_j\right) \\ \qquad 10^{-9} \leqslant x_j \leqslant 1, \quad j = 1,\ldots,n\end{cases}$$

Một phiên bản ngắn gọn hơn của Chương 5: Tối ưu độ cứng giàn
Code 5.1 được viết để người đọc dễ nắm được thuật toán cốt lõi (Code 5.2).

5.2.3. Tối ưu độ cứng giàn sử dụng phương pháp tiệm cận động

Trong lĩnh vực tối ưu hóa kết cấu, phương pháp tiệm cận động được biết đến là một công cụ giải số mạnh mẽ và tổng quát, đặc biệt hiệu quả đối với các bài toán phi tuyến tính cao hoặc bài toán có đa ràng buộc phức tạp (như ràng buộc ứng suất cục bộ, ràng buộc chuyển vị đa điểm). Tuy nhiên, việc áp dụng MMA đòi hỏi chi phí tính toán lớn để xây dựng và giải các bài toán con tại mỗi bước lặp, đồng thời yêu cầu kỹ thuật cài đặt phức tạp. Quy trình thực hiện như sau:

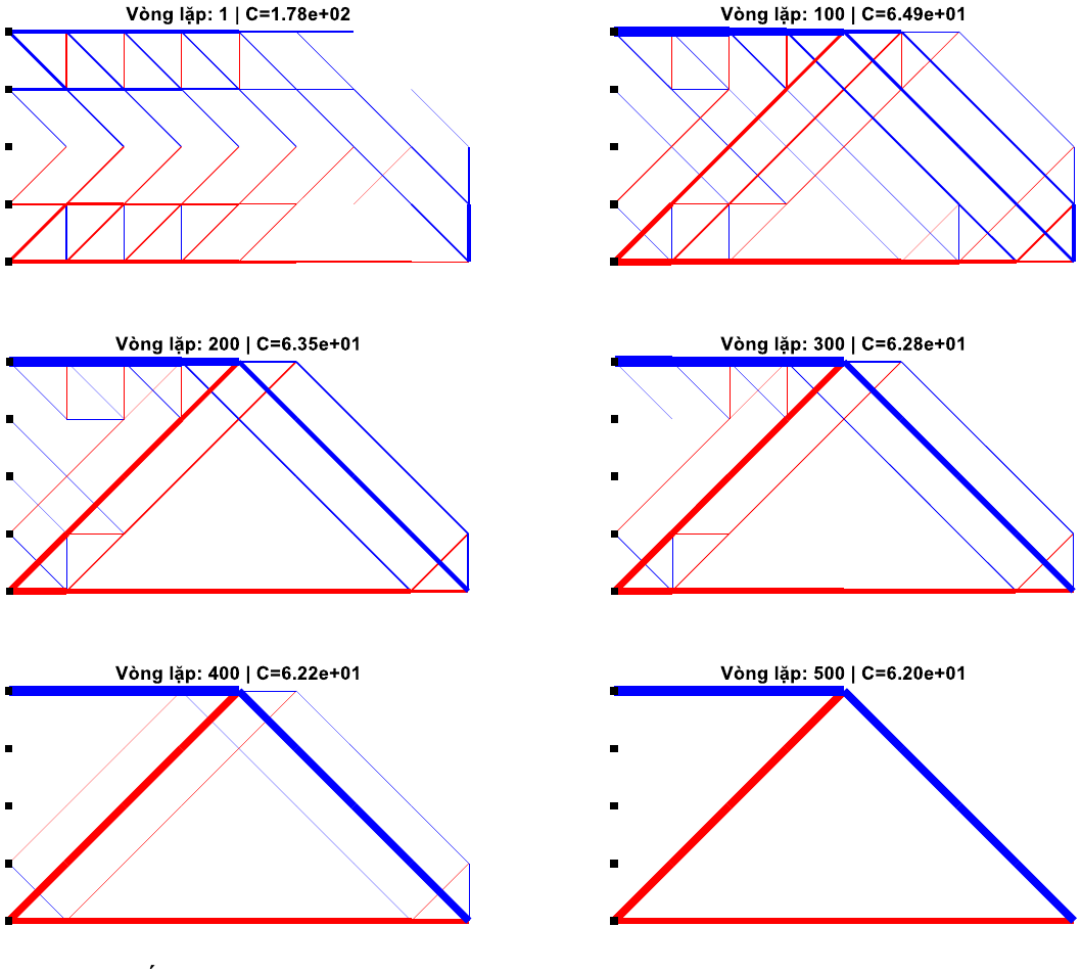
- Bước 1: khởi tạo bằng cách chọn điểm bắt đầu $\mathbf{x}^0$ (thường chọn phân bố đều vật liệu), chọn các tham số tiệm cận ban đầu và đặt chỉ số vòng lặp $k = 0$.
- Bước 2: phân tích phần tử hữu hạn (FE Analysis). Tại bước lặp k, giải phương trình cân bằng $\mathbf{K}(\mathbf{x}^k)\mathbf{u} = \mathbf{F}$ để tìm vector chuyển vị $\mathbf{u}$.
- Bước 3: tính toán độ nhạy bằng cách tính hàm mục tiêu $C(\mathbf{x}^k)$ và đạo hàm bậc



Hình 5.4. Hình thái của hệ giàn sau các bước lặp. Độ dày của các thanh tỷ lệ thuận với diện tích mặt cắt ngang. Màu đỏ biểu thị thanh chịu nén và màu xanh biểu thị thanh chịu kéo, (b) đồ thị độ mềm theo lần lặp.

nhất $\frac{\partial C}{\partial x_j}|_{\mathbf{x}^k}$.

- Bước 4: xây dựng bài toán con MMA bằng cách tạo ra các hàm xấp xỉ lồi cho hàm mục tiêu dựa trên đạo hàm vừa tính, sau đó thiết lập bài toán tối ưu hóa con (sub-problem). Bài toán này đơn giản hơn nhiều so với bài toán gốc và có thể giải nhanh bằng phương pháp đối ngẫu (dual method).
- Bước 5: cập nhật biến thiết kế bằng cách giải bài toán con để tìm ra vector thiết kế mới $\mathbf{x}^{k+1}$, rồi kiểm tra điều kiện hội tụ. Nếu đã hội tụ, thuật toán dừng và kết quả là $\mathbf{x}^{k+1}$; nếu chưa hội tụ, cập nhật $k = k + 1$, điều chỉnh các đường tiệm cận (nếu cần) và quay lại bước phân tích phần tử hữu hạn.

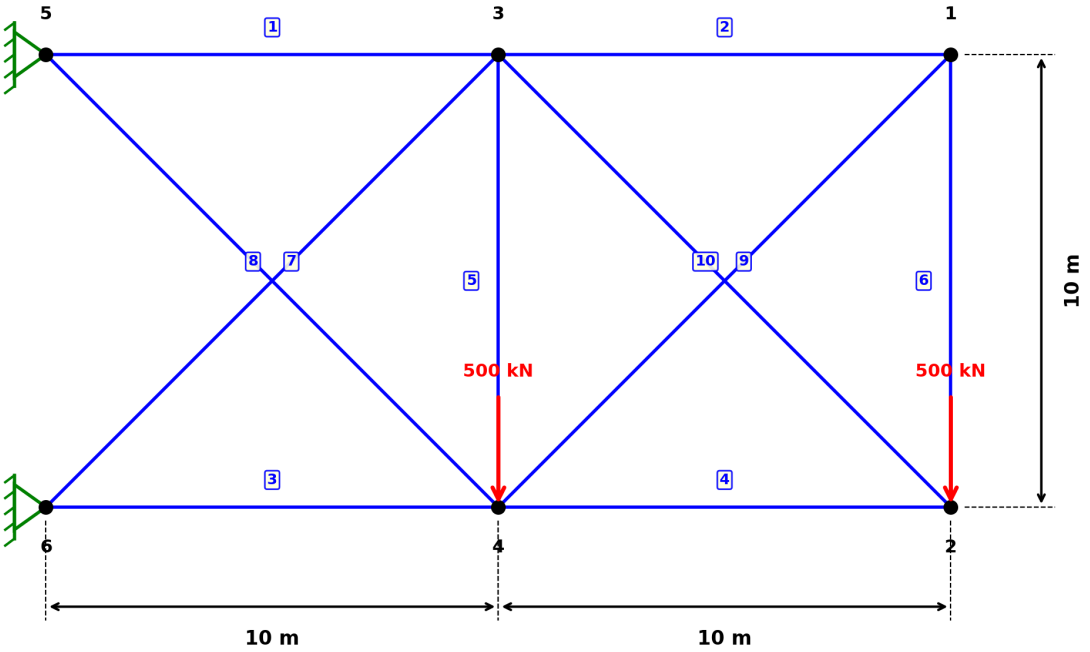


Hình 5.5. Tối ưu hệ giàn Hình 5.3 sau các bước lặp thứ 1, 100, 200, 300, 400 và 500 sử dụng thuật toán MMA.

5.3. Bài tập

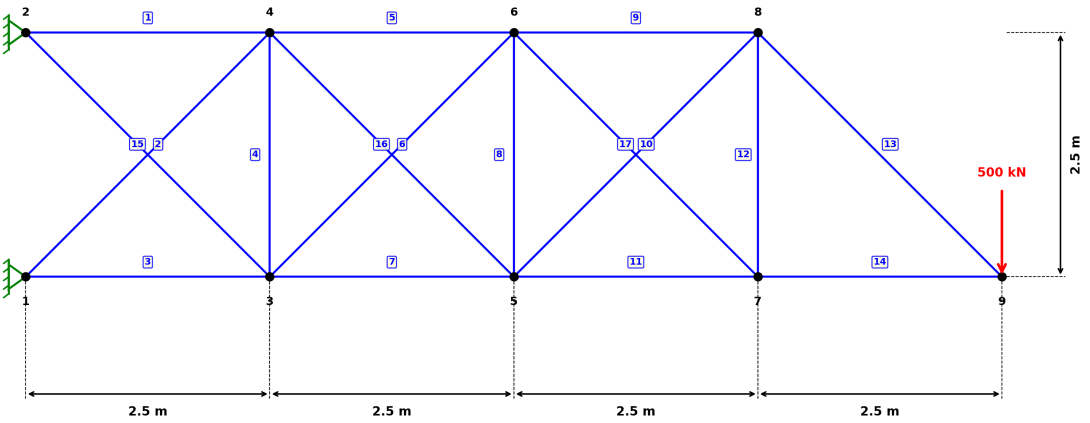
Bài 5.1: Cho kết cấu giàn phẳng 10 thanh như Hình 5.6, với mô-đun đàn hồi $E = 70$ GPa và khối lượng riêng $\rho = 2770$ kg/m^3. Hãy xác định diện tích tiết diện tối ưu của 10 thanh (1 cm$^2 \leq \mathrm{A}_i \leq 50$ cm^2) sao cho khối lượng kết cấu là nhỏ nhất, đồng thời thỏa mãn ràng buộc ứng suất cho phép $[\sigma] = 175$ MPa và chuyển vị cho phép $[\delta] = 50$ mm.

Bài 5.2: Cho kết cấu giàn phẳng 17 thanh như Hình 5.7, với mô-đun đàn hồi $E = 210$ GPa và khối lượng riêng $\rho = 7400$ kg/m^3. Hãy xác định diện tích tiết diện ngang



Hình 5.6.

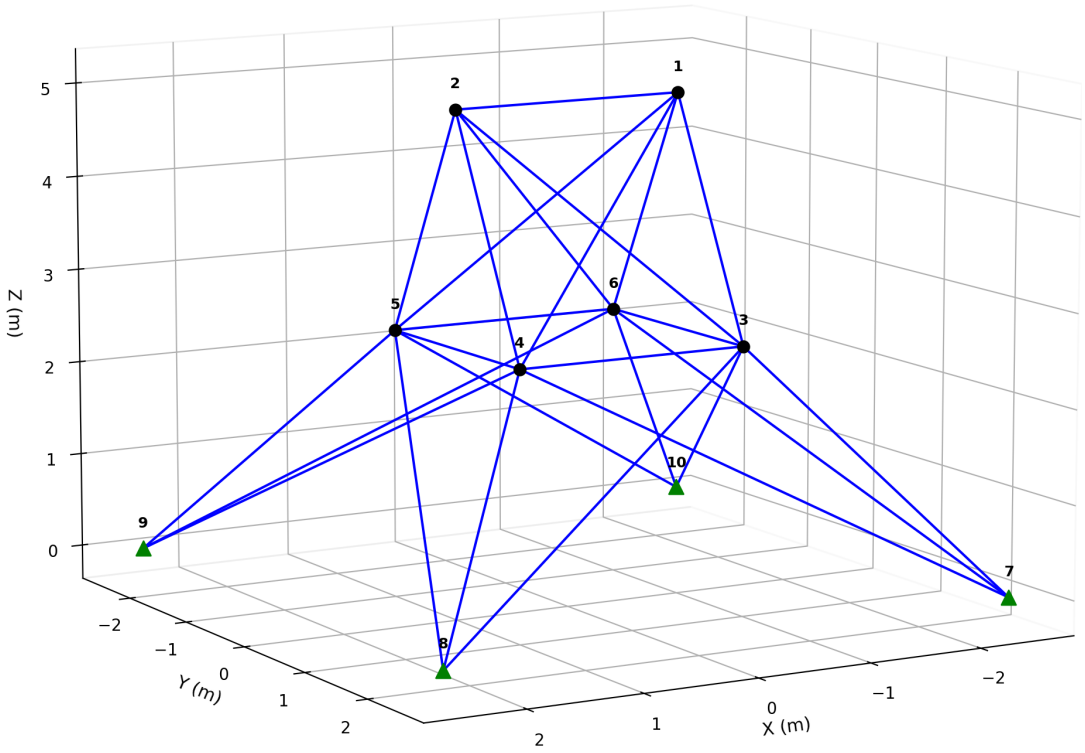
tối ưu của các thanh ($1\ \text{cm}^2 \leq \text{A}_i \leq 300\ \text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất $[\sigma] = 350$ MPa và chuyển vị $[\delta] = 50$ mm tại mọi bậc tự do.



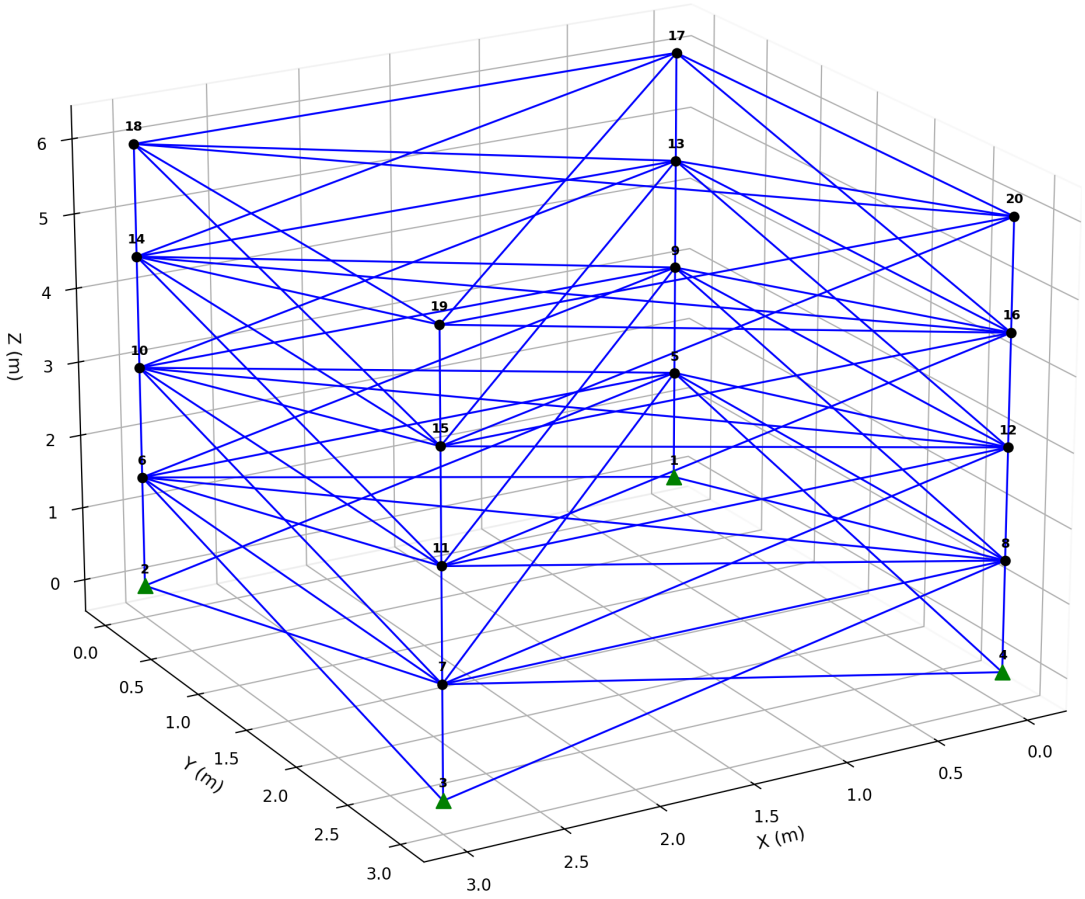
Hình 5.7.

Bài 5.3: Cho kết cấu giàn không gian 25 thanh như Hình 5.8, với mô-đun đàn hồi E = 70 GPa và khối lượng riêng $\rho = 2770\ \text{kg/m}^3$. Hãy xác định diện tích tiết diện tối ưu của 8 nhóm thanh ($0{,}1\ \text{cm}^2 \leq \text{A}_i \leq 22\ \text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn các ràng buộc về ứng suất và chuyển vị $[\delta] = 9$ mm cho cả hai trường hợp tải trọng.

Bài 5.4: Cho kết cấu giàn không gian 72 thanh như Hình 5.9, với mô-đun đàn hồi E = 70 GPa và khối lượng riêng $\rho = 2770\ \text{kg/m}^3$. Hãy xác định diện tích tiết diện tối ưu của 16 nhóm thanh ($1\ \text{cm}^2 \leq \text{A}_i \leq 130\ \text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất $[\sigma] = 175$ MPa và chuyển vị $[\delta] = 6$ mm cho cả hai trường hợp tải trọng.

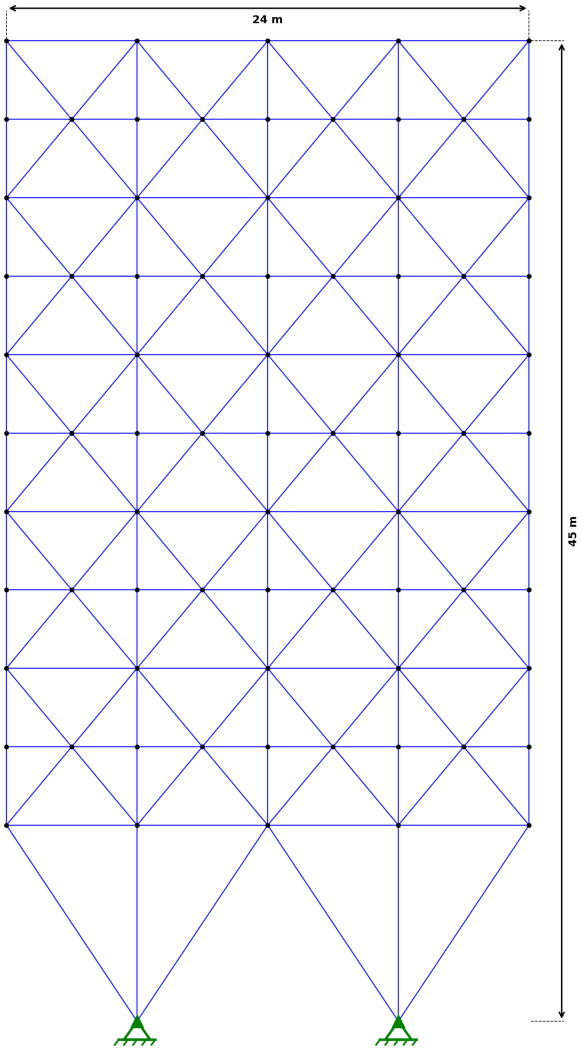


Hình 5.8.



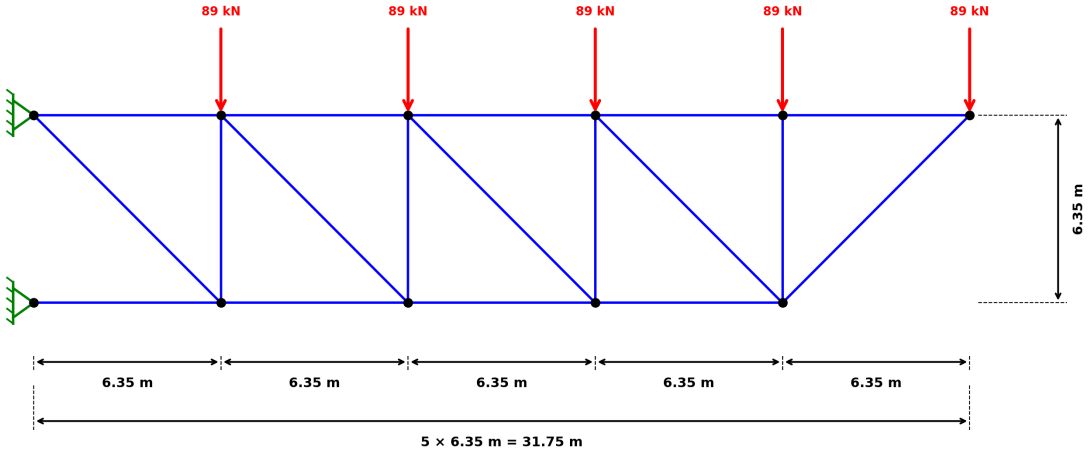
Hình 5.9.

Bài 5.5: Cho kết cấu giàn phẳng 200 thanh dạng tháp như Hình 5.10,với mô-đun đàn hồi $E = 210$ GPa và khối lượng riêng $\rho = 7850$ kg/m^3. Hãy xác định diện tích tiết diện tối ưu của 29 nhóm thanh ($1\ \text{cm}^2 \leq \text{A}_i \leq 130\ \text{cm}^2$) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất $[\sigma] = 70$ MPa cho cả ba trường hợp tải trọng.



Hình 5.10.

Bài 5.6: Cho kết cấu giàn 18 thanh như Hình 5.11, được chế tạo từ vật liệu có trọng lượng riêng $\rho = 2770$ kg/m^3 và mô-đun đàn hồi $E = 69$ GPa. Việc thiết kế tối ưu hóa bao gồm 12 biến độc lập, trong đó các thanh được phân thành 4 nhóm tiết diện biến thiên từ 22 cm^2 đến 116 cm^2, kết hợp cùng biến số về tọa độ ngang và dọc của 4 điểm nút nằm ở biên dưới. Hãy tìm kích thước tiết diện và tọa độ các nút biên tối ưu nhằm cực tiểu hóa tổng khối lượng hệ giàn, đồng thời phải đảm bảo cấu trúc thỏa mãn giới hạn ứng suất kéo nén $|\sigma_i| \leqslant 138$ MPa, không xảy ra hiện tượng mất ổn định Euler ở các thanh chịu nén (trị tuyệt đối của ứng suất nén không được vượt quá ứng suất tới hạn $\sigma_{cr} = 4EA_i/L_i^2$ với A_i và L_i lần lượt là diện tích tiết diện và chiều dài của thanh tương ứng), và giới hạn chuyển vị tại mọi nút không vượt quá 25 mm.


89 kN
89 kN
89 kN
89 kN
89 kN
6.35 m
6.35 m
6.35 m
6.35 m
6.35 m
6.35 m
5 × 6.35 m = 31.75 m

Hình 5.11.

CHƯƠNG 6

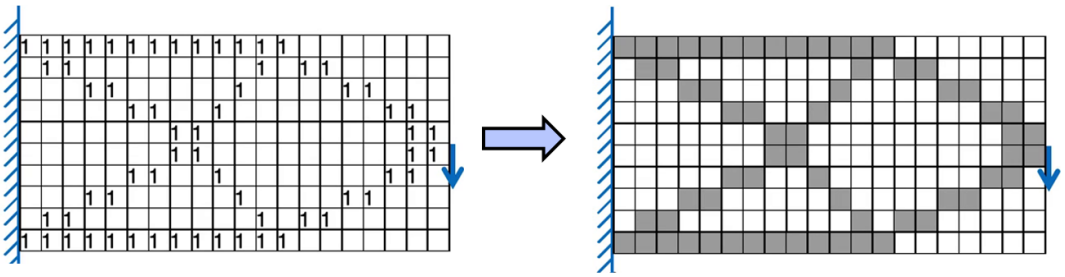
TỐI ƯU CẤU TRÚC

6.1. Phát biểu bài toán tối ưu cấu trúc

Bài toán tối ưu cấu trúc được phát biểu như sau:

$$\begin{aligned} &\text{minimize} && C = \mathbf{u}^T\mathbf{K}\mathbf{u} \\ &\text{s.t.} && \mathbf{K}\mathbf{u} = \mathbf{F} \\ & && \sum_e x_e \leqslant V_0 \\ & && x_e = \begin{cases} 1 & (solid) \\ 0 & (void) \end{cases}, \forall e \end{aligned} \tag{6.1}$$

trong đó C là năng lượng biến dạng (strain energy); $\mathbf{u}$ là trường chuyển vị (displacement); $\mathbf{K}$ là ma trận độ cứng (stiffness matrix) của kết cấu; $\mathbf{F}$ là vector lực (force vector); V_0 là thể tích (volume) tối đa cho phép của kết cấu; x_e là mật độ (density) của phân tố e; nếu một phân tố có giá trị $x_e = 1$ nghĩa là tại vị trí đó có vật liệu, còn nếu $x_e = 0$ nghĩa là tại vị trí đó là lỗ rỗng. Chẳng hạn như ở Hình 6.1, các phần tử có giá trị $x_e = 1$ sẽ có vật liệu (tô màu xám) và những phần tử có giá trị $x_e = 0$ sẽ không có vật liệu (tô màu trắng).



Hình 6.1. Chuyển đổi từ biến mật độ qua kết cấu thực.

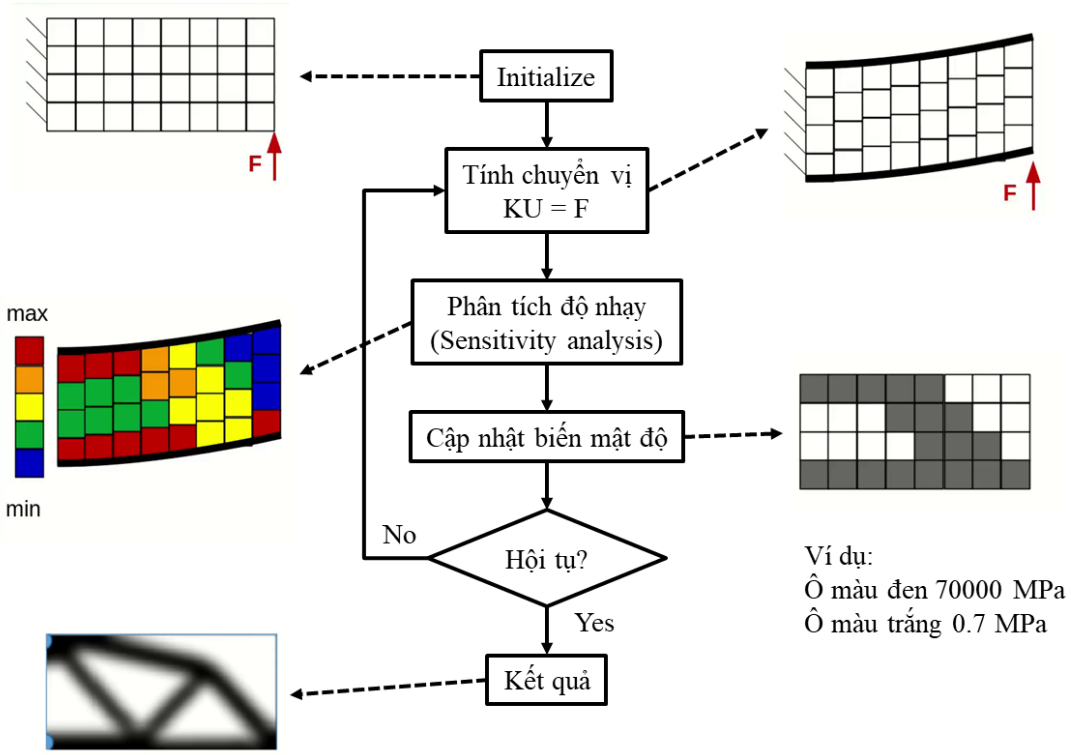
Tuy nhiên bài toán tối ưu cấu trúc với biến mật độ nhị phân rất khó giải. Do đó trong phần tiếp theo của chương này ta sẽ chỉ đề cập đến trường hợp khi mà biến mật độ x_e là một biến liên tục trong miền $[0, 1]$.

$$x_e \in [0, 1], \forall e \tag{6.2}$$

Lưu đồ tính toán bài toán tối ưu cấu trúc được thể hiện ở Hình 6.2. Đầu tiên ta giả định một phân bố mật độ nào đó trong miền thiết kế, chẳng hạn gán tất cả các phần

tử có cùng mật độ bằng với thể tích phần $f = \sum_e x_e / V_0$. Tiếp theo, tính miền chuyển vị $\mathbf{U}$ từ phương trình cân bằng tĩnh học $\mathbf{KU} = \mathbf{F}$. Bước này thông thường sử dụng phương pháp phần tử hữu hạn (Finite Element Method). Tiếp theo là bước phân tích độ nhạy (sensitivity analysis) để cập nhật biến thiết kế mật độ dựa vào thuật toán tiêu chuẩn tối ưu.

Lưu ý để tránh trường hợp ma trận độ cứng bị suy biến, ta không thể gán độ cứng bằng 0 ở các phần tử lỗ rỗng mà phải gán một độ cứng rất nhỏ nào đó: chẳng hạn như với vật liệu nhôm, các phần tử có vật liệu thì mô-đun đàn hồi $E = 70000\,\text{MPa}$, còn lỗ rỗng thì mô-đun đàn hồi $E = 0.7\,\text{MPa}$. Sau khi cập nhật biến thiết kế, ta quay lại bước tính toán chuyển vị bằng phương pháp phần tử hữu hạn. Chương trình tính dừng lại khi kết quả tính toán hội tụ, nghĩa là sự thay đổi của biến thiết kế nhỏ hơn một giá trị định trước nào đó.



Hình 6.2. Lưu đồ tính toán bài toán tối ưu cấu trúc.

6.2. Thuật toán tối ưu cấu trúc SIMP

Phương pháp vật liệu đẳng hướng với sự phạt (Solid Isotropic Material with Penalization SIMP) là thuật toán phổ biến nhất trong bài toán tối ưu cấu trúc. Mỗi phần tử e được gán một mật độ x_e liên hệ với mô-đun đàn hồi E_e theo công thức

$$E_e(x_e) = x_e^p E_0 \tag{6.3}$$

với $p > 1$ là hệ số phạt (penalization factor), thường lấy $p = 3$, E_0 là mô-đun đàn hồi của vật liệu. Ta phát biểu lại bài toán tối ưu cấu trúc như sau

$$\begin{aligned} &\min_{\mathbf{x}} \; C(\mathbf{x}) = \mathbf{u}^T \mathbf{K} \mathbf{u} = \sum_{e=1}^{N} (x_e)^p \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e \\ &\text{s.t:} \; V(\mathbf{x})/V_0 = f \\ &\quad \mathbf{K}\mathbf{u} = \mathbf{F} \\ &\quad \mathbf{0} < \mathbf{x}_{\min} \leqslant \mathbf{x} \leqslant \mathbf{1} \end{aligned} \tag{6.4}$$

trong đó $\mathbf{u}$ và $\mathbf{F}$ lần lượt là vector chuyển vị tổng thể và vector lực tổng thể; $\mathbf{K}$ là ma trận độ cứng tổng thể; $\mathbf{u}_e$ và $\mathbf{k}_e$ lần lượt là vector chuyển vị và ma trận độ cứng của phần tử e; $\mathbf{x}$ là vector biến mật độ; $\mathbf{x}_{\min}$ là vector biến mật độ tối thiểu (phải lớn hơn 0 để tránh ma trận độ cứng bị suy biến); $N = n_{\text{elx}} n_{\text{ely}}$ là tổng số phần tử trong miền thiết kế, p là hệ số phạt (penalization power), thường lấy giá trị $p = 3$; $V(\mathbf{x})$ và V_0 lần lượt là thể tích với biến mật độ $\mathbf{x}$ và thể tích ban đầu của miền thiết kế; f là thể tích phần (volume fraction) được cho trước.

Độ nhạy (sensitivity) của hàm mục tiêu sẽ được sử dụng để tìm cập nhật biến thiết kế

$$\frac{\partial C}{\partial x_e} = -p(x_e)^{p-1} \mathbf{u}_e^T \mathbf{k}_0 \mathbf{u}_e \tag{6.5}$$

Để đảm bảo bài toán tối ưu cấu trúc có nghiệm và để tránh sự hình thành của dạng cấu trúc ô cờ (checkerboard patterns), một số hạn chế (restriction) đối với thiết kế tối ưu phải được đưa vào: chẳng hạn sử dụng bộ lọc độ nhạy (sensitivity filter), hoặc bộ lọc mật độ (density filter). Về bản chất đây là các phương pháp xử lý ảnh (image processing) bằng bộ lọc thông thấp (low-pass filter). Phương pháp này thực hiện bằng cách định nghĩa một miền lân cận (neighborhood) bán kính $r_{\min}$ của phần tử e, khi đó độ nhạy hoặc mật độ của phần tử e được lấy bằng trung bình của tất cả các phần tử trong miền lân cận đó

$$\frac{\widehat{\partial C}}{\partial x_e} = \frac{1}{x_e \sum_{f=1}^{N} \hat{H}_f} \sum_{f=1}^{N} \hat{H}_f x_f \frac{\partial C}{\partial x_e} \tag{6.6}$$

trong đó trọng số (weight factor) $\hat{H}_f$ được xác định bởi

$$\begin{aligned} &\hat{H}_f = r_{\min} - \operatorname{dist}(e, f) \\ &\{f \in N | \operatorname{dist}(e, f) \leqslant r_{\min}\}, \;\; e = 1, ..., N \end{aligned} \tag{6.7}$$

với $\operatorname{dist}(e, f)$ là khoảng cách giữa tâm hai phần tử e và f.

Bước cập nhật biến thiết kế thực chất là một bài toán tìm cực tiểu cho hàm mục tiêu với điều kiện ràng buộc: cho một biến thiết kế $x^{(k)}$, thuật toán cập nhật biến thiết kế tìm ra một biến thiết kế mới $x^{(k+1)}$ làm cho hàm mục tiêu có giá trị nhỏ hơn. Bài báo của Sigmund (2001) sử dụng thuật toán cập nhật biến thiết kế Optimality Criteria (OC) (các công thức (6.8) và (6.9) dưới đây).

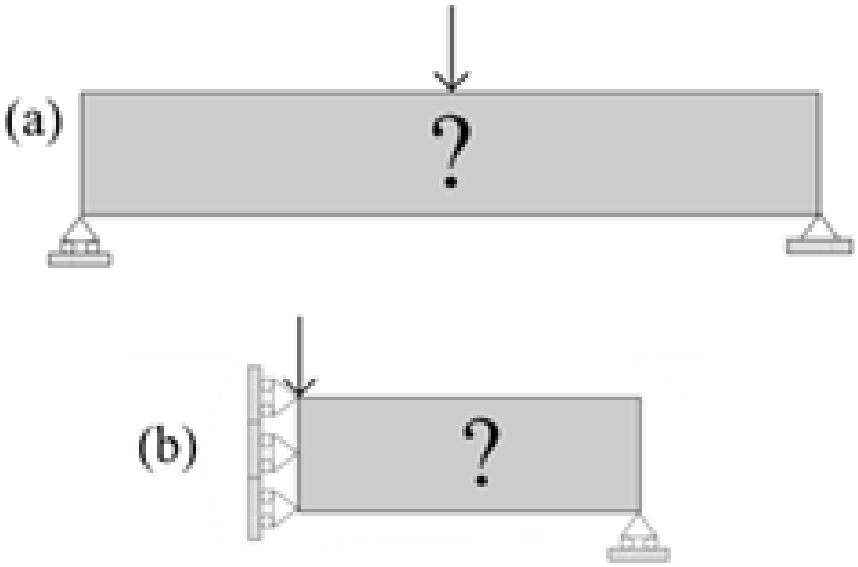
$$x_e^{new} = \begin{cases} \max(x_{\min}, x_e - m) & \text{if} \;\; x_e B_e^{\eta} \leqslant \max(x_{\min}, x_e - m) \\ x_e B_e^{\eta} & \text{if} \;\; \max(x_{\min}, x_e - m) < x_e B_e^{\eta} < \min(1, x_e + m) \\ \min(1, x_e + m) & \text{if} \;\; x_e B_e^{\eta} \geqslant \min(1, x_e + m) \end{cases} \tag{6.8}$$

với m (move) là một giá trị dương; $\eta = 0.5$; và B_e được xác định như sau

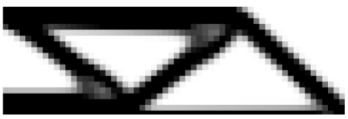
$$B_e = \frac{-\frac{\partial C}{\partial x_e}}{\lambda \frac{\partial V}{\partial x_e}} \tag{6.9}$$

với λ là hệ số Lagrangian (Lagrangian multiplier) được xác định bằng thuật toán bi-sectioning.

Tối ưu cấu trúc dầm đơn giản Hình 6.3 do tải trọng tập trung giữa dầm. Do tính chất đối xứng nên chỉ cần xét một nửa dầm.



Hình 6.3. Tối ưu hóa cấu trúc dầm MBB. (a) Miền thiết kế, (b) Nửa miền thiết kế.



Hình 6.4. Kết quả tối ưu hóa cấu trúc dầm MBB.

CHƯƠNG 7

TỐI ƯU HÓA ĐA MỤC TIÊU

7.1. Giới thiệu

Trong các chương trước, bài toán tối ưu hóa kết cấu được xây dựng dưới dạng một hàm mục tiêu duy nhất. Chẳng hạn, người thiết kế có thể hướng đến việc tối thiểu hóa khối lượng kết cấu, cực đại hóa độ cứng, hoặc giảm chuyển vị tại một vị trí xác định. Tuy nhiên, trong thực tế thiết kế kết cấu, một phương án hiếm khi được đánh giá dựa trên chỉ một tiêu chí. Kỹ sư thường phải đồng thời xem xét nhiều yêu cầu khác nhau, và các yêu cầu này thường có xu hướng mâu thuẫn lẫn nhau, chẳng hạn như kết cấu phải nhẹ nhưng vẫn đủ cứng để hạn chế chuyển vị, chi phí phải thấp nhưng vẫn đảm bảo an toàn và độ bền, hoặc lượng vật liệu sử dụng phải ít nhưng vẫn đáp ứng yêu cầu về ổn định và tuổi thọ khai thác.

Trong những trường hợp như vậy, tối ưu hóa theo một mục tiêu duy nhất thường dẫn đến các thiết kế thiếu cân bằng. Ví dụ, nếu chỉ tập trung tối thiểu hóa khối lượng, nghiệm tối ưu thường nằm sát giới hạn cho phép của các ràng buộc về ứng suất hoặc chuyển vị, khiến kết cấu trở nên nhạy cảm với sai số mô hình hoặc biến động tải trọng. Ngược lại, nếu chỉ cực đại hóa độ cứng, phương án thu được có thể quá nặng và kém hiệu quả về mặt kinh tế.

Vấn đề cốt lõi là không tồn tại một thiết kế đồng thời tốt nhất cho mọi tiêu chí. Việc cải thiện một đặc tính thường phải đánh đổi bằng sự suy giảm của đặc tính khác. Do đó, thay vì tìm kiếm một nghiệm tối ưu duy nhất, kỹ sư cần hiểu rõ mối quan hệ đánh đổi (trade-off) giữa các mục tiêu thiết kế.

Tối ưu hóa đa mục tiêu (Multiobjective Optimization MOO) được phát triển nhằm giải quyết bài toán này. Mục tiêu của MOO không phải là xác định một phương án tối ưu duy nhất, mà là tìm ra một tập hợp các phương án thiết kế hợp lý, thể hiện những mức cân bằng khác nhau giữa các tiêu chí. Từ tập hợp này, người thiết kế có thể lựa chọn phương án phù hợp nhất dựa trên yêu cầu kỹ thuật, điều kiện kinh tế hoặc các yếu tố thực tế nằm ngoài mô hình tính toán.

Cũng cần lưu ý rằng không phải mọi bài toán có nhiều tiêu chí đều cần được mô hình hóa dưới dạng tối ưu đa mục tiêu. Trong nhiều trường hợp, một số tiêu chí chỉ đóng vai trò như các ràng buộc (ví dụ: chuyển vị không vượt quá giá trị cho phép), hoặc có thể được quy đổi thành một đại lượng tổng hợp như chi phí. Việc sử dụng tối ưu đa mục tiêu chỉ thực sự cần thiết khi không thể xác định trước một tiêu chí tổng hợp đủ khả năng phản ánh toàn diện chất lượng thiết kế.

Trong phạm vi chương này, tối ưu hóa đa mục tiêu được trình bày ở mức giới thiệu nhằm làm rõ ý nghĩa kỹ thuật và bản chất của các bài toán có nhiều mục tiêu. Trọng tâm không đặt vào các thuật toán chuyên sâu, mà hướng đến việc giúp người đọc hiểu khái niệm đánh đổi giữa các mục tiêu và cách diễn giải kết quả tối ưu trong quá trình ra quyết định thiết kế kết cấu.

7.2. Phát biểu bài toán đa mục tiêu

Trong bài toán tối ưu hóa kết cấu một mục tiêu đã trình bày ở các chương trước, bài toán tổng quát có dạng:

$$\min_{\mathbf{x}} f(\mathbf{x}) \tag{7.1}$$

trong đó $\mathbf{x}$ là vector biến thiết kế và $f(\mathbf{x})$ là hàm mục tiêu dùng để đánh giá chất lượng của phương án thiết kế. Nghiệm tối ưu là phương án cho giá trị nhỏ nhất của hàm mục tiêu trong miền khả dĩ.

Trong tối ưu hóa đa mục tiêu, thay vì chỉ có một đại lượng cần tối ưu, ta đồng thời xem xét nhiều tiêu chí khác nhau. Khi đó, hàm mục tiêu không còn là một đại lượng vô hướng mà trở thành một vector hàm mục tiêu:

$$\min_{\mathbf{x}} f(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \\ \vdots \\ f_m(\mathbf{x}) \end{bmatrix}, \qquad m \geqslant 2 \tag{7.2}$$

trong đó mỗi hàm $f_i(\mathbf{x})$ biểu diễn một tiêu chí thiết kế riêng biệt. Ví dụ trong bài toán kết cấu, các hàm mục tiêu có thể là khối lượng kết cấu, chuyển vị tại nút hoặc chi phí chế tạo... Các ràng buộc của bài toán, bao gồm ràng buộc ứng suất, chuyển vị, ổn định hoặc điều kiện cân bằng, vẫn giữ nguyên như trong bài toán tối ưu một mục tiêu.

Sự khác biệt cơ bản giữa hai loại bài toán nằm ở chỗ: trong bài toán đa mục tiêu, không thể đồng thời cực tiểu tất cả các hàm mục tiêu. Trong phần lớn các hệ kết cấu, các mục tiêu có mối liên hệ phụ thuộc lẫn nhau. Ví dụ, việc tăng diện tích tiết diện để giảm chuyển vị sẽ làm tăng khối lượng; ngược lại, giảm khối lượng thường dẫn đến độ cứng nhỏ hơn và chuyển vị lớn hơn. Do đó, việc cải thiện một mục tiêu thường kéo theo sự suy giảm của mục tiêu khác.

Hiện tượng này được gọi là sự đánh đổi giữa các mục tiêu. Thay vì tồn tại một nghiệm tối ưu duy nhất như trong bài toán đơn mục tiêu, bài toán đa mục tiêu thường dẫn đến nhiều nghiệm tối ưu tương đương về mặt ý nghĩa thiết kế, mỗi nghiệm thể hiện một mức cân bằng khác nhau giữa các tiêu chí.

Một cách trực quan, có thể xem bài toán đa mục tiêu như quá trình khảo sát toàn bộ không gian thiết kế nhằm trả lời các câu hỏi dạng: cần tăng bao nhiêu khối lượng để giảm một lượng chuyển vị nhất định, hoặc việc cải thiện độ cứng thêm một mức nhỏ có làm chi phí tăng đáng kể hay không. Những thông tin này không thể thu được nếu bài toán được chuyển ngay về dạng một mục tiêu duy nhất từ đầu.

Trong thực tế, việc lựa chọn phương án cuối cùng vẫn luôn tương đương với việc lựa chọn một tiêu chí ưu tiên nào đó. Tuy nhiên, tối ưu hóa đa mục tiêu cho phép quá trình lựa chọn này được thực hiện sau khi đã hiểu rõ mối quan hệ giữa các mục tiêu, thay vì phải giả định trước ngay từ khi xây dựng mô hình toán học.

Trong các mục tiếp theo, chúng ta sẽ giới thiệu khái niệm tối ưu Pareto, là nền tảng để xác định và diễn giải các nghiệm tối ưu trong bài toán đa mục tiêu.

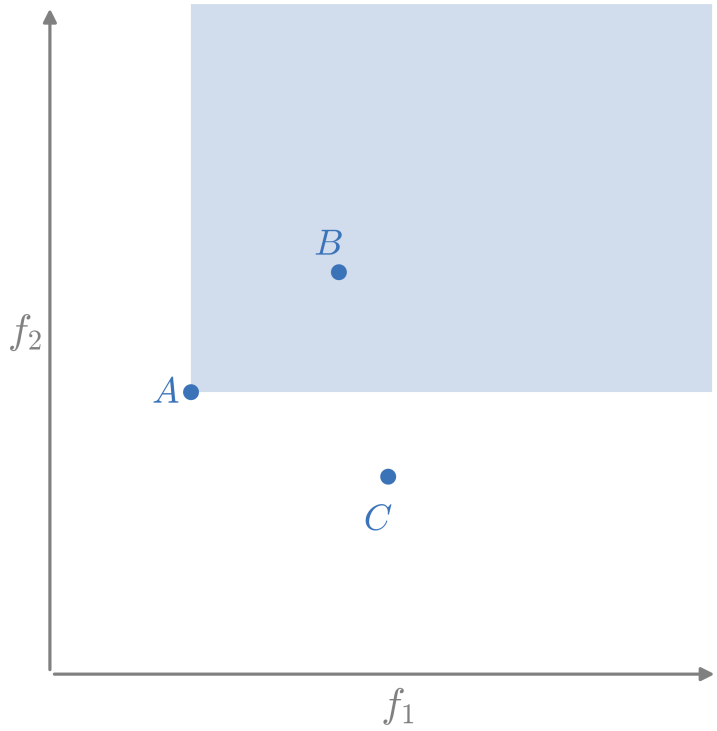
7.3. Khái niệm Pareto tối ưu

Trong bài toán tối ưu hóa một mục tiêu, việc so sánh các phương án thiết kế tương đối đơn giản: phương án nào cho giá trị hàm mục tiêu nhỏ hơn thì được xem là tốt hơn. Khi đó, quá trình tối ưu hướng tới việc tìm ra một nghiệm duy nhất cho giá trị tốt nhất của tiêu chí đã chọn.

Tuy nhiên, trong bài toán đa mục tiêu, tình huống này thay đổi đáng kể. Một phương án có thể tốt hơn ở một tiêu chí nhưng lại kém hơn ở tiêu chí khác. Ví dụ, một kết cấu nhẹ hơn thường có chuyển vị lớn hơn, trong khi một kết cấu cứng hơn lại có khối lượng lớn hơn. Vì vậy, không còn tồn tại một tiêu chuẩn so sánh đơn giản như trong bài toán một mục tiêu.

Để mô tả mối quan hệ này, tối ưu hóa đa mục tiêu sử dụng khái niệm ưu thế (dominance). Xét hai phương án thiết kế A và B. Phương án A được gọi là chi phối (dominate) phương án B nếu A không kém hơn B ở tất cả các mục tiêu và tốt hơn B ít nhất ở một mục tiêu. Trong trường hợp đó, phương án B có thể bị loại bỏ vì tồn tại một phương án khác tốt hơn hoặc tương đương ở mọi khía cạnh.

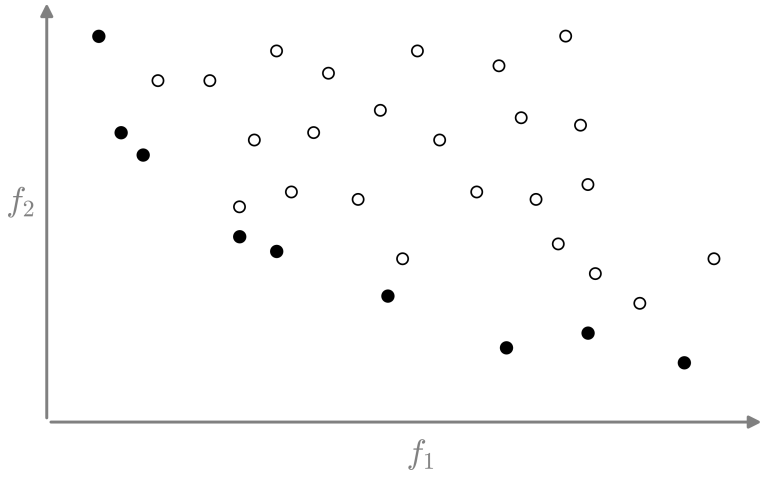
Ngược lại, một phương án được gọi là không bị chi phối (nondominated) nếu không tồn tại phương án nào khác tốt hơn nó theo nghĩa trên. Những nghiệm này đại diện cho các thiết kế mà việc cải thiện thêm một mục tiêu chỉ có thể đạt được bằng cách làm xấu đi ít nhất một mục tiêu khác. Đây chính là bản chất của sự đánh đổi trong thiết kế kỹ thuật.



Hình 7.1. Ba phương án thiết kế A, B và C được biểu diễn theo hai hàm mục tiêu f_1 và f_2. Vùng hình chữ nhật được tô bóng biểu thị các điểm bị phương án thiết kế A chi phối.

Tập hợp tất cả các nghiệm không bị chi phối trong không gian biến thiết kế được gọi là tập Pareto (Pareto set). Khi biểu diễn các nghiệm này trong không gian các hàm mục tiêu, ta thu được biên Pareto (Pareto front). Biên Pareto thể hiện toàn bộ các phương án thiết kế hợp lý mà bài toán có thể đạt được.

Khái niệm Pareto làm thay đổi cách hiểu về tối ưu. Trong bài toán đa mục tiêu, một nghiệm Pareto tối ưu không phải là nghiệm tốt nhất theo mọi tiêu chí, mà là nghiệm mà không thể cải thiện thêm một mục tiêu nào mà không phải đánh đổi ở mục tiêu khác. Do đó, thay vì tồn tại một nghiệm tối ưu duy nhất, bài toán đa mục tiêu thường dẫn đến một tập các nghiệm tối ưu tương đương về mặt ý nghĩa thiết kế.



Hình 7.2. Biểu đồ thể hiện tất cả các điểm đã được đánh giá trong không gian thiết kế theo hai hàm mục tiêu f_1 và f_2. Tập các điểm tô đen không bị bất kỳ điểm nào khác chi phối, do đó tạo thành tập Pareto.

Đối với kỹ sư kết cấu, biên Pareto là một công cụ giúp hiểu rõ hơn hành vi của hệ kết cấu. Hình dạng của đường Pareto cho biết mức độ đánh đổi giữa các tiêu chí. Ví dụ, nếu một đoạn của biên Pareto có độ dốc lớn, chỉ cần tăng nhẹ khối lượng cũng có thể giảm đáng kể chuyển vị. Ngược lại, ở gần các điểm cực trị, việc cải thiện thêm một tiêu chí thường đòi hỏi sự đánh đổi rất lớn ở tiêu chí còn lại. Những thông tin này giúp người thiết kế nhận biết vùng thiết kế hiệu quả trước khi đưa ra lựa chọn cuối cùng.

Vì vậy, mục tiêu của tối ưu hóa đa mục tiêu không phải là lựa chọn ngay một phương án thiết kế duy nhất, mà là xác định biên Pareto để hỗ trợ quá trình ra quyết định. Việc lựa chọn nghiệm cuối cùng thường phụ thuộc vào các yếu tố ngoài mô hình toán học, chẳng hạn như yêu cầu kinh tế, mức độ an toàn mong muốn hoặc kinh nghiệm của người thiết kế.

Khi bài toán có hai biến thiết kế, một công cụ trực quan thường dùng để khảo sát hàm mục tiêu là đường đẳng trị (contour). Đường đẳng trị của một hàm $f(x_1, x_2)$ là tập hợp các điểm trong mặt phẳng thiết kế thỏa mãn $f(x_1, x_2) = c$, với c là một hằng số. Mỗi đường đẳng trị biểu diễn các phương án thiết kế cho cùng một giá trị hàm mục tiêu. Trong bài toán tối ưu đa mục tiêu, việc vẽ đồng thời các đường đẳng trị của nhiều hàm mục tiêu giúp nhận biết hướng cải thiện của từng mục tiêu và vùng có thể xuất hiện các nghiệm Pareto. Nếu hai họ đường đẳng trị có xu hướng kéo nghiệm về các vùng khác nhau, bài toán thể hiện sự đánh đổi rõ rệt giữa các mục tiêu.

Trong mục tiếp theo, các khái niệm trên sẽ được minh họa thông qua một ví dụ tối ưu đa mục tiêu của kết cấu giàn đơn giản, qua đó làm rõ ý nghĩa vật lý của biên Pareto trong thiết kế kết cấu.

7.4. Một số cách tiếp cận trong tối ưu hóa đa mục tiêu

Như đã trình bày ở các mục trước, mục tiêu của tối ưu hóa đa mục tiêu không phải là tìm một nghiệm duy nhất, mà là xác định tập các nghiệm Pareto tối ưu thể hiện sự đánh đổi giữa các tiêu chí thiết kế. Trong thực tế, việc xây dựng biên Pareto có thể được thực hiện bằng nhiều phương pháp khác nhau. Tuy nhiên, trong phạm vi cuốn sách này, chỉ những ý tưởng cơ bản nhất sẽ được giới thiệu nhằm giúp người đọc hiểu nguyên lý chung, thay vì đi sâu vào chi tiết thuật toán.

7.4.1. Phương pháp tổng trọng số (weighted sum method)

Cách tiếp cận đơn giản nhất là chuyển bài toán đa mục tiêu về dạng một mục tiêu duy nhất bằng cách gán trọng số cho từng hàm mục tiêu:

$$\min \; F(x) = w_1 f_1(x) + w_2 f_2(x) + \cdots + w_m f_m(x) \tag{7.3}$$

trong đó w_i là các hệ số trọng số thể hiện mức độ ưu tiên tương đối của từng mục tiêu.

Về bản chất, phương pháp này biến bài toán đa mục tiêu thành một chuỗi các bài toán tối ưu một mục tiêu quen thuộc. Bằng cách thay đổi các giá trị trọng số, ta có thể thu được các nghiệm khác nhau trên biên Pareto.

Ưu điểm của phương pháp là đơn giản, dễ áp dụng và có thể sử dụng trực tiếp các thuật toán tối ưu đã trình bày trong các chương trước. Tuy nhiên, phương pháp này tồn tại một số hạn chế quan trọng. Trước hết, việc lựa chọn trọng số phù hợp thường không dễ dàng vì người thiết kế phải xác định trước mức độ ưu tiên giữa các mục tiêu. Ngoài ra, phương pháp tổng trọng số không thể tìm được đầy đủ các nghiệm Pareto trong trường hợp đường Pareto không lồi, dẫn đến việc một số phương án thiết kế hợp lý có thể bị bỏ qua.

Do những hạn chế này, phương pháp tổng trọng số thường được xem như một cách tiếp cận đơn giản để minh họa ý tưởng đa mục tiêu hơn là một công cụ tổng quát cho mọi bài toán.

7.4.2. Phương pháp ràng buộc epsilon (epsilon-constraint method)

Một cách tiếp cận trực quan hơn là lựa chọn một mục tiêu làm hàm mục tiêu chính, trong khi các mục tiêu còn lại được chuyển thành ràng buộc:

$$\begin{aligned} &\min \;\; f_1(x) \\ &\text{s.t.} \;\; f_2(x) \leqslant \varepsilon_2 \\ &\qquad f_3(x) \leqslant \varepsilon_3, \ldots \end{aligned} \tag{7.4}$$

trong đó ε_i là các giá trị giới hạn được lựa chọn trước.

Bằng cách thay đổi các giá trị ε_i, ta thu được các nghiệm Pareto khác nhau. Cách tiếp cận này phù hợp với tư duy thiết kế kết cấu, vì trong thực tế nhiều yêu cầu kỹ thuật đã tồn tại dưới dạng giới hạn cho phép, ví dụ chuyển vị tối đa hoặc mức ứng suất cho phép.

So với phương pháp tổng trọng số, phương pháp ràng buộc epsilon thường cho phép thu được phân bố nghiệm Pareto đầy đủ hơn và dễ diễn giải hơn về mặt kỹ thuật. Tuy nhiên, việc lựa chọn các giá trị giới hạn phù hợp vẫn cần kinh nghiệm hoặc quá trình thử nghiệm.

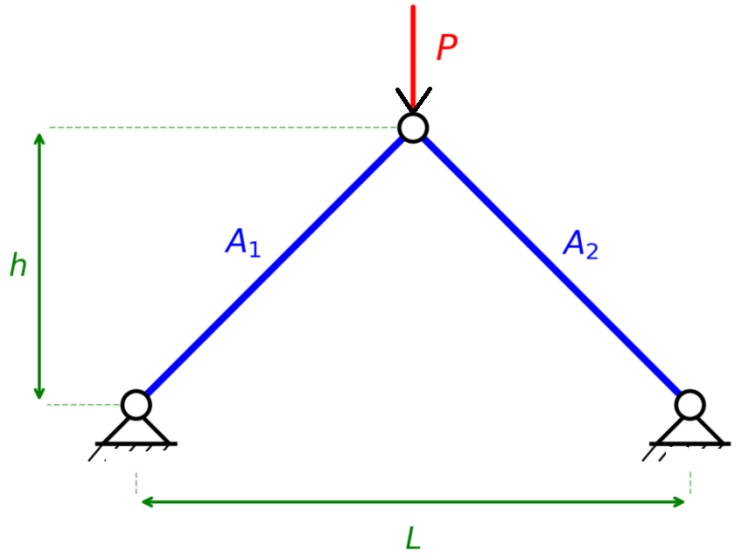
7.4.3. Phương pháp tiến hóa đa mục tiêu

Trong các bài toán kết cấu thực tế với số lượng biến thiết kế lớn hoặc hàm mục tiêu phi tuyến mạnh, việc xây dựng biên Pareto bằng các phương pháp trên có thể trở nên khó khăn. Trong những trường hợp này, các phương pháp tối ưu hóa ngẫu nhiên hoặc tiến hóa, chẳng hạn như thuật toán di truyền đa mục tiêu (multiobjective genetic algorithm), thường được sử dụng.

Các phương pháp này làm việc với một tập nghiệm đồng thời và có khả năng tạo ra nhiều nghiệm Pareto trong một quá trình tính toán duy nhất. Vì lý do này, các thuật toán tiến hóa thường được sử dụng trong tối ưu hóa đa mục tiêu của kết cấu, đặc biệt trong các bài toán tối ưu topo hoặc tối ưu rời rạc.

7.5. Ví dụ minh họa

Cho một hệ giàn phẳng Hình 7.3. Các thanh có diện tích tiết diện lần lượt là A_1 và A_2, với vật liệu có mô đun đàn hồi $E = 200$ GPa, khối lượng riêng $\rho = 7850$ kg/m^3 và ứng suất cho phép $\sigma_{\text{allow}} = 150$ MPa. Xác định biến thiết kế $\mathbf{x} = [A_1, A_2]^T$ trong khoảng $[200, 4000]$ mm^2 sao cho cực tiểu hóa đồng thời hai mục tiêu: khối lượng tổng cộng của hệ $f_1(\mathbf{x})$ và chuyển vị đứng $f_2(\mathbf{x})$ tại điểm đặt lực, đồng thời phải đảm bảo điều kiện ràng buộc về độ bền sao cho ứng suất thực tế trên mỗi thanh không vượt quá ứng suất cho phép.



Hình 7.3. Hệ giàn phẳng dùng trong ví dụ tối ưu hóa đa mục tiêu.

1. Phát biểu bài toán tối ưu đa mục tiêu

$$\min_{\mathbf{x}} \mathbf{F}(\mathbf{x}) = \begin{bmatrix} f_1(\mathbf{x}) \\ f_2(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} \rho(L_1 A_1 + L_2 A_2) \\ |v_C(\mathbf{x})| \end{bmatrix}$$

với các ràng buộc:

$$|\sigma_i(\mathbf{x})| \leqslant \sigma_{\text{allow}} = 150\,\text{MPa}, \quad i = 1, 2$$

$$200\ \text{mm}^2 \leqslant A_i \leqslant 4000\ \text{mm}^2, \quad i = 1, 2$$

2. Chuẩn hóa và hàm tổng trọng số

Bước chuẩn hóa (Utopia–Nadir): giải riêng các bài toán đơn mục tiêu để xác định giá trị tốt nhất và xấu nhất tham chiếu của từng mục tiêu. Điểm Utopia gồm các giá trị tốt nhất đạt được khi tối ưu riêng từng mục tiêu, còn điểm Nadir gồm các giá trị bất lợi tương ứng trên tập nghiệm đang xét. Hai điểm này được dùng để đưa các mục tiêu về cùng thang đo trước khi kết hợp bằng hàm tổng trọng số.

Bài toán	A_1 (mm^2)	A_2 (mm^2)	f_1 (kg)	f_2 (mm)
min f_1 (khối lượng)	235,7	235,7	5,23	1,500

min f_2 (chuyển vị)	4000	4000	88,6	0,088

Giá trị chuẩn hóa:

$$f_1^{\min} = 5,23\,\text{kg}, \quad f_1^{\max} = 88,6\,\text{kg}$$

$$f_2^{\min} = 0,088\,\text{mm}, \quad f_2^{\max} = 1,5\,\text{mm}$$

$$\tilde{f}_i = \frac{f_i - f_i^{\min}}{f_i^{\max} - f_i^{\min}} \in [0,1]$$

Hàm đơn mục tiêu:

$$F(\mathbf{x}, w) = w\tilde{f}_1(\mathbf{x}) + (1-w)\tilde{f}_2(\mathbf{x}), \qquad 0 \leqslant w \leqslant 1$$

3. Quét trọng số và kết quả Pareto
Giải bài toán $\min F(\mathbf{x};, w)$ kèm ràng buộc ứng suất với $w = 0 : 0,1 : 1$

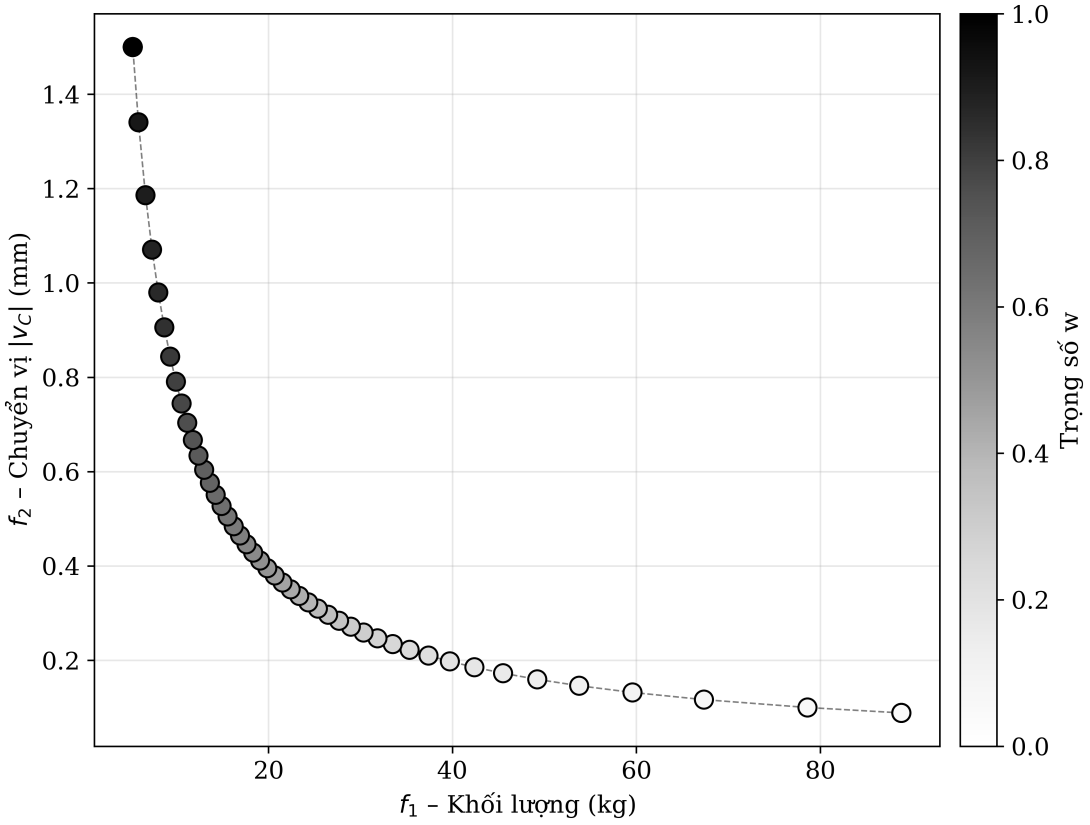
w	A_1 (mm^2)	A_2 (mm^2)	f_1 (kg)	f_2 (mm)
0,0	4000	4000	88,6	0,088
0,1	3458	3458	76,6	0,102
0,2	2450	2450	54,3	0,144
0,3	1789	1789	39,6	0,197
0,4	1302	1302	28,8	0,271
0,5	950	950	21,0	0,371
0,6	690	690	15,3	0,511
0,7	502	502	11,1	0,703
0,8	370	370	8,2	0,953
0,9	282	282	6,3	1,250
1,0	236	236	5,2	1,500

Biên Pareto trong mặt phẳng (f_1, f_2) có dạng đường cong lõm Hình 7.4.
4. Nhận xét ý nghĩa kỹ thuật
Vùng w nhỏ (0 – 0,2): bão hòa chuyển vị: Tăng khối lượng từ 54 đến 89 kg (thêm ~35 kg) chỉ giảm chuyển vị từ 0,14 đến 0,09 mm (giảm khoảng 0,05 mm). Hiệu quả rất kém, đầu tư thêm vật liệu không đáng.

Vùng w lớn (0,8 – 1,0): bão hòa khối lượng: Giảm khối lượng từ 8,2 đến 5,2 kg (tiết kiệm ~3 kg) nhưng chuyển vị tăng từ 0,95 đến 1,50 mm (tăng gấp rưỡi). Tiết kiệm chút vật liệu nhưng kết cấu yếu đi nhanh.

Vùng giữa ($w \approx 0,3$ - $0,6$): đánh đổi hiệu quả: Mỗi kg vật liệu bớt/thêm đều tương ứng thay đổi chuyển vị vừa phải. Đây là vùng thiết kế hợp lý nhất.

Đề xuất phương án: Chọn điểm "đầu gối"(knee point) ở khoảng $w \approx 0,4$ - $0,5$, tương ứng $A_1 = A_2 \approx 950$ - 1300 mm^2, cho khối lượng 21 đến 29 kg và chuyển vị ~0,27–0,37 mm, cân bằng tốt nhất giữa nhẹ và cứng.



Hình 7.4. Biên Pareto trong mặt phẳng (f_1, f_2).

7.6. Bài tập

Bài 7.1: Trong không gian thiết kế, hãy vẽ các đường đẳng trị (contours) của hàm mục tiêu cho bài toán không ràng buộc sau đây và vẽ tập tối ưu Pareto

$$\begin{aligned} \text{minimize} \quad & f_1 = (x_1 - 0,75)^2 + (x_2 - 2)^2 \\ & f_2 = (x_1 - 2,5)^2 + (x_2 - 1,5)^2 \end{aligned}$$

Bài 7.2: Trong không gian thiết kế, hãy vẽ bài toán có ràng buộc sau đây và vẽ tập tối ưu Pareto:

$$\begin{aligned} \text{minimize} \quad & f_1 = (x_1 - 3)^2 + (x_2 - 7)^2 \\ & f_2 = (x_1 - 9)^2 + (x_2 - 8)^2 \\ \text{s.t.} \quad & g_1 = 70 - 4x_2 - 8x_1 \leqslant 0 \\ & g_2 = -2,5x_2 + 3x_1 \leqslant 0 \\ & g_3 = -6,8 + x_1 \leqslant 0 \end{aligned}$$

Bài 7.3: Giải bài toán sau bằng cách sử dụng các điều kiện tối ưu KKT kết hợp với phương pháp tổng trọng số:

$$\begin{aligned} \text{minimize} \quad & f_1 = (x_1 - 3)^2 + (x_2 - 7)^2 \\ & f_2 = (x_1 - 9)^2 + (x_2 - 8)^2 \end{aligned}$$

Hãy viết lời giải cho các biến thiết kế dưới dạng hàm của hai trọng số w_1 và w_2.

Sau đó giải lại bài toán bằng phương pháp ràng buộc epsilon, lần lượt chọn f_1 và f_2 làm hàm mục tiêu chính. Đưa ra nhận xét về việc các trọng số và các giá trị epsilon khác nhau ảnh hưởng như thế nào đến lời giải cuối cùng.

Bài 7.4: Giải bài toán sau bằng các điều kiện tối ưu KKT kết hợp với phương pháp tổng trọng số:

$$\begin{aligned} \text{minimize} \quad & f_1 = 20(x_1 - 0,75)^2 + (2x_2 - 2)^2 \\ & f_2 = 5(x_1 - 1,6)^2 + 2x_2 \\ \text{s.t.} \quad & g_1 = -x_2 \leqslant 0 \end{aligned}$$

Đầu tiên, hãy sử dụng $w_1 = 0,1$ và $w_2 = 0,9$. Sau đó, giải lại bài toán với $w_1 = 0,9$ và $w_2 = 0,1$. Tiếp theo, áp dụng phương pháp ràng buộc epsilon bằng cách cực tiểu hóa f_1 với ràng buộc $f_2 \leqslant \varepsilon$ cho một vài giá trị ε khác nhau. Đưa ra nhận xét về trạng thái hoạt động của ràng buộc trong mỗi trường hợp.

Bài 7.5: Thiết lập bài toán sau và giải bằng phương pháp tổng trọng số và phương pháp ràng buộc epsilon.

Xác định chiều cao và bán kính tối ưu của một hình trụ kín để đồng thời tối đa hóa thể tích và tối thiểu hóa diện tích bề mặt. Giả sử hình trụ có độ dày không đáng kể. Chiều cao phải ít nhất là 0,1 m, và bán kính phải ít nhất bằng một nửa chiều cao. Cả chiều cao và bán kính đều không được lớn hơn 2 m.

Sử dụng điểm bắt đầu là $x^{(0)} = (1, 1)$. Với phương pháp tổng trọng số, sử dụng các vectơ trọng số sau và nhận xét về lời giải: $w = (1, 0); w = (0.75, 0.25); w = (0.5, 0.5); w = (0.25, 0.75); w = (0, 1)$. Với phương pháp ràng buộc epsilon, hãy cực đại hóa thể tích trong khi khống chế diện tích bề mặt bởi một vài giới hạn cho trước, sau đó đổi vai trò hai mục tiêu và so sánh kết quả.

Bài 7.6: Một hệ giàn phẳng hai thanh có hai biến thiết kế là diện tích tiết diện A_1 và A_2. Khối lượng của hệ được viết dưới dạng $f_1(\mathbf{x}) = \rho(L_1A_1 + L_2A_2)$, còn chuyển vị tại nút chịu tải được ký hiệu là $f_2(\mathbf{x}) = |v_C(\mathbf{x})|$. Các biến thiết kế thỏa mãn $A_{\min} \leqslant A_i \leqslant A_{\max}$, $i = 1, 2$, và ứng suất trong mỗi thanh không được vượt quá σ_{allow}. Hãy thiết lập bài toán tối ưu đa mục tiêu nhằm đồng thời giảm khối lượng và giảm chuyển vị. Sau đó chuyển bài toán về dạng một mục tiêu bằng phương pháp tổng trọng số và phương pháp ràng buộc epsilon. Giải thích ý nghĩa kỹ thuật của từng nghiệm thu được trên biên Pareto.

Bài 7.7: Xét một dầm console có tiết diện chữ nhật với chiều rộng b cố định và chiều cao h là biến thiết kế. Hai mục tiêu thiết kế là cực tiểu hóa khối lượng dầm và cực tiểu hóa chuyển vị đầu tự do. Dầm phải thỏa mãn ràng buộc ứng suất uốn cho phép và giới hạn hình học $h_{\min} \leqslant h \leqslant h_{\max}$. Hãy phát biểu bài toán tối ưu đa mục tiêu, sau đó áp dụng phương pháp ràng buộc epsilon bằng cách cực tiểu hóa khối lượng với ràng buộc chuyển vị không vượt quá ε. Thay đổi ε và nhận xét sự đánh đổi giữa khối lượng và độ cứng của dầm.

CHƯƠNG 8

THUẬT TOÁN TỐI ƯU METAHEURISTIC

8.1. Giới thiệu

Thuật ngữ metaheuristic được kết hợp từ meta (vượt ra ngoài/tổng quát) và heuristic (tìm kiếm/khám phá). Về bản chất, đây là một khung thuật toán cung cấp các quy tắc mang tính xác suất để hướng dẫn quá trình tìm kiếm thoát khỏi các cực trị địa phương (local optima).

Khác với các phương pháp giải tích truyền thống vốn dựa vào đạo hàm để xác định hướng đi, metaheuristic vận hành dựa trên cơ chế thử-sai và tính ngẫu nhiên. Do đó, chúng không đảm bảo sẽ tìm ra lời giải chính xác tuyệt đối hoặc tối ưu toán học như các phương pháp truyền thống. Tuy nhiên, đối với các bài toán kỹ thuật phức tạp, nơi hàm mục tiêu có thể không liên tục, có nhiều bẫy cực trị cục bộ hoặc không thể tính được đạo hàm, metaheuristic là công cụ hiệu quả để tìm ra lời giải gần tối ưu (near-optimal) trong một khoảng thời gian tính toán chấp nhận được.

Dựa trên nguồn cảm hứng và cơ chế tìm kiếm, các thuật toán metaheuristic thường được phân thành các nhóm chính sau:

(1) Dựa trên tiến hóa sinh học (Biological Evolution Based): Nhóm này mô phỏng các nguyên lý của học thuyết Darwin về sự tiến hóa và di truyền, tiêu biểu là thuật toán di truyền (Genetic Algorithm GA) mô phỏng quá trình chọn lọc, lai ghép, đột biến; thuật toán tiến hóa vi sai (Differential Evolution DE) tập trung vào sự khác biệt giữa các cá thể.

(2) Dựa trên trí tuệ bầy đàn (Swarm Intelligence Based): Mô phỏng hành vi tập thể và tương tác xã hội để tự tổ chức tìm kiếm, như tối ưu hóa bầy đàn phần tử (Particle Swarm Optimization PSO) dựa trên cách đàn chim di chuyển; thuật toán bầy kiến (Ant Colony Optimization ACO) tìm đường qua dấu vết pheromone.

(3) Dựa trên quá trình vật lý (Physics Process Inspired): Nhóm này dựa trên các quy luật vật lý và nhiệt động lực học, điển hình là thuật toán luyện kim mô phỏng (Simulated Annealing SA) mô phỏng quá trình nung nóng và làm nguội vật liệu; thuật toán tối ưu hóa va chạm các hạt (Colliding Bodies Optimization CBO) dựa trên định luật bảo toàn động lượng.

(4) Dựa trên quá trình hóa học hoặc xã hội (Chemical or Social Process Based): Mô phỏng các phản ứng hóa học hoặc tương tác xã hội, chẳng hạn thuật toán tìm kiếm sự hòa hợp (Harmony Search HS) mô phỏng việc tinh chỉnh nhạc cụ; tối ưu hóa dựa trên việc dạy và học (Teaching Learning Based Optimization TLBO) mô phỏng sự truyền đạt kiến thức trong lớp học.

Trong khuôn khổ chương này, hai thuật toán đại diện được xem xét là thuật toán di truyền (GA) và thuật toán luyện kim mô phỏng (SA). Hai phương pháp này đại diện

cho hai hướng tiếp cận khác nhau trong tối ưu hóa: GA mô phỏng quá trình tiến hóa sinh học và thực hiện tìm kiếm dựa trên quần thể, cho phép khảo sát đồng thời nhiều vùng của không gian thiết kế; trong khi đó SA dựa trên nguyên lý nhiệt động lực học và thực hiện quá trình tìm kiếm dựa trên một lời giải duy nhất, với cơ chế dịch chuyển cục bộ để thoát khỏi cực trị địa phương (local optima). Đây là các phương pháp nền tảng, có độ ổn định cao và đã được chứng minh hiệu quả trong nhiều bài toán tối ưu hóa kết cấu thực tế.

Sự khác biệt cốt lõi giữa các thuật toán metaheuristic và phương pháp tối ưu hóa truyền thống dựa trên đạo hàm nằm ở cơ chế tìm kiếm và yêu cầu về hàm mục tiêu. Các phương pháp truyền thống có tính xác định (deterministic), hội tụ nhanh và đạt độ chính xác cao đối với các hàm mục tiêu trơn và lồi. Tuy nhiên, nhược điểm lớn nhất là phụ thuộc vào điểm xuất phát, dễ bị mắc kẹt tại cực trị địa phương nên khó tìm được lời giải tối ưu toàn cục cho các bài toán phi tuyến phức tạp.

Ngược lại, metaheuristic vận hành theo cơ chế ngẫu nhiên (stochastic), cho phép thăm dò không gian thiết kế rộng rãi mà không yêu cầu thông tin đạo hàm. Nhờ các cơ chế đặc thù như đột biến trong GA hay tiêu chuẩn Metropolis trong SA, metaheuristic có khả năng thoát khỏi các bẫy cực trị địa phương để tìm đến vùng chứa cực trị toàn cầu. Dù vậy, phương pháp này đòi hỏi chi phí tính toán lớn do phải đánh giá hàm mục tiêu nhiều lần, tốc độ hội tụ chậm ở giai đoạn cuối và cần tinh chỉnh kỹ lưỡng các thông số điều khiển (hyperparameters).

8.2. Thuật toán di truyền

8.2.1. Nguyên lý cơ bản

Bản chất của GA là quá trình sàng lọc thông tin di truyền tốt qua nhiều thế hệ:

Cá thể (individual/chromosome): Trong tối ưu hóa kết cấu, một cá thể đại diện cho một phương án thiết kế hoàn chỉnh. Nó được mã hóa thành một chuỗi (thường là nhị phân hoặc số thực) chứa các gen – chính là các thông số như kích thước tiết diện hay tọa độ nút.

Quần thể (population): Tập hợp các phương án thiết kế hiện có. Kích thước quần thể là một tham số quan trọng: nếu quá nhỏ sẽ làm mất đi sự đa dạng di truyền, nếu quá lớn sẽ gây tốn kém chi phí tính toán.

Độ thích nghi (fitness): Đây là thước đo chất lượng của một cá thể. Trong bài toán tối ưu hóa kết cấu thép, độ thích nghi thường được thiết kế tỷ lệ nghịch với hàm mục tiêu (trọng lượng) và có kèm theo mức phạt nếu vi phạm các ràng buộc về ứng suất hay độ võng.

8.2.2. Quy trình vận hành

Thuật toán tiến hóa qua các bước lặp đi lặp lại để không ngừng cải thiện chất lượng quần thể:

1. Khởi tạo và đánh giá: Một quần thể ban đầu được tạo ra ngẫu nhiên để đảm bảo tính bao quát. Mỗi cá thể sau đó được đưa vào mô hình phân tích (như mô hình phần tử hữu hạn) để tính toán độ thích nghi.

2. Chọn lọc (selection): Mục đích là chọn ra những bố mẹ tốt để truyền lại gen cho thế hệ sau. Phương pháp phổ biến nhất là vòng quay may mắn (Roulette Wheel Selection), trong đó xác suất một cá thể được chọn tỷ lệ thuận với độ thích nghi của nó:

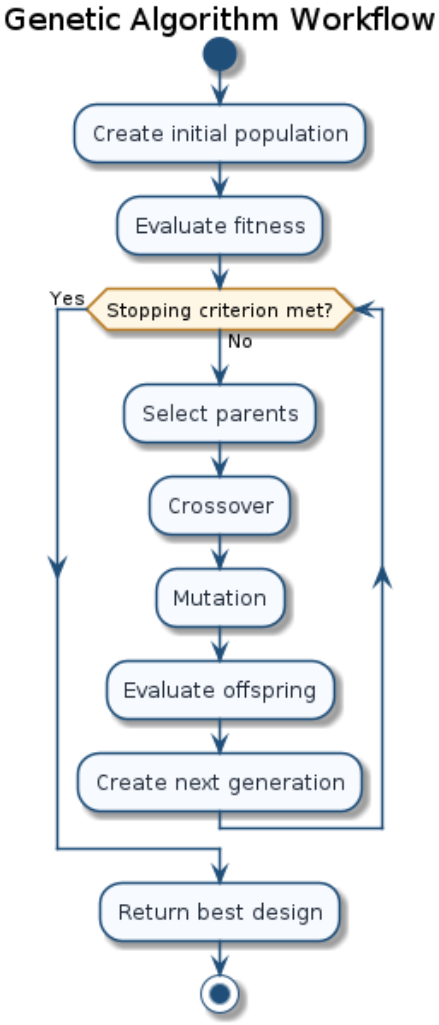
$$P(x_i) = \frac{f(x_i)}{\sum_{j=1}^{N} f(x_j)} \tag{8.1}$$

3. Lai ghép (crossover): Đây là toán tử quan trọng nhất để tạo ra các giải pháp mới bằng cách kết hợp ưu điểm của hai bố mẹ.

Lai ghép đơn điểm: Cắt hai chuỗi gen tại một vị trí ngẫu nhiên và tráo đổi các phần đuôi cho nhau.

Lai ghép đa điểm/đồng nhất: Cho phép trao đổi thông tin di truyền linh hoạt hơn giữa các cá thể con.

4. Đột biến (mutation): Với một xác suất rất nhỏ, các gen trong cá thể con sẽ bị thay đổi ngẫu nhiên (ví dụ: đổi 0 thành 1 trong mã nhị phân). Toán tử này đóng vai trò sống còn trong việc duy trì sự đa dạng, giúp quần thể không bị hội tụ quá sớm vào một cực trị địa phương và có thể "nhảy"sang những vùng không gian mới tiềm năng hơn.



Hình 8.1. Lưu đồ thuật toán di truyền.

8.3. Thuật toán luyện kim mô phỏng

8.3.1. Nguyên lý cơ bản

Trong kỹ thuật luyện kim, để đạt được một cấu trúc tinh thể bền vững và ít khuyết tật nhất (trạng thái năng lượng thấp nhất), vật liệu được nung nóng đến nhiệt độ rất cao, nơi các nguyên tử có thể di chuyển hỗn loạn ở mức năng lượng lớn. Sau đó, vật liệu được làm nguội một cách chậm rãi và có kiểm soát, cho phép các nguyên tử dần

dần sắp xếp vào các vị trí ổn định nhất.

Trong tối ưu hóa kết cấu, quá trình này được ánh xạ như sau:

Trạng thái vật chất: Tương ứng với một phương án thiết kế.

Năng lượng: Tương ứng với giá trị hàm mục tiêu (chẳng hạn trọng lượng kết cấu). Trạng thái năng lượng thấp nhất chính là lời giải tối ưu.

Nhiệt độ (T): Một tham số điều khiển mức độ chấp nhận các thay đổi ngẫu nhiên.

8.3.2. Quy trình vận hành

Chìa khóa giúp SA thoát khỏi các bẫy cực trị địa phương là khả năng chấp nhận các lời giải "tệ hơn"một cách tạm thời. Cơ chế này dựa trên tiêu chuẩn Metropolis như sau:

Nếu lời giải mới tốt hơn lời giải hiện tại ($\Delta E < 0$), thuật toán sẽ chấp nhận nó ngay lập tức.

Nếu lời giải mới tệ hơn ($\Delta E > 0$), nó vẫn có thể được chấp nhận với một xác suất P được tính theo công thức:

$$P(\Delta E) = \exp\left(-\frac{\Delta E}{kT}\right) \tag{8.2}$$

Ở nhiệt độ cao giai đoạn đầu, xác suất chấp nhận các lời giải tệ hơn là rất lớn, giúp thuật toán tự do khám phá (exploration) toàn bộ không gian thiết kế. Khi nhiệt độ giảm dần, xác suất này thấp đi, thuật toán trở nên khắt khe hơn và tập trung vào việc khai thác (exploitation) vùng lời giải tiềm năng để hội tụ về cực trị toàn cục.

8.4. Ví dụ minh họa

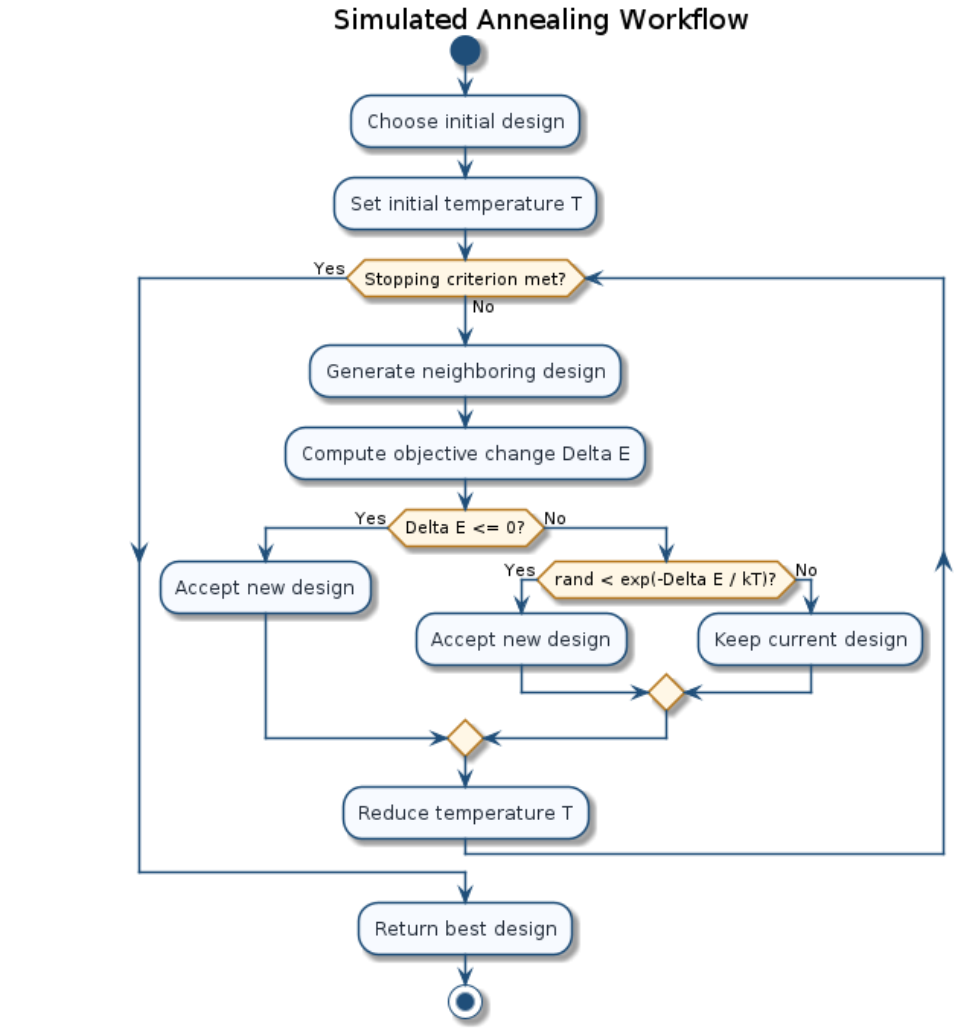
Ví dụ 8.1: Bài toán đặt ra mục tiêu tối ưu hóa khối lượng của một dầm công xôn bằng thép S270 ($E = 210$ GPa, $\sigma_{yield} = 270$ MPa) có tổng chiều dài $L = 3$ m, chịu tải trọng tập trung $F = 500$ kN tại đầu tự do. Kết cấu dầm được chia thành 5 phân đoạn bằng nhau, mỗi đoạn dài $0,6$ m với mặt cắt ngang là ống tròn rỗng, được xác định bởi 10 biến thiết kế tương ứng với bán kính trong r_{in} và bán kính ngoài r_{out} của từng đoạn. Quá trình tối ưu hóa bằng thuật toán luyện kim mô phỏng phải đảm bảo các điều kiện biên và ràng buộc kỹ thuật, bao gồm: độ võng tại đầu dầm không vượt quá 2 cm, ứng suất lớn nhất tại mọi vị trí không vượt quá giới hạn chảy của vật liệu, và kích thước hình học phải thỏa mãn tính lắp ghép (khả năng chồng lấn để hàn nối giữa các đoạn). Ngoài ra, các biến số thiết kế cần tuân thủ giới hạn thực tế với bán kính ngoài từ 1 đến 30 cm và chiều dày thành ống nằm trong khoảng $0,1$ đến 5 cm để đảm bảo tính khả thi trong chế tạo.

Dùng thuật toán luyện kim mô phỏng

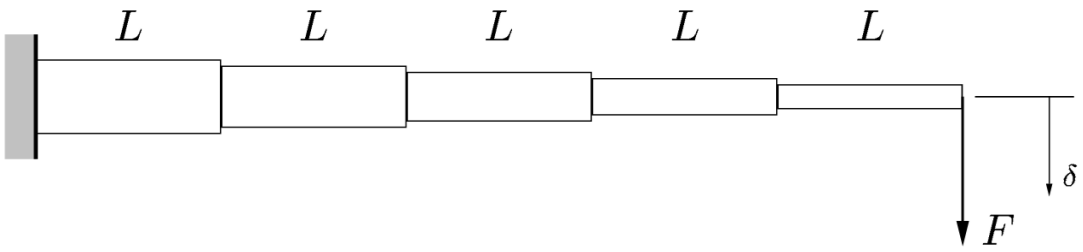
Ví dụ 8.2: Tối ưu tiết diện cho một khung thép đơn giản gồm 4 nhịp và 8 tầng ở Hình 8.6. Mô hình gồm một kết cấu khung thép có 4 nhịp, khoảng cách nhịp 5 m và 8 tầng, chiều cao tầng 3 m. Mỗi tầng chịu tải phân bố đều 40 kN/m trên mỗi nhịp. Thép cấp bền Grade 36 được sử dụng cho toàn bộ kết cấu.

Quá trình tính toán phần tử hữu hạn bằng SAP2000 OAPI với công cụ Steel Design của SAP2000 được sử dụng cho quá trình thiết kế và tối ưu. Thuật toán luyện kim mô phỏng được sử dụng trong quá trình tối ưu, và việc thiết kế được thực hiện theo phương pháp LRFD.

Các cột và dầm trong kết cấu được nhóm lại sao cho tiết diện thay đổi sau mỗi hai tầng. Do đó, tổng cộng có 4 nhóm cột và 4 nhóm dầm. Các lựa chọn tiết diện khác nhau sẽ được áp dụng cho từng nhóm, và tiết diện của các nhóm này sẽ được tối ưu trong quá trình tối ưu hóa.



Hình 8.2. Lưu đồ thuật toán luyện kim mô phỏng.

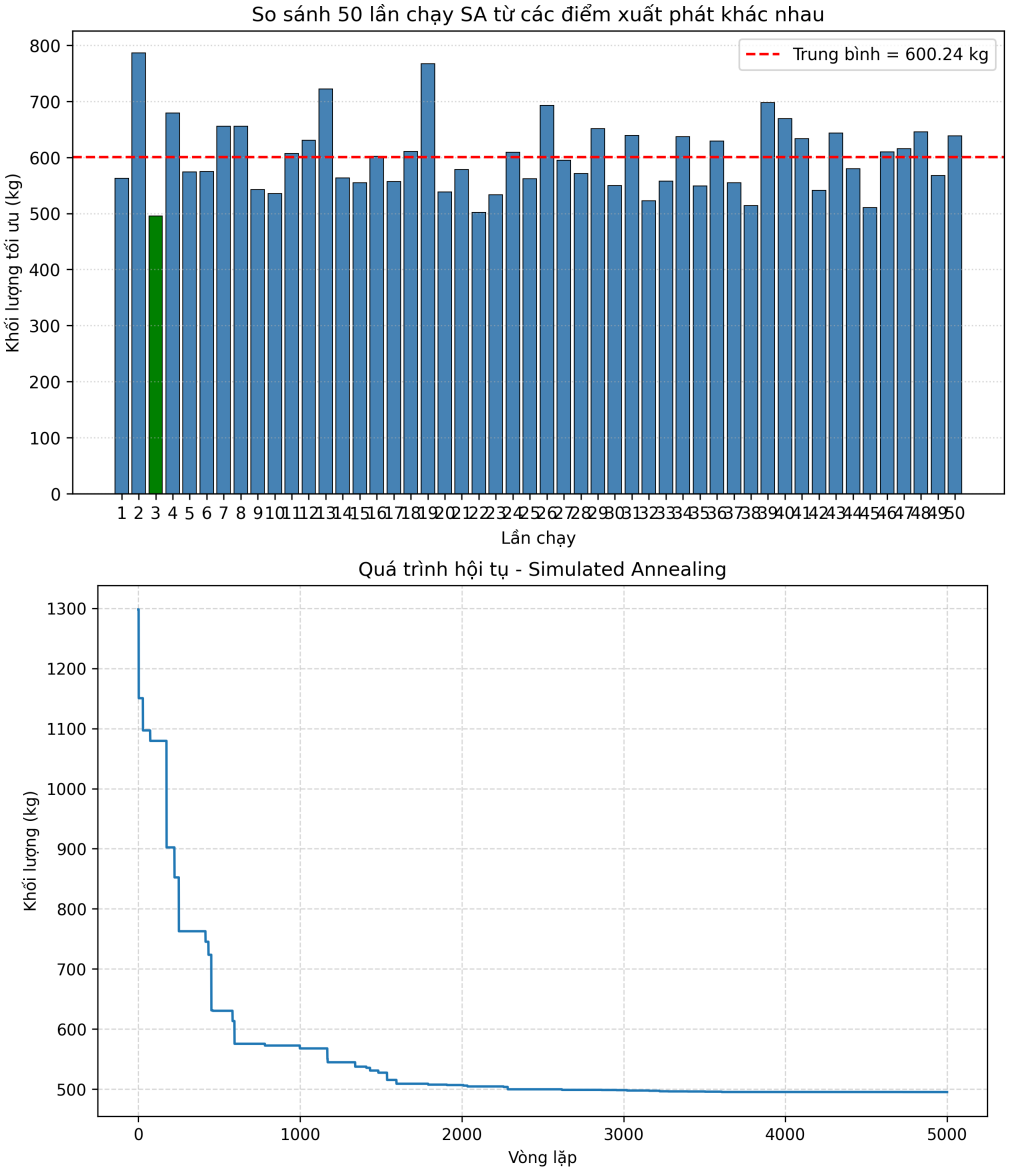


Hình 8.3.

Danh sách tiết diện sử dụng trong quá trình tối ưu gồm các tiết diện chữ W được chọn từ catalog AISC. Danh sách này bao gồm các tiết diện có kích thước từ W12 đến W33. Việc lựa chọn tiết diện phù hợp cho từng nhóm sẽ được thực hiện từ danh sách này, và tiết diện tối ưu nhất sẽ được xác định bởi thuật toán tối ưu.

Hàm mục tiêu của bài toán tối ưu là tối thiểu hóa tổng khối lượng của kết cấu

$$f(x) = \sum_{i=1}^{n} \rho_i A_i L_i \tag{8.3}$$



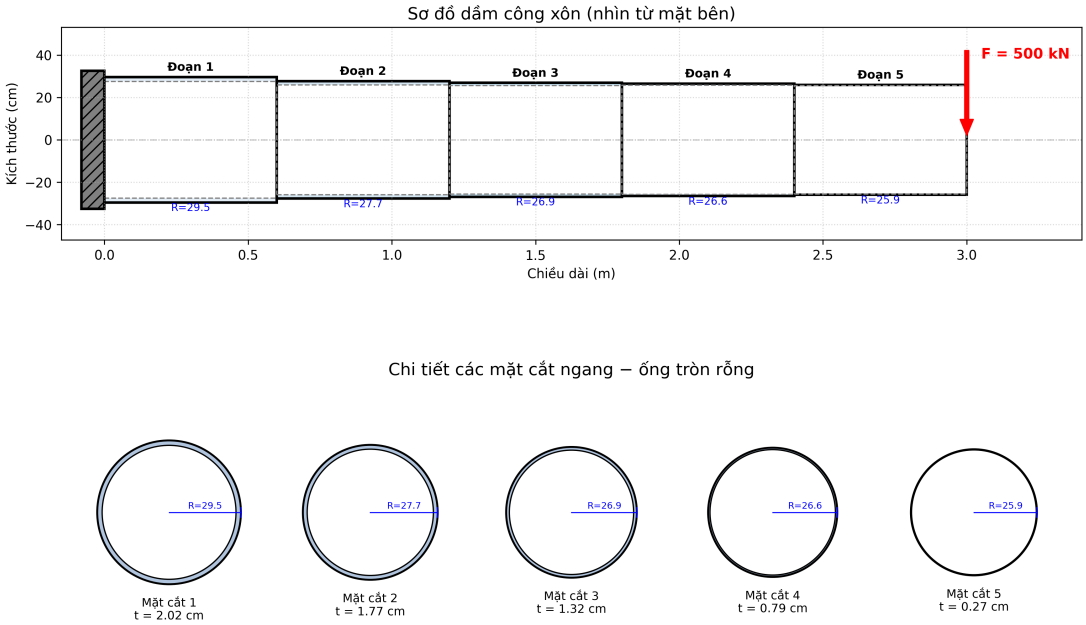
Hình 8.4. Kết quả tối ưu dầm công-xôn bằng thuật toán luyện kim mô phỏng.

trong đó, ρ_i là trọng lượng riêng của tiết diện, A_i là diện tích tiết diện và L_i là chiều dài phần tử.

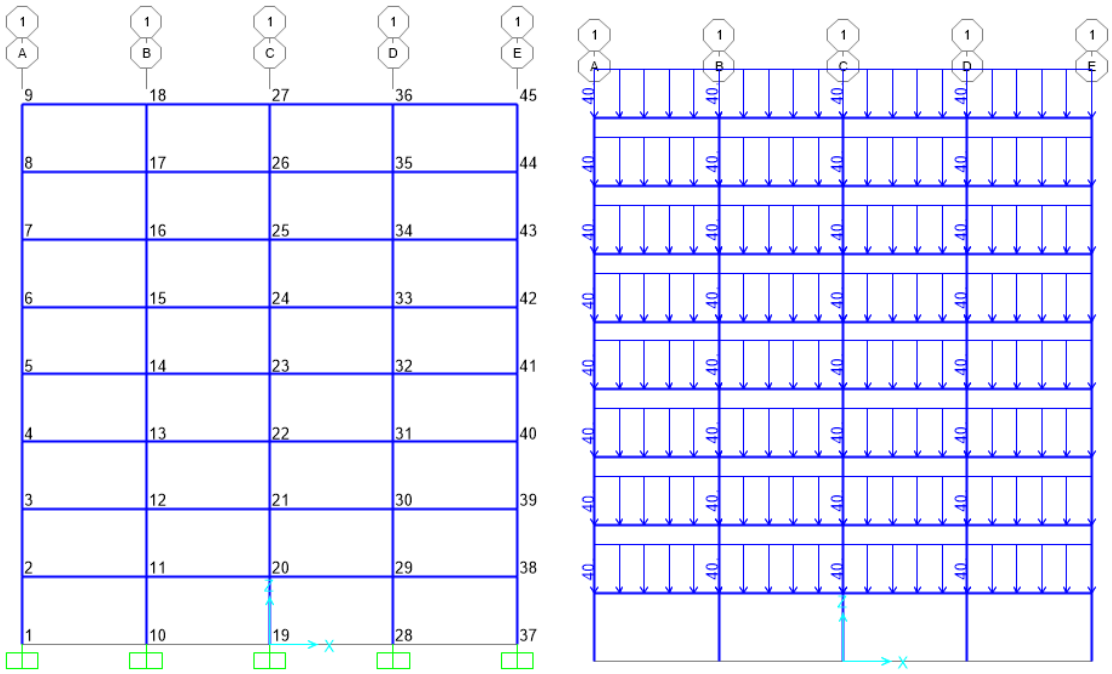
Để tính khối lượng kết cấu, một trường hợp tải có tên là weight được tạo trong mô hình. Do trường hợp tải này không chứa các lực khác, tổng phản lực gối theo phương trục z sẽ bằng trọng lượng của kết cấu.

Các ràng buộc sau được kiểm tra bằng công cụ Steel Design của SAP2000 trong quá trình tối ưu. Phương thức VerifyPassed() trả về số lượng phần tử khung không thỏa mãn yêu cầu tiêu chuẩn thiết kế. Trong nền, tất cả các kiểm tra từ chiều dài ổn định đến các phương trình tương tác tổng quát đều được thực hiện.

Như đã đề cập, thuật toán luyện kim mô phỏng là một phương pháp ngẫu nhiên. Vì vậy, việc thu được các kết quả khác nhau ở mỗi lần chạy là điều bình thường. Tuy nhiên, trong các nghiên cứu, tính ổn định của thuật toán thường được đánh giá thông qua việc thực hiện nhiều lần chạy để đánh giá tốt hơn hiệu năng tổng thể của thuật toán.

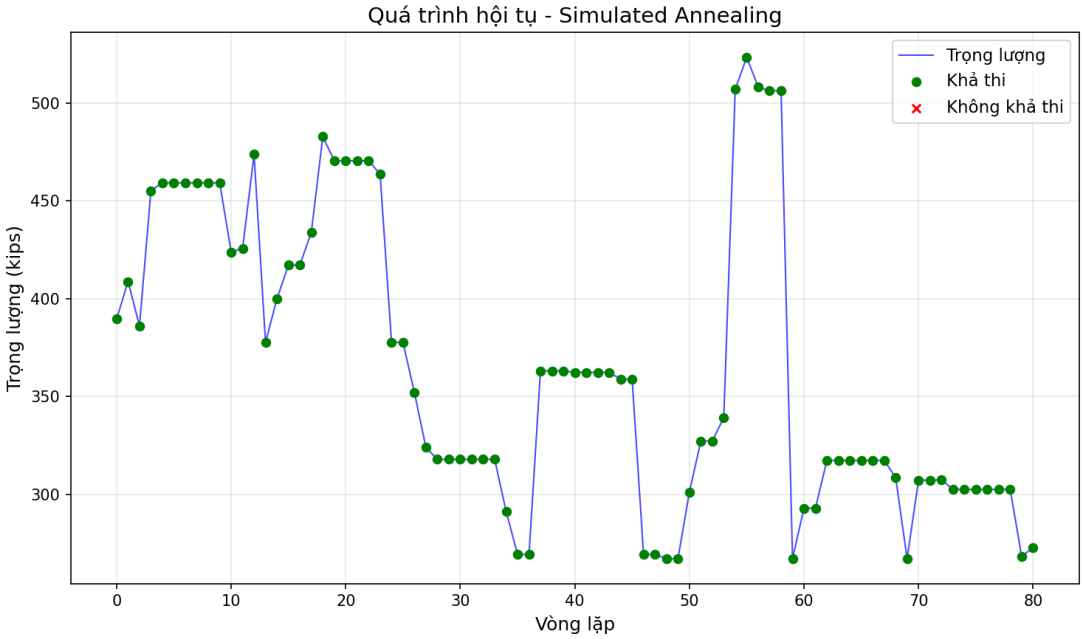


Hình 8.5. Chi tiết tiết diện tối ưu của dầm công-xôn.

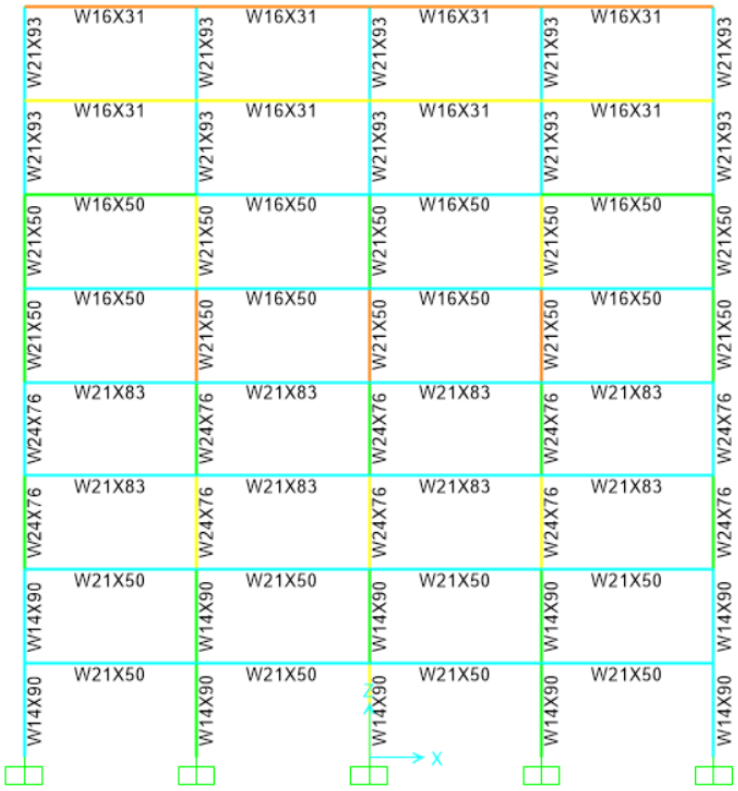


(a) Mô hình khung thép (b) Tải trọng

Hình 8.6. Mô hình khung thép và tải trọng.



Hình 8.7. Quá trình tối ưu hóa của thuật toán luyện kim mô phỏng.



Hình 8.8. Khung tối ưu.

Trong thực tế, nhiều nghiên cứu tối ưu kết cấu không sử dụng OAPI mà thay vào đó tự lập trình kiểm tra ràng buộc thiết kế. Cách tiếp cận này giúp tăng đáng kể tốc độ tính toán, bằng cách áp dụng trực tiếp các phương trình tương tác sau để kiểm tra điều kiện bền của thanh (Tiêu chuẩn AISC 360-16):

$$\begin{aligned} \frac{P_u}{\phi_c P_n} \geq 0.2; \quad c_1 &= \frac{P_u}{\phi_c P_n} + \frac{8}{9}\left(\frac{M_{ux}}{\phi_b M_{nx}} + \frac{M_{uy}}{\phi_b M_{ny}}\right) \\ \frac{P_u}{\phi_c P_n} < 0.2; \quad c_2 &= \frac{P_u}{\phi_c P_n} + \left(\frac{M_{ux}}{\phi_b M_{nx}} + \frac{M_{uy}}{\phi_b M_{ny}}\right) \\ \forall i :; c_i &\leq 1.0 \end{aligned} \tag{8.4}$$

8.5. Bài tập

Bài 8.1: Cho kết cấu giàn không gian 120 thanh dạng vòm như Hình 8.9, với mô-đun đàn hồi $E = 210$ GPa, khối lượng riêng $\rho = 7850$ kg/m^3 và giới hạn chảy $\mathrm{f}_y = 400$ MPa. Hãy xác định diện tích tiết diện tối ưu của 7 nhóm thanh (5 cm$^2 \leq \mathrm{A}_i \leq 130$ cm^2) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn ràng buộc ứng suất kéo $\sigma \leq 0.6\mathrm{f}_y$, ràng buộc mất ổn định theo tiêu chuẩn AISC cho các thanh chịu nén, và chuyển vị $[\delta] = 5$ mm tại mọi bậc tự do. Xem chi tiết trong Code 8.3.

Kiểm tra ràng buộc ứng suất và mất ổn định theo tiêu chuẩn AISC (ASD 1989):

Ứng suất kéo cho phép được xác định là $\sigma_{allow}^{tension} = 0.6 f_y$. Đối với các thanh chịu nén, ứng suất nén cho phép σ_{allow}^{comp} phụ thuộc vào hệ số mảnh $\lambda_i = \frac{kL_i}{r_i}$, trong đó L_i là chiều dài thanh, $k = 1$ là hệ số chiều dài hiệu dụng, r_i là bán kính quán tính, được xấp xỉ liên hệ với diện tích mặt cắt ngang A_i qua công thức thực nghiệm: $r_i = 0,4993 \times (A_i \times 10^4)^{0,6777}/100$.

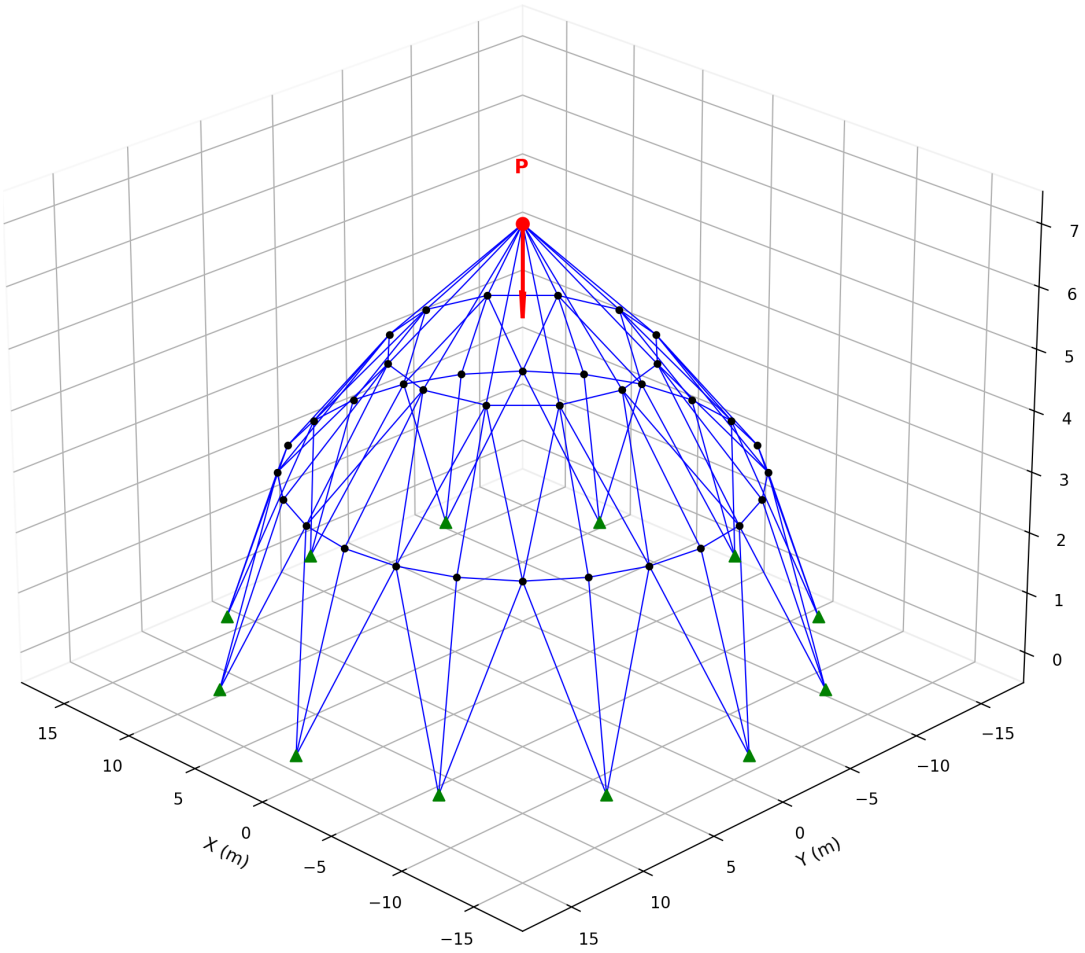
Tùy thuộc vào giá trị độ mảnh tới hạn $C_c = \sqrt{\frac{2\pi^2 E}{f_y}}$, ứng suất nén cho phép được tính theo hai trường hợp:

Nếu $\lambda_i < C_c$ (Mất ổn định phi đàn hồi): $\sigma_{allow}^{comp} = \dfrac{\left[1 - \frac{\lambda_i^2}{2C_c^2}\right] f_y}{\frac{5}{3} + \frac{3\lambda_i}{8C_c} - \frac{\lambda_i^3}{8C_c^3}}$

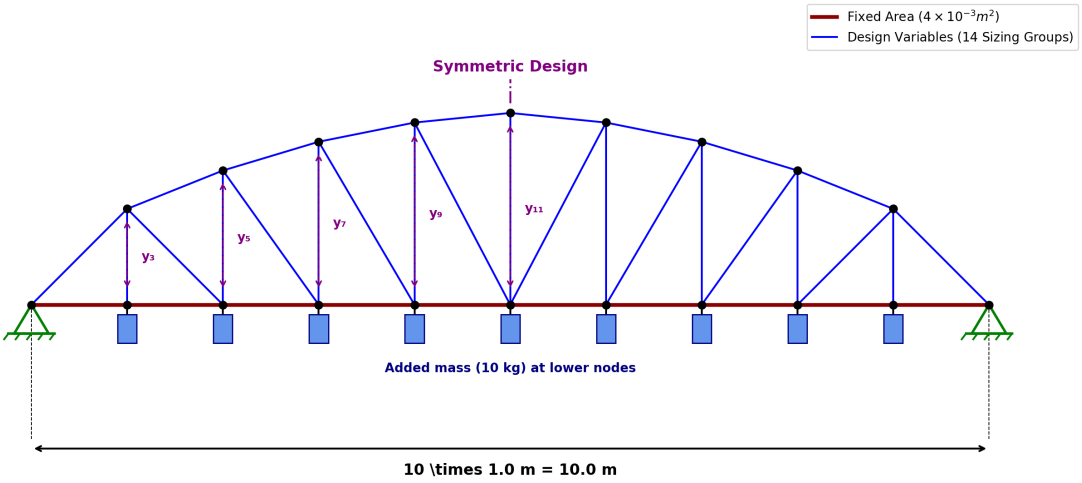
Nếu $\lambda_i \geqslant C_c$ (Mất ổn định đàn hồi Euler): $\sigma_{allow}^{comp} = \frac{12\pi^2 E}{23\lambda_i^2}$ Yêu cầu tối ưu đảm bảo ứng suất thực tế trong các thanh không vượt quá giới hạn trên $\left(\frac{|\sigma_i|}{\sigma_{allow}} \leqslant 1\right)$.

Bài 8.2: Cho kết cấu giàn 37 thanh như Hình 8.10 với mô-đun đàn hồi $E = 2.1 \times 10^{11}$ Pa, khối lượng riêng $\rho = 7800$ kg/m^3, và các khối lượng tập trung 10 kg được gắn thêm tại các nút ở mép cánh trên. Bốn thanh biên dưới (gồm 10 đoạn liên kết liên tục) có tiết diện cố định $A = 4 \times 10^{-3}$ m^2. Hãy xác định đồng thời hình dạng (tọa độ y của 5 nút cánh trên, biến thiên đối xứng trong khoảng 0.5 m $\leqslant y_i \leqslant 5.0$ m) và kích thước tiết diện tối ưu của 14 nhóm thanh còn lại (10^{-4} m$^2 \leqslant A_i \leqslant 100 \times 10^{-4}$ m^2) sao cho khối lượng kết cấu là nhỏ nhất. Quá trình thiết kế phải đảm bảo hệ kết cấu thỏa mãn ràng buộc tần số dao động riêng tại ba bậc tự do đầu tiên: $f_1 \geqslant 20$ Hz, $f_2 \geqslant 40$ Hz và $f_3 \geqslant 60$ Hz. Xem chi tiết trong Code 8.4.

Bài 8.3: Cho kết cấu giàn không gian 52 thanh dạng tháp như Hình 8.11, với mô-đun đàn hồi $E = 2.1 \times 10^{11}$ Pa, khối lượng riêng $\rho = 7800$ kg/m^3, và khối lượng tập trung 50 kg tại 13 nút phía trong tháp (3 chiều khối lượng). Hãy xác định đồng thời hình dạng (5 biến tọa độ không gian: Z_A; X_B, Z_B, X_F, Z_F của 2 vòng trung gian) và tiết diện tối ưu của 8 nhóm thanh ($10^{-4} \leqslant A_i \leqslant 10^{-3}$ m^2) sao cho khối lượng kết cấu là nhỏ nhất, thỏa mãn đồng thời hai ràng buộc tần số dao động riêng: $f_1 \leqslant 16$ Hz và

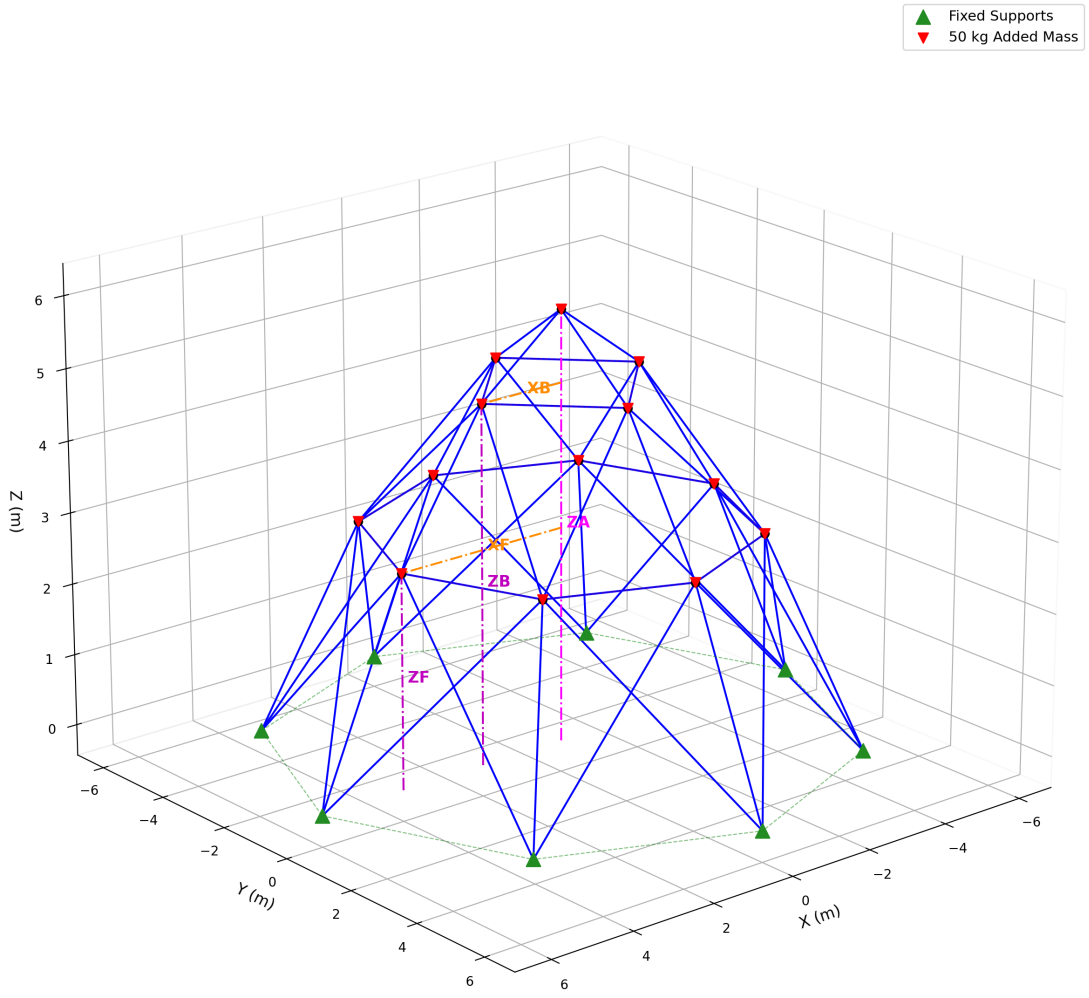


Hình 8.9.



Hình 8.10.

$f_2 \geqslant 28$ Hz. Xem chi tiết trong Code 8.5.



Hình 8.11.

Bài 8.4: Đặt 4 khối lượng 500 kg tại các nút tự do của hệ giàn Bài 5.1. Giải bài toán tối ưu với ràng buộc tần số dao động riêng: $f_1 \geq 7$ Hz, $f_2 \geq 15$ Hz, $f_3 \geq 20$ Hz.

PHỤ LỤC CODE

Tối ưu hóa kết cấu bằng phương pháp đồ thị

Code 2.1: Vẽ Hình 2.2

```
clc; clear; close all;

% --- 1. Thông số ban đầu ---
A1_opt = 0.25; % Tọa độ điểm tối ưu
A2_opt = 0.25;
L_axis = 1.2;  % Giới hạn trục

% --- 2. Khởi tạo đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 650, 550]);
hold on; axis off; axis equal;
axis([-0.1 L_axis+0.1 -0.15 L_axis+0.1]);
set(gca, 'FontSize', 12);

% --- 3. Vẽ vùng khả thi (Màu xám) ---
% Tạo biên cong và đa giác vùng khả thi
t = linspace(0, pi/2, 50);
r_curve = 0.95;
x_curve = A1_opt + r_curve * cos(t);
y_curve = (A2_opt + r_curve * sin(t)) .* (1 - 0.15*t);
fill([A1_opt, 1.2, x_curve, A1_opt, A1_opt], ...
     [A2_opt, A2_opt, y_curve, 1.2, A2_opt], ...
     [0.85 0.85 0.85], 'EdgeColor', 'none');

% --- 4. Vẽ biên ràng buộc ---
plot([A1_opt, A1_opt], [A2_opt, 1.2], 'k-', 'LineWidth', 2); % Biên dọc
plot([A1_opt, 1.2], [A2_opt, A2_opt], 'k-', 'LineWidth', 2); % Biên ngang

% --- 5. Vẽ hàm mục tiêu & Gradient ---
delta = 0.25; len_line = 0.35;
cx = A1_opt + delta; cy = A2_opt + delta;
% Đường đồng mức (dốc -1)
plot([cx - len_line, cx + len_line], [cy + len_line, cy - len_line], 'k-', 'LineWidth', 1.5);
% Vector gradient (vuông góc)
quiver(cx, cy, -0.15, -0.15, 'k', 'LineWidth', 1.2, 'MaxHeadSize', 0.8, 'AutoScale', 'off');
text(cx + 0.15, cy + 0.05, '$A_1 + A_2 = \mathrm{const.}$', 'Interpreter', 'latex', 'FontSize', 14);

% --- 6. Vẽ trục tọa độ ---
plot([0 L_axis], [0 0], 'k-', 'LineWidth', 1); % Trục hoành
text(L_axis, -0.05, '$A_1$', 'Interpreter', 'latex', 'FontSize', 14);
plot([0 0], [0 L_axis], 'k-', 'LineWidth', 1); % Trục tung
text(-0.08, L_axis, '$A_2$', 'Interpreter', 'latex', 'FontSize', 14);
% Mũi tên trục
plot(L_axis, 0, 'k>', 'MarkerFaceColor', 'k', 'MarkerSize', 5);
plot(0, L_axis, 'k^', 'MarkerFaceColor', 'k', 'MarkerSize', 5);

% --- 7. Điểm tối ưu ---
plot(A1_opt, A2_opt, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 5);
text(A1_opt + 0.02, A2_opt + 0.04, 'tối ưu', 'FontSize', 12);

% --- 8. Chú thích trục & Công thức ---
% Giống và ghi chú A1*
plot([A1_opt A1_opt], [0 -0.03], 'k-');
plot([A1_opt A1_opt], [0 A2_opt], 'k:', 'LineWidth', 0.5);
text(A1_opt - 0.1, -0.15, '$A_1^* = \frac{F \cos \alpha}{\sigma_0}$', 'Interpreter', 'latex', 'FontSize',
    ↪ 14);

```

```
% Giống và ghi chú A2*
plot([0 -0.03], [A2_opt A2_opt], 'k-');
plot([0 A1_opt], [A2_opt A2_opt], 'k:', 'LineWidth', 0.5);
text(-0.45, A2_opt, '$A_2^* = \frac{F \sin \alpha}{\sigma_0}$', 'Interpreter', 'latex', 'FontSize', 14);

hold off;saveas(gcf, 'figure_2_2.png');
```

Code 2.2: Vẽ Hình 2.5

```
rt3 = sqrt(3);
% --- 1. Thiết lập thông số & Lưới ---
x1_int = rt3/4; x2_int = 1/rt3;          % Giao điểm trục
x1_opt = rt3/7; x2_opt = rt3/7;          % Điểm tối ưu
[X1, X2] = meshgrid(linspace(1e-3, 0.9, 800), linspace(1e-3, 0.9, 800));
F = 4./(3*X1) + 1./X2;                   % Hàm mục tiêu

% --- 2. Vẽ đồ thị ---
figure('Color', 'w'); hold on; box off; pbaspect([1 1 1]);

% Miền chấp nhận (Tam giác xám)
fill([0, 0, x1_int], [0, x2_int, 0], [0.85 0.85 0.85], 'EdgeColor', 'none');

% Đường ràng buộc
x_line = linspace(0, x1_int, 200);
plot(x_line, x2_int - (4/3)*x_line, 'k', 'LineWidth', 2);

% Đường đồng mức & Nhãn
[C, h] = contour(X1, X2, F, [7, 9.43, 14], 'k', 'LineWidth', 1);
clabel(C, h, 'FontSize', 10, 'LabelSpacing', 300);

% Đánh dấu điểm tối ưu
plot(x1_opt, x2_opt, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x1_opt + 0.02, x2_opt + 0.02, 'opt.', 'FontSize', 10);

% --- 3. Định dạng trục ---
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 10);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 10);
axis([0 0.9 0 0.9]);

% Cài đặt Ticks & Font LaTeX
set(gca, 'XTick', x1_int, 'XTickLabel', {'$\frac{\sqrt{3}}{4}$'}, ...
         'YTick', x2_int, 'YTickLabel', {'$\frac{1}{\sqrt{3}}$'}, ...
         'TickLabelInterpreter', 'latex', 'FontSize', 10);
hold off;
```

Code 2.3: Vẽ Hình 2.8

```
% --- 1. Thiet lap du lieu ---
x1_left = linspace(0.001, 0.499, 200); x1_right = linspace(0.501, 1.0, 200); x1_all = linspace(0, 1, 400)
    ↪ ;

% Cac ham rang buoc & Tinh toan
sigma1 = @(x) (2.*x.*(x-1))./(1-2.*x); y_sigma1 = sigma1(x1_right);
sigma2 = @(x) (1/sqrt(2))-x;           y_sigma2 = sigma2(x1_all);
sigma3 = @(x) (2.*x.^2)./(1-2.*x);     y_sigma3 = sigma3(x1_left);

% --- 2. Ve do thi ---
figure; set(gcf, 'Position', [100, 100, 600, 525]); hold on; axis([0 1 0 1]); box on;

% Ve cac duong rang buoc (Net dam + Net dut + Text)
plot(x1_right, y_sigma1, 'k-', 'LineWidth', 2); plot(x1_right-0.01, y_sigma1, 'k:', 'LineWidth', 1);
text(0.75, 0.95, '$\sigma_1=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 12);

plot(x1_all, y_sigma2, 'k-', 'LineWidth', 2); plot(x1_all-0.01, y_sigma2-0.01, 'k:', 'LineWidth', 1);
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 12);

plot(x1_left, y_sigma3, 'k-', 'LineWidth', 2); plot(x1_left-0.005, y_sigma3+0.01, 'k:', 'LineWidth', 1);
text(0.4, 0.95, '$\sigma_3=-\sigma_0$', 'Interpreter', 'latex', 'FontSize', 12);

% Ve Diem toi uu A
x_A = (2 + sqrt(2))/4; y_A = sqrt(2)/4;
plot(x_A, y_A, 'k+', 'MarkerSize', 10, 'LineWidth', 2);
text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 14);

```

```
% Ve cac duong dong muc
C_opt = 3*x_A + y_A;
plot(x1_all, C_opt - 3*x1_all, 'k-', 'LineWidth', 0.8);
for os = [-1.5, -0.8, 0.8, 1.5], plot(x1_all, (C_opt+os) - 3*x1_all, 'k-', 'LineWidth', 0.5); end
text(0.5, 0.5, '$3x_1 + x_2 = \mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 12, 'BackgroundColor'
    ↪ , 'w');

% Nhan truc & Thiet lap cuoi cung
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);
set(gca, 'TickDir', 'in', 'Layer', 'top'); hold off;
```

Code 2.4: Vẽ Hình 2.9

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập dữ liệu ---
% Tạo lưới điểm cho từng vùng
x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2

% --- 2. Các hàm ràng buộc ---
sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
sigma2_func = @(x) (1 / sqrt(2)) - x;
sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);

% --- 3. Tính các điểm đặc biệt ---
k = 1 / sqrt(2);
x_A = k / (4*k - 1);
y_A = sigma2_func(x_A);
x_B = 0.5;
y_B = 0;

% --- 4. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
hold on; axis([0 1 0 1]); box on;

% Độ lệch nét đứt (tạo bóng)
off = 0.015;

% --- Vẽ Sigma 3 (Trái) ---
y_s3 = sigma3_func(x1_left);
mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
x3_plot = x1_left(mask_s3);
y3_plot = y_s3(mask_s3);
% 1. Đường bóng
plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2);
text(0.09, 0.92, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 2 (Thẳng) ---
y_s2 = sigma2_func(x1_all);
% 1. Đường bóng
plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2);
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 1 (Phải) ---
y_s1 = sigma1_func(x1_right);
mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
x1_plot = x1_right(mask_s1);
y1_plot = y_s1(mask_s1);
% 1. Đường bóng
plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2);
text(0.37, 0.55, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Chi tiết khác ---
% Điểm A
plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
```

```
text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');

% Điểm B
plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_B + 0.01, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');

% Đường đồng mức
contours = [1.1, 1.6, 2.1, 2.6];
for i = 1:length(contours)
    C = contours(i);
    plot(x1_all, C - 2*x1_all, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
end

% Nhãn phương trình
text(0.69, 0.75, '$2x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% Trục và Định dạng
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in');
set(gca, 'XTick', 0:0.1:1.1);
set(gca, 'YTick', 0:0.1:1.1);
set(gca, 'LineWidth', 1);
hold off;
```

Code 2.5: Vẽ Hình 2.10

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập dữ liệu ---
% Tạo lưới điểm cho từng vùng
x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2

% --- 2. Các hàm ràng buộc ---
sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
sigma2_func = @(x) (1 / sqrt(2)) - x;
sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);

% --- 3. Tính các điểm đặc biệt ---
% Điểm A: Giao của Sigma 1 & 2
k = 1 / sqrt(2);
x_A = k / (4*k - 1);
y_A = sigma2_func(x_A);

% Điểm B: Giao Sigma 1 với trục hoành
x_B = 0.5;
y_B = 0;

% --- 4. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
hold on; axis([0 1 0 1]); box on;

% Độ lệch nét đứt (tạo bóng)
off = 0.015;

% --- Vẽ Sigma 3 (Trái) ---
y_s3 = sigma3_func(x1_left);
mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
x3_plot = x1_left(mask_s3);
y3_plot = y_s3(mask_s3);
% 1. Đường bóng
plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2);
text(0.08, 0.92, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 2 (Thẳng) ---
y_s2 = sigma2_func(x1_all);
% 1. Đường bóng
plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2);
```

```
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 1 (Phải) ---
y_s1 = sigma1_func(x1_right);
mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
x1_plot = x1_right(mask_s1);
y1_plot = y_s1(mask_s1);
% 1. Đường bóng
plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1);
% 2. Đường chính
plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2);
text(0.33, 0.95, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Các chi tiết khác ---
% Điểm A
plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');

% Điểm B
plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_B + 0.02, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');

% --- Đường đồng mức (Contours) ---
C_opt = sqrt(2); % Giá trị tối ưu
contours = [C_opt, 1.8, 2.2, 2.6];

for i = 1:length(contours)
    C = contours(i);
    y_contour = C - 2*sqrt(2)*x1_all; % Phương trình contour
    plot(x1_all, y_contour, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
end

% Nhãn hàm mục tiêu
text(0.65, 0.8, '$2^{1.5}x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% Trục và Định dạng
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in');
set(gca, 'XTick', 0:0.1:1.1);
set(gca, 'YTick', 0:0.1:1.1);
set(gca, 'LineWidth', 1);

hold off;
```

Code 2.6: Vẽ Hình 2.11

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập dữ liệu ---
% Tạo lưới điểm cho từng vùng ràng buộc
x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2

% --- 2. Các hàm ràng buộc ---
sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
sigma2_func = @(x) (1 / sqrt(2)) - x;
sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);

% --- 3. Tính điểm đặc biệt ---
% Điểm A: Giao của Sigma 1 & 2
k = 1 / sqrt(2);
x_A = k / (4*k - 1);
y_A = sigma2_func(x_A);

% Điểm B: Giao Sigma 1 với trục hoành
x_B = 0.5;
y_B = 0;

% --- 4. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
hold on; axis([0 1 0 1]); box on;

```

```
% Độ lệch tạo bóng (shadow)
off = 0.015;

% --- Vẽ Sigma 3 (Trái) ---
y_s3 = sigma3_func(x1_left);
mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
x3_plot = x1_left(mask_s3);
y3_plot = y_s3(mask_s3);
plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2); % Chính
text(0.09, 0.92, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 2 (Thẳng) ---
y_s2 = sigma2_func(x1_all);
plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2); % Chính
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 1 (Phải) ---
y_s1 = sigma1_func(x1_right);
mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
x1_plot = x1_right(mask_s1);
y1_plot = y_s1(mask_s1);
plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2); % Chính
text(0.34, 0.65, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Các điểm A, B ---
plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_A + 0.02, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');

plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_B + 0.01, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');

% --- Đường đồng mức (Hàm mục tiêu: f = 4x1 + x2) ---
C_opt = 4*x_A + y_A;        % Giá trị tối ưu tại A
contours = [C_opt, 2.5, 3.1, 3.7];

for i = 1:length(contours)
    C = contours(i);
    y_contour = C - 4*x1_all; % Phương trình: x2 = C - 4*x1
    plot(x1_all, y_contour, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
end

% Nhãn hàm mục tiêu
text(0.79, 0.65, '$4x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Định dạng trục ---
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in', 'LineWidth', 1);
set(gca, 'XTick', 0:0.1:1.1, 'YTick', 0:0.1:1.1);

hold off;
```

Code 2.7: Vẽ Hình 2.12

```
% --- Khởi tạo ---
clc; clear; close all;

% --- 1. Thiết lập dữ liệu ---
% Tạo lưới điểm cho từng vùng ràng buộc
x1_left = linspace(0.001, 0.249, 200);  % Vùng Sigma 3
x1_right = linspace(0.252, 1.0, 500);   % Vùng Sigma 1
x1_all = linspace(0, 1.0, 200);         % Vùng Sigma 2

% --- 2. Các hàm ràng buộc ---
sigma1_func = @(x) (2.*x - 4.*x.^2) ./ (4.*x - 1);
sigma2_func = @(x) (1 / sqrt(2)) - x;
sigma3_func = @(x) (4 .* x.^2) ./ (1 - 4.*x);

% --- 3. Tính toán điểm đặc biệt ---
% Điểm A: Giao của Sigma 1 & Sigma 2
k = 1 / sqrt(2);
x_A = k / (4*k - 1);
```

```
y_A = sigma2_func(x_A);

% Điểm B: Giao Sigma 1 với trục hoành
x_B = 0.5;
y_B = 0;

% Điểm C: Giao Sigma 2 với trục hoành (x = 1/sqrt(2))
x_C = 1 / sqrt(2);
y_C = 0;

% --- 4. Vẽ đồ thị ---
figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 700, 600]);
hold on; axis([0 1 0 1]); box on;

% Độ lệch tạo bóng (shadow)
off = 0.015;

% --- Vẽ Sigma 3 (Trái) ---
y_s3 = sigma3_func(x1_left);
mask_s3 = (y_s3 >= -0.1) & (y_s3 <= 1.2);
x3_plot = x1_left(mask_s3);
y3_plot = y_s3(mask_s3);
plot(x3_plot - off, y3_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x3_plot, y3_plot, 'k-', 'LineWidth', 1.2); % Chính
text(0.09, 0.89, '$\sigma_3=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 2 (Thẳng) ---
y_s2 = sigma2_func(x1_all);
plot(x1_all - off/2, y_s2 - off/2, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x1_all, y_s2, 'k-', 'LineWidth', 1.2); % Chính
text(0.02, 0.72, '$\sigma_2=\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Vẽ Sigma 1 (Phải) ---
y_s1 = sigma1_func(x1_right);
mask_s1 = (y_s1 >= -0.1) & (y_s1 <= 1.2);
x1_plot = x1_right(mask_s1);
y1_plot = y_s1(mask_s1);
plot(x1_plot - off, y1_plot, ':', 'Color', [0.2 0.2 0.2], 'LineWidth', 1); % Bóng
plot(x1_plot, y1_plot, 'k-', 'LineWidth', 1.2); % Chính
text(0.34, 0.79, '$\sigma_1=2\sigma_0$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Đánh dấu các điểm A, B, C ---
plot(x_A, y_A, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_A + 0.01, y_A + 0.02, 'A', 'FontSize', 12, 'FontWeight', 'bold');

plot(x_B, y_B, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_B + 0.01, 0.02, 'B', 'FontSize', 12, 'FontWeight', 'bold');

plot(x_C, y_C, 'ko', 'MarkerSize', 4, 'MarkerFaceColor', 'k');
text(x_C + 0.01, 0.02, 'C', 'FontSize', 12, 'FontWeight', 'bold');

% --- Đường đồng mức (Hàm mục tiêu: f = x1 + x2) ---
% Hệ số góc -1, song song với Sigma 2
contours = [0.9, 1.1, 1.3, 1.5];
for i = 1:length(contours)
    C = contours(i);
    % Phương trình: x2 = C - x1
    plot(x1_all, C - x1_all, '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.5);
end

% Nhãn hàm mục tiêu
text(0.7, 0.82, '$x_1+x_2=\mathrm{const}$', 'Interpreter', 'latex', 'FontSize', 10);

% --- Định dạng trục ---
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'TickDir', 'in', 'LineWidth', 1);
set(gca, 'XTick', 0:0.1:1.1, 'YTick', 0:0.1:1.1);

hold off;
```

Code 2.8: Vẽ Hình 2.13

```
% --- Khởi tạo ---
clc; clear; close all;
```

```
3
4  % --- 1. Thiết lập lưới dữ liệu ---
5  x1_vals = linspace(0.01, 1.6, 500);
6  x2_vals = linspace(0, 5.5, 500);
7  [X1, X2] = meshgrid(x1_vals, x2_vals);
8
9  % --- 2. Tính toán ràng buộc (Constraint) ---
10 % Phương trình: (4x1^2 + 200x2^2 + 40x1x2) / (x1^2 * (2x1 + 11x2)^2) = 1
11 Numer = 4.*X1.^2 + 200.*X2.^2 + 40.*X1.*X2;
12 Denom = X1.^2 .* (2.*X1 + 11.*X2).^2;
13 Constraint = Numer ./ Denom;
14
15 % Trích xuất tọa độ đường cong từ dữ liệu contour
16 C_data = contourc(x1_vals, x2_vals, Constraint, [1 1]);
17
18 % Xử lý dữ liệu để vẽ liên tục
19 idx = 1;
20 x_curve = [];
21 y_curve = [];
22 while idx < size(C_data, 2)
23     len = C_data(2, idx);
24     x_curve = [x_curve, C_data(1, idx+1 : idx+len), NaN];
25     y_curve = [y_curve, C_data(2, idx+1 : idx+len), NaN];
26     idx = idx + len + 1;
27 end
28
29 % --- 3. Vẽ đồ thị ---
30 figure('Color', 'w', 'Units', 'pixels', 'Position', [100, 100, 650, 550]);
31 hold on; axis([0 1.5 0 5]); box on;
32
33 % --- A. Vẽ hàm mục tiêu (11x1 + x2 = C) ---
34 C_values = [2, 5, 8, 11, 14, 17];
35
36 for i = 1:length(C_values)
37     C = C_values(i);
38     y_line = C - 11 * x1_vals; % Phương trình đường thẳng
39
40     % Chỉ vẽ phần nằm trong giới hạn trục
41     mask = y_line >= 0 & y_line <= 5.5;
42     if any(mask)
43         plot(x1_vals(mask), y_line(mask), '-', 'Color', [0.5 0.5 0.5], 'LineWidth', 0.8);
44     end
45 end
46
47 % Nhãn hàm mục tiêu
48 text(0.55, 4.3, '$11x_1+x_2=\mathrm{const.}$', 'Interpreter', 'latex', 'FontSize', 11);
49
50 % --- B. Vẽ đường cong ràng buộc ---
51 off = 0.008; % Độ lệch tạo bóng
52
53 % 1. Đường bóng (Nét đứt)
54 plot(x_curve - off, y_curve, ':', 'Color', [0.4 0.4 0.4], 'LineWidth', 1);
55
56 % 2. Đường chính (Nét liền đậm)
57 plot(x_curve, y_curve, 'k-', 'LineWidth', 1.5);
58
59 % Nhãn ràng buộc
60 text(1.28, 3.5, '$\mathbf{u}^T\mathbf{u}=\delta_0^2$', 'Interpreter', 'latex', ...
61      'FontSize', 12, 'FontWeight', 'bold');
62
63 % --- 4. Định dạng trục ---
64 xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
65 ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);
66
67 % Tùy chỉnh Tick marks
68 set(gca, 'TickDir', 'in');
69 set(gca, 'XTick', 0:0.5:1.5);
70 set(gca, 'YTick', 0:0.5:5);
71 set(gca, 'LineWidth', 1, 'FontSize', 10);
72
73 hold off;
```

Code 2.9: Vẽ Hình 2.14

```
1 clear; clc; close all;
```

```

% Thiết lập cửa sổ hiển thị chính
figure('Color', 'w', 'Units', 'pixels', 'Position', [50, 50, 1000, 500]);

%% ========================================================================
% HÌNH 1: CẬN CẢNH TRUNG BÌNH (BÊN TRÁI)
% ========================================================================

subplot(1, 2, 1); hold on; box on; axis([0.95 1.1 0 0.2]);

% 1. Khởi tạo lưới dữ liệu và tính toán hàm ràng buộc
x1_vals = linspace(0.94, 1.11, 400);
x2_vals = linspace(0, 0.21, 400);
[X1, X2] = meshgrid(x1_vals, x2_vals);

% Công thức ràng buộc: (4x1^2 + 200x2^2 + 40x1x2) / (x1^2 * (2x1 + 11x2)^2)
Numer = 4.*X1.^2 + 200.*X2.^2 + 40.*X1.*X2;
Denom = X1.^2 .* (2.*X1 + 11.*X2).^2;
Constraint = Numer ./ Denom;

% Trích xuất đường đẳng mức (Level Set) tại giá trị bằng 1
C_data = contourc(x1_vals, x2_vals, Constraint, [1 1]);
idx = 1; x_curve = []; y_curve = [];
while idx < size(C_data, 2)
    len = C_data(2, idx);
    x_curve = [x_curve, C_data(1, idx+1 : idx+len), NaN];
    y_curve = [y_curve, C_data(2, idx+1 : idx+len), NaN];
    idx = idx + len + 1;
end

% 2. Vẽ các đường mức hàm mục tiêu (11x1 + x2 = C)
% Phân bổ các đường thẳng dựa trên giao điểm trục hoành
x_intercepts = 0.96 : 0.04 : 1.12;
C_vals_1 = 11 * x_intercepts;

for C = C_vals_1
    y_line = C - 11 * x1_vals;
    mask = y_line >= -0.01 & y_line <= 0.21; % Giới hạn vẽ trong khung hình
    if any(mask)
        plot(x1_vals(mask), y_line(mask), '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.8);
    end
end

% 3. Hiển thị đường ràng buộc (kèm hiệu ứng bóng đổ)
off1 = 0.002;
plot(x_curve - off1, y_curve, ':', 'Color', [0.4 0.4 0.4], 'LineWidth', 1);
plot(x_curve, y_curve, 'k-', 'LineWidth', 1.5);

% Định dạng trục và nhãn
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);
set(gca, 'TickDir', 'in', 'LineWidth', 1, 'FontSize', 12, ...
    'XTick', 0.95:0.05:1.1, 'YTick', 0:0.02:0.2);

%% ========================================================================
% HÌNH 2: CẬN CẢNH CHI TIẾT (BÊN PHẢI)
% ========================================================================

subplot(1, 2, 2); hold on; box on; axis([0.995 0.997 0.005 0.025]);

% 1. Khởi tạo lưới dữ liệu siêu mịn cho khu vực tối ưu
x1_vals_2 = linspace(0.994, 0.998, 500);
x2_vals_2 = linspace(0.004, 0.026, 500);
[X1_2, X2_2] = meshgrid(x1_vals_2, x2_vals_2);

Numer_2 = 4.*X1_2.^2 + 200.*X2_2.^2 + 40.*X1_2.*X2_2;
Denom_2 = X1_2.^2 .* (2.*X1_2 + 11.*X2_2).^2;
Constraint_2 = Numer_2 ./ Denom_2;

% Trích xuất đường cong ràng buộc mịn
C_data_2 = contourc(x1_vals_2, x2_vals_2, Constraint_2, [1 1]);
idx = 1; x_curve_2 = []; y_curve_2 = [];
while idx < size(C_data_2, 2)
    len = C_data_2(2, idx);
    x_curve_2 = [x_curve_2, C_data_2(1, idx+1 : idx+len), NaN];
```

```
    y_curve_2 = [y_curve_2, C_data_2(2, idx+1 : idx+len), NaN];
    idx = idx + len + 1;
end

% 2. Vẽ hệ thống hàm mục tiêu song song dày đặc
x_targets = 0.995 : 0.0004 : 0.998;
C_vals_2 = 11 * x_targets + 0.01545; % Căn chỉnh tại độ cao y = 0.015

for C = C_vals_2
    y_line = C - 11 * x1_vals_2;
    mask = y_line >= 0 & y_line <= 0.03;
    if any(mask)
        plot(x1_vals_2(mask), y_line(mask), '-', 'Color', [0.4 0.4 0.4], 'LineWidth', 0.8);
    end
end

% 3. Hiển thị đường ràng buộc mịn
off2 = 0.00005;

plot(x_curve_2 - off2, y_curve_2, ':', 'Color', [0.4 0.4 0.4], 'LineWidth', 1);
plot(x_curve_2, y_curve_2, 'k-', 'LineWidth', 1.5);

% 4. Đánh dấu điểm tối ưu A
x_A = 0.995259; y_A = 0.0170132;
plot(x_A, y_A, 'k+', 'MarkerSize', 9, 'LineWidth', 1.5);
text(x_A + 0.0001, y_A + 0.001, 'A', 'FontSize', 12, 'FontWeight', 'bold');

% Định dạng trục và nhãn chi tiết
xlabel('$x_1$', 'Interpreter', 'latex', 'FontSize', 14);
ylabel('$x_2$', 'Interpreter', 'latex', 'FontSize', 14);
set(gca, 'TickDir', 'in', 'LineWidth', 1, 'FontSize', 12);
xticks([0.995 0.9955 0.996 0.9965 0.997]);
yticks([0.005 0.01 0.015 0.02 0.025]);
xtickformat('%.4f');

hold off;
```

Cơ sở toán học quy hoạch lồi

Code 3.1: Vẽ Hình 3.3

```
%% TRỰC QUAN HÓA TÍNH LỒI - LÕM
clc; clear; close all;

% Dữ liệu lưới cho bề mặt 3D
[X, Y] = meshgrid(linspace(-2, 2, 60));

%% HÌNH 1: HÀM LỒI CHẶT f(x) = x^2 (Parabol)
figure('Color', 'w', 'Position', [50, 400, 500, 400]);
x1 = linspace(-2, 2, 100);
y1 = x1.^2;
plot(x1, y1, 'b-', 'LineWidth', 2.5);
grid on; box on;
title('\bf $$f(x_1) = x_1^2$$', 'Interpreter', 'latex', 'FontSize', 14);
xlabel('$$x_1$$', 'Interpreter', 'latex', 'FontSize', 12);
ylabel('$$f$$', 'Interpreter', 'latex', 'FontSize', 12);
set(gca, 'FontSize', 10);

%% HÌNH 2: HÀM LỒI (KHÔNG CHẶT)
% f(x1, x2) = x1^2 (Máng xối)
figure('Color', 'w', 'Position', [560, 400, 500, 400]);
Z1 = X.^2;
surf(X, Y, Z1, 'EdgeColor', 'none', 'FaceAlpha', 0.85);
hold on;
colormap('parula');
% Đường chỉ thị đáy phẳng (trục dọc)
plot3([0 0], [-2 2], [0 0], 'r-', 'LineWidth', 3);
view(45, 20);
axis square; grid on; box on;
camlight; lighting gouraud;
title('\bf $$f(x_1, x_2) = x_1^2$$', 'Interpreter', 'latex', 'FontSize', 14);
xlabel('$$x_1$$', 'Interpreter', 'latex');
ylabel('$$x_2$$', 'Interpreter', 'latex');
```

```
zlabel('$$f$$', 'Interpreter', 'latex');
rotate3d on;

%% HÌNH 3: HÀM KHÔNG LỒI - KHÔNG LÕM f(x1, x2) = x1 * x2 (Yên ngựa)
figure('Color', 'w', 'Position', [1070, 400, 500, 400]);
Z2 = X .* Y;
surf(X, Y, Z2, 'EdgeColor', 'none', 'FaceAlpha', 0.9);
hold on;
colormap('jet');
view(15, 15);
axis square; grid on; box on;
camlight; lighting gouraud;
title('\bf $$f(x_1, x_2) = x_1 x_2$$', 'Interpreter', 'latex', 'FontSize', 14);
xlabel('$$x_1$$', 'Interpreter', 'latex');
ylabel('$$x_2$$', 'Interpreter', 'latex');
zlabel('$$f$$', 'Interpreter', 'latex');
rotate3d on;
```

Code 3.2: Vẽ Hình 3.5 (Python)

```python
import numpy as np
import matplotlib.pyplot as plt

# --- Cấu hình Matplotlib ---
# Thiết lập font LaTeX (Computer Modern)
plt.rcParams['mathtext.fontset'] = 'cm'
plt.rcParams['font.family'] = 'serif'

# --- 1. Chuẩn bị dữ liệu ---
x = np.linspace(-0.2, 1.3, 400)
y = np.linspace(-0.2, 1.3, 400)
X, Y = np.meshgrid(x, y)
Z = X**2 + Y**2  # Hàm mục tiêu

# --- 2. Khởi tạo hình vẽ ---
fig, ax = plt.subplots(figsize=(7, 7))

# --- 3. Vẽ đồ thị ---
# Đường đồng mức (Contours)
contours = ax.contour(X, Y, Z, levels=np.arange(0, 2.6, 0.5), cmap='viridis')

# Đường ràng buộc (Constraint): x1 + x2 = 1
x_line = np.linspace(-0.2, 1.2, 100)
y_line = 1 - x_line
line_color = '#1f77b4'
ax.plot(x_line, y_line, color=line_color, linewidth=2)

# Điểm tối ưu (Optimal point)
ax.plot(0.5, 0.5, 'o', color=line_color, markersize=8, zorder=5)

# Vector Gradient (Hướng giảm)
ax.arrow(0.5, 0.5, -0.15, -0.15,
         head_width=0.04, head_length=0.06,
         fc='black', ec='black', linewidth=2, zorder=5)

# --- 4. Thêm nhãn chú thích (Annotations) ---

# Nhãn hàm mục tiêu
ax.text(0.8, 1.15, r'$g_0(\mathbf{x}) = x_1^2 + x_2^2$',
        fontsize=14, color='black',
        bbox=dict(facecolor='white', edgecolor='none', alpha=0.7))

# Nhãn ràng buộc g1(x)
ax.text(0.75, 0.3, r'$g_1(\mathbf{x}) = 1 - x_1 - x_2 \leq 0$',
        fontsize=14, color=line_color, ha='left')

# Nhãn điểm tối ưu
ax.text(0.52, 0.52, r'$\mathbf{x}^* (0.5, 0.5)$',
        fontsize=14, color=line_color,
        ha='left', va='bottom', zorder=10)

# Nhãn Gradient
ax.text(0.3, 0.3, r'$-\nabla g_0$', fontsize=13, ha='right')

# --- 5. Hoàn thiện hiển thị ---
```

```
# Thiết lập trục và lưới
ax.set_xlabel('$x_1$', fontsize=14)
ax.set_ylabel('$x_2$', fontsize=14)
ax.set_xlim([-0.2, 1.3])
ax.set_ylim([-0.2, 1.3])
ax.grid(True, linestyle='--', alpha=0.7)
ax.set_aspect('equal')

plt.tight_layout()
plt.show()
```

Code 3.3: Vẽ Hình 3.6 (Python)

```
import numpy as np
import matplotlib.pyplot as plt

# --- 1. Cấu hình hệ thống hiển thị ---
plt.rcParams['font.family'] = 'serif'
plt.rcParams['mathtext.fontset'] = 'cm'
plt.rcParams['font.size'] = 11

# --- 2. Khởi tạo dữ liệu ---
x = np.linspace(0, 11, 400)
y = np.linspace(0, 11, 400)
X, Y = np.meshgrid(x, y)
Z = (X - 5)**2 + (Y - 4)**2  # Hàm mục tiêu f(x)

# --- 3. Thiết lập khung hình ---
# Sử dụng kích thước 6x6 để đảm bảo khung hình vuông
fig, ax = plt.subplots(figsize=(6, 6), dpi=200)

# --- 4. Vẽ các thành phần hình học ---
# a) Miền khả dĩ (Feasible Region)
x_fill = np.linspace(0, 4, 100)
y_fill = (10 - x_fill)/2
ax.fill_between(x_fill, 0, y_fill, color='#d9d9d9', alpha=0.7)
ax.text(1.5, 2, 'Miền khả dĩ', fontsize=11, ha='center', color='black')

# b) Đường biên ràng buộc (Boundaries)
ax.plot(x, (10 - x)/2, 'k-', linewidth=1.2)                  # Biên g1
ax.axvline(x=4, color='k', linewidth=1.2, ymin=0, ymax=1)    # Biên g2

# c) Đường đồng mức (Contour Lines)
levels = [0.5, 2, 5, 9, 14, 20]
contours = ax.contour(X, Y, Z, levels=levels, colors='black', linewidths=0.6, alpha=0.7)

# d) Các điểm đặc biệt
# Điểm tối ưu x*
ax.plot(4, 3, 'ko', markersize=5)
ax.text(2.8, 2.6, r'$x^*(4,3)$', fontsize=11, fontweight='bold')

# Tâm hàm mục tiêu T
ax.plot(5, 4, 'ko', markersize=3)
ax.text(5.1, 4.0, r'$T(5,4)$', fontsize=11)

# --- 5. Biểu diễn Vector Gradient ---
x0, y0 = 4, 3
arrow_width = 0.008
head_width = 4

# Vector -grad(f)
ax.quiver(x0, y0, 1, 1, angles='xy', scale_units='xy', scale=1,
          color='#d62728', width=arrow_width, headwidth=head_width, label=r'$-\nabla f$')
ax.text(4.9, 3.5, r'$-\nabla f$', color='#d62728', fontsize=11)

# Vector grad(g1)
s1 = 0.6
ax.quiver(x0, y0, 1*s1, 2*s1, angles='xy', scale_units='xy', scale=1,
          color='#1f77b4', width=arrow_width, headwidth=head_width)
ax.text(4.0, 4.3, r'$\nabla g_1$', color='#1f77b4', fontsize=11)

# Vector grad(g2)
s2 = 1.2
ax.quiver(x0, y0, 1*s2, 0*s2, angles='xy', scale_units='xy', scale=1,
          color='#1f77b4', width=arrow_width, headwidth=head_width)
```

```python
ax.text(5.2, 2.6, r'$\nabla g_2$', color='#1f77b4', fontsize=11)

# --- 6. Hoàn thiện bố cục và chú thích ---
# Thiết lập giới hạn trục (vuông vức 0-11)
ax.set_xlim(0, 11)
ax.set_ylim(0, 11)

# Tên trục
ax.set_xlabel(r'$x_1$', fontsize=12, loc='right')
ax.set_ylabel(r'$x_2$', fontsize=12, loc='top')

# Nhãn phương trình
ax.text(1.5, 4.6, r'$g_1=0$', fontsize=11, rotation=0)
ax.text(6.5, 1.0, r'$g_1(x) = x_1 + 2x_2 - 10 \leq 0$', fontsize=10)
ax.text(4.2, 0.5, r'$g_2=0$', fontsize=10)
ax.text(0.5, 8.5, r'$g_2(x) = x_1 - 4 \leq 0$', fontsize=10)
ax.text(6.5, 8.5, r'$f(x) = (x_1 - 5)^2 + (x_2 - 4)^2$', fontsize=10)

# Cấu hình hiển thị trục (Center spines)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
ax.spines['left'].set_position('zero')
ax.spines['bottom'].set_position('zero')

# Thiết lập tỷ lệ 1:1 (Aspect Ratio)
ax.set_aspect('equal')

plt.tight_layout()
plt.show()
```

Code 3.4: Vẽ Hình 3.7

```
% SCRIPT: 3D_KKT_Visualization_Final.m
% MÔ TẢ:
%   Minh họa trực quan điều kiện Karush-Kuhn-Tucker (KKT) trong không gian 3D.
%   Biểu diễn điểm tiếp xúc giữa mặt cầu mục tiêu f(x) và mặt phẳng ràng buộc g(x).
%   Hiển thị tính đồng phương của các vector gradient tại điểm tối ưu.
%
%% 1. Khởi tạo môi trường và Cấu hình đồ họa
clear; clc; close all;

figure('Color', 'w', 'Position', [100, 100, 900, 800]);
hold on;
grid on; box on; axis equal;

%% 2. Định nghĩa Thông số Bài toán
% Điểm tối ưu cục bộ (Local Optimum / Stationary Point)
Opt = [1, 1, 1];

% Bán kính mặt cầu tại điểm tối ưu
R_opt = sqrt(3);

%% 3. Trực quan hóa Mặt phẳng Ràng buộc (Constraint Plane)
% Tạo lưới tọa độ cho mặt phẳng
[X_plane, Y_plane] = meshgrid(linspace(-1, 4.5, 50));

% Phương trình mặt phẳng: z = 3 - x - y
Z_plane = 3 - X_plane - Y_plane;

% Vẽ mặt phẳng ràng buộc
surf(X_plane, Y_plane, Z_plane, 'FaceColor', [0.7 0.7 0.9], ...
     'FaceAlpha', 0.5, 'EdgeColor', 'none');

% Ghi chú phương trình ràng buộc g(x)
text(-2.9, 2.4, 1.5, '$g(x): x + y + z = 3$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', 'k', 'FontWeight', 'bold');

%% 4. Trực quan hóa Hàm mục tiêu (Objective Function)
% Tạo dữ liệu mặt cầu
[Xs, Ys, Zs] = sphere(80);

% Vẽ mặt cầu mục tiêu
surf(Xs*R_opt, Ys*R_opt, Zs*R_opt, 'FaceColor', [1 0.8 0.4], ...
     'FaceAlpha', 0.4, 'EdgeColor', 'none');
```

```

% Ghi chú phương trình hàm mục tiêu f(x)
text(-1.16, -3.58, 1.82, '$f(x): x^2 + y^2 + z^2 = 3$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', [0.8 0.4 0], 'FontWeight', 'bold');

%% 5. Hiển thị Vector Gradient tại Điểm tối ưu
% Thiết lập hệ số tỉ lệ (Scaling factors) để hiển thị vector rõ ràng hơn
scale_g = 1.8;  % Hệ số tỉ lệ cho vector gradient ràng buộc
scale_f = 2.8;  % Hệ số tỉ lệ cho vector gradient hàm mục tiêu

% --- Vector Gradient của Ràng buộc (Nabla g) ---
% Vẽ vector pháp tuyến mặt phẳng (Màu đỏ)
quiver3(Opt(1), Opt(2), Opt(3), scale_g, scale_g, scale_g, ...
    'r', 'LineWidth', 4, 'MaxHeadSize', 0.5, 'AutoScale', 'off');

% Nhãn cho vector Nabla g
text(Opt(1)+scale_g, Opt(2)+scale_g, Opt(3)+scale_g - 0.2, '$\nabla g$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', 'r', 'FontWeight', 'bold');

% --- Vector Gradient của Hàm mục tiêu (Nabla f) ---
% Vẽ vector pháp tuyến mặt cầu (Màu xanh dương)
quiver3(Opt(1), Opt(2), Opt(3), scale_f, scale_f, scale_f, ...
    'b', 'LineWidth', 3, 'MaxHeadSize', 0.5, 'AutoScale', 'off');

% Nhãn cho vector Nabla f
text(Opt(1)+scale_f, Opt(2)+scale_f, Opt(3)+scale_f, '$\nabla f$', ...
    'Interpreter', 'latex', 'FontSize', 16, 'Color', 'b', 'FontWeight', 'bold');

%% 6. Cấu hình Ánh sáng và Góc nhìn (Camera & Lighting)
% Thiết lập ánh sáng Gouraud để tăng chiều sâu 3D
camlight left; camlight right;
lighting gouraud;

% Đặt góc nhìn cố định
view(48, 37);

% Thiết lập giới hạn không gian hiển thị
xlim([-1 5]); ylim([-1 5]); zlim([-1 5]);

hold off;
```

Các thuật toán xấp xỉ lồi tường minh tuần tự

Code 4.1: Quy hoạch tuyến tính tuần tự SLP

```
% TỐI ƯU HÓA BẰNG QUY HOẠCH TUYẾN TÍNH TUẦN TỰ (SLP)
% lưu ý cần Optimization Toolbox
%     min   f(x) = (x1 - 5)^2 + (x2 - 4)^2
%     s.t.  g1(x): x1 + 2*x2 - 10 <= 0
%           g2(x): x1 - 4 <= 0
clc; clear;
% 1. Khởi tạo thông số ban đầu
x = [0; 0];           % Điểm khởi tạo x0
tol = 1e-6;           % Sai số cho phép để dừng thuật toán
max_iter = 50;        % Số vòng lặp tối đa
move_limit = 0.5;     % Giới hạn dịch chuyển
% 2. Vòng lặp chính SLP (Sequential Linear Programming)
for k = 1:max_iter
    % Bước A: Tính toán giá trị tại điểm lặp hiện tại và đạo hàm
    f_val = (x(1)-5)^2 + (x(2)-4)^2;
    grad_f = [2*(x(1)-5); 2*(x(2)-4)];
    % Tính giá trị các hàm ràng buộc và ma trận đạo hàm các ràng buộc
    g_val = [x(1) + 2*x(2) - 10; x(1) - 4];
    grad_g = [1, 2; 1, 0];
    % Bước B: Thiết lập và giải bài toán con quy hoạch tuyến tính tuần tự
    lb = [-move_limit; -move_limit]; % Giới hạn dưới của d
    ub = [move_limit; move_limit];   % Giới hạn trên của d
    opts = optimoptions('linprog', 'Display', 'off');
    [d, ~, exitflag] = linprog(grad_f, grad_g, -g_val, [], [], lb, ub, opts);
    % Bước C: Cập nhật nghiệm và kiểm tra điều kiện hội tụ
    x = x + d;
    if norm(d) < tol
        break;
```

```
    end
end
% 3. Hiển thị kết quả tối ưu
fprintf('Nghiệm tối ưu x*: [%.4f; %.4f]\n', x);
fprintf('Giá trị f(x*):    %.4f\n', (x(1)-5)^2 + (x(2)-4)^2);
```

Code 4.2: Quy hoạch toàn phương tuần tự SQP

```
% TỐI ƯU HÓA BẰNG QUY HOẠCH TOÀN PHƯƠNG TUẦN TỰ (SQP)
% lưu ý cần Optimization Toolbox
%     min   f(x) = (x1 - 5)^2 + (x2 - 4)^2
%     s.t.  g1(x): x1 + 2*x2 - 10 <= 0
%           g2(x): x1 - 4 <= 0
clc; clear;
% 1. Khởi tạo thông số ban đầu
x = [0; 0];           % Điểm khởi tạo x0
tol = 1e-6;           % Sai số cho phép để dừng thuật toán
max_iter = 50;        % Số vòng lặp tối đa
H = eye(2);           % Khởi tạo ma trận Hessian
% 2. Vòng lặp chính SQP (Sequential Quadratic Programming)
for k = 1:max_iter
    % Bước A: Tính toán giá trị tại điểm lặp hiện tại và đạo hàm
    f_val = (x(1)-5)^2 + (x(2)-4)^2;
    grad_f = [2*(x(1)-5);
              2*(x(2)-4)];
    % Tính giá trị các hàm ràng buộc và ma trận đạo hàm các ràng buộc
    g_val = [x(1) + 2*x(2) - 10; x(1) - 4];
    grad_g = [1, 2; 1, 0];
    % Bước B: Thiết lập và giải bài toán con quy hoạch toàn phương
    opts = optimoptions('quadprog', 'Display', 'off');
    [d, ~, exitflag] = quadprog(H, grad_f, grad_g, -g_val, [],[],[],[],[], opts);
    % Bước C: Cập nhật nghiệm và kiểm tra điều kiện hội tụ
    x = x + d;
    if norm(d) < tol
        break;
    end
end
% 3. Hiển thị kết quả tối ưu
fprintf('Nghiệm tối ưu x*:         [%0.4f; %0.4f]\n', x);
fprintf('Giá trị hàm mục tiêu f*:  %0.4f\n', (x(1)-5)^2 + (x(2)-4)^2);
```

Code 4.3: Vẽ Hình 4.1

```
% ========================================================================
% Xấp xỉ CONLIN
% ========================================================================

clc; clear; close all;

% 1. ĐỊNH NGHĨA HÀM SỐ VÀ ĐẠO HÀM
g = @(x) x + x.^2 - (1/40).*x.^4;
dg = @(x) 1 + 2.*x - (1/10).*x.^3;

% Tạo miền giá trị x để vẽ
x = linspace(0.1, 9, 500);
y_true = g(x);

% 2. TÍNH TOÁN TẠI X1 = 1
x1 = 1;
g1 = g(x1);
dg1 = dg(x1);
gL1 = g1 + dg1 .* (x - x1);               % Tuyến tính
gR1 = g1 + dg1 .* (x1 ./ x) .* (x - x1);  % Nghịch đảo

% 3. TÍNH TOÁN TẠI X2 = 6
x2 = 6;
g2 = g(x2);
dg2 = dg(x2);
gL2 = g2 + dg2 .* (x - x2);               % Tuyến tính
gR2 = g2 + dg2 .* (x2 ./ x) .* (x - x2);  % Nghịch đảo

% 4. VẼ ĐỒ THỊ
figure('Color', 'w', 'Position', [100, 100, 800, 500]);
hold on; grid on; box on;
```

```
32
33 % --- Vẽ các đường ---
34 plot(x, y_true, 'k-', 'LineWidth', 2);     % Hàm gốc
35 plot(x, gL1, 'k--', 'LineWidth', 1.5);     % CONLIN tại x=1 (Tuyến tính)
36 plot(x, gR1, 'k:', 'LineWidth', 1.2);      % So sánh tại x=1
37 plot(x, gR2, 'b-.', 'LineWidth', 1.5);     % CONLIN tại x=6 (Nghịch đảo)
38 plot(x, gL2, 'b:', 'LineWidth', 1.2);      % So sánh tại x=6
39
40 % --- Đánh dấu điểm ---
41 plot(x1, g1, 'ro', 'MarkerFaceColor', 'r');
42 plot(x2, g2, 'ro', 'MarkerFaceColor', 'r');
43
44 % 5. TRANG TRÍ & CHÚ THÍCH (TEXT NGANG)
45 ylim([-5 18]);
46 xlim([0 9]);
47 xlabel('x');
48 ylabel('g(x)');
49
50 % Chú thích trực tiếp (Text nằm ngang mặc định)
51 text(4, 15, 'g', 'FontSize', 16, 'FontWeight', 'bold');
52
53 % Khu vực x = 1
54 text(2.6, 8.5, 'g^C = g^L', 'FontSize', 16); % Đã bỏ Rotation
55 text(2.6, 4.5, 'g^R', 'FontSize', 16);       % Đã bỏ Rotation
56
57 % Khu vực x = 6
58 text(7.2, 5, 'g^C = g^R', 'FontSize', 16);
59 text(6.8, -2, 'g^L', 'FontSize', 16);
60
61 hold off;
```

Code 4.4: Vẽ Hình 4.2

```
1  % ========================================================================
2  % Phương pháp MMA
3  % ========================================================================
4
5  clc; clear; close all;
6
7  % 1. ĐỊNH NGHĨA HÀM SỐ VÀ ĐẠO HÀM
8  g = @(x) x + x.^2 - (1/40).*x.^4;
9  dg = @(x) 1 + 2.*x - (1/10).*x.^3;
10
11 % Điểm khảo sát x0 = 1
12 x0 = 1;
13 g0 = g(x0);      % g(1) = 1.975
14 dg0 = dg(x0);    % g'(1) = 2.9
15
16 % Miền giá trị x để vẽ
17 x = linspace(-6, 6, 2000); % Tăng điểm ảnh để vẽ mượt hơn
18 y_true = g(x);
19
20 % 2. CÁC GIÁ TRỊ TIỆM CẬN TRÊN (U)
21 U_values = [1.5, 4, 20, 10000];
22 line_styles = {'--', ':', '-.', '-'};
23 line_colors = {'k', 'k', 'k', 'k'};
24
25 % 3. VẼ ĐỒ THỊ
26 figure('Color', 'w', 'Position', [100, 100, 800, 600]);
27 hold on; grid on; box on;
28
29 % Vẽ hàm gốc g(x)
30 plot(x, y_true, 'k-', 'LineWidth', 2, 'DisplayName', 'Hàm gốc g(x)');
31
32 % Vòng lặp tính và vẽ MMA
33 for i = 1:length(U_values)
34     U = U_values(i);
35
36     % Tính toán hệ số (đạo hàm dương -> dùng U)
37     p = (U - x0)^2 * dg0;
38     r = g0 - p / (U - x0);
39     g_mma = r + p ./ (U - x);
40
41     % --- XỬ LÝ RIÊNG THEO YÊU CẦU ---
```

```
    % Với U = 1.5
    if U == 1.5
        g_mma(x >= 1.5) = NaN;
    end

    % Với U = 4
        if U == 4
         g_mma(abs(x - U) < 0.05) = NaN; % Chỉ ngắt tại điểm gián đoạn
    end

    % Giới hạn hiển thị trục y để hình không bị biến dạng
    g_mma(g_mma > 50 | g_mma < -50) = NaN;

    % Thiết lập Label
    if U == 10000
        legend_label = sprintf('Xấp xỉ MMA với U^0=10^4');
    else
        legend_label = sprintf('Xấp xỉ MMA với U^0=%.1f', U);
    end

    % Vẽ đường cong
    plot(x, g_mma, line_styles{i}, 'Color', line_colors{i}, ...
         'LineWidth', 1.2, 'DisplayName', legend_label);

    % Vẽ đường tiệm cận đứng minh họa cho U=1.5 và U=4
    if U <= 4
         xline(U, 'k:', 'LineWidth', 0.5, 'HandleVisibility', 'off');
    end
end

% 4. TRANG TRÍ
xlim([-6 6]);
ylim([-10 40]);
xlabel('x');
ylabel('g(x)');

% Đánh dấu điểm x0
plot(x0, g0, 'ko', 'MarkerFaceColor', 'k', 'MarkerSize', 16, 'HandleVisibility', 'off');
text(x0, g0-3, 'x^0=1', 'HorizontalAlignment', 'center');

legend('Location', 'northwest');
hold off;
```

Tối ưu độ cứng giàn

Code 5.1: Vẽ Hình 5.4

```
% ---------------------------------------------------------------
% TỐI ƯU HÓA CẤU TRÚC GIÀN
% Phương pháp: Optimality Criteria (OC)
% ---------------------------------------------------------------
clear; clc; close all;

%% 1. CẤU HÌNH & TẠO LƯỚI
E        = 1.0;
Vol_frac = 0.10;
xmin     = 1e-9;
xmax     = 1.0;

% Thông số lưới (8x4)
nelx = 8;
nely = 4;
dx   = 1;
dy   = 1;

% Tạo tọa độ nút
[X, Y] = meshgrid(0:dx:nelx*dx, 0:dy:nely*dy);
nodes  = [X(:), Y(:)];
num_nodes = size(nodes, 1);

% Tạo phần tử thanh (Ground Structure)
bars = [];
node_map = reshape(1:num_nodes, nely+1, nelx+1);
```

```

for i = 1:nelx
    for j = 1:nely
        nBL = node_map(j, i);   nBR = node_map(j, i+1);
        nTL = node_map(j+1, i); nTR = node_map(j+1, i+1);
        % Kết nối: Ngang, Dọc, Chéo (X)
        bars = [bars; nBL nBR; nTL nTR; nBL nTL; nBR nTR; nBL nTR; nTL nBR];
    end
end

% Loại bỏ các thanh trùng lặp và sắp xếp
bars = unique(sort(bars, 2), 'rows');
num_bars = size(bars, 1);

% Tính toán đặc tính hình học (Chiều dài, Cos, Sin)
d = nodes(bars(:,2), :) - nodes(bars(:,1), :);
L = sqrt(sum(d.^2, 2));
C = d(:,1) ./ L;
S = d(:,2) ./ L;

% Hàm tính ma trận độ cứng phần tử
GetKe = @(Area, len, c, s) (E * Area / len) * ...
    [ c^2  c*s -c^2 -c*s;
      c*s  s^2 -c*s -s^2;
     -c^2 -c*s  c^2  c*s;
     -c*s -s^2  c*s  s^2];

% --- Tối ưu: Tính trước Ke đơn vị (Area=1) cho tất cả các thanh ---
Ke0_store = cell(num_bars, 1);
for i = 1:num_bars
    Ke0_store{i} = GetKe(1, L(i), C(i), S(i));
end

%% 2. ĐIỀU KIỆN BIÊN & TẢI TRỌNG
total_dof = num_nodes * 2;
tol = 1e-6; % Sai số cho phép khi tìm tọa độ

% Ngàm cứng các nút tại biên trái (x = 0)
fixed_nodes = find(abs(nodes(:,1)) < tol);
fixed_dofs  = [2*fixed_nodes-1; 2*fixed_nodes];
free_dofs   = setdiff(1:total_dof, fixed_dofs(:));

% Tải trọng tại góc dưới bên phải
load_node = find(abs(nodes(:,1) - nelx*dx) < tol & abs(nodes(:,2)) < tol);
F = zeros(total_dof, 1);
F(2*load_node) = -1;

%% 3. THIẾT LẬP HIỂN THỊ
max_iter = 200;
% Các mốc vòng lặp muốn hiển thị
loop_snapshots = [1, 20, 30, 50, 70, max_iter];

figure('Name', 'Qua trinh Toi uu hoa', 'Color', 'w', 'Position', [100 100 1000 700]);

% Khởi tạo biến thiết kế (Diện tích mặt cắt)
x = Vol_frac * xmax * ones(num_bars, 1);

%% 4. VÒNG LẶP TỐI ƯU HÓA
change = 1;
iter = 0;
move = 0.3; % Giới hạn bước nhảy (Move limit)
C_hist = []; % Lịch sử độ mềm

fprintf('%-10s %-15s %-15s\n', 'Iter', 'Compliance', 'Change');

while (change > 1e-6) && (iter < max_iter)
    iter = iter + 1;
    x_old = x;

    % --- PHÂN TÍCH FEM ---
    K = sparse(total_dof, total_dof);
    for i = 1:num_bars
        % Ke = Ke0 * x(i) (Vì vật liệu tuyến tính)
        Ke = Ke0_store{i} * x(i);

```

```
        idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
        K(idx, idx) = K(idx, idx) + Ke;
    end

    % Giải hệ phương trình cân bằng: K*u = F
    u = zeros(total_dof, 1);
    u(free_dofs) = K(free_dofs, free_dofs) \ F(free_dofs);

    % Tính độ mềm (Compliance)
    compliance = F' * u;
    C_hist = [C_hist; compliance];

    % --- TÍNH ĐỘ NHẠY (SENSITIVITY) ---
    dc = zeros(num_bars, 1);
    for i = 1:num_bars
        idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
        ue = u(idx);
        % dc = -ue^T * Ke0 * ue
        dc(i) = -ue' * Ke0_store{i} * ue;
    end

    % --- CẬP NHẬT BIẾN THIẾT KẾ (OPTIMALITY CRITERIA) ---
    l1 = 0; l2 = 1e9;
    V_limit = Vol_frac * sum(xmax * L);

    % Phương pháp chia đôi (Bisection) để tìm nhân tử Lagrange
    while (l2 - l1)/(l1 + l2) > 1e-6
        l_mid = 0.5 * (l2 + l1);
        B = -dc ./ (l_mid * L);

        % Cập nhật x với move limit và giới hạn biên
        x_new = max(xmin, max(x - move, min(xmax, min(x + move, x .* sqrt(B)))));

        if sum(x_new .* L) > V_limit
            l1 = l_mid;
        else
            l2 = l_mid;
        end
    end

    change = max(abs(x_new - x));
    x = x_new;

    % In tiến độ ra màn hình
    fprintf('%-10d %-15.4e %-15.4e\n', iter, compliance, change);

    % --- VẼ HÌNH SNAPSHOT ---
    % Vẽ nếu iter nằm trong danh sách HOẶC là vòng lặp cuối cùng (hội tụ)
    is_snapshot = ismember(iter, loop_snapshots);
    is_final    = (change <= 1e-6) || (iter == max_iter);

    if is_snapshot || is_final
        % Xác định vị trí subplot
        if is_snapshot
            idx_in_list = find(loop_snapshots == iter, 1);
        else
            % Nếu hội tụ sớm, vẽ vào ô cuối cùng
            idx_in_list = length(loop_snapshots);
        end

        if idx_in_list <= 6 % Chỉ vẽ nếu còn chỗ trong lưới 3x2
            subplot(3, 2, idx_in_list);
            cla; hold on; axis equal; axis off;
            title(sprintf('Iter: %d | C = %.2e', iter, compliance));

            % Sắp xếp để vẽ thanh nhỏ trước, thanh lớn sau
            [~, sort_ids] = sort(x);

            for k = 1:num_bars
                bar_id = sort_ids(k);
                val = x(bar_id);

                % Chỉ vẽ các thanh có diện tích đáng kể
                if val > 0.01
                    n1 = nodes(bars(bar_id,1), :);
```

```
                    n2 = nodes(bars(bar_id,2), :);

                    % Tính biến dạng dọc trục để tô màu
                    dof_idx = [2*bars(bar_id,1)-1, 2*bars(bar_id,1), ...
                               2*bars(bar_id,2)-1, 2*bars(bar_id,2)];
                    u_local = u(dof_idx);

                    axial_def = (u_local(3) - u_local(1)) * C(bar_id) + ...
                                (u_local(4) - u_local(2)) * S(bar_id);

                    if axial_def > 1e-12
                        col = 'b'; % Xanh: Kéo
                    elseif axial_def < -1e-12
                        col = 'r'; % Đỏ: Nén
                    else
                        col = [0.2 0.2 0.2]; % Xám: Không chịu lực
                    end

                    % Độ dày nét vẽ tỷ lệ với diện tích mặt cắt
                    width = 6 * val / xmax;
                    plot([n1(1) n2(1)], [n1(2) n2(2)], '-', 'Color', col, 'LineWidth', width);
                end
            end
            % Vẽ các nút cố định
            plot(nodes(fixed_nodes,1), nodes(fixed_nodes,2), 'ks', ...
                'MarkerFaceColor', 'k', 'MarkerSize', 5);
            drawnow;
        end
    end
end

fprintf('Hoan thanh! Vong lap: %d, Do mem cuoi cung: %.4e\n', iter, compliance);

%% 5. ĐỒ THỊ HỘI TỤ
figure('Name', 'Lich su hoi tu', 'Color', 'w');
plot(1:iter, C_hist, '-o', 'Color', 'b', 'LineWidth', 1.5, 'MarkerSize', 4, 'MarkerFaceColor', 'b');
grid on;
xlabel('Vòng lặp (Iterations)');
ylabel('Độ mềm (Compliance)');
title(['Hội tụ tại vòng lặp ', num2str(iter)]);
```

Code 5.2: Bản rút gọn thuật toán tối ưu độ cứng giàn

```
% TỐI ƯU HÓA CẤU TRÚC GIÀN
clear; clc; close all;

%% 1. THIẾT LẬP & TẠO LƯỚI
nelx = 8; nely = 4; Vol = 0.1; E = 1; % Thông số cơ bản
[X,Y] = meshgrid(0:nelx, 0:nely); nodes = [X(:) Y(:)];
num_nodes = size(nodes,1);
bars = []; map = reshape(1:num_nodes, nely+1, nelx+1);

% Tạo kết nối (Ground Structure)
for i=1:nelx, for j=1:nely
    n = [map(j,i) map(j,i+1); map(j+1,i) map(j+1,i+1)];
    bars = [bars; n(1,1) n(1,2); n(2,1) n(2,2); n(1,1) n(2,1); ...
            n(1,2) n(2,2); n(1,1) n(2,2); n(2,1) n(1,2)];
end; end
bars = unique(sort(bars,2),'rows'); nb = size(bars,1);

% Tính toán hình học
d = nodes(bars(:,2),:) - nodes(bars(:,1),:);
L = sqrt(sum(d.^2,2)); C = d(:,1)./L; S = d(:,2)./L;

%% 2. FEM & TẢI TRỌNG
dofs = 2*num_nodes;
fixed = find(nodes(:,1)==0);
free = setdiff(1:dofs, [2*fixed-1; 2*fixed]);
load_n = find(nodes(:,1)==nelx & nodes(:,2)==0);
F = sparse(2*load_n, 1, -1, dofs, 1);

% Hàm độ cứng phần tử
KeFunc = @(a,l,c,s) (E*a/l)*[c^2 c*s -c^2 -c*s; c*s s^2 -c*s -s^2; ...
                             -c^2 -c*s c^2 c*s; -c*s -s^2 c*s s^2];
```

```

%% 3. VÒNG LẶP TỐI ƯU (OPTIMALITY CRITERIA)
x = Vol*ones(nb,1); loop = 0; change = 1;
while change > 1e-6 && loop < 200
    loop = loop + 1; xold = x;

    % Phân tích phần tử hữu hạn
    K = sparse(dofs, dofs);
    for i=1:nb
        idx = [2*bars(i,1)-1:2*bars(i,1), 2*bars(i,2)-1:2*bars(i,2)];
        K(idx,idx) = K(idx,idx) + KeFunc(x(i), L(i), C(i), S(i));
    end
    U = zeros(dofs,1); U(free) = K(free,free) \ F(free);

    % Tính độ nhạy
    dc = zeros(nb,1);
    for i=1:nb
        idx = [2*bars(i,1)-1:2*bars(i,1), 2*bars(i,2)-1:2*bars(i,2)];
        dc(i) = -U(idx)' * KeFunc(1, L(i), C(i), S(i)) * U(idx);
    end

    % Cập nhật biến thiết kế
    l1=0; l2=1e9; move=0.2;
    while (l2-l1)/(l1+l2) > 1e-6
        lm = 0.5*(l1+l2); xnew = max(1e-9, max(x-move, min(1, min(x+move, x.*sqrt(-dc./(lm*L))))));
        if sum(xnew.*L) > Vol*sum(L), l1=lm; else, l2=lm; end
    end
    change = max(abs(xnew-x)); x = xnew;
    fprintf('Iter: %3d | Obj: %.4f | Vol: %.3f\n', loop, F'*U, sum(x.*L)/sum(L));
end

%% 4. HIỂN THỊ KẾT QUẢ
figure('Color','w'); axis equal off; hold on;
title(['Compliance: ' num2str(F'*U)]);
for i=1:nb
    if x(i) > 0.01 % Chỉ vẽ thanh có kích thước đáng kể
        plot(nodes(bars(i,:),1), nodes(bars(i,:),2), 'k', 'LineWidth', 5*x(i));
    end
end
plot(nodes(fixed,1), nodes(fixed,2), 'k^', 'MarkerFaceColor','k'); % Vẽ gối
```

Code 5.3: Vẽ Hình 5.5

```
% ------------------------------------------------------------
% TỐI ƯU HÓA CẤU TRÚC GIÀN DÙNG THUẬT TOÁN MMA
% ------------------------------------------------------------

clear; clc; close all;

%% 1. CẤU HÌNH & TẠO LƯỚI
E = 1e7;
Vol_frac = 0.10;

xmin = 1e-3;
xmax = 1.0;

% Thông số lưới (8x4)
nelx = 8; nely = 4; dx = 100; dy = 100;

[X, Y] = meshgrid(0:dx:nelx*dx, 0:dy:nely*dy);
nodes = [X(:), Y(:)];
num_nodes = size(nodes, 1);

bars = [];
node_map = reshape(1:num_nodes, nely+1, nelx+1);
for i = 1:nelx
    for j = 1:nely
        nBL = node_map(j, i);   nBR = node_map(j, i+1);
        nTL = node_map(j+1, i); nTR = node_map(j+1, i+1);
        bars = [bars; nBL nBR; nTL nTR; nBL nTL; nBR nTR; nBL nTR; nTL nBR];
    end
end
bars = unique(sort(bars, 2), 'rows');
num_bars = size(bars, 1);
```

```
32
33 dxy = nodes(bars(:,2), :) - nodes(bars(:,1), :);
34 L = sqrt(sum(dxy.^2, 2));
35 C = dxy(:,1) ./ L;
36 S = dxy(:,2) ./ L;
37
38 GetKe = @(Area, len, c, s) (E * Area / len) * ...
39     [ c^2  c*s -c^2 -c*s;  c*s  s^2 -c*s -s^2;
40      -c^2 -c*s  c^2  c*s; -c*s -s^2  c*s  s^2];
41
42 %% 2. ĐIỀU KIỆN BIÊN & TẢI TRỌNG
43 total_dof = num_nodes * 2;
44
45 fixed_nodes = find(nodes(:,1) < 1e-6);          % biên trái ngàm (fix x,y)
46 fixed_dofs  = [2*fixed_nodes-1; 2*fixed_nodes];
47 free_dofs   = setdiff(1:total_dof, fixed_dofs(:));
48
49 load_node = find(nodes(:,1)==nelx*dx & nodes(:,2)==0);
50 F = zeros(total_dof, 1);
51 F(2*load_node) = -10000;
52
53 %% 3. THIẾT LẬP HIỂN THỊ SNAPSHOTS
54 max_iter = 500;
55
56 loop_snapshots = [1, 100, 200, 300, 400, 500];
57 figure('Name', 'Toi uu hoa gian (MMA) - Snapshots', ...
58        'Color', 'w', 'Position', [50 50 1000 800]);
59
60 %% 4. THIẾT LẬP MMA
61 n = num_bars;   % số biến thiết kế
62 m = 1;          % 1 ràng buộc thể tích
63
64 x     = Vol_frac * xmax * ones(n,1);
65 xold1 = x;
66 xold2 = x;
67
68 low = xmin*ones(n,1);
69 upp = xmax*ones(n,1);
70
71 % Tham số MMA
72 a0 = 1;
73 a  = zeros(m,1);
74
75 % ---- SIẾT CHẶT: tăng "kỷ luật" ràng buộc ----
76 c_mma = 1e5*ones(m,1);     % thử 1e5; nếu cần siết hơn: 1e6
77 d_mma = zeros(m,1);
78
79 V_limit = Vol_frac * sum(xmax * L);
80
81 % Lưu lịch sử
82 C_hist = nan(max_iter,1);
83 g_hist = nan(max_iter,1);
84
85 fprintf('Dang chay toi uu hoa (MMA - siet chat)...\n');
86
87 %% 5. VÒNG LẶP TỐI ƯU HÓA (MMA)
88 C0 = [];   % scale compliance
89
90 for iter = 1:max_iter
91     % --- PHÂN TÍCH FEM ---
92     K = sparse(total_dof, total_dof);
93     for i = 1:n
94         Ke = GetKe(x(i), L(i), C(i), S(i));
95         idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
96         K(idx, idx) = K(idx, idx) + Ke;
97     end
98
99     u = zeros(total_dof, 1);
100    u(free_dofs) = K(free_dofs, free_dofs) \ F(free_dofs);
101
102    compliance = F' * u;
103
104    if iter == 1
105        C0 = max(compliance, 1e-12);
106    end
```

```
107
108     C_hist(iter) = compliance;
109
110     % --- ĐỘ NHẠY MỤC TIÊU: dC/dx ---
111     dc = zeros(n,1);
112     for i = 1:n
113         Ke0 = GetKe(1, L(i), C(i), S(i));   % tuyến tính theo Area
114         idx = [2*bars(i,1)-1, 2*bars(i,1), 2*bars(i,2)-1, 2*bars(i,2)];
115         ue = u(idx);
116         dc(i) = -ue' * Ke0 * ue;
117     end
118
119     % --- RÀNG BUỘC THỂ TÍCH: g(x) <= 0 ---
120     g = sum(x.*L)/V_limit - 1;
121     g_hist(iter) = g;
122
123     dgdx = (L / V_limit)';   % 1 x n
124
125     % --- SCALE MỤC TIÊU cho MMA ---
126     f0val = compliance / C0;
127     df0dx = dc / C0;
128
129     fval = g;                % m x 1
130     dfdx = dgdx;             % m x n
131
132     % --- SIẾT MOVE LIMIT bằng low/upp trước khi gọi MMA ---
133     move = 0.05;
134     range = move*(xmax - xmin);
135     low = max(xmin, x - range);
136     upp = min(xmax, x + range);
137
138     % --- GỌI MMA SUBPROBLEM ---
139     [xmma,~,~,~,~,~,~,~,~,low,upp] = mmasub( ...
140         m,n,iter,x,xmin*ones(n,1),xmax*ones(n,1),xold1,xold2, ...
141         f0val,df0dx,fval,dfdx,low,upp,a0,a,c_mma,d_mma);
142
143     % --- (TUỲ CHỌN) ÉP FEASIBLE MỖI VÒNG: KHÔNG VƯỢT THỂ TÍCH ---
144     if sum(xmma.*L) > V_limit
145         xmma = xmma * (V_limit / sum(xmma.*L));
146         xmma = min(xmax, max(xmin, xmma));
147     end
148
149     % --- CẬP NHẬT ---
150     xold2 = xold1;
151     xold1 = x;
152     x = xmma;
153
154     change = max(abs(x - xold1));
155
156     % --- VẼ SNAPSHOT ---
157     if ismember(iter, loop_snapshots)
158         idx_in_list = find(loop_snapshots == iter);
159         subplot(3,2,idx_in_list);
160         hold on; axis equal; axis off;
161         title(sprintf('Vòng lặp: %d | C=%.2e', iter, compliance));
162
163         [~, sort_ids] = sort(x);
164         for k = 1:n
165             bar_id = sort_ids(k);
166             val = x(bar_id);
167             if val > 0.01
168                 n1 = nodes(bars(bar_id,1), :);
169                 n2 = nodes(bars(bar_id,2), :);
170
171                 dof_idx = [2*bars(bar_id,1)-1, 2*bars(bar_id,1), 2*bars(bar_id,2)-1, 2*bars(bar_id,2)];
172                 u_local = u(dof_idx);
173                 axial_def = (u_local(3) - u_local(1)) * C(bar_id) + ...
174                             (u_local(4) - u_local(2)) * S(bar_id);
175
176                 if axial_def > 1e-12
177                     col = 'b'; % kéo
178                 elseif axial_def < -1e-12
179                     col = 'r'; % nén
180                 else
181                     col = 'k';
```

```
                end

                width = 6 * val / xmax;
                plot([n1(1) n2(1)], [n1(2) n2(2)], '-', 'Color', col, 'LineWidth', width);
            end
        end
        plot(nodes(fixed_nodes,1), nodes(fixed_nodes,2), 'ks', 'MarkerFaceColor', 'k', 'MarkerSize', 4);
        drawnow;
    end

    fprintf('iter=%3d | C=%.3e | g=%.3e | change=%.3e\n', iter, compliance, g, change);

    % --- TIÊU CHÍ DỪNG: vừa change nhỏ vừa ràng buộc gần 0 ---
    if (change < 1e-6) && (abs(g) < 1e-6)
        fprintf('Hoi tu! iter=%d | C=%.3e | g=%.3e\n', iter, compliance, g);
        break;
    end
end

iter_end = find(~isnan(C_hist), 1, 'last');
fprintf('Hoan thanh! Vong lap cuoi: %d, C: %.3e, g: %.3e\n', ...
    iter_end, C_hist(iter_end), g_hist(iter_end));
```

Function kèm theo Code 5.3: mmasub.m

```
%-------------------------------------------------------------
function [xmma,ymma,zmma,lam,xsi,eta,mu,zet,s,low,upp] = ...
mmasub(m,n,iter,xval,xmin,xmax,xold1,xold2, ...
f0val,df0dx,fval,dfdx,low,upp,a0,a,c,d);
%
%    This function mmasub performs one MMA-iteration, aimed at
%    solving the nonlinear programming problem:
%
%      Minimize  f_0(x) + a_0*z + sum( c_i*y_i + 0.5*d_i*(y_i)^2 )
%    subject to  f_i(x) - a_i*z - y_i <= 0,  i = 1,...,m
%                xmin_j <= x_j <= xmax_j,    j = 1,...,n
%                z >= 0,   y_i >= 0,         i = 1,...,m
%*** INPUT:
%
%   m    = The number of general constraints.
%   n    = The number of variables x_j.
%  iter  = Current iteration number ( =1 the first time mmasub is called).
%  xval  = Column vector with the current values of the variables x_j.
%  xmin  = Column vector with the lower bounds for the variables x_j.
%  xmax  = Column vector with the upper bounds for the variables x_j.
%  xold1 = xval, one iteration ago (provided that iter>1).
%  xold2 = xval, two iterations ago (provided that iter>2).
%  f0val = The value of the objective function f_0 at xval.
%  df0dx = Column vector with the derivatives of the objective function
%          f_0 with respect to the variables x_j, calculated at xval.
%  fval  = Column vector with the values of the constraint functions f_i,
%          calculated at xval.
%  dfdx  = (m x n)-matrix with the derivatives of the constraint functions
%          f_i with respect to the variables x_j, calculated at xval.
%          dfdx(i,j) = the derivative of f_i with respect to x_j.
%  low   = Column vector with the lower asymptotes from the previous
%          iteration (provided that iter>1).
%  upp   = Column vector with the upper asymptotes from the previous
%          iteration (provided that iter>1).
%  a0    = The constants a_0 in the term a_0*z.
%  a     = Column vector with the constants a_i in the terms a_i*z.
%  c     = Column vector with the constants c_i in the terms c_i*y_i.
%  d     = Column vector with the constants d_i in the terms 0.5*d_i*(y_i)^2.
%
%*** OUTPUT:
%
%  xmma  = Column vector with the optimal values of the variables x_j
%          in the current MMA subproblem.
%  ymma  = Column vector with the optimal values of the variables y_i
%          in the current MMA subproblem.
%  zmma  = Scalar with the optimal value of the variable z
%          in the current MMA subproblem.
%  lam   = Lagrange multipliers for the m general MMA constraints.
%  xsi   = Lagrange multipliers for the n constraints alfa_j - x_j <= 0.
%  eta   = Lagrange multipliers for the n constraints x_j - beta_j <= 0.
```

```
%   mu   = Lagrange multipliers for the m constraints -y_i <= 0.
%  zet   = Lagrange multiplier for the single constraint -z <= 0.
%   s    = Slack variables for the m general MMA constraints.
%  low   = Column vector with the lower asymptotes, calculated and used
%          in the current MMA subproblem.
%  upp   = Column vector with the upper asymptotes, calculated and used
%          in the current MMA subproblem.
%
%epsimin = sqrt(m+n)*10^(-9);
epsimin = 10^(-7);
raa0 = 0.00001;
move = 0.5;
albefa = 0.1;
asyinit = 0.5;
asyincr = 1.2;
asydecr = 0.7;
eeen = ones(n,1);
eeem = ones(m,1);
zeron = zeros(n,1);

% Calculation of the asymptotes low and upp :
if iter < 2.5
  low = xval - asyinit*(xmax-xmin);
  upp = xval + asyinit*(xmax-xmin);
else
  zzz = (xval-xold1).*(xold1-xold2);
  factor = eeen;
  factor(find(zzz > 0)) = asyincr;
  factor(find(zzz < 0)) = asydecr;
  low = xval - factor.*(xold1 - low);
  upp = xval + factor.*(upp - xold1);
  lowmin = xval - 10*(xmax-xmin);
  lowmax = xval - 0.01*(xmax-xmin);
  uppmin = xval + 0.01*(xmax-xmin);
  uppmax = xval + 10*(xmax-xmin);
  low = max(low,lowmin);
  low = min(low,lowmax);
  upp = min(upp,uppmax);
  upp = max(upp,uppmin);
end

% Calculation of the bounds alfa and beta :

zzz1 = low + albefa*(xval-low);
zzz2 = xval - move*(xmax-xmin);
zzz  = max(zzz1,zzz2);
alfa = max(zzz,xmin);
zzz1 = upp - albefa*(upp-xval);
zzz2 = xval + move*(xmax-xmin);
zzz  = min(zzz1,zzz2);
beta = min(zzz,xmax);

% Calculations of p0, q0, P, Q and b.

xmami = xmax-xmin;
xmamieps = 0.00001*eeen;
xmami = max(xmami,xmamieps);
xmamiinv = eeen./xmami;
ux1 = upp-xval;
ux2 = ux1.*ux1;
xl1 = xval-low;
xl2 = xl1.*xl1;
uxinv = eeen./ux1;
xlinv = eeen./xl1;
%
p0 = zeron;
q0 = zeron;
p0 = max(df0dx,0);
q0 = max(-df0dx,0);
pq0 = 0.001*(p0 + q0) + raa0*xmamiinv;
p0 = p0 + pq0;
q0 = q0 + pq0;
p0 = p0.*ux2;
q0 = q0.*xl2;
%
```

```
P = sparse(m,n);
Q = sparse(m,n);
P = max(dfdx,0);
Q = max(-dfdx,0);
PQ = 0.001*(P + Q) + raa0*eeem*xmamiinv';
P = P + PQ;
Q = Q + PQ;
P = P * spdiags(ux2,0,n,n);
Q = Q * spdiags(xl2,0,n,n);
b = P*uxinv + Q*xlinv - fval ;
%
%%% Solving the subproblem by a primal-dual Newton method
[xmma,ymma,zmma,lam,xsi,eta,mu,zet,s] = ...
subsolv(m,n,epsimin,low,upp,alfa,beta,p0,q0,P,Q,a0,a,b,c,d);
```

Function kèm theo Code 5.3: subsolv.m

```
%-----------------------------------------------------------
function [xmma,ymma,zmma,lamma,xsimma,etamma,mumma,zetmma,smma] = ...
subsolv(m,n,epsimin,low,upp,alfa,beta,p0,q0,P,Q,a0,a,b,c,d);
%
% This function subsolv solves the MMA subproblem:
%
% minimize   SUM[ p0j/(uppj-xj) + q0j/(xj-lowj) ] + a0*z +
%          + SUM[ ci*yi + 0.5*di*(yi)^2 ],
%
% subject to SUM[ pij/(uppj-xj) + qij/(xj-lowj) ] - ai*z - yi <= bi,
%            alfaj <=  xj <=  betaj,  yi >= 0,  z >= 0.
%
% Input:  m, n, low, upp, alfa, beta, p0, q0, P, Q, a0, a, b, c, d.
% Output: xmma,ymma,zmma, slack variables and Lagrange multiplers.
%
een = ones(n,1);
eem = ones(m,1);
epsi = 1;
epsvecn = epsi*een;
epsvecm = epsi*eem;
x = 0.5*(alfa+beta);
y = eem;
z = 1;
lam = eem;
xsi = een./(x-alfa);
xsi = max(xsi,een);
eta = een./(beta-x);
eta = max(eta,een);
mu  = max(eem,0.5*c);
zet = 1;
s = eem;
itera = 0;
while epsi > epsimin
  epsvecn = epsi*een;
  epsvecm = epsi*eem;
  ux1 = upp-x;
  xl1 = x-low;
  ux2 = ux1.*ux1;
  xl2 = xl1.*xl1;
  uxinv1 = een./ux1;
  xlinv1 = een./xl1;
  plam = p0 + P'*lam ;
  qlam = q0 + Q'*lam ;
  gvec = P*uxinv1 + Q*xlinv1;
  dpsidx = plam./ux2 - qlam./xl2 ;
  rex = dpsidx - xsi + eta;
  rey = c + d.*y - mu - lam;
  rez = a0 - zet - a'*lam;
  relam = gvec - a*z - y + s - b;
  rexsi = xsi.*(x-alfa) - epsvecn;
  reeta = eta.*(beta-x) - epsvecn;
  remu = mu.*y - epsvecm;
  rezet = zet*z - epsi;
  res = lam.*s - epsvecm;
  residu1 = [rex' rey' rez]';
  residu2 = [relam' rexsi' reeta' remu' rezet res']';
  residu = [residu1' residu2']';
  residunorm = sqrt(residu'*residu);
```

```
  residumax = max(abs(residu));
  ittt = 0;
  while residumax > 0.9*epsi & ittt < 200
    ittt=ittt + 1;
    itera=itera + 1;
    ux1 = upp-x;
    xl1 = x-low;
    ux2 = ux1.*ux1;
    xl2 = xl1.*xl1;
    ux3 = ux1.*ux2;
    xl3 = xl1.*xl2;
    uxinv1 = een./ux1;
    xlinv1 = een./xl1;
    uxinv2 = een./ux2;
    xlinv2 = een./xl2;
    plam = p0 + P'*lam ;
    qlam = q0 + Q'*lam ;
    gvec = P*uxinv1 + Q*xlinv1;
    GG = P*spdiags(uxinv2,0,n,n) - Q*spdiags(xlinv2,0,n,n);
    dpsidx = plam./ux2 - qlam./xl2 ;
    delx = dpsidx - epsvecn./(x-alfa) + epsvecn./(beta-x);
    dely = c + d.*y - lam - epsvecm./y;
    delz = a0 - a'*lam - epsi/z;
    dellam = gvec - a*z - y - b + epsvecm./lam;
    diagx = plam./ux3 + qlam./xl3;
    diagx = 2*diagx + xsi./(x-alfa) + eta./(beta-x);
    diagxinv = een./diagx;
    diagy = d + mu./y;
    diagyinv = eem./diagy;
    diaglam = s./lam;
    diaglamyi = diaglam+diagyinv;
    if m < n
      blam = dellam + dely./diagy - GG*(delx./diagx);
      bb = [blam' delz]';
      Alam = spdiags(diaglamyi,0,m,m) + GG*spdiags(diagxinv,0,n,n)*GG';
      AA = [Alam     a
            a'    -zet/z ];
      solut = AA\bb;
      dlam = solut(1:m);
      dz = solut(m+1);
      dx = -delx./diagx - (GG'*dlam)./diagx;
    else
      diaglamyiinv = eem./diaglamyi;
      dellamyi = dellam + dely./diagy;
      Axx = spdiags(diagx,0,n,n) + GG'*spdiags(diaglamyiinv,0,m,m)*GG;
      azz = zet/z + a'*(a./diaglamyi);
      axz = -GG'*(a./diaglamyi);
      bx = delx + GG'*(dellamyi./diaglamyi);
      bz  = delz - a'*(dellamyi./diaglamyi);
      AA = [Axx   axz
            axz'  azz ];
      bb = [-bx' -bz]';
      solut = AA\bb;
      dx  = solut(1:n);
      dz = solut(n+1);
      dlam = (GG*dx)./diaglamyi - dz*(a./diaglamyi) + dellamyi./diaglamyi;
    end
%
    dy = -dely./diagy + dlam./diagy;
    dxsi = -xsi + epsvecn./(x-alfa) - (xsi.*dx)./(x-alfa);
    deta = -eta + epsvecn./(beta-x) + (eta.*dx)./(beta-x);
    dmu  = -mu + epsvecm./y - (mu.*dy)./y;
    dzet = -zet + epsi/z - zet*dz/z;
    ds   = -s + epsvecm./lam - (s.*dlam)./lam;
    xx  = [ y'  z  lam'  xsi'  eta'  mu'  zet  s']';
    dxx = [dy' dz dlam' dxsi' deta' dmu' dzet ds']';
%
    stepxx = -1.01*dxx./xx;
    stmxx  = max(stepxx);
    stepalfa = -1.01*dx./(x-alfa);
    stmalfa = max(stepalfa);
    stepbeta = 1.01*dx./(beta-x);
    stmbeta = max(stepbeta);
    stmalbe  = max(stmalfa,stmbeta);
    stmalbexx = max(stmalbe,stmxx);
```

```
    stminv = max(stmalbexx,1);
    steg = 1/stminv;
%
    xold   =   x;
    yold   =   y;
    zold   =   z;
    lamold =  lam;
    xsiold =  xsi;
    etaold =  eta;
    muold  =  mu;
    zetold =  zet;
    sold   =   s;
%
    itto = 0;
    resinew = 2*residunorm;
    while resinew > residunorm & itto < 50
    itto = itto+1;
    x   =   xold + steg*dx;
    y   =   yold + steg*dy;
    z   =   zold + steg*dz;
    lam = lamold + steg*dlam;
    xsi = xsiold + steg*dxsi;
    eta = etaold + steg*deta;
    mu  = muold  + steg*dmu;
    zet = zetold + steg*dzet;
    s   =   sold + steg*ds;
    ux1 = upp-x;
    xl1 = x-low;
    ux2 = ux1.*ux1;
    xl2 = xl1.*xl1;
    uxinv1 = een./ux1;
    xlinv1 = een./xl1;
    plam = p0 + P'*lam ;
    qlam = q0 + Q'*lam ;
    gvec = P*uxinv1 + Q*xlinv1;
    dpsidx = plam./ux2 - qlam./xl2 ;
    rex = dpsidx - xsi + eta;
    rey = c + d.*y - mu - lam;
    rez = a0 - zet - a'*lam;
    relam = gvec - a*z - y + s - b;
    rexsi = xsi.*(x-alfa) - epsvecn;
    reeta = eta.*(beta-x) - epsvecn;
    remu = mu.*y - epsvecm;
    rezet = zet*z - epsi;
    res = lam.*s - epsvecm;
    residu1 = [rex' rey' rez]';
    residu2 = [relam' rexsi' reeta' remu' rezet res']';
    residu = [residu1' residu2']';
    resinew = sqrt(residu'*residu);
    steg = steg/2;
    end
  residunorm=resinew;
  residumax = max(abs(residu));
  steg = 2*steg;
  end
  if ittt > 198
    epsi
    ittt
  end
epsi = 0.1*epsi;
end
xmma   =   x;
ymma   =   y;
zmma   =   z;
lamma =  lam;
xsimma =  xsi;
etamma =  eta;
mumma  =  mu;
zetmma =  zet;
smma   =   s;
%-------------------------------------------------------------
```

Code 5.4: Giàn 10 thanh

```
function Truss_Data=TrussData_10bar
```

```
2
3  % Modulus of Elasticity (Pa)
4  E=7e10;  % 70 GPa
5
6  % Node coordinates (m)
7  % L = 10 m
8  dnkoor=10*[2 1 0;2 0 0;1 1 0;1 0 0;0 1 0;0 0 0];
9
10 % Bar connections between nodes
11 % Bar 1: 5-3   Bar 2: 3-1   Bar 3: 6-4   Bar 4: 4-2
12 % Bar 5: 4-3   Bar 6: 2-1   Bar 7: 5-4   Bar 8: 6-3
13 % Bar 9: 3-2   Bar 10: 4-1
14 eldn=[5 3;3 1;6 4;4 2;4 3;2 1;5 4;6 3;3 2;4 1];
15
16 % Support conditions (1=fixed, 0=free) [x y z]
17 % Nodes 5 and 6 are fixed supports
18 MesKos=[0 0 1;0 0 1;0 0 1;0 0 1;1 1 1;1 1 1];
19
20 % Loading (N) ---> Load(node,:)=[Fx Fy Fz]
21 % Downward loads at nodes 2 and 4
22 yuk=zeros(size(dnkoor));
23 yuk(2,:)=[0 -500000 0];  % 500 kN
24 yuk(4,:)=[0 -500000 0];  % 500 kN
25
26 % Allowable Stress (Pa)
27 maxGER=1.75e8;  % 175 MPa
28
29 % Allowed Displacement (m)
30 maxYER=0.05;  % 50 mm
31
32 % Unit weight (kg/m^3)
33 GS=2770;
34
35 % Design variable bounds (m^2)
36 LB=ones(1,10)*1e-4;    % 1 cm^2
37 UB=ones(1,10)*50e-4;    % 50 cm^2
38
39 Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
40     'E',E,'yuk',yuk,'maxGER',maxGER,'maxYER',maxYER,'GS',GS,...
41     'LB',LB,'UB',UB);
```

Code 5.5: Giàn 17 thanh

```
1  function Truss_Data=TrussData_17bar
2
3  % Modulus of Elasticity (Pa)
4  E=2.1e11;  % 210 GPa (steel)
5
6  % Node coordinates (m)
7  % Unit spacing = 2.5 m
8  dnkoor=2.5*[0 0 0; 0 1 0; 1 0 0; 1 1 0; 2 0 0; 2 1 0; 3 0 0; 3 1 0; 4 0 0];
9
10 % Bar connections between nodes
11 eldn=[2 4;2 3;1 3;3 4; 4 6;4 5;3 5;5 6;6 8;6 7;5 7;7 8;8 9;7 9;1 4;3 6;5 8];
12
13 % Support conditions (1=fixed, 0=free) [x y z]
14 % Nodes 1 and 2 are fixed supports
15 MesKos=[1 1 1;1 1 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1;0 0 1];
16
17 % Loading (N) ---> Load(node,:)=[Fx Fy Fz]
18 yuk=zeros(size(dnkoor));
19 yuk(9,:)=[0 -500000 0];  % 500 kN
20
21 % Allowable Stress (Pa)
22 maxGER=3.5e8;  % 350 MPa
23
24 % Allowed Displacement (m)
25 maxYER=0.05;  % 50 mm
26
27 % Unit weight (kg/m^3)
28 GS=7400;  % steel
29
30 % Design variable bounds (m^2)
31 LB=ones(1,17)*1e-4;    % 1 cm^2
32 UB=ones(1,17)*300e-4;  % 300 cm^2
```

```

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk',yuk,'maxGER',maxGER,'maxYER',maxYER,'GS',GS,...
    'LB',LB,'UB',UB);
```

Code 5.6: Giàn 25 thanh

```
function Truss_Data=TrussData_25bar

% Modulus of Elasticity (Pa)
E=7e10;  % 70 GPa (aluminum)

% Node coordinates (m)
dnkoor=[-1.0   0     5.0;
         1.0   0     5.0;
        -1.0   1.0   2.5;
         1.0   1.0   2.5;
         1.0  -1.0   2.5;
        -1.0  -1.0   2.5;
        -2.5   2.5   0;
         2.5   2.5   0;
         2.5  -2.5   0;
        -2.5  -2.5   0];

% Bar connections between nodes
eldn=[1 2;
      1 4;
      2 3;
      1 5;
      2 6;
      2 5;
      2 4;
      1 3;
      1 6;
      3 6;
      4 5;
      3 4;
      5 6;
      3 10;
      6 7;
      4 9;
      5 8;
      3 8;
      4 7;
      6 9;
      5 10;
      3 7;
      4 8;
      5 9;
      6 10];

% Bar grouping (8 groups, 8 design variables)
% Group 1: bar 1          (top cross)
% Group 2: bars 2-5       (top-to-mid diagonal)
% Group 3: bars 6-9       (top-to-mid diagonal)
% Group 4: bars 10-11     (mid horizontal)
% Group 5: bars 12-13     (mid horizontal)
% Group 6: bars 14-17     (mid-to-base diagonal)
% Group 7: bars 18-21     (mid-to-base diagonal)
% Group 8: bars 22-25     (mid-to-base vertical)
groups={[1],[2:5],[6:9],[10:11],[12:13],[14:17],[18:21],[22:25]};

% Support conditions (1=fixed, 0=free) [x y z]
% Nodes 7-10 are fixed supports
MesKos=[0 0 0;0 0 0;0 0 0;0 0 0;0 0 0;0 0 0;1 1 1;1 1 1;1 1 1;1 1 1];

% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
% Loading #1
yuk1=zeros(size(dnkoor));
yuk1(1,:)=[0      90000 -25000];   % 90 kN, -25 kN
yuk1(2,:)=[0     -90000 -25000];   % -90 kN, -25 kN

% Loading #2
yuk2=zeros(size(dnkoor));
yuk2(1,:)=[5000   45000 -25000];   % 5 kN, 45 kN, -25 kN
```

```
yuk2(2,:)=[0      45000 -25000];   % 45 kN, -25 kN
yuk2(3,:)=[2500   0      0];       % 2.5 kN
yuk2(6,:)=[2500   0      0];       % 2.5 kN

% Allowable Stress (Pa) - per group (recalculated for SI coordinates)
maxBAS=[220; 78; 121; 220; 220; 48; 49; 82]*1e6;  % MPa (compression)
maxCEK=275e6;  % 275 MPa (tension)

% Allowed Displacement (m)
maxYER=0.009;  % 9 mm

% Unit weight (kg/m^3)
GS=2770;  % aluminum

% Design variable bounds (m^2)
LB=ones(1,8)*0.1e-4;    % 0.1 cm^2
UB=ones(1,8)*22e-4;     % 22 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk1',yuk1,'yuk2',yuk2,...
    'maxBAS',maxBAS','maxCEK',maxCEK,'maxYER',maxYER,'GS',GS,...
    'LB',LB,'UB',UB);
```

Code 5.7: Giàn 72 thanh

```
function Truss_Data=TrussData_72bar

% Modulus of Elasticity (Pa)
E=7e10;  % 70 GPa (aluminum)

% Node coordinates (m)
% 4-story tower: each story height = 1.5 m, plan size = 3.0 x 3.0 m
dnkoor=[0   0   0;
3.0 0   0;
3.0 3.0 0;
0   3.0 0;
0   0   1.5;
3.0 0   1.5;
3.0 3.0 1.5;
0   3.0 1.5;
0   0   3.0;
3.0 0   3.0;
3.0 3.0 3.0;
0   3.0 3.0;
0   0   4.5;
3.0 0   4.5;
3.0 3.0 4.5;
0   3.0 4.5;
0   0   6.0;
3.0 0   6.0;
3.0 3.0 6.0;
0   3.0 6.0];

% Bar connections between nodes
eldn=[1 5;
2 6;
3 7;
4 8;
2 5;
1 6;
2 7;
3 6;
3 8;
4 7;
1 8;
4 5;
5 6;
6 7;
7 8;
8 5;
5 7;
6 8;
5 9;
6 10;
7 11;
```

```
8 12;
9 6;
5 10;
6 11;
7 10;
7 12;
8 11;
5 12;
8 9;
9 10;
10 11;
11 12;
12 9;
9 11;
10 12;
9 13;
10 14;
11 15;
12 16;
10 13;
9 14;
10 15;
11 14;
11 16;
12 15;
9 16;
12 13;
13 14;
14 15;
15 16;
13 16;
13 15;
14 16;
13 17;
14 18;
15 19;
16 20;
14 17;
13 18;
14 19;
15 18;
15 20;
16 19;
13 20;
16 17;
17 18;
18 19;
19 20;
20 17;
17 19;
18 20];

% Bar grouping (16 groups, 16 design variables)
% Group 1:  bars 1-4     (story 1 vertical)
% Group 2:  bars 5-12    (story 1 diagonal)
% Group 3:  bars 13-16   (story 1 horizontal)
% Group 4:  bars 17-18   (story 1 cross)
% Group 5:  bars 19-22   (story 2 vertical)
% Group 6:  bars 23-30   (story 2 diagonal)
% Group 7:  bars 31-34   (story 2 horizontal)
% Group 8:  bars 35-36   (story 2 cross)
% Group 9:  bars 37-40   (story 3 vertical)
% Group 10: bars 41-48   (story 3 diagonal)
% Group 11: bars 49-52   (story 3 horizontal)
% Group 12: bars 53-54   (story 3 cross)
% Group 13: bars 55-58   (story 4 vertical)
% Group 14: bars 59-66   (story 4 diagonal)
% Group 15: bars 67-70   (story 4 horizontal)
% Group 16: bars 71-72   (story 4 cross)
groups={[1:4],[5:12],[13:16],[17:18],...
        [19:22],[23:30],[31:34],[35:36],...
        [37:40],[41:48],[49:52],[53:54],...
        [55:58],[59:66],[67:70],[71:72]};

% Support conditions (1=fixed, 0=free) [x y z]
```

```
% Nodes 1-4 are fixed supports
MesKos=[1 1 1;
1 1 1;
1 1 1;
1 1 1;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0];

% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
% Loading #1
yuk1=zeros(size(dnkoor));
yuk1(17,:)=[25000 25000 -25000];  % 25 kN each direction

% Loading #2
yuk2=zeros(size(dnkoor));
yuk2(17,:)=[0 0 -25000];  % 25 kN downward
yuk2(18,:)=[0 0 -25000];
yuk2(19,:)=[0 0 -25000];
yuk2(20,:)=[0 0 -25000];

% Allowable Stress (Pa)
maxGER=1.75e8;  % 175 MPa

% Allowed Displacement (m)
maxYER=0.006;  % 6 mm

% Unit weight (kg/m^3)
GS=2770;  % aluminum

% Design variable bounds (m^2)
LB=ones(1,16)*1e-4;    % 1 cm^2
UB=ones(1,16)*130e-4;  % 130 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk1',yuk1,'yuk2',yuk2,...
    'maxGER',maxGER,'maxYER',maxYER,'GS',GS,...
    'groups',{groups},'LB',LB,'UB',UB);
```

Code 5.8: Giàn 200 thanh

```
function Truss_Data=TrussData_200bar

% Modulus of Elasticity (Pa)
E=2.1e11;  % 210 GPa (steel)

% Node coordinates (m)
% Planar truss (z=0), scale: 120 in = 3.0 m
S = 3.0;  % panel width (m)
H = 3.6;  % row spacing (m) = 144 in

dnkoor=[0    15*S  0;  % 1
S*2  15*S  0;  % 2
S*4  15*S  0;  % 3
S*6  15*S  0;  % 4
S*8  15*S  0;  % 5
0      15*S-H    0;  % 6
S*1  15*S-H    0;  % 7
S*2  15*S-H    0;  % 8
S*3  15*S-H    0;  % 9
S*4  15*S-H    0;  % 10
S*5  15*S-H    0;  % 11
```

```
S*6  15*S-H    0;  % 12
S*7  15*S-H    0;  % 13
S*8  15*S-H    0;  % 14
0      15*S-2*H  0;  % 15
S*2  15*S-2*H  0;  % 16
S*4  15*S-2*H  0;  % 17
S*6  15*S-2*H  0;  % 18
S*8  15*S-2*H  0;  % 19
0      15*S-3*H  0;  % 20
S*1  15*S-3*H  0;  % 21
S*2  15*S-3*H  0;  % 22
S*3  15*S-3*H  0;  % 23
S*4  15*S-3*H  0;  % 24
S*5  15*S-3*H  0;  % 25
S*6  15*S-3*H  0;  % 26
S*7  15*S-3*H  0;  % 27
S*8  15*S-3*H  0;  % 28
0      15*S-4*H  0;  % 29
S*2  15*S-4*H  0;  % 30
S*4  15*S-4*H  0;  % 31
S*6  15*S-4*H  0;  % 32
S*8  15*S-4*H  0;  % 33
0      15*S-5*H  0;  % 34
S*1  15*S-5*H  0;  % 35
S*2  15*S-5*H  0;  % 36
S*3  15*S-5*H  0;  % 37
S*4  15*S-5*H  0;  % 38
S*5  15*S-5*H  0;  % 39
S*6  15*S-5*H  0;  % 40
S*7  15*S-5*H  0;  % 41
S*8  15*S-5*H  0;  % 42
0      15*S-6*H  0;  % 43
S*2  15*S-6*H  0;  % 44
S*4  15*S-6*H  0;  % 45
S*6  15*S-6*H  0;  % 46
S*8  15*S-6*H  0;  % 47
0      15*S-7*H  0;  % 48
S*1  15*S-7*H  0;  % 49
S*2  15*S-7*H  0;  % 50
S*3  15*S-7*H  0;  % 51
S*4  15*S-7*H  0;  % 52
S*5  15*S-7*H  0;  % 53
S*6  15*S-7*H  0;  % 54
S*7  15*S-7*H  0;  % 55
S*8  15*S-7*H  0;  % 56
0      15*S-8*H  0;  % 57
S*2  15*S-8*H  0;  % 58
S*4  15*S-8*H  0;  % 59
S*6  15*S-8*H  0;  % 60
S*8  15*S-8*H  0;  % 61
0      15*S-9*H  0;  % 62
S*1  15*S-9*H  0;  % 63
S*2  15*S-9*H  0;  % 64
S*3  15*S-9*H  0;  % 65
S*4  15*S-9*H  0;  % 66
S*5  15*S-9*H  0;  % 67
S*6  15*S-9*H  0;  % 68
S*7  15*S-9*H  0;  % 69
S*8  15*S-9*H  0;  % 70
0      15*S-10*H 0;  % 71
S*2  15*S-10*H 0;  % 72
S*4  15*S-10*H 0;  % 73
S*6  15*S-10*H 0;  % 74
S*8  15*S-10*H 0;  % 75
S*2  0         0;  % 76 (support)
S*6  0         0]; % 77 (support)

% Bar connections between nodes
eldn=[1 2;
2 3;
3 4;
4 5;
1 6;
1 7;
2 7;
```

```
2 8;
2 9;
3 9;
3 10;
3 11;
4 11;
4 12;
4 13;
5 13;
5 14;
6 7;
7 8;
8 9;
9 10;
10 11;
11 12;
12 13;
13 14;
6 15;
7 15;
7 16;
8 16;
9 16;
9 17;
10 17;
11 17;
11 18;
12 18;
13 18;
13 19;
14 19;
15 16;
16 17;
17 18;
18 19;
15 20;
15 21;
16 21;
16 22;
16 23;
17 23;
17 24;
17 25;
18 25;
18 26;
18 27;
19 27;
19 28;
20 21;
21 22;
21 23;
23 24;
24 25;
25 26;
26 27;
27 28;
20 29;
21 29;
21 30;
22 30;
23 30;
23 31;
24 31;
25 31;
25 32;
26 32;
27 32;
27 33;
28 33;
29 30;
30 31;
31 32;
32 33;
29 34;
29 35;
```

```
30 35;
30 36;
30 37;
31 37;
31 38;
31 39;
32 39;
32 40;
32 41;
33 41;
33 42;
34 35;
35 36;
36 37;
37 38;
38 39;
39 40;
40 41;
41 42;
34 43;
35 43;
35 44;
36 44;
37 44;
37 45;
38 45;
39 45;
39 46;
40 46;
41 46;
41 47;
42 47;
43 44;
44 45;
45 46;
46 47;
43 48;
43 49;
44 49;
44 50;
44 51;
45 51;
45 52;
45 53;
46 53;
46 54;
46 55;
47 55;
47 56;
48 49;
49 50;
50 51;
51 52;
52 53;
53 54;
54 55;
55 56;
48 57;
49 57;
49 58;
50 58;
51 58;
51 59;
52 59;
53 59;
53 60;
54 60;
55 60;
55 61;
56 61;
57 58;
58 59;
59 60;
60 61;
57 62;
```

```
57 63;
58 63;
58 64;
58 65;
59 65;
59 66;
59 67;
60 67;
60 68;
60 69;
61 69;
61 70;
62 63;
63 64;
64 65;
65 66;
66 67;
67 68;
68 69;
69 70;
62 71;
63 71;
63 72;
64 72;
65 72;
65 73;
66 73;
67 73;
67 74;
68 74;
69 74;
69 75;
70 75;
71 72;
72 73;
73 74;
74 75;
71 76;
72 76;
73 76;
73 77;
74 77;
75 77];

% Bar grouping (29 groups, 29 design variables)
% G1:  top chord
% G2:  radial verticals (odd panels)
% G3:  horizontal ring
% G4:  panel-edge horizontals (repeating)
% G5:  radial verticals (even panels)
% G6:  diagonals (web members)
% Pattern repeats every 2 panel levels
groups={[1:4],...                                                    % G1
        [5,8,11,14,17],...                                           % G2
        [19,20,21,22,23,24],...                                      % G3
        [18,25,56,63,94,101,132,139,170,177],...                     % G4
        [26,29,32,35,38],...                                         % G5
        [6,7,9,10,12,13,15,16,27,28,30,31,33,34,36,37],...           % G6
        [39,40,41,42],...                                             % G7
        [43,46,49,52,55],...                                         % G8
        [57,58,59,60,61,62],...                                      % G9
        [64,67,70,73,76],...                                         % G10
        [44,45,47,48,50,51,53,54,65,66,68,69,71,72,74,75],...        % G11
        [77,78,79,80],...                                             % G12
        [81,84,87,90,93],...                                         % G13
        [95,96,97,98,99,100],...                                     % G14
        [102,105,108,111,114],...                                    % G15
        [82,83,85,86,88,89,91,92,103,104,106,107,109,110,112,113],... % G16
        [115,116,117,118],...                                        % G17
        [119,122,125,128,131],...                                    % G18
        [133,134,135,136,137,138],...                                % G19
        [140,143,146,149,152],...                                    % G20
        [120,121,123,124,126,127,129,130,141,142,144,145,147,148,150,151],...% G21
        [153,154,155,156],...                                        % G22
        [157,160,163,166,169],...                                    % G23
```

```
        [171,172,173,174,175,176],...                                    % G24
        [178,181,184,187,190],...                                        % G25
        [158,159,161,162,164,165,167,168,179,180,182,183,185,186,188,189],...% G26
        [191,192,193,194],...                                            % G27
        [195,197,198,200],...                                            % G28
        [196,199]};                                                      % G29

% Support conditions (1=fixed, 0=free) [x y z]
% All nodes: z fixed (planar truss), nodes 76-77 fully fixed
MesKos = zeros(77,3);
MesKos(:,3) = 1;            % z fixed for all (planar)
MesKos(76,:) = [1 1 1];     % support
MesKos(77,:) = [1 1 1];     % support

% Loading (N) ---> Load(node,:)=[Fx Fy Fz]
% Loading #1: horizontal loads at left-edge nodes
yuk1=zeros(size(dnkoor));
left_nodes = [1, 6, 15, 20, 29, 34, 43, 48, 57, 62, 71];
for i = 1:length(left_nodes)
    yuk1(left_nodes(i),:) = [5000 0 0];  % 5 kN horizontal
end

% Loading #2: vertical loads at most nodes
yuk2=zeros(size(dnkoor));
loaded_nodes = [1,2,3,4,5,6,8,10,12,14,15,16,17,18,19,...
    20,22,24,26,28,29,30,31,32,33,...
    34,36,38,40,42,43,44,45,46,47,...
    48,50,52,54,56,57,58,59,60,61,...
    62,64,66,68,70,71,72,73,74,75];
for i = 1:length(loaded_nodes)
    yuk2(loaded_nodes(i),:) = [0 -45000 0];  % 45 kN downward
end

% Loading #3: combined
yuk3 = yuk1 + yuk2;

% Allowable Stress (Pa)
maxGER=7e7;  % 70 MPa

% Unit weight (kg/m^3)
GS=7850;  % steel

% Design variable bounds (m^2)
LB=ones(1,29)*1e-4;    % 1 cm^2
UB=ones(1,29)*130e-4;  % 130 cm^2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,...
    'E',E,'yuk1',yuk1,'yuk2',yuk2,'yuk3',yuk3,...
    'maxGER',maxGER,'GS',GS,...
    'groups',{groups},'LB',LB,'UB',UB);
```

Code 5.9: Giàn 18 thanh

```
function Truss_Data=TrussData_18bar

% Modulus of Elasticity (Pa)
E=6.9e10; % 69 GPa (~1e7 psi)

% Bar connections between nodes
eldn=[1 2   ;
1   3   ;
2   3   ;
2   4   ;
3   4   ;
3   5   ;
4   5   ;
4   6   ;
5   6   ;
5   7   ;
6   7   ;
6   8   ;
7   8   ;
7   9   ;
8   9   ;
8   10  ;
```

```
9   10  ;
9   11];

% Support conditions
MesKos=[0   0   1   ;
0   0   1   ;
0   0   1   ;
0   0   1   ;
0   0   1   ;
0   0   1   ;
0   0   1   ;
0   0   1   ;
0   0   1   ;
1   1   1   ;
1   1   1   ];

% Loading     ----- > Load(dn_no,:)=[x y z] (N)
yuk=zeros(size(MesKos));
yuk(1,:)=[0 -89000 0]; % 89 kN (~20000 lbs)
yuk(2,:)=[0 -89000 0];
yuk(4,:)=[0 -89000 0];
yuk(6,:)=[0 -89000 0];
yuk(8,:)=[0 -89000 0];

% Allowable Stress (Pa)
maxGER=1.38e8; % 138 MPa (~20000 psi)

% Allowed Displacement (m)
maxYER=0.025; % 25 mm (~1.0 in)

% Unit density (kg/m^3)
GS=2770; % 2770 kg/m^3 (~0.1 lb/in^3)

% Design variable bounds
LB=ones(1,12);
LB(1,1:4)=LB(1,1:4)*22 * 1e-4; % 22 cm2 (m2)
LB(1,5)=LB(1,5)*20; %m (x3)
LB(1,6)=LB(1,6)*-6; %m (y3)
LB(1,7)=LB(1,7)*13; %m (x5)
LB(1,8)=LB(1,8)*-6; %m (y5)
LB(1,9)=LB(1,9)*7; %m (x7)
LB(1,10)=LB(1,10)*-6; %m (y7)
LB(1,11)=LB(1,11)*1; %m (x9)
LB(1,12)=LB(1,12)*-6; %m (y9)

UB=ones(1,12);
UB(1,1:4)=UB(1,1:4)*116 * 1e-4; % 116 cm2 (m2)
UB(1,5)=UB(1,5)*31; %m (x3)
UB(1,6)=UB(1,6)*6; %m (y3)
UB(1,7)=UB(1,7)*25; %m (x5)
UB(1,8)=UB(1,8)*6; %m (y5)
UB(1,9)=UB(1,9)*18; %m (x7)
UB(1,10)=UB(1,10)*6; %m (y7)
UB(1,11)=UB(1,11)*12; %m (x9)
UB(1,12)=UB(1,12)*6; %m (y9)

% Base node coordinates (m)
% Coordinates for nodes 3,5,7,9 will be overwritten by optimizer
dnkoor = [
    31.75  6.35 0; % Node 1
    25.40  6.35 0; % Node 2
    19.69 -5.72 0; % Node 3 (var x3, y3 - using default initial limit)
    19.05  6.35 0; % Node 4
    13.34 -5.72 0; % Node 5 (var x5, y5)
    12.70  6.35 0; % Node 6
     6.99 -5.72 0; % Node 7 (var x7, y7)
     6.35  6.35 0; % Node 8
     0.64 -5.72 0; % Node 9 (var x9, y9)
     0.00  6.35 0; % Node 10
     0.00  0.00 0  % Node 11
];

% Bar grouping (4 areas, 8 node coordinates) -> 12 Design variables
% Area 1: bars 1,4,8,12,16 (Top chord)
% Area 2: bars 2,6,10,14,18 (Bottom chord)
```

```
% Area 3: bars 3,7,11,15 (Diagonals / verticals)
% Area 4: bars 5,9,13,17 (Diagonals)
groups={[1,4,8,12,16],[2,6,10,14,18],[3,7,11,15],[5,9,13,17]};

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,'E',E,'yuk',yuk,'maxGER',maxGER,'maxYER',
    ↪ maxYER,'GS',GS,'LB',LB,'UB',UB,'groups',{groups});
```

Tối ưu tô-pô

Code 6.1: SIMP

```
% --- DONG LENH CHAY O TREN CUNG ---
top99(60,20,0.5,3.0,1.5,1)
% ---------------------------------

% Dinh nghia cac function o ben duoi
function top99(nelx,nely,volfrac,penal,rmin,nLoadCase)

% KHOI TAO
x(1:nely,1:nelx) = volfrac;
loop = 0;
change = 1;
passive = [];

% BAT DAU VONG LAP TOI UU
while change > 0.01
    loop = loop + 1;
    xold = x;

    % Phan tich phan tu huu han
    [U] = FE(nelx,nely,x,penal,nLoadCase);

    % Tinh toan ham muc tieu
    [KE] = localK;
    c = 0;
    dc = zeros(nely,nelx);
    for ely = 1:nely
        for elx = 1:nelx
            n1 = (nely+1)*(elx-1)+ely;
            n2 = (nely+1)* elx   +ely;
            Ue = U([2*n1-1;2*n1; 2*n2-1;2*n2; 2*n2+1;2*n2+2; 2*n1+1;2*n1+2],1);
            c = c + x(ely,elx)^penal*Ue'*KE*Ue;
            dc(ely,elx) = -penal*x(ely,elx)^(penal-1)*Ue'*KE*Ue;
        end
    end

    % Loc do nhay
    [dc] = check(nelx,nely,rmin,x,dc);

    % Cap nhat bien thiet ke
    [x] = OC(nelx,nely,x,volfrac,dc,passive);

    % In ket qua
    change = max(max(abs(x-xold)));
    disp([' It.: ' sprintf('%4i',loop) ' Obj.: ' sprintf('%10.4f',c) ...
          ' Vol.: ' sprintf('%6.3f',sum(sum(x))/(nelx*nely)) ...
          ' ch.: ' sprintf('%6.3f',change )])

    % Ve bieu do
    colormap(gray); imagesc(-x); axis equal; axis tight; axis off;drawnow;
end
end

%%%%%%%%%% CAC HAM CON (LOCAL FUNCTIONS) %%%%%%%%%%

function [dcn] = check(nelx,nely,rmin,x,dc)
dcn = zeros(nely,nelx);
for i = 1:nelx
  for j = 1:nely
    sum = 0;
    for k = max(i-floor(rmin),1):min(i+floor(rmin),nelx)
      for l = max(j-floor(rmin),1):min(j+floor(rmin),nely)
        fac = rmin-sqrt((i-k)^2+(j-l)^2);
        sum = sum+max(0,fac);
```

```
        dcn(j,i) = dcn(j,i) + max(0,fac)*x(l,k)*dc(l,k);
      end
    end
    dcn(j,i) = dcn(j,i)/(x(j,i)*sum);
  end
end
end

function [U] = FE(nelx,nely,x,penal,nLoadCase)
[KE] = localK;
K = sparse(2*(nelx+1)*(nely+1), 2*(nelx+1)*(nely+1));
F = sparse(2*(nely+1)*(nelx+1),nLoadCase);
U = sparse(2*(nely+1)*(nelx+1),nLoadCase);

for elx = 1:nelx
    for ely = 1:nely
        n1 = (nely+1)*(elx-1)+ely;
        n2 = (nely+1)* elx   +ely;
        edof = [2*n1-1; 2*n1; 2*n2-1; 2*n2; 2*n2+1; 2*n2+2; 2*n1+1; 2*n1+2];
        K(edof,edof) = K(edof,edof) + x(ely,elx)^penal*KE;
    end
end

% DIEU KIEN BIEN (MBB Beam)
F(2,1) = -1;
fixeddofs = union([1:2:2*(nely+1)],[2*(nelx+1)*(nely+1)]);

alldofs = [1:2*(nely+1)*(nelx+1)];
freedofs = setdiff(alldofs,fixeddofs);

U(freedofs,:) = K(freedofs,freedofs) \ F(freedofs,:);
U(fixeddofs,:) = 0;
end

function [KE] = localK
E = 1.;
nu = 0.3;
k = [ 1/2-nu/6   1/8+nu/8 -1/4-nu/12 -1/8+3*nu/8 ...
     -1/4+nu/12 -1/8-nu/8  nu/6       1/8-3*nu/8 ];
KE = E/(1-nu^2)*[ k(1) k(2) k(3) k(4) k(5) k(6) k(7) k(8)
                  k(2) k(1) k(8) k(7) k(6) k(5) k(4) k(3)
                  k(3) k(8) k(1) k(6) k(7) k(4) k(5) k(2)
                  k(4) k(7) k(6) k(1) k(8) k(3) k(2) k(5)
                  k(5) k(6) k(7) k(8) k(1) k(2) k(3) k(4)
                  k(6) k(5) k(4) k(3) k(2) k(1) k(8) k(7)
                  k(7) k(4) k(5) k(2) k(3) k(8) k(1) k(6)
                  k(8) k(3) k(2) k(5) k(4) k(7) k(6) k(1)];
end

function [xnew] = OC(nelx,nely,x,volfrac,dc,passive)
l1 = 0; l2 = 100000; move = 0.1;
while (l2-l1 > 1e-4)
    lmid = 0.5*(l2+l1);
    xnew = max(0.001,max(x-move,min(1,min(x+move,x.*sqrt(-dc./lmid)))));
    if ~isempty(passive)
        xnew(passive==1) = 0.001;
    end
    if sum(sum(xnew)) - volfrac*nelx*nely > 0
        l1 = lmid;
    else
        l2 = lmid;
    end
end
end
```

Tối ưu đa mục tiêu

Code 7.3: Vẽ Hình 7.4

```
"""
============================================================
Ví dụ 7.x - Tối ưu đa mục tiêu giàn hai thanh
Phương pháp tổng trọng số (Weighted-Sum) → Biên Pareto
============================================================
```

```
Hai mục tiêu:
  f1 = khối lượng giàn (kg)
  f2 = |v_C| - chuyển vị đứng tại nút C (mm)

Ràng buộc:
  |σ_i| ≤ σ_allow   (i = 1, 2)
  A_min ≤ A_i ≤ A_max

Biến thiết kế: x = [A1, A2]  (m²)
============================================================
"""
import sys, io
sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding="utf-8")

import numpy as np
from scipy.optimize import minimize
import matplotlib
matplotlib.use("Agg")                    # backend không cần GUI
import matplotlib.pyplot as plt
matplotlib.rcParams.update({
    "font.size": 12,
    "font.family": "serif",
    "axes.grid": True,
    "grid.alpha": 0.3,
})

# ============================================================
# 1. DỮ LIỆU BÀI TOÁN
# ============================================================
L   = 2.0           # m  - khoảng cách AB
h   = 1.0           # m  - chiều cao nút C
P   = 50e3          # N  - tải trọng
E   = 200e9         # Pa - mô đun đàn hồi
rho = 7850          # kg/m³ - khối lượng riêng
sig_allow = 150e6   # Pa - ứng suất cho phép
A_min = 200e-6      # m²  (= 200 mm²)
A_max = 4000e-6     # m²  (= 4000 mm²)

# Tọa độ nút (m)
node_A = np.array([0.0, 0.0])
node_B = np.array([L,   0.0])
node_C = np.array([L/2, h  ])

# Chiều dài thanh
L1 = np.linalg.norm(node_C - node_A)   # |AC|
L2 = np.linalg.norm(node_C - node_B)   # |BC|

# Cosin chỉ phương (từ gối đến nút C)
c1 = (node_C[0] - node_A[0]) / L1
s1 = (node_C[1] - node_A[1]) / L1
c2 = (node_C[0] - node_B[0]) / L2
s2 = (node_C[1] - node_B[1]) / L2

print(f"Chiều dài thanh: L1 = {L1:.4f} m, L2 = {L2:.4f} m")
print(f"Thanh AC: cosθ = {c1:.4f}, sinθ = {s1:.4f}")
print(f"Thanh BC: cosθ = {c2:.4f}, sinθ = {s2:.4f}")
print()

# ============================================================
# 2. PHÂN TÍCH KẾT CẤU (Phương pháp độ cứng trực tiếp)
# ============================================================
def analyze(A1, A2):
    """
    Phân tích giàn hai thanh bằng phương pháp độ cứng.
    Trả về: (u_C, v_C, σ1, σ2)
      u_C, v_C : chuyển vị nút C theo x, y (m)
      σ1, σ2   : ứng suất dọc trong thanh AC, BC (Pa)
    """
    # Ma trận độ cứng phần tử (2×2, đã rút gọn về DOF nút C)
    k1 = (E * A1 / L1) * np.array([[c1**2, c1*s1],
                                   [c1*s1, s1**2]])
    k2 = (E * A2 / L2) * np.array([[c2**2, c2*s2],
                                   [c2*s2, s2**2]])
    # Ma trận độ cứng tổng thể
    K = k1 + k2
```

```
81
82      # Vector tải
83      F = np.array([0.0, -P])
84
85      # Giải hệ phương trình: K · d = F
86      d = np.linalg.solve(K, F)
87      u_C, v_C = d
88
89      # Biến dạng dọc → ứng suất
90      delta1 = c1 * u_C + s1 * v_C          # biến dạng thanh AC
91      sigma1 = E * delta1 / L1
92
93      delta2 = c2 * u_C + s2 * v_C          # biến dạng thanh BC
94      sigma2 = E * delta2 / L2
95
96      return u_C, v_C, sigma1, sigma2
97
98  # ============================================================
99  # 3. HÀM MỤC TIÊU VÀ RÀNG BUỘC
100 # ============================================================
101 def f1_mass(x):
102     """Khối lượng giàn (kg)."""
103     A1, A2 = x
104     return rho * (L1 * A1 + L2 * A2)
105
106 def f2_disp(x):
107     """Chuyển vị đứng tuyệt đối tại C (mm)."""
108     A1, A2 = x
109     _, v_C, _, _ = analyze(A1, A2)
110     return abs(v_C) * 1e3              # đổi m → mm
111
112 def stress_con1(x):
113     """g1 = σ_allow - |σ1| ≥ 0"""
114     A1, A2 = x
115     _, _, sig1, _ = analyze(A1, A2)
116     return sig_allow - abs(sig1)
117
118 def stress_con2(x):
119     """g2 = σ_allow - |σ2| ≥ 0"""
120     A1, A2 = x
121     _, _, _, sig2 = analyze(A1, A2)
122     return sig_allow - abs(sig2)
123
124 # ============================================================
125 # 4. CHUẨN HÓA MỤC TIÊU (utopia-nadir)
126 # ============================================================
127 bounds = [(A_min, A_max), (A_min, A_max)]
128 cons   = [{"type": "ineq", "fun": stress_con1},
129           {"type": "ineq", "fun": stress_con2}]
130 x0     = np.array([1e-3, 1e-3])        # điểm xuất phát (1000 mm²)
131
132 # Tối ưu riêng f1
133 res1 = minimize(f1_mass, x0, method="SLSQP",
134                 bounds=bounds, constraints=cons)
135 f1_min = f1_mass(res1.x)
136 f2_at_f1min = f2_disp(res1.x)
137
138 # Tối ưu riêng f2
139 res2 = minimize(f2_disp, x0, method="SLSQP",
140                 bounds=bounds, constraints=cons)
141 f2_min = f2_disp(res2.x)
142 f1_at_f2min = f1_mass(res2.x)
143
144 # Giá trị nadir (cận trên) cho chuẩn hóa
145 f1_max = f1_at_f2min
146 f2_max = f2_at_f1min
147
148 print("-- Kết quả tối ưu đơn mục tiêu --")
149 print(f"  min f1 → f1 = {f1_min:.3f} kg, f2 = {f2_at_f1min:.4f} mm")
150 print(f"  min f2 → f1 = {f1_at_f2min:.3f} kg, f2 = {f2_min:.4f} mm")
151 print(f"  Chuẩn hóa: f1 ∈ [{f1_min:.3f}, {f1_max:.3f}] kg")
152 print(f"             f2 ∈ [{f2_min:.4f}, {f2_max:.4f}] mm\n")
153
154 # ============================================================
155 # 5. GIẢI BẰNG PHƯƠNG PHÁP TỔNG TRỌNG SỐ
```

```python
# ============================================================
def weighted_obj(x, w):
    """F(x;w) = w·f~1 + (1-w)·f~2"""
    f1_norm = (f1_mass(x) - f1_min) / (f1_max - f1_min + 1e-30)
    f2_norm = (f2_disp(x) - f2_min) / (f2_max - f2_min + 1e-30)
    return w * f1_norm + (1 - w) * f2_norm

# Quét w từ 0 đến 1
n_pts = 51                                  # nhiều điểm hơn cho đường mượt
weights = np.linspace(0, 1, n_pts)

results = []                                # [(w, A1, A2, f1, f2)]
for w in weights:
    res = minimize(weighted_obj, x0, args=(w,), method="SLSQP",
                   bounds=bounds, constraints=cons,
                   options={"ftol": 1e-12, "maxiter": 500})
    A1_opt, A2_opt = res.x
    f1_val = f1_mass(res.x)
    f2_val = f2_disp(res.x)
    results.append((w, A1_opt, A2_opt, f1_val, f2_val))

results = np.array(results)

# In bảng kết quả (mỗi bước 0.1)
print("-- Bảng kết quả Pareto --")
print(f"{'w':>5s} {'A1(mm²)':>10s} {'A2(mm²)':>10s} "
      f"{'f1(kg)':>10s} {'f2(mm)':>10s}")
print("-" * 50)
for row in results:
    w, A1, A2, f1, f2 = row
    if np.isclose(w * 10 % 1, 0, atol=0.02) or w == weights[-1]:
        print(f"{w:5.2f} {A1*1e6:10.1f} {A2*1e6:10.1f} "
              f"{f1:10.3f} {f2:10.4f}")
print()

# ============================================================
# 6. VẼ ĐỒ THỊ
# ============================================================
fig, ax = plt.subplots(figsize=(8, 6))

# -- Biên Pareto ----------------------------------
f1_arr = results[:, 3]
f2_arr = results[:, 4]
sc = ax.scatter(f1_arr, f2_arr, c=results[:, 0], cmap="Greys",
                edgecolors="k", s=80, zorder=5)
ax.plot(f1_arr, f2_arr, "k--", lw=0.8, alpha=0.5)


cbar = fig.colorbar(sc, ax=ax, pad=0.02)
cbar.set_label("Trọng số w")
ax.set_xlabel("$f_1$ - Khối lượng (kg)")
ax.set_ylabel("$f_2$ - Chuyển vị $|v_C|$ (mm)")

plt.tight_layout()
import os
script_dir = os.path.dirname(os.path.abspath(__file__))
save_path = os.path.join(script_dir, "pareto_front_two_bar_truss.png")
plt.savefig(save_path, dpi=600, bbox_inches="tight")

print(f"✓ Đồ thị đã lưu: {save_path}")

# ============================================================
# 7. NHẬN XÉT BIÊN PARETO
# ============================================================
print("""
-- NHẬN XÉT --
• Đường Pareto cho thấy mối quan hệ đánh đổi giữa khối lượng
  và chuyển vị: giảm khối lượng → tăng chuyển vị và ngược lại.
• Vùng w nhỏ (ưu tiên chuyển vị): tăng thêm khối lượng chỉ
  giảm rất ít chuyển vị → "bão hòa" về chuyển vị.
• Vùng w lớn (ưu tiên khối lượng): giảm thêm một ít khối lượng
  sẽ khiến chuyển vị tăng mạnh → "bão hòa" về khối lượng.
• Vùng giữa (w ≈ 0.3-0.6): mỗi đơn vị khối lượng tiết kiệm
  được "đổi" lượng chuyển vị tăng hợp lý → vùng thiết kế hiệu quả.
• Phương án thiết kế đề xuất: chọn điểm "đầu gối" (knee point)
```

```
  trên đường Pareto, thường ở w ≈ 0.4-0.5, cân bằng tốt nhất
  giữa hai mục tiêu.
""")
```

Các thuật toán tối ưu metaheuristic

Code 8.1: Ví dụ 8.1

```
"""
Ví dụ 6: Tối ưu hóa dầm công xôn bằng thuật toán Simulated Annealing (SA)

Mô tả bài toán:
  - Dầm công xôn dài L = 3m, chia thành 5 đoạn, mỗi đoạn dài 0.6m
  - Mặt cắt ngang hình tròn rỗng (r_outer, r_inner)
  - Vật liệu: Thép S270 (E = 210 GPa, σ_yield = 270 MPa)
  - Lực tập trung F = 500 kN tại đầu tự do

Mục tiêu:
  - Tối thiểu hóa Khối lượng dầm

Biến thiết kế (10 biến):
  - r_outer[1..5]: bán kính ngoài của 5 đoạn, 1 ≤ r_outer ≤ 30 cm (≤ L/10)
  - r_inner[1..5]: bán kính trong của 5 đoạn (cm)

Ràng buộc:
  1. Chuyển vị tại đầu dầm ≤ 2 cm
  2. r_outer[i] > r_inner[i]  (mặt cắt hợp lệ)
  3. r_outer[i] > r_inner[i+1] và r_outer[i+1] > r_inner[i]  (thành ống chồng lấn → hàn được)
  4. Ứng suất max < σ_yield = 270 MPa
  5. Bán kính ngoài ≤ L/10 = 30 cm
  6. Chiều dày thành ống: 0.1 cm ≤ (r_outer - r_inner) ≤ 5 cm
"""

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from scipy.interpolate import CubicSpline
import os

# ============================================================
# 1. THÔNG SỐ BÀI TOÁN
# ============================================================

L = 3.0                     # Chiều dài dầm (m)
NUM_SEGMENTS = 5            # Số đoạn
SEG_LEN = L / NUM_SEGMENTS  # Chiều dài mỗi đoạn (m)

E = 210e9                   # Mô-đun đàn hồi Young (Pa)
RHO = 7850                  # Khối lượng riêng thép (kg/m^3)
SIGMA_YIELD = 270e6         # Giới hạn chảy (Pa)
MAX_DISP = 0.02             # Chuyển vị cho phép tại đầu dầm (m)
F = 500000                  # Lực tập trung tại đầu tự do (N) = 500 kN

# Giới hạn biến thiết kế (cm)
R_OUT_MAX = L * 100 / 10       # r_outer ≤ L/10 = 30 cm
R_OUT_MIN = 1.0                # Bán kính ngoài tối thiểu (cm)
T_MIN = 0.1                    # Chiều dày thành ống tối thiểu (cm) = 1 mm
T_MAX = 5.0                    # Chiều dày thành ống tối đa (cm) = 5 cm

# ============================================================
# 2. CÁC HÀM TÍNH TOÁN KẾT CẤU (Phần tử hữu hạn dầm)
# ============================================================

def area(r_out, r_in):
    """Diện tích mặt cắt tròn rỗng."""
    return np.pi * (r_out**2 - r_in**2)

def moment_of_inertia(r_out, r_in):
    """Mô-men quán tính mặt cắt tròn rỗng."""
    return np.pi * (r_out**4 - r_in**4) / 4

def beam_stiffness_matrix(r_outer, r_inner):
    """
```

```
    Lắp ghép ma trận độ cứng tổng thể cho dầm.
    Mỗi nút có 2 bậc tự do: chuyển vị đứng (v) và góc xoay (θ).
    Tổng cộng: (NUM_SEGMENTS + 1) nút × 2 = 12 bậc tự do.
    """
    n_dof = 2 * (NUM_SEGMENTS + 1)
    K = np.zeros((n_dof, n_dof))

    for i in range(NUM_SEGMENTS):
        I_i = moment_of_inertia(r_outer[i], r_inner[i])
        le = SEG_LEN  # chiều dài phần tử

        # Ma trận độ cứng phần tử dầm Euler-Bernoulli (4×4)
        k_e = (E * I_i) * np.array([
            [ 12/le**3,  6/le**2, -12/le**3,  6/le**2],
            [  6/le**2,  4/le,     -6/le**2,  2/le   ],
            [-12/le**3, -6/le**2,  12/le**3, -6/le**2],
            [  6/le**2,  2/le,     -6/le**2,  4/le   ]
        ])

        # Ghép vào ma trận tổng thể
        dofs = [2*i, 2*i+1, 2*i+2, 2*i+3]
        for j in range(4):
            for k in range(4):
                K[dofs[j], dofs[k]] += k_e[j, k]

    return K

def solve_displacement(r_outer, r_inner):
    """
    Giải hệ phương trình K·u = F để tính chuyển vị.
    Điều kiện biên: ngàm tại đầu trái (v₁ = 0, θ₁ = 0).
    Trả về: vector chuyển vị đầy đủ (bao gồm cả nút ngàm).
    """
    K = beam_stiffness_matrix(r_outer, r_inner)

    # Loại bỏ 2 bậc tự do đầu tiên (ngàm)
    K_red = K[2:, 2:]

    # Vector tải trọng (lực F tại nút thứ 2, tức bậc tự do đầu tiên sau ngàm)
    F_vec = np.zeros(K_red.shape[0])
    F_vec[0] = F

    # Giải hệ phương trình
    u_red = np.linalg.solve(K_red, F_vec)

    # Ghép lại vector chuyển vị đầy đủ (thêm 2 giá trị = 0 tại ngàm)
    u_full = np.zeros(2 * (NUM_SEGMENTS + 1))
    u_full[2:] = u_red
    return u_full

def tip_displacement(r_outer, r_inner):
    """Chuyển vị đứng tại đầu tự do của dầm (nút cuối)."""
    u = solve_displacement(r_outer, r_inner)
    return u[-2]   # bậc tự do chuyển vị đứng tại nút cuối

def max_stress(r_outer, r_inner):
    """
    Tính ứng suất lớn nhất trong dầm theo công thức uốn: σ = M·c / I.
    Với dầm công xôn chịu lực tập trung tại đầu: M(x) = F·(L - x).
    """
    sigma_max = 0.0
    for i in range(NUM_SEGMENTS):
        x_mid = (i + 0.5) * SEG_LEN         # vị trí giữa đoạn
        M = F * (L - x_mid)                  # mô-men uốn tại giữa đoạn
        c = r_outer[i]                       # khoảng cách đến thớ ngoài cùng
        I_i = moment_of_inertia(r_outer[i], r_inner[i])
        sigma = abs(M * c / I_i)
        sigma_max = max(sigma_max, sigma)
    return sigma_max

# ============================================================
# 3. HÀM MỤC TIÊU VÀ RÀNG BUỘC
# ============================================================

def objective(params):
```

```
    """
    Hàm mục tiêu: Khối lượng dầm (kg).
    params = [r_out1..r_out5, r_in1..r_in5] (đơn vị: cm)
    """
    r_out = params[:NUM_SEGMENTS] / 100   # chuyển sang mét
    r_in = params[NUM_SEGMENTS:] / 100
    weight = 0.0
    for i in range(NUM_SEGMENTS):
        weight += area(r_out[i], r_in[i]) * SEG_LEN * RHO
    return weight

def is_feasible(params):
    """
    Kiểm tra tất cả ràng buộc. Trả về True nếu thiết kế thỏa mãn.

    Ràng buộc:
      (1) Chuyển vị đầu dầm ≤ MAX_DISP
      (2) r_outer[i] > r_inner[i]  ∀i
      (3) r_outer[i+1] > r_inner[i]  ∀i (ràng buộc hàn)
      (4) Ứng suất max < SIGMA_YIELD
      (5) r_outer[i] ≤ R_OUT_MAX (đường kính ngoài ≤ L/10)
      (6) T_MIN ≤ (r_outer[i] - r_inner[i]) ≤ T_MAX (chiều dày thành ống)
    """
    r_out = params[:NUM_SEGMENTS]
    r_in = params[NUM_SEGMENTS:]
    r_out_m = r_out / 100   # chuyển sang mét
    r_in_m = r_in / 100

    # (2) Ràng buộc bán kính: r_outer > r_inner
    if np.any(r_out <= r_in):
        return False

    # (5) Ràng buộc giới hạn trên bán kính ngoài
    if np.any(r_out > R_OUT_MAX) or np.any(r_out < R_OUT_MIN):
        return False

    # (6) Ràng buộc chiều dày thành ống
    thickness = r_out - r_in
    if np.any(thickness < T_MIN) or np.any(thickness > T_MAX):
        return False

    # (3) Ràng buộc hàn: thành ống hai đoạn kề nhau phải chồng lấn
    for i in range(NUM_SEGMENTS - 1):
        if r_out[i] <= r_in[i + 1]:       # r_outer[i] > r_inner[i+1]
            return False
        if r_out[i + 1] <= r_in[i]:       # r_outer[i+1] > r_inner[i]
            return False

    # (1) Ràng buộc chuyển vị
    disp = abs(tip_displacement(r_out_m, r_in_m))
    if disp > MAX_DISP:
        return False

    # (4) Ràng buộc ứng suất
    sigma = max_stress(r_out_m, r_in_m)
    if sigma > SIGMA_YIELD:
        return False

    return True

# ============================================================
# 4. THUẬT TOÁN SIMULATED ANNEALING (SA)
# ============================================================

def simulated_annealing(x0, n_iter=5000, T0=100.0, alpha=0.99):
    """
    Thuật toán Simulated Annealing (Luyện kim mô phỏng).

    Tham số:
      x0     : Nghiệm ban đầu (vector biến thiết kế)
      n_iter : Số vòng lặp tối đa
      T0     : Nhiệt độ ban đầu
      alpha  : Tốc độ làm nguội (0 < alpha < 1)

    Thuật toán:
```

```
    1. Bắt đầu với nghiệm x_current, tính f(x_current)
    2. Lặp:
       a. Tạo nghiệm lân cận x_new = x_current + nhiễu ngẫu nhiên
       b. Nếu x_new không thỏa ràng buộc → bỏ qua
       c. Tính Δf = f(x_new) - f(x_current)
       d. Nếu Δf < 0 → chấp nhận (nghiệm tốt hơn)
       e. Nếu Δf ≥ 0 → chấp nhận với xác suất p = exp(-Δf / T)
       f. Giảm nhiệt độ: T = T0 × alpha^k
    3. Trả về nghiệm tốt nhất tìm được
    """
    # --- Khởi tạo ---
    x_current = x0.copy()
    f_current = objective(x_current)
    x_best = x_current.copy()
    f_best = f_current

    # Tìm điểm xuất phát khả thi (nếu x0 vi phạm ràng buộc)
    if not is_feasible(x_current):
        print("Nghiệm ban đầu vi phạm ràng buộc. Tìm điểm khả thi...")
        for _ in range(10000):
            # Tạo nghệm ngẫu nhiên trong giới hạn biến
            r_out_trial = np.random.uniform(R_OUT_MIN, R_OUT_MAX, NUM_SEGMENTS)
            r_out_trial = np.sort(r_out_trial)[::-1]  # giảm dần (hợp lý cho dầm công xôn)
            t_trial = np.random.uniform(T_MIN, T_MAX, NUM_SEGMENTS)
            r_in_trial = r_out_trial - t_trial
            x_trial = np.concatenate([r_out_trial, r_in_trial])
            if is_feasible(x_trial):
                x_current = x_trial.copy()
                x_best = x_trial.copy()
                f_current = objective(x_trial)
                f_best = f_current
                print(f"  Đã tìm được điểm khả thi, khối lượng = {f_best:.2f} kg")
                break
        else:
            print("  CẢNH BÁO: Không tìm được điểm khả thi.")

    # --- Vòng lặp SA ---
    history = []            # lưu lịch sử f_best qua các vòng lặp
    step_size = 1.0         # kích thước bước nhiễu
    n_reject = 0            # đếm số lần liên tiếp bị từ chối

    for k in range(n_iter):
        # Nhiệt độ hiện tại: T_k = T0 × alpha^k
        T_k = T0 * (alpha ** k)

        # Tự điều chỉnh bước: giảm bước nếu bị từ chối quá nhiều
        if n_reject > 50:
            step_size *= 0.9
            n_reject = 0

        # Bước 2a: Tạo nghiệm lân cận
        perturbation = np.random.uniform(-step_size, step_size, len(x_current))
        x_new = x_current + perturbation

        # Giới hạn biên cho biến thiết kế
        x_new[:NUM_SEGMENTS] = np.clip(x_new[:NUM_SEGMENTS], R_OUT_MIN, R_OUT_MAX)
        x_new[NUM_SEGMENTS:] = np.clip(
            x_new[NUM_SEGMENTS:],
            x_new[:NUM_SEGMENTS] - T_MAX,   # r_inner ≥ r_outer - T_MAX
            x_new[:NUM_SEGMENTS] - T_MIN    # r_inner ≤ r_outer - T_MIN
        )
        x_new = np.maximum(x_new, 0.1)      # đảm bảo bán kính trong > 0

        # Bước 2b: Kiểm tra ràng buộc
        if not is_feasible(x_new):
            n_reject += 1
            history.append(f_best)
            continue

        # Bước 2c: Tính chênh lệch hàm mục tiêu
        f_new = objective(x_new)
        delta_f = f_new - f_current

        # Bước 2d-e: Quyết định chấp nhận hay từ chối
        if delta_f < 0:
```

```
            # Nghiệm tốt hơn → luôn chấp nhận
            accept = True
        else:
            # Nghiệm kém hơn → chấp nhận với xác suất p = exp(-Δf / T)
            p = np.exp(-delta_f / T_k)
            accept = (np.random.random() < p)

        if accept:
            x_current = x_new.copy()
            f_current = f_new
            n_reject = 0

            # Cập nhật nghiệm tốt nhất
            if f_current < f_best:
                x_best = x_current.copy()
                f_best = f_current
        else:
            n_reject += 1

        # Lưu lịch sử
        history.append(f_best)

        # In tiến trình mỗi 500 vòng
        if k % 500 == 0:
            print(f"  Vòng lặp {k:5d} | f_best = {f_best:.2f} kg | T = {T_k:.6f}")

    return x_best, f_best, history

# ============================================================
# 5. HIỂN THỊ KẾT QUẢ
# ============================================================

def plot_history(history, save_dir):
    """Vẽ biểu đồ lịch sử hội tụ của thuật toán SA."""
    plt.figure(figsize=(10, 6))
    plt.plot(history, linewidth=1.5)
    plt.xlabel('Vòng lặp')
    plt.ylabel('Khối lượng (kg)')
    plt.title('Quá trình hội tụ - Simulated Annealing')
    plt.grid(True, linestyle='--', alpha=0.5)
    plt.savefig(os.path.join(save_dir, 'optimization_history.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: optimization_history.png")

def plot_deformed_beam(params, save_dir):
    """Vẽ hình dạng biến dạng của dầm sau tối ưu."""
    r_out_m = params[:NUM_SEGMENTS] / 100
    r_in_m = params[NUM_SEGMENTS:] / 100

    # Giải chuyển vị
    u_full = solve_displacement(r_out_m, r_in_m)

    # Lấy chuyển vị đứng tại các nút (đảo dấu để hiển thị võng xuống)
    v_nodes = -u_full[0::2]
    x_nodes = np.linspace(0, L, NUM_SEGMENTS + 1)

    # Nội suy mịn bằng spline bậc 3
    cs = CubicSpline(x_nodes, v_nodes)
    x_smooth = np.linspace(0, L, 200)
    v_smooth = cs(x_smooth)

    # --- Vẽ đồ thị ---
    plt.figure(figsize=(12, 5))

    # Dầm ban đầu (chưa biến dạng)
    plt.plot(x_nodes, np.zeros_like(x_nodes), 'k-', lw=2, label='Dầm ban đầu')

    # Dầm biến dạng
    plt.plot(x_smooth, v_smooth, 'r-', lw=2, label='Dầm biến dạng')
    plt.plot(x_nodes, v_nodes, 'ro', ms=6)

    # Ghi chú chuyển vị tại từng nút
    for x, v in zip(x_nodes, v_nodes):
        plt.annotate(f"{abs(v)*100:.2f} cm", (x, v),
                     textcoords="offset points", xytext=(0, -15),
```

```
                    ha='center', fontsize=9,
                    bbox=dict(boxstyle="round,pad=0.3",
                              fc="white", ec="gray", alpha=0.8))

    # Mũi tên lực
    plt.arrow(L, 0, 0, v_nodes[-1] / 2,
              head_width=0.05, head_length=abs(v_nodes[-1] / 4),
              fc='blue', ec='blue', lw=1.5)
    plt.text(L + 0.05, v_nodes[-1] / 2, f"F = {F/1000:.0f} kN",
             color='blue', fontsize=10, ha='left', va='center')

    plt.grid(True, linestyle='--', alpha=0.5)
    plt.title('Chuyển vị dầm sau tối ưu')
    plt.xlabel('Chiều dài (m)')
    plt.ylabel('Chuyển vị (m)')
    plt.legend(loc='upper right')

    # Giới hạn trục
    v_min = min(v_nodes)
    y_range = abs(v_min) if abs(v_min) > 0.001 else 0.001
    plt.xlim(0, L + 0.2)
    plt.ylim(v_min * 1.3, y_range * 0.3)

    plt.savefig(os.path.join(save_dir, 'deformed_beam.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: deformed_beam.png")

def plot_beam_shape(params, save_dir):
    """
    Vẽ hình dạng dầm tối ưu:
      - Subplot 1: Sơ đồ dầm nhìn từ mặt bên (side view)
      - Subplot 2: Các mặt cắt ngang hình tròn rỗng (cross sections)
    """
    r_out = params[:NUM_SEGMENTS]   # cm
    r_in = params[NUM_SEGMENTS:]    # cm

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(14, 9),
                                   gridspec_kw={'height_ratios': [1, 1.5]})
    plt.subplots_adjust(hspace=0.45)

    # ------------------------------------------------------------------
    # SUBPLOT 1: SƠ ĐỒ DẦM (SIDE VIEW)
    # ------------------------------------------------------------------
    colors = plt.cm.Blues(np.linspace(0.3, 0.7, NUM_SEGMENTS))
    current_x = 0
    for i in range(NUM_SEGMENTS):
        # Vùng thành ống (tô màu)
        rect_fill_top = patches.Rectangle(
            (current_x, r_in[i]), SEG_LEN, r_out[i] - r_in[i],
            facecolor=colors[i], edgecolor='none', alpha=0.4)
        rect_fill_bot = patches.Rectangle(
            (current_x, -r_out[i]), SEG_LEN, r_out[i] - r_in[i],
            facecolor=colors[i], edgecolor='none', alpha=0.4)
        ax1.add_patch(rect_fill_top)
        ax1.add_patch(rect_fill_bot)

        # Biên ngoài (nét liền)
        rect_outer = patches.Rectangle(
            (current_x, -r_out[i]), SEG_LEN, r_out[i] * 2,
            linewidth=2, edgecolor='black', facecolor='none')
        # Biên trong (nét đứt)
        rect_inner = patches.Rectangle(
            (current_x, -r_in[i]), SEG_LEN, r_in[i] * 2,
            linewidth=1, edgecolor='gray', linestyle='--', facecolor='none')

        ax1.add_patch(rect_outer)
        ax1.add_patch(rect_inner)

        # Nhãn đoạn
        ax1.text(current_x + SEG_LEN / 2, r_out[i] + 3,
                 f"Đoạn {i+1}", ha='center', fontsize=9, fontweight='bold')
        # Kích thước bán kính ngoài
        ax1.text(current_x + SEG_LEN / 2, -r_out[i] - 4,
                 f"R={r_out[i]:.1f}", ha='center', fontsize=8, color='blue')

```

```
        current_x += SEG_LEN

    # Vẽ ký hiệu ngàm (hatch)
    ax1.add_patch(patches.Rectangle(
        (-0.08, -max(r_out) * 1.1), 0.08, max(r_out) * 2.2,
        facecolor='gray', edgecolor='black', hatch='///', linewidth=2))

    # Mũi tên lực tại đầu tự do
    r_max = max(r_out)
    ax1.annotate('', xy=(L, 0), xytext=(L, r_max * 1.5),
                 arrowprops=dict(facecolor='red', edgecolor='red',
                                 shrink=0.05, width=3, headwidth=10))
    ax1.text(L + 0.05, r_max * 1.3, f'F = {F/1000:.0f} kN',
             color='red', fontsize=11, fontweight='bold', ha='left')

    # Đường trục dầm
    ax1.axhline(y=0, color='gray', linestyle='-.', linewidth=0.8, alpha=0.5)

    ax1.set_xlim(-0.15, L + 0.4)
    ax1.set_ylim(-r_max * 1.6, r_max * 1.8)
    ax1.set_title('Sơ đồ dầm công xôn (nhìn từ mặt bên)', fontsize=13)
    ax1.set_xlabel('Chiều dài (m)')
    ax1.set_ylabel('Kích thước (cm)')
    ax1.grid(True, linestyle=':', alpha=0.5)

    # ------------------------------------------------------------------
    # SUBPLOT 2: MẶT CẮT NGANG (CROSS SECTIONS) -- hình tròn rỗng
    # ------------------------------------------------------------------
    spacing = max(r_out) * 2.8   # khoảng cách giữa các mặt cắt
    for i in range(NUM_SEGMENTS):
        cx = i * spacing   # tâm mặt cắt

        # Vòng tròn ngoài (tô xám)
        circle_out = patches.Circle(
            (cx, 0), r_out[i],
            facecolor='lightsteelblue', edgecolor='black', linewidth=1.5)
        # Vòng tròn trong (đục trắng)
        circle_in = patches.Circle(
            (cx, 0), r_in[i],
            facecolor='white', edgecolor='black', linewidth=1)

        ax2.add_patch(circle_out)
        ax2.add_patch(circle_in)

        # Nhãn mặt cắt
        t_i = r_out[i] - r_in[i]
        ax2.text(cx, -r_out[i] - 5,
                 f"Mặt cắt {i+1}\nt = {t_i:.2f} cm",
                 ha='center', fontsize=9, va='top')

        # Đường ghi kích thước bán kính ngoài
        ax2.plot([cx, cx + r_out[i]], [0, 0], 'b-', linewidth=0.8)
        ax2.plot(cx + r_out[i], 0, 'b|', markersize=8)
        ax2.text(cx + r_out[i] / 2, 1.5,
                 f"R={r_out[i]:.1f}", ha='center', fontsize=7, color='blue')

    total_width = (NUM_SEGMENTS - 1) * spacing
    ax2.set_xlim(-r_max * 1.5, total_width + r_max * 1.5)
    ax2.set_ylim(-r_max * 2, r_max * 1.8)
    ax2.set_aspect('equal')
    ax2.axis('off')
    ax2.set_title('Chi tiết các mặt cắt ngang - ống tròn rỗng', fontsize=13)

    plt.savefig(os.path.join(save_dir, 'beam_cross_sections.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: beam_cross_sections.png")

def print_results(params, f_best):
    """In kết quả tối ưu ra console."""
    r_out = params[:NUM_SEGMENTS]
    r_in = params[NUM_SEGMENTS:]
    r_out_m = r_out / 100
    r_in_m = r_in / 100

    print("\n" + "=" * 50)
```

```
    print("KẾT QUẢ TỐI ƯU")
    print("=" * 50)
    print(f"Khối lượng tối ưu: {f_best:.2f} kg")
    print(f"\nKích thước mặt cắt tối ưu (cm):")
    print(f"  {'Đoạn':<8} {'r_outer':>10} {'r_inner':>10} {'t (dày)':>10}")
    print(f"  {'-'*38}")
    for i in range(NUM_SEGMENTS):
        t_i = r_out[i] - r_in[i]
        print(f"  {i+1:<8} {r_out[i]:>10.2f} {r_in[i]:>10.2f} {t_i:>10.2f}")

    disp = abs(tip_displacement(r_out_m, r_in_m))
    sigma = max_stress(r_out_m, r_in_m)
    print(f"\nChuyển vị đầu dầm:  {disp*100:.2f} cm  (giới hạn: {MAX_DISP*100:.0f} cm)")
    print(f"Ứng suất lớn nhất:  {sigma/1e6:.2f} MPa (giới hạn: {SIGMA_YIELD/1e6:.0f} MPa)")
    print("=" * 50)

def save_results(params, f_best, save_dir):
    """Lưu kết quả ra file text."""
    r_out = params[:NUM_SEGMENTS]
    r_in = params[NUM_SEGMENTS:]
    r_out_m = r_out / 100
    r_in_m = r_in / 100

    path = os.path.join(save_dir, 'optimization_results.txt')
    with open(path, 'w', encoding='utf-8') as f:
        f.write(f"Khối lượng tối ưu: {f_best:.2f} kg\n\n")
        f.write(f"Kích thước mặt cắt tối ưu (cm):\n")
        for i in range(NUM_SEGMENTS):
            f.write(f"  Đoạn {i+1}: r_outer = {r_out[i]:.2f}, r_inner = {r_in[i]:.2f}\n")
        disp = abs(tip_displacement(r_out_m, r_in_m))
        sigma = max_stress(r_out_m, r_in_m)
        f.write(f"\nChuyển vị đầu dầm: {disp*100:.2f} cm\n")
        f.write(f"Ứng suất lớn nhất: {sigma/1e6:.2f} MPa\n")
    print(f"  Đã lưu: optimization_results.txt")

# ============================================================
# 6. CHƯƠNG TRÌNH CHÍNH
# ============================================================

def generate_random_initial(seed=None):
    """
    Tạo nghiệm ban đầu ngẫu nhiên trong miền khả thi.
    r_outer giảm dần (hợp lý cho dầm công xôn), chiều dày trong [T_MIN, T_MAX].
    """
    if seed is not None:
        np.random.seed(seed)

    # Sinh r_outer ngẫu nhiên rồi sắp giảm dần
    r_out = np.random.uniform(R_OUT_MIN + T_MAX, R_OUT_MAX, NUM_SEGMENTS)
    r_out = np.sort(r_out)[::-1]

    # Sinh chiều dày ngẫu nhiên
    t = np.random.uniform(T_MIN, T_MAX, NUM_SEGMENTS)
    r_in = r_out - t

    return np.concatenate([r_out, r_in])

def main():
    save_dir = os.path.dirname(os.path.abspath(__file__))

    # --- Thông số chạy nhiều lần ---
    N_RUNS = 50          # số lần chạy SA
    N_ITER = 5000        # số vòng lặp mỗi lần
    T0 = 100.0           # nhiệt độ ban đầu
    ALPHA = 0.99         # tốc độ làm nguội

    # In thông tin bài toán
    print("=" * 60)
    print("TỐI ƯU HÓA DẦM CÔNG XÔN - SIMULATED ANNEALING")
    print(f"Chạy {N_RUNS} lần từ các điểm xuất phát ngẫu nhiên khác nhau")
    print("=" * 60)
    print(f"Chiều dài dầm:  L = {L} m ({NUM_SEGMENTS} đoạn)")
    print(f"Lực tập trung:  F = {F/1000:.0f} kN")
    print(f"Vật liệu:       Thép S270 (σ_y = {SIGMA_YIELD/1e6:.0f} MPa)")
    print(f"Biến thiết kế:  {2*NUM_SEGMENTS} biến (r_outer, r_inner)")
```

```
    print(f"Giới hạn:       r_outer ≤ {R_OUT_MAX} cm, "
          f"chiều dày {T_MIN}-{T_MAX} cm")

    # --- Chạy SA nhiều lần ---
    all_results = []     # [(x_best, f_best, history), ...]
    all_f_best = []

    for run in range(N_RUNS):
        print(f"\n{'-'*60}")
        print(f"LẦN CHẠY {run + 1}/{N_RUNS}")
        print(f"{'-'*60}")

        # Tạo nghiệm ban đầu ngẫu nhiên
        x0 = generate_random_initial(seed=run * 42 + 7)

        # Kiểm tra khả thi
        w0 = objective(x0)
        feas = is_feasible(x0)
        print(f"  Nghiệm ban đầu: Khối lượng = {w0:.2f} kg, Khả thi: {feas}")

        # Chạy SA
        x_best, f_best, history = simulated_annealing(
            x0, n_iter=N_ITER, T0=T0, alpha=ALPHA
        )

        all_results.append((x_best, f_best, history))
        all_f_best.append(f_best)
        print(f"  → Kết quả: Khối lượng = {f_best:.2f} kg")

    # --- Thống kê tổng hợp ---
    all_f_best = np.array(all_f_best)
    idx_best = np.argmin(all_f_best)
    idx_worst = np.argmax(all_f_best)

    print(f"\n{'='*60}")
    print(f"THỐNG KÊ TỔNG HỢP ({N_RUNS} lần chạy)")
    print(f"{'='*60}")
    print(f"\n  {'Lần':<6} {'Khối lượng (kg)':>16}")
    print(f"  {'-'*24}")
    for i, f in enumerate(all_f_best):
        marker = " ← TỐT NHẤT" if i == idx_best else ""
        print(f"  {i+1:<6} {f:>16.2f}{marker}")

    print(f"\n  {'-'*30}")
    print(f"  Tốt nhất  (lần {idx_best+1}): {all_f_best[idx_best]:.2f} kg")
    print(f"  Kém nhất  (lần {idx_worst+1}): {all_f_best[idx_worst]:.2f} kg")
    print(f"  Trung bình:          {np.mean(all_f_best):.2f} kg")
    print(f"  Độ lệch chuẩn:      {np.std(all_f_best):.2f} kg")
    print(f"  Hệ số biến thiên:   {np.std(all_f_best)/np.mean(all_f_best)*100:.1f}%")
    print(f"{'='*60}")

    # --- In chi tiết nghiệm tốt nhất ---
    x_global_best, f_global_best, history_best = all_results[idx_best]
    print(f"\nCHI TIẾT NGHIỆM TỐT NHẤT (lần chạy {idx_best + 1}):")
    print_results(x_global_best, f_global_best)
    save_results(x_global_best, f_global_best, save_dir)

    # --- Vẽ đồ thị cho nghiệm tốt nhất ---
    print("\nĐang vẽ đồ thị cho nghiệm tốt nhất...")
    plot_history(history_best, save_dir)
    plot_deformed_beam(x_global_best, save_dir)
    plot_beam_shape(x_global_best, save_dir)

    # --- Vẽ biểu đồ so sánh các lần chạy ---
    plt.figure(figsize=(10, 5))
    plt.bar(range(1, N_RUNS + 1), all_f_best,
            color=['green' if i == idx_best else 'steelblue'
                   for i in range(N_RUNS)],
            edgecolor='black', linewidth=0.5)
    plt.axhline(y=np.mean(all_f_best), color='red', linestyle='--',
                linewidth=1.5, label=f'Trung bình = {np.mean(all_f_best):.2f} kg')
    plt.xlabel('Lần chạy')
    plt.ylabel('Khối lượng tối ưu (kg)')
    plt.title(f'So sánh {N_RUNS} lần chạy SA từ các điểm xuất phát khác nhau')
```

```
    plt.legend()
    plt.grid(True, axis='y', linestyle=':', alpha=0.5)
    plt.xticks(range(1, N_RUNS + 1))
    plt.savefig(os.path.join(save_dir, 'multi_run_comparison.png'),
                dpi=300, bbox_inches='tight')
    print(f"  Đã lưu: multi_run_comparison.png")

    print("\nHoàn thành!")

if __name__ == "__main__":
    main()
```

Kết quả chạy Code 8.1

```
Khối lượng tối ưu: 495.25 kg

Kích thước mặt cắt tối ưu (cm):
  Đoạn 1: r_outer = 29.54, r_inner = 27.52
  Đoạn 2: r_outer = 27.72, r_inner = 25.95
  Đoạn 3: r_outer = 26.86, r_inner = 25.54
  Đoạn 4: r_outer = 26.56, r_inner = 25.77
  Đoạn 5: r_outer = 25.91, r_inner = 25.65

Chuyển vị đầu dầm: 0.08 cm
Ứng suất lớn nhất: 269.99 MPa
```

Code 8.2: Ví dụ 8.2

```
"""
Tối ưu hóa kết cấu khung thép bằng thuật toán Simulated Annealing (SA)
kết hợp với SAP2000 qua COM API (OAPI).

Bài toán:
    - Hàm mục tiêu: Cực tiểu hóa trọng lượng kết cấu
    - Biến thiết kế: Tiết diện thép W-section cho 8 nhóm thanh
    - Ràng buộc: Tất cả các thanh phải thỏa mãn tiêu chuẩn thiết kế thép AISC 360

Tài liệu tham khảo:
    - AISC 360-16, Chapter H, Equations H1-1a & H1-1b
    - Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983).
      "Optimization by Simulated Annealing". Science, 220(4598), 671-680.
"""

import os
import sys
import random
import math
import numpy as np
import win32com.client
import time
import matplotlib.pyplot as plt

# ============================================================================
# PHẦN 1: CẤU HÌNH VÀ THAM SỐ
# ============================================================================

# --- Danh sách tiết diện thép W-section (AISC) ---
# Mỗi tiết diện ký hiệu theo dạng: WdXw
#   d = chiều cao tiết diện (inch)
#   w = trọng lượng trên đơn vị chiều dài (lb/ft)
W_SECTIONS = [
    "W12X14", "W12X16", "W12X19", "W12X22", "W12X26", "W12X30", "W12X35", "W12X40",
    "W12X45", "W12X50", "W12X53", "W12X58", "W12X65", "W12X72", "W12X79", "W12X87",
    "W12X96", "W12X106", "W12X120", "W12X136", "W12X152", "W12X170", "W12X190", "W12X210",
    "W12X230", "W12X252", "W12X279", "W12X305", "W12X336", "W14X22", "W14X26", "W14X30",
    "W14X34", "W14X38", "W14X43", "W14X48", "W14X53", "W14X61", "W14X68", "W14X74", "W14X82",
    "W14X90", "W14X99", "W14X109", "W14X120", "W14X132", "W14X145", "W14X159", "W14X176",
    "W14X193", "W14X211", "W14X233", "W14X257", "W14X283", "W14X311", "W14X342", "W14X370",
    "W14X398", "W14X426", "W14X455", "W14X500", "W14X550", "W14X605", "W14X665", "W14X730",
    "W16X26", "W16X31", "W16X36", "W16X40", "W16X45", "W16X50", "W16X57", "W16X67", "W16X77",
    "W16X89", "W16X100", "W18X35", "W18X40", "W18X46", "W18X50", "W18X55", "W18X60", "W18X65",
    "W18X71", "W18X76", "W18X86", "W18X97", "W18X106", "W18X119", "W18X130", "W18X143", "W18X158",
    "W18X175", "W21X44", "W21X48", "W21X50", "W21X55", "W21X57", "W21X62", "W21X68", "W21X73",
    "W21X83", "W21X93", "W21X101", "W21X111", "W21X122", "W21X132", "W21X147", "W21X166",
```

```
    "W21X182", "W21X201", "W24X55", "W24X62", "W24X68", "W24X76", "W24X84", "W24X94",
    "W24X103", "W24X104", "W24X117", "W24X131", "W24X146", "W24X162", "W24X176", "W24X192",
    "W24X207", "W24X229", "W24X250", "W24X279", "W24X306", "W24X335", "W24X370", "W27X84",
    "W27X94", "W27X102", "W27X114", "W27X129", "W27X146", "W27X161", "W27X178", "W27X194",
    "W27X217", "W27X235", "W27X258", "W27X281", "W27X307", "W27X336", "W27X368", "W27X539",
    "W30X90", "W30X99", "W30X108", "W30X116", "W30X124", "W30X132", "W30X148", "W30X173",
    "W30X191", "W30X211", "W30X235", "W30X261", "W30X292", "W30X326", "W30X357", "W30X391",
    "W33X118", "W33X130", "W33X141", "W33X152", "W33X169", "W33X201", "W33X221",
    "W33X241", "W33X263", "W33X291", "W33X318", "W33X354", "W33X387"
]

# --- Tham số mô hình SAP2000 ---
MODEL_FILE = "model_v24_2.sdb"                                  # Tên file mô hình SAP2000
CURRENT_DIR = os.path.dirname(os.path.abspath(__file__))        # Thư mục chứa file code
MODEL_PATH = os.path.join(CURRENT_DIR, MODEL_FILE)              # Đường dẫn đầy đủ đến mô hình
JOINTS = ["1", "10", "19", "28", "37"]                          # Các nút gối tựa (dùng để tính trọng lượng)
NUM_ELEMENTS = 8                                                # Số nhóm thanh cần tối ưu (G1 → G8)

# --- Tham số thuật toán Simulated Annealing ---
INITIAL_TEMP = 100      # Nhiệt độ ban đầu T_0
COOLING_RATE = 0.98      # Hệ số làm nguội α (T_mới = α × T_cũ)
MIN_TEMP = 0.1           # Nhiệt độ dừng T_min
MAX_ITER = 80            # Số vòng lặp tối đa
PENALTY_FACTOR = 1000    # Hệ số phạt cho lời giải không khả thi
RESTART_THRESHOLD = 10   # Số vòng không cải thiện trước khi khởi động lại

# --- Thiết lập Random Seed để kết quả luôn giống nhau mỗi lần chạy ---
RANDOM_SEED = 42
random.seed(RANDOM_SEED)

# ============================================================================
# PHẦN 2: GIAO TIẾP VỚI SAP2000 (OAPI)
# ============================================================================

def start_sap():
    """
    Kết nối hoặc khởi tạo phiên làm việc SAP2000 qua COM API.

    Returns:
        sap_object: Đối tượng ứng dụng SAP2000
        sap_model:  Đối tượng mô hình SAP2000
    """
    try:
        # Thử kết nối đến SAP2000 đang mở
        try:
            sap_object = win32com.client.GetActiveObject("CSI.SAP2000.API.SapObject")
            print("Đã kết nối đến SAP2000 đang chạy")
        except:
            # Nếu chưa mở, khởi tạo SAP2000 mới
            print("Đang khởi tạo SAP2000 mới...")
            sap_object = win32com.client.Dispatch("CSI.SAP2000.API.SapObject")

        # Khởi động ứng dụng SAP2000
        ret = sap_object.ApplicationStart()
        if ret != 0:
            raise Exception("Không thể khởi động SAP2000")

        # Khởi tạo mô hình rỗng
        sap_model = sap_object.SapModel
        ret = sap_model.InitializeNewModel()
        if ret != 0:
            raise Exception("Không thể khởi tạo mô hình mới")

        return sap_object, sap_model

    except Exception as e:
        print(f"Lỗi khi khởi động SAP2000: {str(e)}")
        raise

def assign_sections(sap_model, solution):
    """
    Gán tiết diện W-section cho từng nhóm thanh trong mô hình.

    Mỗi phần tử trong lời giải là chỉ số (index) tương ứng với
    một tiết diện trong danh sách W_SECTIONS.
```

```
122
123     Args:
124         sap_model: Đối tượng mô hình SAP2000
125         solution:  Mảng [s_1, s_2, ..., s_8], trong đó s_i là chỉ số tiết diện cho nhóm G_i
126     """
127     for i in range(len(solution)):
128         group_name = f"G{i+1}"                          # Tên nhóm thanh: G1, G2, ..., G8
129         section_name = W_SECTIONS[solution[i]]           # Tra tiết diện từ danh sách
130         # ItemType=1: gán cho tất cả thanh trong nhóm
131         sap_model.FrameObj.SetSection(group_name, section_name, ItemType=1)
132
133 def open_model(sap_model):
134     """
135     Mở file mô hình SAP2000 và lưu bản sao tạm thời.
136
137     Returns:
138         sap_model: Đối tượng mô hình SAP2000 đã mở
139     """
140     # Mở file mô hình gốc
141     ret = sap_model.File.OpenFile(MODEL_PATH)
142     if ret != 0:
143         raise Exception(f"Không thể mở file mô hình: {MODEL_FILE}")
144
145     # Lưu bản sao tạm thời (tránh ghi đè file gốc)
146     temp_path = os.path.join(CURRENT_DIR, "model_temp3.sdb")
147     sap_model.File.Save(temp_path)
148
149     return sap_model
150
151 def run_analysis(sap_model):
152     """
153     Chạy phân tích kết cấu trong SAP2000.
154     """
155     sap_model.Analyze.RunAnalysis()
156
157 def check_design(sap_model):
158     """
159     Chạy kiểm tra thiết kế thép theo AISC 360.
160
161     SAP2000 sẽ kiểm tra tất cả thanh theo tiêu chuẩn AISC:
162     - Kiểm tra bền (strength)
163     - Kiểm tra ổn định (stability)
164     - Tỷ số demand/capacity (D/C ratio)
165
166     Returns:
167         num_not_passed: Số thanh không thỏa mãn (0 = tất cả đạt)
168     """
169     # Chạy thiết kế thép
170     sap_model.DesignSteel.StartDesign()
171
172     # Kiểm tra kết quả: có bao nhiêu thanh không đạt?
173     ret, num_not_passed, failed, not_checked, failed_frames = \
174         sap_model.DesignSteel.VerifyPassed()
175
176     return num_not_passed
177
178 def get_weight(sap_model):
179     """
180     Tính tổng trọng lượng kết cấu từ phản lực gối tựa.
181
182     Trọng lượng = Tổng phản lực đứng (F3) tại tất cả gối tựa,
183     lấy từ trường hợp tải "weight" (trọng lượng bản thân).
184
185     Returns:
186         weight: Tổng trọng lượng kết cấu (kips)
187     """
188     # Chọn trường hợp tải "weight" để xuất kết quả
189     sap_model.Results.Setup.SetCaseSelectedForOutput("weight")
190
191     # Khởi tạo biến đầu ra cho hàm JointReact
192     NumberResults = 0
193     Obj = []; Elm = []; LoadCase = []; StepType = []; StepNum = []
194     F1 = []; F2 = []; F3 = []; M1 = []; M2 = []; M3 = []
195
196     # Tính tổng phản lực đứng tại tất cả gối tựa
```

```
    weight = 0
    for joint in JOINTS:
        result = sap_model.Results.JointReact(
            joint, 1, NumberResults, Obj, Elm, LoadCase,
            StepType, StepNum, F1, F2, F3, M1, M2, M3
        )
        # result[9] chứa mảng F3 (phản lực theo phương đứng)
        weight += result[9][0]

    return weight

# ============================================================================
# PHẦN 3: ĐỊNH NGHĨA BÀI TOÁN TỐI ƯU
# ============================================================================

def objective_function(sap_model, solution):
    """
    Đánh giá lời giải: tính trọng lượng và kiểm tra ràng buộc.

    Quy trình:
        1. Mở mô hình SAP2000
        2. Gán tiết diện theo lời giải
        3. Chạy phân tích kết cấu
        4. Kiểm tra thiết kế thép (ràng buộc)
        5. Tính trọng lượng kết cấu (hàm mục tiêu)

    Args:
        sap_model: Đối tượng mô hình SAP2000
        solution:  Mảng chỉ số tiết diện [s1, s2, ..., s8]

    Returns:
        weight:         Trọng lượng kết cấu (kips)
        num_not_passed: Số thanh không thỏa mãn ràng buộc
    """
    # Bước 1: Mở mô hình và lưu bản sao tạm
    sap_model = open_model(sap_model)

    # Bước 2: Gán tiết diện cho từng nhóm thanh
    assign_sections(sap_model, solution)

    # Bước 3: Chạy phân tích kết cấu
    run_analysis(sap_model)

    # Bước 4: Kiểm tra ràng buộc thiết kế thép (AISC 360)
    num_not_passed = check_design(sap_model)

    # Bước 5: Tính trọng lượng kết cấu
    weight = get_weight(sap_model)

    # In kết quả
    feasible = (num_not_passed == 0)
    sections = [W_SECTIONS[i] for i in solution]
    print(f"  Tiết diện: {sections}")
    print(f"  Trọng lượng: {weight:.2f} kips | Khả thi: {feasible}")

    return weight, num_not_passed

def penalized_cost(weight, num_not_passed):
    """
    Tính chi phí có phạt (penalized cost) cho lời giải.

    Phương pháp hàm phạt (penalty function):
        - Nếu lời giải khả thi (tất cả thanh đạt): chi phí = trọng lượng
        - Nếu lời giải KHÔNG khả thi: chi phí = trọng lượng × PENALTY_FACTOR

    Hệ số phạt lớn (= 1000) đảm bảo lời giải vi phạm ràng buộc
    hầu như luôn bị loại bỏ trong quá trình tối ưu.

    Args:
        weight:         Trọng lượng kết cấu
        num_not_passed: Số thanh vi phạm ràng buộc

    Returns:
        cost: Chi phí sau khi áp dụng hàm phạt
    """
```

```python
    if num_not_passed > 0:
        return weight * PENALTY_FACTOR  # Phạt nặng nếu vi phạm
    else:
        return weight                   # Giữ nguyên nếu khả thi

# ============================================================================
# PHẦN 4: THUẬT TOÁN SIMULATED ANNEALING (SA)
# ============================================================================

def generate_initial_solution():
    """
    Tạo lời giải ban đầu.

    Sử dụng tiết diện bảo thủ (W14X90, index = 41) cho tất cả nhóm thanh
    để đảm bảo lời giải ban đầu là khả thi, giúp thuật toán hội tụ nhanh hơn.

    Returns:
        Mảng [41, 41, ..., 41] có NUM_ELEMENTS phần tử
    """
    return [41 for _ in range(NUM_ELEMENTS)]

def generate_neighbor(current_solution):
    """
    Tạo lời giải lân cận bằng cách thay đổi ngẫu nhiên 1-2 tiết diện.

    Cơ chế tạo lân cận (neighborhood structure):
        1. Chọn ngẫu nhiên 1 hoặc 2 nhóm thanh
        2. Thay thế tiết diện của mỗi nhóm bằng một tiết diện ngẫu nhiên khác

    Thay đổi 1-2 phần tử giúp cân bằng giữa:
        - Khám phá (exploration): thay đổi đủ lớn để thoát cực trị địa phương
        - Khai thác (exploitation): thay đổi đủ nhỏ để cải thiện dần lời giải

    Args:
        current_solution: Lời giải hiện tại [$s_1$, $s_2$, ..., $s_8$]

    Returns:
        neighbor: Lời giải lân cận
    """
    neighbor = current_solution.copy()

    # Chọn ngẫu nhiên số phần tử cần thay đổi (1 hoặc 2)
    num_changes = random.randint(1, 2)

    for _ in range(num_changes):
        # Chọn ngẫu nhiên nhóm thanh cần thay đổi
        idx = random.randint(0, NUM_ELEMENTS - 1)

        # Chọn tiết diện mới (khác tiết diện hiện tại)
        old_section = neighbor[idx]
        new_section = old_section
        while new_section == old_section:
            new_section = random.randint(0, len(W_SECTIONS) - 1)

        neighbor[idx] = new_section

    return neighbor

def acceptance_probability(old_cost, new_cost, temperature):
    """
    Tính xác suất chấp nhận lời giải mới theo tiêu chuẩn Metropolis.

    Công thức Boltzmann:
        - Nếu f(x_mới) < f(x_cũ):  P = 1.0        (luôn chấp nhận lời giải tốt hơn)
        - Nếu f(x_mới) ≥ f(x_cũ):  P = exp(-(f_mới - f_cũ) / T)

    Ý nghĩa:
        - Nhiệt độ T cao → P lớn → dễ chấp nhận lời giải xấu hơn (khám phá)
        - Nhiệt độ T thấp → P nhỏ → chỉ chấp nhận lời giải tốt hơn (khai thác)

    Args:
        old_cost:    Chi phí lời giải hiện tại f(x_cũ)
        new_cost:    Chi phí lời giải mới f(x_mới)
        temperature: Nhiệt độ hiện tại T

```

```
    Returns:
        Xác suất chấp nhận P ∈ [0, 1]
    """
    # Trường hợp 1: Lời giải mới TỐT HƠN → luôn chấp nhận
    if new_cost < old_cost:
        return 1.0

    # Trường hợp 2: Lời giải mới XẤU HƠN → chấp nhận theo xác suất Boltzmann
    delta = new_cost - old_cost           # Δf = f_mới - f_cũ > 0
    return math.exp(-delta / temperature)  # P = exp(-Δf / T)

def simulated_annealing(sap_model):
    """
    Vòng lặp chính của thuật toán Simulated Annealing.

    Pseudocode:
        1. Tạo lời giải ban đầu x₀, tính f(x₀)
        2. Đặt T = T₀, x* = x₀ (lời giải tốt nhất)
        3. Lặp cho đến khi T < T_min hoặc đạt MAX_ITER:
            3.1. Tạo lời giải lân cận x' từ x hiện tại
            3.2. Tính f(x') và chi phí có phạt
            3.3. Nếu random() < P(f(x), f(x'), T):
                    Chấp nhận: x ← x'
                    Nếu f(x') < f(x*): cập nhật x* ← x'
            3.4. Giảm nhiệt độ: T ← α × T
        4. Trả về x* (lời giải tốt nhất)

    Args:
        sap_model: Đối tượng mô hình SAP2000

    Returns:
        best_solution: Mảng tiết diện tối ưu
        best_weight:   Trọng lượng tối ưu
        best_feasible: True nếu lời giải tốt nhất thỏa mãn ràng buộc
    """

    # ----- Bước 1: Khởi tạo lời giải ban đầu -----
    print("=" * 60)
    print("THUẬT TOÁN SIMULATED ANNEALING")
    print("=" * 60)

    current_x = generate_initial_solution()
    print(f"\nLời giải ban đầu: {[W_SECTIONS[i] for i in current_x]}")

    # Đánh giá lời giải ban đầu
    current_w, current_violations = objective_function(sap_model, current_x)
    current_cost = penalized_cost(current_w, current_violations)

    print(f"\nTrọng lượng ban đầu: {current_w:.2f} kips")
    print(f"Khả thi: {current_violations == 0}")

    # ----- Bước 2: Khởi tạo lời giải tốt nhất -----
    best_x = current_x.copy()
    best_cost = current_cost
    best_w = current_w
    best_violations = current_violations

    # ----- Khởi tạo biến theo dõi -----
    temperature = INITIAL_TEMP             # Nhiệt độ hiện tại T
    iteration = 0                           # Bộ đếm vòng lặp
    no_improve_count = 0                    # Bộ đếm vòng không cải thiện
    history = []                            # Lịch sử các lời giải tốt nhất

    # Dữ liệu cho đồ thị hội tụ
    weight_list = [current_w]
    iter_list = [0]
    feasible_list = [current_violations == 0]

    # ----- Bước 3: Vòng lặp chính -----
    print(f"\n{'='*60}")
    print(f"Bắt đầu tối ưu hóa: T₀={INITIAL_TEMP}, α={COOLING_RATE}, MaxIter={MAX_ITER}")
    print(f"{'='*60}")

    while temperature > MIN_TEMP and iteration < MAX_ITER:
        iteration += 1
```

```
422
423         # --- Bước 3.1: Tạo lời giải lân cận ---
424         neighbor_x = generate_neighbor(current_x)
425
426         # --- Bước 3.2: Đánh giá lời giải mới ---
427         print(f"\n--- Vòng lặp {iteration} | T = {temperature:.2f} ---")
428         neighbor_w, neighbor_violations = objective_function(sap_model, neighbor_x)
429         neighbor_cost = penalized_cost(neighbor_w, neighbor_violations)
430
431         # --- Bước 3.3: Quyết định chấp nhận theo tiêu chuẩn Metropolis ---
432         P = acceptance_probability(current_cost, neighbor_cost, temperature)
433         r = random.random()    # Số ngẫu nhiên r ∈ [0, 1)
434
435         if r < P:
436             # CHẤP NHẬN lời giải mới
437             current_x = neighbor_x
438             current_cost = neighbor_cost
439             current_w = neighbor_w
440             current_violations = neighbor_violations
441
442             result_str = "✓ CHẤP NHẬN"
443             if neighbor_cost >= current_cost:
444                 result_str += f" (P={P:.4f}, r={r:.4f})"
445
446             # Cập nhật lời giải tốt nhất nếu cần
447             if neighbor_cost < best_cost or \
448                (neighbor_cost == best_cost and neighbor_violations < best_violations):
449                 best_x = neighbor_x.copy()
450                 best_cost = neighbor_cost
451                 best_w = neighbor_w
452                 best_violations = neighbor_violations
453                 no_improve_count = 0
454
455                 history.append((best_x.copy(), best_w, best_violations == 0))
456                 print(f"  ★ LỜI GIẢI TỐT NHẤT MỚI! W = {best_w:.2f} kips")
457             else:
458                 no_improve_count += 1
459         else:
460             # TỪ CHỐI lời giải mới
461             no_improve_count += 1
462             result_str = f"× TỪ CHỐI (P={P:.4f}, r={r:.4f})"
463
464         print(f"  {result_str}")
465
466         # Lưu dữ liệu cho đồ thị
467         weight_list.append(current_w)
468         iter_list.append(iteration)
469         feasible_list.append(current_violations == 0)
470
471         # --- Bước 3.4: Giảm nhiệt độ ---
472         temperature *= COOLING_RATE    # T ← α × T
473
474         # Khởi động lại từ lời giải tốt nhất nếu không cải thiện quá lâu
475         if no_improve_count >= RESTART_THRESHOLD:
476             print(f"\n  ↻ Khởi động lại từ lời giải tốt nhất "
477                   f"(không cải thiện {RESTART_THRESHOLD} vòng)")
478             current_x = best_x.copy()
479             current_cost = best_cost
480             current_w = best_w
481             current_violations = best_violations
482             no_improve_count = 0
483
484     # ----- Bước 4: Kết quả cuối cùng -----
485     best_feasible = (best_violations == 0)
486
487     print(f"\n{'='*60}")
488     print(f"KẾT QUẢ TỐI ƯU HÓA")
489     print(f"{'='*60}")
490     print(f"Tiết diện tối ưu: {[W_SECTIONS[i] for i in best_x]}")
491     print(f"Trọng lượng:      {best_w:.2f} kips")
492     print(f"Khả thi:          {best_feasible}")
493     print(f"Số vòng lặp:      {iteration}")
494
495     # In lịch sử cải thiện
```

```
    if history:
        print(f"\nLịch sử cải thiện lời giải tốt nhất:")
        for idx, (sol, w, feasible) in enumerate(history):
            print(f"  {idx+1}. W = {w:.2f} kips | Khả thi: {feasible}")
            print(f"     Tiết diện: {[W_SECTIONS[i] for i in sol]}")

    # Vẽ đồ thị hội tụ
    plot_convergence(iter_list, weight_list, feasible_list)

    return best_x, best_w, best_feasible

# ============================================================================
# PHẦN 5: KẾT QUẢ VÀ TRỰC QUAN HÓA
# ============================================================================

def plot_convergence(iter_list, weight_list, feasible_list):
    """
    Vẽ đồ thị quá trình hội tụ của thuật toán SA.

    - Điểm xanh (●): lời giải khả thi
    - Điểm đỏ  (×): lời giải không khả thi
    - Đường xanh:    trọng lượng theo vòng lặp
    """
    fig, ax = plt.subplots(figsize=(10, 6))

    # Vẽ đường trọng lượng
    ax.plot(iter_list, weight_list, 'b-', linewidth=1, alpha=0.7, label='Trọng lượng')

    # Phân loại điểm khả thi / không khả thi
    f_iters = [iter_list[i] for i, f in enumerate(feasible_list) if f]
    f_weights = [weight_list[i] for i, f in enumerate(feasible_list) if f]
    nf_iters = [iter_list[i] for i, f in enumerate(feasible_list) if not f]
    nf_weights = [weight_list[i] for i, f in enumerate(feasible_list) if not f]

    # Vẽ điểm phân tán
    ax.scatter(f_iters, f_weights, c='green', marker='o', s=30,
               label='Khả thi', zorder=5)
    ax.scatter(nf_iters, nf_weights, c='red', marker='x', s=30,
               label='Không khả thi', zorder=5)

    # Định dạng đồ thị
    ax.set_xlabel('Vòng lặp', fontsize=12)
    ax.set_ylabel('Trọng lượng (kips)', fontsize=12)
    ax.set_title('Quá trình hội tụ - Simulated Annealing', fontsize=14)
    ax.legend(fontsize=11)
    ax.grid(True, alpha=0.3)

    plt.tight_layout()
    plt.savefig(os.path.join(CURRENT_DIR, 'optimization_progress.png'), dpi=150)
    plt.show()
    print(f"Đã lưu đồ thị: optimization_progress.png")

def print_summary(solution, weight, feasible):
    """In bảng tóm tắt kết quả tối ưu."""
    print(f"\n{'='*60}")
    print(f"TÓM TẮT KẾT QUẢ TỐI ƯU HÓA")
    print(f"{'='*60}")
    print(f"{'Nhóm thanh':<12} {'Tiết diện':<15}")
    print(f"{'-'*27}")
    for i, idx in enumerate(solution):
        print(f"  G{i+1:<9} {W_SECTIONS[idx]:<15}")
    print(f"{'-'*27}")
    print(f"Trọng lượng tối ưu: {weight:.2f} kips")
    print(f"Tất cả thanh đạt:  {feasible}")
    print(f"{'='*60}")

# ============================================================================
# PHẦN 6: CHƯƠNG TRÌNH CHÍNH
# ============================================================================

def main():
    """
    Chương trình chính: khởi động SAP2000, chạy tối ưu hóa, lưu kết quả.
    """
    try:
```

```
        # Bước 1: Khởi động SAP2000
        print("Đang kết nối SAP2000...")
        sap_object, sap_model = start_sap()

        # Bước 2: Chạy tối ưu hóa bằng Simulated Annealing
        best_solution, best_weight, best_feasible = simulated_annealing(sap_model)

        # Bước 3: Áp dụng lời giải tốt nhất và lưu mô hình
        if best_feasible:
            print("\nĐang áp dụng lời giải tối ưu vào mô hình...")
            objective_function(sap_model, best_solution)

            output_path = os.path.join(CURRENT_DIR, "model_optimized.sdb")
            sap_model.File.Save(output_path)
            print(f"Đã lưu mô hình tối ưu: {output_path}")
        else:
            print("\n! Cảnh báo: Lời giải tốt nhất KHÔNG khả thi. Không lưu mô hình.")

        # In tóm tắt
        print_summary(best_solution, best_weight, best_feasible)

    except Exception as e:
        print(f"\nLỗi: {str(e)}")

    finally:
        # Đóng SAP2000
        if 'sap_object' in locals():
            try:
                print("\nĐang đóng SAP2000...")
                sap_object.ApplicationExit(False)
                print("Đã đóng SAP2000 thành công")
            except:
                print("Lỗi khi đóng SAP2000")

if __name__ == "__main__":
    main()
```

Code 8.3: Giàn 120 thanh

```
function Truss_Data=TrussData_120bar

% Modulus of Elasticity (psi)
E=30450000;

% Node coordinates
dnkoor=[0 0 275.59;
273.26  0 196.85;
236.6501  136.63  196.85;
136.63  236.6501  196.85;
0 273.26  196.85;
-136.63 236.6501  196.85;
-236.6501 136.63  196.85;
-273.26 0 196.85;
-236.6501 -136.63 196.85;
-136.63 -236.6501 196.85;
0 -273.26 196.85;
136.63  -236.6501 196.85;
236.6501  -136.63 196.85;
492.12  0 118.11;
475.3514  127.37  118.11;
426.1884  246.06  118.11;
347.9814  347.9814  118.11;
246.06  426.1884  118.11;
127.37  475.3514  118.11;
0 492.12  118.11;
-127.37 475.3514  118.11;
-246.06 426.1884  118.11;
-347.9814 347.9814  118.11;
-426.1884 246.06  118.11;
-475.3514 127.37  118.11;
-492.12 0 118.11;
-475.3514 -127.37 118.11;
-426.1884 -246.06 118.11;
-347.9814 -347.9814 118.11;
-246.06 -426.1884 118.11;
```

```
-127.37 -475.3514 118.11;
0 -492.12 118.11;
127.37  -475.3514 118.11;
246.06  -426.1884 118.11;
347.9814  -347.9814 118.11;
426.1884  -246.06 118.11;
475.3514  -127.37 118.11;
625.59  0 0;
541.7768  312.795 0;
312.795 541.7768  0;
0 625.59  0;
-312.795  541.7768  0;
-541.7768 312.795 0;
-625.59 0 0;
-541.7768 -312.795  0;
-312.795  -541.7768 0;
0 -625.59 0;
312.795 -541.7768 0;
541.7768  -312.795  0];

% Bar connections between nodes
eldn=[1 2;
1 3;
1 4;
1 5;
1 6;
1 7;
1 8;
1 9;
1 10;
1 11;
1 12;
1 13;
2 3;
3 4;
4 5;
5 6;
6 7;
7 8;
8 9;
9 10;
10  11;
11  12;
12  13;
13  2;
2 14;
3 16;
4 18;
5 20;
6 22;
7 24;
8 26;
9 28;
10  30;
11  32;
12  34;
13  36;
2 15;
3 15;
3 17;
4 17;
4 19;
5 19;
5 21;
6 21;
6 23;
7 23;
7 25;
8 25;
8 27;
9 27;
9 29;
10  29;
10  31;
11  31;
```

```
11  33;
12  33;
12  35;
13  35;
13  37;
2 37;
14  15;
15  16;
16  17;
17  18;
18  19;
19  20;
20  21;
21  22;
22  23;
23  24;
24  25;
25  26;
26  27;
27  28;
28  29;
29  30;
30  31;
31  32;
32  33;
33  34;
34  35;
35  36;
36  37;
37  14;
14  38;
16  39;
18  40;
20  41;
22  42;
24  43;
26  44;
28  45;
30  46;
32  47;
34  48;
36  49;
15  38;
15  39;
17  39;
17  40;
19  40;
19  41;
21  41;
21  42;
23  42;
23  43;
25  43;
25  44;
27  44;
27  45;
29  45;
29  46;
31  46;
31  47;
33  47;
33  48;
35  48;
35  49;
37  49;
37  38];

% Support conditions
MesKos=[0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
```

```
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
0 0 0;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1;
1 1 1];

% Loading     ----- > Load(dn_no,:)=[x y z] (lbs)
yuk=zeros(size(dnkoor));
yuk(1,:)=[0 0 -13490];
for i=2:14
yuk(i,:)=[0 0 -6744];
end
for i=15:37
yuk(i,:)=[0 0 -2248];
end

% Steel yield strength (psi)
fy=58000;

% Allowable Stress
maxYER=0.1969;%inch

% Unit weight
GS=0.288;%lb/in^3

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,'E',E,'yuk',yuk,'fy',fy','maxYER',maxYER,'
    ↪ GS',GS);
```

Code 8.4: Giàn 37 thanh modal

```
function Truss_Data=TrussData_37bar_modal

% Modulus of Elasticity
E=2.10e11;

% Node coordinates
dnkoor=zeros(20,3);
dnkoor=[0       0       0;
```

```
1       1       0;;
1       0       0;
2       1       0;;
2       0       0;
3       1       0;;
3       0       0;
4       1       0;;
4       0       0;
5       1       0;;
5       0       0;
6       1       0;;
6       0       0;
7       1       0;;
7       0       0;
8       1       0;;
8       0       0;
9       1       0;;
9       0       0;
10      0       0];

% Bar connections between nodes
eldn=[1 3       ;
2       3       ;
3       4       ;
3       5       ;
4       5       ;
5       6       ;
5       7       ;
6       7       ;
7       8       ;
7       9       ;
8       9       ;
9       10      ;
9       11      ;
10      11      ;
10      13      ;
11      13      ;
12      13      ;
12      15      ;
13      15      ;
14      15      ;
14      17      ;
15      17      ;
16      17      ;
16      19      ;
17      19      ;
18      19      ;
19      20      ;
1       2       ;
2       4       ;
4       6       ;
6       8       ;
8       10      ;
10      12      ;
12      14      ;
14      16      ;
16      18      ;
18      20      ];

% Support conditions
MesKos=[1       1       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
```

```
0       0       1       ;
0       0       1       ;
0       0       1       ;
0       0       1       ;
1       1       1       ];

% Fixed cross sections
A=zeros(size(eldn,1),1);
A(28:37)=4e-3; %m2

% Frequency limits (Hz)
limfre(1)=20;
limfre(2)=40;
limfre(3)=60;

% Unit weight
GS=7800;%kg/m^3

% added masses
add_mass=zeros(size(dnkoor,1),1);
add_mass(2:2:18)=10; %kg

% Bar grouping (14 symmetric groups for the 27 active members)
% Elements 28-37 are constant and excluded from active design groups
groups={[1,27], [4,25], [7,22], [10,19], [13,16], ... % Lower chords (5 groups)
        [2,26], [5,23], [8,20], [11,17], [14],    ... % Verticals (5 groups)
        [3,24], [6,21], [9,18], [12,15]};             % Diagonals (4 groups)

% Design variable bounds (19 variables total)
% Variables 1-5: Shape variables (Y-coordinates of upper nodes in meters)
% Variables 6-19: Sizing variables (Cross-sectional areas of the 14 groups in m^2)
LB = [ones(1,5)*0.5, ones(1,14)*1e-4];   % Y_min = 0.5m, A_min = 1 cm2
UB = [ones(1,5)*5.0, ones(1,14)*100e-4]; % Y_max = 5.0m, A_max = 100 cm2

Truss_Data=struct('dnkoor',dnkoor,'MesKos',MesKos,'eldn',eldn,'E',E,'GS',GS,'add_mass',add_mass,'limfre',
    ↪ limfre,'A',A,'groups',{groups},'LB',LB,'UB',UB);
```

Code 8.5: Giàn 52 thanh modal

```
function Truss_Data=TrussData_52bar_modal

% Modulus of Elasticity
E=2.10e11;

% Bar connections between nodes
eldn=[1 4 ;
1 5 ;
1 2 ;
1 3 ;
4 10  ;
5 12  ;
2 6 ;
3 8 ;
4 11  ;
11  5 ;
5 13  ;
2 13  ;
2 7 ;
3 7 ;
3 9 ;
4 9 ;
4 5 ;
5 2 ;
2 3 ;
3 4 ;
10  11  ;
11  12  ;
12  13  ;
13  6 ;
6 7 ;
7 8 ;
8 9 ;
9 10  ;
10  18  ;
11  19  ;
```

```
12  20  ;
13  21  ;
6 14  ;
7 15  ;
8 16  ;
9 17  ;
10  19  ;
12  19  ;
12  21  ;
6 21  ;
6 15  ;
8 15  ;
8 17  ;
10  17  ;
11  18  ;
11  20  ;
13  20  ;
13  14  ;
7 14  ;
7 16  ;
9 16  ;
9 18  ];

% Support conditions
MesKos=[0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
0 0 0 ;
1 1 1 ;
1 1 1 ;
1 1 1 ;
1 1 1 ;
1 1 1 ;
1 1 1 ;
1 1 1 ;
1 1 1 ];

% Frequency limits (Hz)
limfre(1)=15.916;
limfre(2)=28.648;

% Unit weight
GS=7800;%kg/m^3

% added masses
add_mass=zeros(size(MesKos,1));
add_mass(1:13)=50; %kg

Truss_Data=struct('MesKos',MesKos,'eldn',eldn,'E',E,'GS',GS,'add_mass',add_mass,'limfre',limfre);
```

TÀI LIỆU THAM KHẢO

Haftka, Raphael T., and Zafer Gürdal. Elements of structural optimization. Vol. 11. Springer Science & Business Media, 2012.

Kirsch, Uri. Structural optimization: fundamentals and applications. Springer Science & Business Media, 2012.

Bendsoe, Martin Philip, and Ole Sigmund. Topology optimization: theory, methods, and applications. Springer Science & Business Media, 2013.

Christensen, Peter W., and Anders Klarbring. An introduction to structural optimization. Vol. 153. Springer Science & Business Media, 2008.

Arora, Jasbir Singh. Introduction to optimum design. Elsevier, 2004.

Rao, Singiresu S. Engineering optimization: theory and practice. John Wiley & Sons, 2019.

2019_Rao_Engineering Optimization Theory and Practice

Martins, Joaquim RRA, and Andrew Ning. Engineering design optimization. Cambridge University Press, 2021.

Đặng Xuân Phương, Thiết kế tối ưu trong kỹ thuật và Cơ khí với sự hỗ trợ của máy tính, NXB Khoa học và Kỹ thuật, 2017

Lê Hồng Quân, Các phương pháp tối ưu thiết kế, Nhà xuất bản Xây dựng, 2022.

...